A/B Testing: Statistical Foundations and Practical Implementation

Diogo Ribeiro
ESMAD - Instituto Politécnico do Porto

November 19, 2025

Contents

List of Figures

List of Tables

Preface

The idea for this book crystallized during a late-night debugging session at Mysense.ai, where I serve as Lead Data Scientist. We were investigating why an A/B test that should have been straightforward—comparing two checkout button designs—had produced confusing results. The statistical significance was marginal, the effect size was tiny, yet business stakeholders were pressing for a decision. As I walked through the analysis with the team, I realized that the real challenge wasn't the mathematics or the code—it was the gap between statistical rigor and practical decision-making under uncertainty.

This experience, repeated countless times across industry and academia, revealed a fundamental problem: most practitioners learn A/B testing from one of two extremes. Either they study rigorous statistical theory without adequate grounding in real-world constraints, or they learn practical "recipes" without understanding the underlying assumptions and limitations. Both approaches leave critical gaps. The theorist may design a perfectly valid experiment that's impossible to implement in production. The practitioner may run hundreds of tests without recognizing systematic biases that invalidate their conclusions.

This book attempts to bridge that divide.

Why This Book, Why Now

The practice of experimentation has undergone a quiet revolution over the past two decades. What began as a specialized technique used primarily in pharmaceutical trials and agricultural research has become the cornerstone of decision-making at technology companies, e-commerce platforms, government agencies, and organizations of all types. Google runs tens of thousands of experiments annually. Amazon tests nearly every product change. Netflix uses experimentation to optimize everything from recommendation algorithms to thumbnail images. Even traditional industries—retail, finance, healthcare—increasingly embrace data-driven experimentation.

Yet this democratization of A/B testing has created new challenges. The barrier to running an experiment has dropped dramatically—modern platforms make it trivially easy to split traffic and measure outcomes. But the barrier to running a *good* experiment, one that produces trustworthy insights and leads to sound decisions, remains high. Statistical literacy has not kept pace with experimental practice.

The consequences manifest in multiple ways. I've seen teams celebrate "winning" variants that were merely lucky noise. I've watched organizations make million-dollar decisions based on p-values they didn't understand. I've encountered experiments that violated basic statistical assumptions yet were treated as gospel. Most concerning, I've observed a growing cynicism about experimentation itself when poorly designed tests produce conflicting or irreproducible results.

These experiences, accumulated through my dual roles—as Lead Data Scientist at Mysense.ai, where I design and analyze experiments for real products and real customers, and as Researcher and Instructor at ESMAD (Instituto Politécnico do Porto), where I teach the statistical foundations to the next generation of data scientists—convinced me that a different kind of book was needed.

Who This Book Is For

I wrote this book with several overlapping audiences in mind, all united by a need to understand both the "how" and the "why" of A/B testing.

Data Scientists and Analysts working in industry will find practical guidance grounded in statistical rigor. You face daily pressures to move fast, to test everything, to "let the data decide." This book will help you move quickly *and* correctly, understanding when standard methods apply and when you need something more sophisticated. You'll learn to spot the subtle implementation issues that invalidate experiments, to communicate uncertainty clearly to stakeholders, and to push back on unreasonable demands ("Can we just peek at the results?") with solid statistical reasoning.

Graduate Students in statistics, data science, computer science, or related fields will find a modern treatment of experimental design that connects classical theory to contemporary practice. The mathematical development is rigorous—I don't shy away from proofs when they illuminate understanding—but always motivated by practical questions. You'll see how the Central Limit Theorem justifies real-world approximations, why Bayesian methods handle certain problems elegantly, and where theory meets the messiness of production systems.

Researchers in academia or industry who design experiments will discover how classical experimental design principles translate to digital contexts. If you're familiar with randomized controlled trials in clinical or social science settings, you'll learn how online experimentation differs—both the new opportunities (instant results, massive sample sizes) and new challenges (network effects, novelty effects, implementation bugs).

Product Managers and Technical Leaders who make decisions based on experimental evidence need to understand what the numbers really mean. This book will help you ask the right questions: Is this sample size adequate? Are we measuring the right thing? What assumptions underlie this analysis? You'll develop intuition for when to trust experimental results and when to dig deeper.

Practitioners Transitioning from Other Fields who have statistical training but are new to online experimentation will find familiar concepts (hypothesis testing, confidence intervals, power analysis) applied in novel contexts. You'll learn the domain-specific challenges—how to handle sample ratio mismatches, why sequential testing requires special care, when to use variance reduction techniques.

What unites all these audiences is a need for both depth and breadth—statistical foundations solid enough to support critical decisions, practical guidance specific enough to implement tomorrow, and judgment sophisticated enough to navigate the grey areas where theory doesn't provide clear answers.

What Makes This Book Different

Several principles guided the development of this book:

Theory Motivated by Practice. Every statistical concept is introduced in response to a real problem. Why do we need the Central Limit Theorem? Because we want to make probability statements about sample means. Why study multiple testing corrections? Because without them, we fool ourselves with false discoveries. The mathematics serves the practice, not the other way around.

Practice Grounded in Theory. Conversely, every practical technique is connected to its theoretical foundation. When I present a Python function for calculating sample size, I also derive the formula and explain the assumptions. When we implement Bayesian A/B testing, we understand why conjugate priors work and when they're appropriate. You won't just learn what to do—you'll understand why it works and when it doesn't.

Code as First-Class Citizen. This is not a book where code appears as an afterthought. The implementations in Python and R are production-quality, documented, and ready to adapt. I've run every code example, debugged the edge cases, and provided not just the happy path but also error handling and validation. Because I've been the person at 2 AM trying to debug a broken experiment, I know that implementation details matter.

Honest About Limitations. Statistics education often presents methods as if they solve problems cleanly. Real experimentation is messier. This book acknowledges the gaps: where assumptions are likely violated, when approximations break down, which questions statistics cannot answer. I share failures alongside successes, because learning what doesn't work is as valuable as learning what does.

Modern Methodologies. While grounded in classical statistics, this book embraces contemporary approaches. We cover Bayesian methods seriously, not as a curiosity but as a practical alternative with real advantages in certain contexts. We explore multi-armed bandits, sequential testing, and variance reduction techniques that have emerged from modern practice. We address challenges—network effects, novelty effects, metric dilution—that classical texts don't consider.

How to Use This Book

The book is designed for multiple modes of engagement:

As a Course Text: Graduate courses can proceed through chapters sequentially, using exercises to reinforce learning. I've structured the material to build progressively, with each chapter assuming mastery of previous content. Problem sets range from mathematical derivations to simulation studies to analysis of real datasets.

As a Reference: Practitioners can dive directly into specific chapters as needs arise. Comprehensive cross-referencing, a detailed index, and self-contained chapters enable targeted learning. Need to understand multiple testing corrections? Chapter 6 provides everything required. Want to implement Bayesian A/B testing? Chapter 7 is your guide.

As Professional Development: Working data scientists can deepen their understanding systematically. Perhaps you've been running A/B tests for years using standard tools but want to understand the statistical foundations more rigorously. Or you're comfortable with frequentist methods but curious about Bayesian approaches. The book supports both deepening and broadening your expertise.

Each chapter includes learning objectives, worked examples with complete solutions, production-ready code, exercises at varying difficulty levels, and suggestions for further reading. The appendices provide mathematical proofs for those seeking rigor, compre-

hensive code libraries for those seeking implementation, and statistical tables for quick reference.

A Personal Note

Writing this book has been a journey of synthesis—bringing together years of experience across industry and academia, reconciling practical constraints with theoretical ideals, and distilling complex ideas into accessible explanations. My work at Mysense.ai has provided invaluable real-world context: the pressure to move fast, the complexity of production systems, the challenge of communicating uncertainty to stakeholders who want certainty. My teaching and research at ESMAD has forced me to articulate foundations clearly, to answer "why" questions rigorously, and to stay current with methodological developments.

This dual perspective—practitioner and educator, industry and academia—has shaped every page. I've tried to write the book I wish had existed when I began my career, one that would have saved me from numerous mistakes and accelerated my understanding of this powerful methodology.

Experimentation, at its best, represents a commitment to learning from reality rather than assuming we already know. It's an acknowledgment that our intuitions, however informed, are fallible—and that careful measurement can surprise us. This intellectual humility, coupled with methodological rigor, enables genuine discovery.

My hope is that this book helps you design better experiments, analyze results more critically, make sounder decisions, and ultimately contribute to the growing culture of evidence-based decision making. Whether you're optimizing a website, evaluating a policy intervention, or conducting scientific research, the principles of rigorous experimentation apply.

The field of A/B testing continues to evolve. New methods emerge, new challenges arise, new applications appear. But the fundamentals—randomization, statistical inference, careful interpretation—endure. Master these foundations, and you'll be equipped not just for today's problems but for tomorrow's challenges.

Diogo Ribeiro
Lead Data Scientist, Mysense.ai
Researcher and Instructor, ESMAD
Instituto Politécnico do Porto
Porto, Portugal
November 19, 2025

Chapter 1

Introduction to A/B Testing

1.1 Learning Objectives

After completing this chapter, readers will be able to:

- Define A/B testing and explain its fundamental principles
- Trace the historical evolution from agricultural experiments to digital applications
- Distinguish A/B testing from other experimental methodologies
- Understand core terminology and concepts used throughout the book
- Recognize different types of A/B tests and their appropriate applications
- Implement basic A/B tests using Python and statistical libraries
- Identify real-world scenarios where A/B testing adds value

1.2 What is A/B Testing?

1.2.1 Definition and Core Concepts

Definition 1.1 (A/B Test)**.** An A/B test is a randomized controlled experiment in which a population is randomly divided into two or more groups to compare the effects of different treatments, interventions, or variations on a measured outcome. The fundamental structure includes:

- A **control group** (A) receiving the existing or baseline treatment
- One or more **treatment groups** (B, C, etc.) receiving new variations
- **Random assignment** ensuring unbiased allocation
- **Quantifiable outcomes** measured consistently across groups

At its core, A/B testing operationalizes the scientific method for decision-making under uncertainty. Rather than relying on intuition, expert opinion, or observational

data—all valuable but potentially misleading—we directly test hypotheses through controlled experimentation. This approach answers a deceptively simple question: *Does this change actually work?*

The elegance of A/B testing lies in its simplicity. By randomly assigning subjects to groups and measuring outcomes, we isolate the causal effect of our intervention from confounding variables. If users assigned to the new checkout flow convert at a higher rate than those in the control group, and this difference exceeds what we'd expect from random variation, we have evidence that the change caused the improvement.

1.2.2 From Agricultural Fields to Digital Platforms

The intellectual foundations of A/B testing trace back over a century, evolving through several distinct phases that reveal both continuity and transformation in experimental thinking.

Early Pioneers: Fisher and Agricultural Science

In the 1920s, Sir Ronald Fisher revolutionized experimental design while working at Rothamsted Experimental Station in England. Fisher confronted a practical problem: how to determine which fertilizers, crop varieties, and farming techniques produced the best yields when countless variables—soil quality, weather, pest pressure—varied unpredictably across fields and seasons.

Fisher's solution introduced three principles that remain foundational today [**?**]:

1. **Randomization:** Rather than systematically assigning treatments (which might confound treatment effects with field characteristics), Fisher advocated random assignment to eliminate systematic bias.

2. **Replication:** Testing each treatment on multiple plots enabled statistical estimation of variability and increased confidence in results.

3. **Blocking:** Grouping similar experimental units (e.g., adjacent plots) and randomizing within blocks reduced noise and improved precision.

These principles solved the agricultural scientist's dilemma: how to draw valid causal inferences in the face of uncontrolled variation. They work equally well for digital experimentation, though the scale and speed have changed dramatically.

Clinical Trials and Medical Research

The mid-20th century saw experimental design principles migrate to medical research. The 1948 streptomycin tuberculosis trial is often cited as the first modern randomized controlled trial (RCT) [**?**]. Medical researchers recognized that comparing treatments without randomization introduced selection bias—sicker patients might systematically receive more aggressive treatments, confounding causal inference.

Clinical trials formalized concepts still central to A/B testing:

- **Blinding:** Keeping subjects (and often researchers) unaware of treatment assignment to prevent expectation effects

- **Intention-to-treat analysis:** Analyzing all randomized subjects regardless of compliance
- **Power analysis:** Calculating required sample sizes before conducting experiments
- **Interim monitoring:** Checking results during trials for ethical reasons, requiring special statistical methods

The Digital Revolution

The internet transformed experimentation. What took months or years in agricultural or medical research could now happen in hours. Early online experiments at Amazon, Google, and Microsoft in the late 1990s and early 2000s demonstrated that digital platforms offered unprecedented advantages:

- **Massive scale:** Millions of subjects available
- **Low cost:** Marginal cost of adding one more subject approaches zero
- **Rapid iteration:** Tests complete in days or weeks, not months or years
- **Precise measurement:** Every click, view, and conversion tracked automatically
- **Perfect randomization:** Computers execute random assignment flawlessly

However, digital experimentation also introduced new challenges that Fisher never contemplated:

- **Implementation bugs:** Code errors can invalidate experiments
- **Network effects:** One user's treatment may affect others
- **Novelty effects:** Users respond differently to new features initially
- **Metric dilution:** Measuring too many outcomes increases false positives
- **Continuous monitoring:** "Peeking" at results invalidates classical statistics

Modern A/B testing inherits Fisher's foundational principles while adapting them to digital contexts. This synthesis—classical rigor meets contemporary scale—defines the field today.

1.2.3 A/B Testing vs. Other Methodologies

Understanding A/B testing requires distinguishing it from alternative approaches to learning and decision-making.

Observational Studies

Observational studies analyze existing data without intervention. For example, examining whether users who engage with a particular feature have higher retention. While valuable for hypothesis generation, observational studies suffer from confounding: perhaps engaged users were already more loyal, and feature usage is a symptom rather than a cause.

Example 1.1 (Simpson's Paradox)**.** An e-commerce platform observes that customers who use the mobile app spend 50% more than website-only customers. However, randomizing users to receive an app download prompt shows no effect on spending. Why?

The observational data confounds app usage with customer type. High-value customers independently discovered and adopted the app. The app didn't cause higher spending; higher-spending customers chose to use it. Randomization eliminates this confounding.

Multi-Armed Bandits

Multi-armed bandit (MAB) algorithms dynamically allocate more traffic to better-performing variants, optimizing cumulative performance during the test. Unlike A/B tests with fixed allocation, bandits adapt.

Consider testing K variants with unknown success rates $\theta_1, \ldots, \theta_K$. The regret after T trials is:

$$R(T) = T \cdot \max_k \theta_k - \sum_{t=1}^{T} \mathbb{E}[r_t] \tag{1.1}$$

where r_t is the reward at trial t.

MAB algorithms like Thompson Sampling or Upper Confidence Bound (UCB) minimize regret by balancing exploration (testing uncertain variants) and exploitation (favoring likely winners).

Trade-offs:

- A/B tests provide clean statistical inference but sacrifice performance during testing
- MABs optimize during testing but complicate inference (non-uniform assignment probabilities)
- A/B tests suit scenarios where learning is the goal
- MABs suit scenarios where optimization during testing matters (e.g., ad selection)

We explore bandits in depth in Chapter 8.

Quasi-Experimental Methods

When randomization is infeasible, quasi-experimental designs attempt causal inference from observational data under assumptions. Methods include:

- **Difference-in-differences:** Comparing trends before/after intervention in treatment vs. control groups
- **Regression discontinuity:** Exploiting arbitrary cutoffs (e.g., test score thresholds) for assignment

- **Instrumental variables:** Using variables that affect treatment but not outcomes directly
- **Propensity score matching:** Creating matched pairs with similar characteristics

These methods complement A/B testing when experimentation is impractical but carry stronger assumptions.

1.3 Why A/B Testing Matters

1.3.1 The Business Case for Experimentation

A/B testing delivers measurable value through better decision-making, risk mitigation, and organizational learning.

Quantifiable Return on Investment

Consider a concrete example. An e-commerce company with 1 million monthly visitors and 3% conversion rate generates 30,000 orders monthly. An A/B test finds that a new checkout flow increases conversion to 3.3%—a 10% relative lift.

Annual impact:

$$\text{Additional conversions} = 1{,}000{,}000 \times 12 \times (0.033 - 0.030) = 36{,}000 \tag{1.2}$$

$$\text{Revenue (at \$50 AOV)} = 36{,}000 \times \$50 = \$1{,}800{,}000 \tag{1.3}$$

If the test cost $10,000 to design, implement, and analyze, the ROI is 180x. Even conservative estimates assuming 5% lift still yield $900,000 annually.

De-Risking Product Changes

Not all changes improve outcomes. Experimentation prevents costly mistakes:

Example 1.2 (Microsoft Bing)**.** Microsoft tested displaying ads more prominently, hypothesizing increased revenue. The test showed short-term revenue gains but significant user satisfaction declines and reduced search quality metrics. Without testing, full deployment would have damaged the product and brand [**?**].

Failures are valuable. Learning that an idea doesn't work before full rollout saves development resources and protects user experience.

Compound Returns from Iteration

Individual experiments often yield small improvements—2-5% lifts. The power comes from compounding many improvements over time.

If a team runs one experiment weekly, each improving a key metric by 2%, annual compound growth is:

$$(1.02)^{52} - 1 \approx 1.83 = 183\% \tag{1.4}$$

This calculation oversimplifies (not all tests succeed, metrics may not be independent), but illustrates how systematic experimentation drives dramatic cumulative gains.

1.3.2 Evidence-Based Culture

Beyond individual tests, A/B testing transforms organizational decision-making.

Objective Evaluation of Ideas

Experimentation democratizes decision-making. Good ideas can come from anywhere—junior engineers, customer support, users themselves—and all receive objective evaluation. This meritocracy of ideas contrasts with authority-based or seniority-based decision-making.

Learning from Surprises

The most valuable experiments challenge assumptions. When results contradict expert opinion or conventional wisdom, we learn something fundamental about user behavior, market dynamics, or product mechanics.

Example 1.3 (Netflix Personalization)**.** Netflix hypothesized that showing personalized movie thumbnails would increase engagement. Testing revealed the effect was far larger than anticipated—personalized images increased viewing by 20-30% for some content. This insight led to broader personalization initiatives worth billions in subscriber value [?].

1.3.3 Examples from Industry Leaders

Google: Search Result Optimization

Google runs over 10,000 experiments annually on Search alone. A famous 2009 experiment tested displaying 10, 20, or 30 search results per page. The hypothesis: more results would reduce pagination and improve user experience.

Results showed that increasing results to 30 caused a 20% increase in page load time (from 0.4s to 0.9s). This seemingly small delay significantly reduced searches per user—users abandoned searches rather than wait. The test demonstrated that even half-second delays meaningfully impact behavior at scale.

Amazon: Everything is Testable

Amazon's culture of experimentation is legendary. The company tests nearly every product change before deployment. Examples include:

- Testing "Customers who bought this also bought" recommendation algorithms
- Optimizing product page layouts
- Testing checkout flows and payment options
- Evaluating search result ranking algorithms

Amazon founder Jeff Bezos famously said, "Our success at Amazon is a function of how many experiments we do per year, per month, per week, per day."

Facebook: Continuous Optimization

Facebook treats its platform as a continuous experiment. The company tests features on small user subsets before gradual rollout. Notable examples:

- News Feed ranking algorithms optimizing for engagement
- Ad formats and placement testing
- Like button variations and reactions
- Video autoplay behavior

However, Facebook also illustrates ethical challenges—a 2014 "emotional contagion" study manipulating News Feed content sparked controversy about informed consent in experimentation.

1.4 Basic Terminology and Concepts

Precise language enables clear thinking about experiments. This section establishes terminology used throughout the book.

1.4.1 Experimental Units and Assignment

Definition 1.2 (Experimental Unit)**.** The experimental unit is the entity randomized to treatment. Common units include:

- **Users:** Individual people (most common in digital experiments)
- **Sessions:** Individual visits or browsing sessions
- **Page views:** Individual page loads
- **Clusters:** Groups (e.g., geographic regions, schools, stores)

The choice of experimental unit affects both implementation and analysis. User-level randomization ensures consistency across sessions but requires persistent identifiers. Session-level randomization is simpler but creates inconsistent experiences for repeat visitors.

1.4.2 Control and Treatment Groups

Definition 1.3 (Control Group)**.** The control group (often called variant A) receives the existing or baseline experience. It serves as the comparison benchmark for measuring treatment effects.

Definition 1.4 (Treatment Group)**.** Treatment groups (B, C, etc.) receive new variations being tested. In simple A/B tests, there is one treatment group; multivariate tests have multiple.

Why maintain a control? Even if we believe a change will improve outcomes, we need the counterfactual—what would have happened without the change? External factors (seasonality, marketing campaigns, news events) affect all users. Comparing treatment to control isolates the causal effect of our intervention.

1.4.3 Metrics and Key Performance Indicators

Definition 1.5 (Metric)**.** A metric is a quantifiable measure of user behavior or business outcome. Metrics can be:

- **Binary:** Conversion (yes/no), click (yes/no)
- **Continuous:** Revenue per user, time on site, pages viewed
- **Count:** Number of purchases, search queries, shares

Effective metrics share several properties:

1. **Measurable:** Can be tracked reliably and consistently
2. **Attributable:** Can be linked to experimental units
3. **Timely:** Observable within reasonable experimental duration
4. **Sensitive:** Responds to interventions we care about
5. **Aligned:** Correlates with long-term business goals

Metric Hierarchies

Mature experimentation programs organize metrics hierarchically:

- **Primary metric:** The main outcome for decision-making (e.g., revenue per user)
- **Secondary metrics:** Supporting measures providing context (e.g., conversion rate, average order value)
- **Guardrail metrics:** Measures we monitor but don't want to degrade (e.g., page load time, error rate)

This hierarchy prevents "metric shopping"—testing many metrics until finding significance by chance.

1.4.4 Statistical Significance

Definition 1.6 (Statistical Significance)**.** A result is statistically significant if the observed difference between groups is unlikely to have occurred by chance alone under the null hypothesis of no effect. Conventionally, we use significance level $\alpha = 0.05$, meaning we tolerate a 5% chance of false positives.

The p-value quantifies this: $p = P(\text{observe data this extreme} \mid H_0 \text{ true})$. If $p < \alpha$, we reject the null hypothesis and conclude a statistically significant effect exists.

Critical distinction: Statistical significance differs from practical significance. A tiny effect might be statistically significant with enough data but too small to matter. Conversely, a meaningful effect might not reach significance with limited data.

1.4.5 Effect Size

Definition 1.7 (Effect Size)**.** Effect size quantifies the magnitude of difference between groups, independent of sample size. Common measures include:

- **Absolute difference:** $\hat{p}_B - \hat{p}_A$ (e.g., 10.5% - 10.0% = 0.5 percentage points)
- **Relative lift:** $\frac{\hat{p}_B - \hat{p}_A}{\hat{p}_A}$ (e.g., 0.5% / 10.0% = 5% relative increase)
- **Cohen's d:** $d = \frac{\bar{x}_B - \bar{x}_A}{s_{\text{pooled}}}$ (standardized difference for continuous outcomes)

Effect sizes communicate practical importance and facilitate comparison across experiments. A 5% relative lift in conversion rate from 10% baseline has different business impact than the same lift from 1% baseline, even if both are statistically significant.

1.5 Types of A/B Tests

Experimentation encompasses various designs suited to different objectives and constraints.

1.5.1 Simple A/B Tests

The canonical design compares one treatment against a control:

- 50% of users see control (A)
- 50% see treatment (B)
- Measure and compare a primary metric

When to use:

- Testing a single change (button color, headline, algorithm)
- Clear hypothesis about improvement
- Want clean statistical inference
- First test of a new idea

1.5.2 A/B/n Tests (Multiple Variants)

Testing multiple variations simultaneously:

- Control (A): 50%
- Treatment 1 (B): 25%
- Treatment 2 (C): 25%

Trade-offs:

- **Efficiency:** Test multiple ideas with same traffic

- **Complexity:** More comparisons increase multiple testing burden
- **Power:** Splitting traffic reduces power for each comparison

Proper multiple testing corrections (Chapter 6) are essential to maintain valid error rates.

1.5.3 Multivariate Tests (Factorial Designs)

Factorial designs test multiple factors simultaneously. A 2^k factorial tests k binary factors:

Example 1.4 (2×2 Factorial Design)**.** Testing email subject line (A/B) and send time (morning/evening):

	Morning	**Evening**
Subject A	25%	25%
Subject B	25%	25%

This design estimates:

- Main effect of subject line
- Main effect of send time
- Interaction: does subject line effect depend on time?

Advantages:

- Test multiple factors with one experiment
- Detect interactions between factors
- More efficient than separate tests

Challenges:

- Complexity grows exponentially (2^k combinations)
- Interpretation harder with many factors
- Requires larger samples for interaction detection

Chapter 8 covers factorial designs comprehensively.

1.5.4 Sequential Testing

Traditional A/B tests fix sample size in advance and analyze once at the end. Sequential testing allows monitoring during the experiment with appropriate statistical controls.

Group sequential methods: Pre-specify analysis times (e.g., 25%, 50%, 75%, 100% of planned sample) and use spending functions to control Type I error.

Advantages:

- Stop early if strong evidence emerges

- Ethical in some contexts (e.g., medical trials)
- Reduce time to decision

Challenges:

- Requires special statistical methods
- "Peeking" at results invalidates standard p-values
- More complex implementation

1.5.5 Multi-Armed Bandits

Rather than fixed allocation, bandit algorithms dynamically shift traffic toward better performers:

Algorithm 1 Simple ϵ-Greedy Bandit

```
Initialize: N_i = 0, μ̂_i = 0 for variants i = 1, ..., K
for each user t = 1, 2, ... do
    With probability ε: select random variant (explore)
    With probability 1 − ε: select arg max_i μ̂_i (exploit)
    Observe outcome r_t
    Update estimates for selected variant
end for
```

When bandits outperform A/B tests:

- Many variants to test (e.g., ad creative)
- Performance during testing matters (opportunity cost high)
- Priors available from similar tests
- Willing to trade inference clarity for optimization

1.6 Real-World Examples

1.6.1 Website Conversion Optimization

E-commerce Checkout Flow

An online retailer redesigns the checkout process, reducing steps from 5 to 3 by combining shipping and billing information.

Hypothesis: Fewer steps will reduce abandonment and increase completion rate.

Metrics:

- Primary: Checkout completion rate (conversions / checkout initiations)
- Secondary: Revenue per session, average order value
- Guardrails: Form errors, customer support contacts

Results:

- Completion rate increased from 68% to 71.5% ($p < 0.01$)
- Revenue per session up 6.2%
- No increase in form errors or support contacts

Business impact: With 100,000 monthly checkout initiations, 3.5 percentage point improvement yields 3,500 additional monthly conversions. At $80 average order value: $280,000 monthly revenue increase, or $3.36M annually.

Listing 1.1: Basic A/B Test Analysis for Conversion Rates

```
import numpy as np
from scipy import stats
import pandas as pd

def analyze_conversion_test(control_conversions, control_total,
                            treatment_conversions, treatment_total,
                            alpha=0.05):
    """
␣␣␣␣Analyze␣A/B␣test␣for␣conversion␣rates.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣control_conversions␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣conversions␣in␣control
␣␣␣␣control_total␣:␣int
␣␣␣␣␣␣␣␣Total␣users␣in␣control
␣␣␣␣treatment_conversions␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣conversions␣in␣treatment
␣␣␣␣treatment_total␣:␣int
␣␣␣␣␣␣␣␣Total␣users␣in␣treatment
␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣Significance␣level␣(default␣0.05)

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Test␣results␣including␣statistics␣and␣interpretation
␣␣␣␣"""
    # Calculate proportions
    p_control = control_conversions / control_total
    p_treatment = treatment_conversions / treatment_total

    # Pooled proportion for null hypothesis
    p_pooled = (control_conversions + treatment_conversions) / \
               (control_total + treatment_total)

    # Standard error under null
    se_null = np.sqrt(p_pooled * (1 - p_pooled) *
                      (1/control_total + 1/treatment_total))

    # Z-statistic
    z_stat = (p_treatment - p_control) / se_null

    # Two-sided p-value
    p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))

```

```
    # Confidence interval (using unpooled variance)
    se_unpooled = np.sqrt(p_control*(1-p_control)/control_total +
                          p_treatment*(1-p_treatment)/treatment_total)
    z_crit = stats.norm.ppf(1 - alpha/2)
    ci_lower = (p_treatment - p_control) - z_crit * se_unpooled
    ci_upper = (p_treatment - p_control) + z_crit * se_unpooled

    # Calculate effect sizes
    absolute_lift = p_treatment - p_control
    relative_lift = absolute_lift / p_control if p_control > 0 else np.
        nan

    # Determine statistical significance
    is_significant = p_value < alpha

    results = {
        'control_rate': p_control,
        'treatment_rate': p_treatment,
        'absolute_lift': absolute_lift,
        'relative_lift': relative_lift,
        'z_statistic': z_stat,
        'p_value': p_value,
        'ci_lower': ci_lower,
        'ci_upper': ci_upper,
        'is_significant': is_significant,
        'alpha': alpha
    }

    return results

# Example: Checkout flow test
results = analyze_conversion_test(
    control_conversions=6800,
    control_total=10000,
    treatment_conversions=7150,
    treatment_total=10000
)

print("A/B Test Results")
print("=" * 50)
print(f"Control conversion rate: {results['control_rate']:.1%}")
print(f"Treatment conversion rate: {results['treatment_rate']:.1%}")
print(f"\nAbsolute lift: {results['absolute_lift']:.1%}")
print(f"Relative lift: {results['relative_lift']:.1%}")
print(f"\nZ-statistic: {results['z_statistic']:.3f}")
print(f"P-value: {results['p_value']:.4f}")
print(f"95% CI: [{results['ci_lower']:.1%}, {results['ci_upper']:.1%}]"
    )
print(f"\nStatistically significant: {results['is_significant']}")
```

1.6.2 Email Marketing Campaigns

Subject Line Testing

An e-commerce company tests three email subject lines for a promotional campaign:

- A (Control): "Weekend Sale - 20% Off"

- B: "Flash Sale: Save 20% This Weekend Only"
- C: "Your Exclusive 20% Weekend Discount"

Experimental design:

- Randomly assign 100,000 subscribers equally to three variants
- Send simultaneously (Saturday 9 AM)
- Measure open rate, click-through rate, conversion rate

Listing 1.2: Multi-Variant Email Test Analysis

```
def analyze_email_test(variants_data, metric='open_rate', alpha=0.05):
    """
␣␣␣␣Analyze␣multi-variant␣email␣test␣with␣Bonferroni␣correction.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣variants_data␣:␣dict
␣␣␣␣␣␣␣␣Dictionary␣mapping␣variant␣names␣to␣(successes,␣total)␣tuples
␣␣␣␣metric␣:␣str
␣␣␣␣␣␣␣␣Metric␣name␣for␣reporting
␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣Family-wise␣error␣rate

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣DataFrame␣:␣Comparison␣of␣all␣variants␣with␣adjusted␣p-values
␣␣␣␣"""
    variant_names = list(variants_data.keys())
    n_variants = len(variant_names)
    n_comparisons = n_variants * (n_variants - 1) // 2

    # Bonferroni correction
    alpha_adjusted = alpha / n_comparisons

    results = []

    # Pairwise comparisons
    for i in range(n_variants):
        for j in range(i + 1, n_variants):
            var1, var2 = variant_names[i], variant_names[j]

            successes1, total1 = variants_data[var1]
            successes2, total2 = variants_data[var2]

            # Perform test
            test_results = analyze_conversion_test(
                successes1, total1, successes2, total2, alpha_adjusted
            )

            results.append({
                'comparison': f'{var2}␣vs␣{var1}',
                f'{var1}_{metric}': test_results['control_rate'],
                f'{var2}_{metric}': test_results['treatment_rate'],
                'absolute_diff': test_results['absolute_lift'],
```

```
                'relative_lift': test_results['relative_lift'],
                'p_value': test_results['p_value'],
                'significant': test_results['is_significant']
            })

    df = pd.DataFrame(results)

    print(f"Email Test Results - {metric}")
    print("=" * 70)
    print(f"Number of comparisons: {n_comparisons}")
    print(f"Bonferroni-adjusted alpha: {alpha_adjusted:.4f}")
    print()

    return df

# Example data
email_data = {
    'A_Control': (4200, 33333),      # Open rate ~12.6%
    'B_Flash': (4500, 33333),         # Open rate ~13.5%
    'C_Exclusive': (4800, 33334)      # Open rate ~14.4%
}

results_df = analyze_email_test(email_data, metric='open_rate')
print(results_df.to_string(index=False))
```

1.6.3 Product Feature Testing

Recommendation Algorithm

A streaming platform tests a new content recommendation algorithm against the existing system.

Hypothesis: Machine learning-based personalized recommendations will increase viewing time compared to popularity-based recommendations.

Experimental design:

- User-level randomization (consistent experience across sessions)
- 50/50 split: 5 million users per variant
- Duration: 4 weeks (capture weekly patterns, reduce novelty effects)

Metrics:

- Primary: Hours watched per user per week
- Secondary: Content diversity (unique titles watched), completion rate
- Guardrails: Churn rate, customer satisfaction scores

Results demonstrate heterogeneous treatment effects:

User Segment	Control	Treatment	Lift
New users ($<$ 1 month)	8.2 hrs	10.1 hrs	+23%
Established (1-6 months)	12.5 hrs	13.8 hrs	+10%
Long-term ($>$ 6 months)	15.3 hrs	15.1 hrs	-1%
Overall	12.4 hrs	13.2 hrs	+6.5%

New algorithm benefits new and moderately engaged users but provides no value (or slight harm) to long-term users who have strong preferences. This insight suggests opportunities for segment-specific optimization.

1.6.4 Pricing Experiments

Pricing is perhaps the most sensitive area for experimentation, requiring careful ethical and business considerations.

Subscription Tier Testing

A SaaS company tests two pricing strategies:

- Control: $29/month, $290/year (17% annual discount)
- Treatment: $29/month, $249/year (28% annual discount)

Hypothesis: Larger annual discount will increase annual plan adoption without significantly reducing revenue.

Key considerations:

- **Ethics:** All users see both options; no one charged different prices for same product
- **Segment:** Test on new signups only (existing customers unaffected)
- **Duration:** 8 weeks to capture full subscription cycle

Listing 1.3: Revenue Analysis for Pricing Test

```
def analyze_pricing_test(control_monthly, control_annual,
                         treatment_monthly, treatment_annual,
                         avg_months_subscribed=12):
    """
␣␣␣␣Analyze␣pricing␣experiment␣considering␣customer␣lifetime␣value.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣control_monthly␣,␣treatment_monthly␣:␣tuple
␣␣␣␣␣␣␣␣(count␣,␣price)␣for␣monthly␣subscribers
␣␣␣␣control_annual␣,␣treatment_annual␣:␣tuple
␣␣␣␣␣␣␣␣(count␣,␣price)␣for␣annual␣subscribers
␣␣␣␣avg_months_subscribed␣:␣int
␣␣␣␣␣␣␣␣Average␣subscription␣duration␣in␣months

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Revenue␣metrics␣and␣comparison
␣␣␣␣"""
    # Unpack data
    c_monthly_count, c_monthly_price = control_monthly
    c_annual_count, c_annual_price = control_annual
    t_monthly_count, t_monthly_price = treatment_monthly
    t_annual_count, t_annual_price = treatment_annual

    # Calculate annual plan adoption rate
    control_total = c_monthly_count + c_annual_count
```

```
    treatment_total = t_monthly_count + t_annual_count

    c_annual_rate = c_annual_count / control_total
    t_annual_rate = t_annual_count / treatment_total

    # Calculate average revenue per customer (annualized)
    c_avg_revenue = (
        (c_monthly_count * c_monthly_price * avg_months_subscribed +
         c_annual_count * c_annual_price) / control_total
    )

    t_avg_revenue = (
        (t_monthly_count * t_monthly_price * avg_months_subscribed +
         t_annual_count * t_annual_price) / treatment_total
    )

    # Statistical test on annual adoption rate
    from scipy.stats import chi2_contingency

    contingency_table = [
        [c_annual_count, c_monthly_count],
        [t_annual_count, t_monthly_count]
    ]
    chi2, p_value, dof, expected = chi2_contingency(contingency_table)

    results = {
        'control_annual_rate': c_annual_rate,
        'treatment_annual_rate': t_annual_rate,
        'annual_adoption_lift': (t_annual_rate - c_annual_rate) /
            c_annual_rate,
        'control_avg_revenue': c_avg_revenue,
        'treatment_avg_revenue': t_avg_revenue,
        'revenue_diff': t_avg_revenue - c_avg_revenue,
        'revenue_diff_pct': (t_avg_revenue - c_avg_revenue) /
            c_avg_revenue,
        'chi2_statistic': chi2,
        'p_value': p_value,
        'significant': p_value < 0.05
    }

    return results

# Example: Pricing test data
pricing_results = analyze_pricing_test(
    control_monthly=(2800, 29),      # 2800 monthly at $29
    control_annual=(1200, 290),      # 1200 annual at $290
    treatment_monthly=(2400, 29),    # 2400 monthly at $29
    treatment_annual=(1600, 249)     # 1600 annual at $249
)

print("Pricing Test Results")
print("=" * 60)
print(f"Control annual adoption: {pricing_results['control_annual_rate
    ']:.1%}")
print(f"Treatment annual adoption: {pricing_results['
    treatment_annual_rate']:.1%}")
print(f"Lift in annual adoption: {pricing_results['annual_adoption_lift
    ']:+.1%}")
```

```
print()
print(f"Control avg revenue: ${pricing_results['control_avg_revenue']:.2f}")
print(f"Treatment avg revenue: ${pricing_results['treatment_avg_revenue']:.2f}")
print(f"Revenue difference: ${pricing_results['revenue_diff']:+.2f} "
      f"({pricing_results['revenue_diff_pct']:+.1%})")
print()
print(f"Chi-squared: {pricing_results['chi2_statistic']:.2f}, "
      f"p-value: {pricing_results['p_value']:.4f}")
print(f"Statistically significant: {pricing_results['significant']}")
```

Results interpretation: Treatment increased annual plan adoption from 30% to 40%, but the lower price point resulted in slight decrease in average revenue per customer (from $319.20 to $314.10 annually). However, higher adoption of annual plans improves retention and reduces churn, making this a strategic win despite short-term revenue impact.

1.7 When A/B Testing Works (and When It Doesn't)

1.7.1 Ideal Scenarios for A/B Testing

A/B testing excels when:

1. **Sufficient traffic:** Adequate users/events to detect meaningful effects
2. **Clear metrics:** Quantifiable outcomes observable in reasonable timeframe
3. **Reversible changes:** Can easily roll back if needed
4. **Limited network effects:** Users' experiences are independent
5. **Ethical feasibility:** Randomization and experimentation are appropriate

1.7.2 When A/B Testing Struggles

Insufficient Sample Size

If your website receives 100 visitors weekly and conversion rate is 2%, detecting a 25% relative improvement (from 2% to 2.5%) requires approximately 6,300 users per variant—over a year of testing.

Long-Term Effects

Some interventions show different short-term and long-term effects. A/B tests typically run weeks to months. Evaluating impacts on 5-year customer lifetime value requires different approaches.

Rare Events

Testing impacts on events occurring once per 10,000 users requires enormous samples. Consider proxy metrics or specialized methods.

Fundamental Redesigns

Radical changes create such different experiences that incremental testing may not apply. Qualitative research, gradual rollouts, and careful monitoring complement experimentation.

Network Effects

Social networks, marketplaces, and communication platforms violate the "no interference" assumption. One user's treatment affects others. Specialized designs (Chapter 8) address this.

1.8 Summary

A/B testing provides a rigorous framework for learning from data and making evidence-based decisions. Its core principles—randomization, controlled comparison, statistical inference—enable causal conclusions that observational studies cannot support.

Key takeaways:

- A/B testing evolved from Fisher's agricultural experiments to become central to digital product development
- Randomization eliminates confounding and enables causal inference
- Proper terminology (control/treatment, metrics, significance, effect size) enables precise thinking
- Multiple test types (simple A/B, multivariate, sequential, bandits) suit different objectives
- Real-world applications span e-commerce, marketing, product features, and pricing
- Understanding both capabilities and limitations guides appropriate application

The subsequent chapters develop the statistical theory, implementation practices, and analytical techniques necessary for rigorous experimentation. We begin in Chapter 2 with probability and statistical foundations.

1.9 Further Reading

- **?**: Original exposition of experimental design principles
- **?**: Comprehensive modern treatment from industry leaders
- **?**: Classical text bridging theory and practice
- **?**: Rigorous causal inference framework

1.10 Exercises

1.10.1 Conceptual Questions

1. **Understanding Randomization**

   (a) Explain in your own words why randomization is essential for causal inference.

   (b) Describe a scenario where observational data would be misleading but a randomized experiment would reveal the truth.

(c) What could go wrong if we assigned users to variants based on the first letter of their username instead of random assignment?

2. **Metrics and Business Impact**

   (a) A test shows 10% increase in clicks but no change in revenue. What might explain this? Would you ship the change?

   (b) Propose a complete metric hierarchy (primary, secondary, guardrails) for testing a new mobile app checkout flow.

   (c) Explain the difference between statistical significance and practical significance with a concrete example.

3. **Experimental Design**

   (a) When would you choose user-level over session-level randomization? Provide three scenarios for each.

   (b) A social network wants to test a new feature but worries about network effects. How would this concern affect their experimental design?

   (c) Describe the trade-offs between A/B testing and multi-armed bandits for selecting email subject lines.

1.10.2 Quantitative Problems

1. **Sample Size Intuition**

   (a) Your website has 10,000 daily visitors. How long must a test run to accumulate 20,000 users per variant?

   (b) If detecting a 5% effect requires 10,000 users per variant, approximately how many users are needed to detect a 2.5% effect (half as large)?

   (c) Explain why detecting small effects requires disproportionately more data.

2. **Effect Size Calculations**

   (a) Control: 8.2% conversion. Treatment: 9.0% conversion. Calculate absolute and relative lift.

   (b) A test increases average order value from \$45 to \$48. Express as absolute difference, relative lift, and Cohen's d (assume SD = \$15).

   (c) Why might the same 10% relative improvement have different business value at different baseline rates?

1.10.3 Programming Exercises

1. **Basic Analysis**

   (a) Use the provided code to analyze: Control (450 conversions / 5000 users), Treatment (520 conversions / 5000 users). Interpret all outputs.

   (b) Modify the code to calculate 90% confidence intervals instead of 95%. How does this change affect the width?

(c) Implement a function that determines if an observed difference could plausibly be due to chance (p-value interpretation).

2. **Simulation Study**

   (a) Simulate 10,000 A/B tests where the null hypothesis is true (no effect). What proportion show $p < 0.05$? Why?

   (b) Simulate tests with 10% true relative lift. What proportion correctly detect this effect (statistical power)?

   (c) Create a visualization showing how power changes with sample size for a fixed effect size.

3. **Real Data Analysis**

   (a) Download a public A/B test dataset (e.g., from Kaggle). Perform complete analysis including data validation, statistical testing, and business recommendation.

   (b) Implement pre-analysis checks: sample ratio mismatch detection, temporal patterns, outlier identification.

   (c) Create a reusable analysis template you could apply to future experiments.

1.10.4 Critical Thinking

1. **Case Analysis**

   (a) Microsoft Bing's experiment (mentioned earlier) showed short-term revenue gains but long-term user satisfaction declines. Design an experimental protocol that would have detected this issue earlier.

   (b) A company runs 100 experiments simultaneously. Several show statistically significant improvements. What concerns should you have? How would you address them?

   (c) Critics argue that A/B testing leads to incremental optimization rather than breakthrough innovation. How would you respond? When is this criticism valid?

2. **Ethical Considerations**

   (a) A dating app wants to test whether showing fewer high-quality matches increases engagement (by creating scarcity). Is this ethical? What principles guide your answer?

   (b) When, if ever, should informed consent be required for A/B tests in commercial products?

   (c) Design ethical guidelines for experimentation in your domain of interest.

Chapter 2

Statistical Foundations

2.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply probability theory fundamentals to experimental design and analysis
- Work with probability distributions essential for A/B testing
- Understand and apply the Central Limit Theorem to justify normal approximations
- Construct and interpret confidence intervals for common metrics
- Distinguish between frequentist and Bayesian inference frameworks
- Formulate and test statistical hypotheses rigorously
- Calculate and interpret effect size measures for practical significance
- Implement statistical analyses using Python and R

2.2 Introduction

A/B testing rests on a foundation of probability theory and statistical inference. While modern software makes running experiments trivially easy, understanding the underlying mathematics is essential for:

1. **Correct Implementation:** Knowing when assumptions are violated
2. **Proper Interpretation:** Understanding what results actually mean
3. **Sound Decisions:** Distinguishing statistical significance from practical importance
4. **Effective Communication:** Explaining uncertainty to stakeholders

This chapter develops these foundations systematically, connecting abstract theory to concrete experimental practice. We emphasize not just the mechanics of calculation but the reasoning that justifies each method.

2.3 Probability Theory Fundamentals

Probability provides the language for reasoning about uncertainty—the fundamental challenge in A/B testing where we observe samples but want to make inferences about populations.

2.3.1 Probability Spaces and Random Variables

Definition 2.1 (Probability Space)**.** A probability space is a triple $(\Omega, \mathcal{F}, P)$ where:

- Ω is the sample space (set of all possible outcomes)
- $\mathcal{F}$ is a σ-algebra of events (subsets of Ω)
- $P : \mathcal{F} \to [0, 1]$ is a probability measure satisfying:
  1. $P(\Omega) = 1$ (normalization)
  2. For disjoint events $A_1, A_2, \ldots$: $P(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} P(A_i)$ (countable additivity)

Example 2.1 (A/B Test Sample Space)**.** In a simple A/B test where we observe whether a user converts:

- $\Omega = \{\text{convert}, \text{no convert}\}$
- $\mathcal{F} = \{\emptyset, \{\text{convert}\}, \{\text{no convert}\}, \Omega\}$
- $P(\{\text{convert}\}) = p$, $P(\{\text{no convert}\}) = 1 - p$

For n users, Ω becomes the set of all possible sequences of conversions and non-conversions, containing 2^n elements.

Definition 2.2 (Random Variable)**.** A random variable X is a measurable function $X : \Omega \to \mathbb{R}$ that maps outcomes to real numbers. We distinguish:

- **Discrete random variables:** Taking countable values with probability mass function (PMF) $p(x) = P(X = x)$
- **Continuous random variables:** Taking uncountable values with probability density function (PDF) $f(x)$ where $P(a \leq X \leq b) = \int_a^b f(x)dx$

2.3.2 Expected Value and Variance

The expected value and variance are the most important summary statistics of a distribution, characterizing location and spread respectively.

Definition 2.3 (Expected Value)**.** For discrete X:

$$\mathbb{E}[X] = \sum_x x \cdot p(x) \tag{2.1}$$

For continuous X:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x)dx \tag{2.2}$$

The expected value represents the long-run average if we could repeat the experiment infinitely many times.

Definition 2.4 (Variance and Standard Deviation)**.**

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \tag{2.3}$$

The standard deviation is $\sigma = \sqrt{\text{Var}(X)}$.

Variance measures the expected squared deviation from the mean—how spread out the distribution is. Standard deviation has the same units as X, making it more interpretable.

Essential Properties for A/B Testing

Proposition 2.1 (Linearity of Expectation)**.** For random variables $X_1, \ldots, X_n$ and constants $a_1, \ldots, a_n, b$:

$$\mathbb{E}\left[b + \sum_{i=1}^{n} a_i X_i\right] = b + \sum_{i=1}^{n} a_i \mathbb{E}[X_i] \tag{2.4}$$

This holds **regardless of dependence** between variables—a remarkable fact that greatly simplifies calculations.

Proposition 2.2 (Variance of Linear Combinations)**.** For independent random variables $X_1, \ldots, X_n$ and constants $a_1, \ldots, a_n, b$:

$$\text{Var}\left(b + \sum_{i=1}^{n} a_i X_i\right) = \sum_{i=1}^{n} a_i^2 \text{Var}(X_i) \tag{2.5}$$

Note: The constant b vanishes (shifting doesn't change spread), and independence is crucial.

Corollary 2.1 (Variance of Sample Mean)**.** For $X_1, \ldots, X_n$ independent with $\mathbb{E}[X_i] = \mu$ and $\text{Var}(X_i) = \sigma^2$:

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\sum_{i=1}^{n} \text{Var}(X_i) = \frac{\sigma^2}{n} \tag{2.6}$$

This fundamental result shows uncertainty decreases with $\sqrt{n}$, not n. Quadrupling sample size only halves standard error.

Example 2.2 (Computing Means and Variances)**.** Consider an A/B test where users spend time on site. Let X_i represent time spent by user i, with individual $\mathbb{E}[X_i] = 5$ minutes and $\text{Var}(X_i) = 4$ minutes^2.

For $n = 100$ users:

$$\mathbb{E}[\bar{X}] = \mathbb{E}\left[\frac{1}{100}\sum_{i=1}^{100} X_i\right] = \frac{1}{100}\sum_{i=1}^{100} \mathbb{E}[X_i] = 5 \text{ minutes} \tag{2.7}$$

$$\text{Var}(\bar{X}) = \frac{4}{100} = 0.04 \text{ minutes}^2 \tag{2.8}$$

$$\text{SD}(\bar{X}) = \sqrt{0.04} = 0.2 \text{ minutes} \tag{2.9}$$

The sample mean has the same expected value as individual observations, but much less variability.

Listing 2.1: Simulating Expected Value and Variance

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_style("whitegrid")
np.random.seed(42)

# Simulate time spent on site (exponential distribution)
true_mean = 5  # minutes
n_users = 100
n_simulations = 10000

# Individual observations
individual_times = np.random.exponential(true_mean, size=(n_simulations
    , n_users))

# Sample means
sample_means = individual_times.mean(axis=1)

# Theoretical values
theoretical_var_individual = true_mean**2  # For exponential
theoretical_var_sample_mean = theoretical_var_individual / n_users
theoretical_sd_sample_mean = np.sqrt(theoretical_var_sample_mean)

# Empirical values
empirical_mean = sample_means.mean()
empirical_sd = sample_means.std()

# Visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Individual observations
axes[0].hist(individual_times[0, :], bins=50, density=True, alpha=0.7,
             label='Sample data', color='steelblue')
x = np.linspace(0, 25, 1000)
axes[0].plot(x, stats.expon.pdf(x, scale=true_mean), 'r-', linewidth=2,
            label=f'Theoretical PDF')
axes[0].axvline(true_mean, color='green', linestyle='--', linewidth=2,
               label=f'True mean = {true_mean}')
axes[0].set_xlabel('Time Spent (minutes)', fontsize=12)
axes[0].set_ylabel('Density', fontsize=12)
axes[0].set_title('Individual User Times', fontsize=14, fontweight='
    bold')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Sample means
axes[1].hist(sample_means, bins=50, density=True, alpha=0.7,
            label=f'Simulated sample means (n={n_simulations:,})',
               color='steelblue')
axes[1].axvline(empirical_mean, color='red', linestyle='-', linewidth
    =2,
               label=f'Empirical mean = {empirical_mean:.3f}')
axes[1].axvline(true_mean, color='green', linestyle='--', linewidth=2,
               label=f'True mean = {true_mean}')
axes[1].set_xlabel('Sample Mean (minutes)', fontsize=12)
```

```
axes[1].set_ylabel('Density', fontsize=12)
axes[1].set_title(f'Distribution of Sample Means (n={n_users})',
                  fontsize=14, fontweight='bold')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('expectation_variance_demo.pdf', dpi=300, bbox_inches='
    tight')
plt.show()

# Print comparison
print(f"Theoretical SD of sample mean: {theoretical_sd_sample_mean:.4f}
    ")
print(f"Empirical SD of sample mean:    {empirical_sd:.4f}")
print(f"Ratio (should be ~1):           {empirical_sd/
    theoretical_sd_sample_mean:.4f}")
```

2.3.3 The Central Limit Theorem

The Central Limit Theorem (CLT) is arguably the most important result in statistics. It justifies using normal approximations in A/B testing, enabling inference even when individual observations are far from normally distributed.

Theorem 2.1 (Central Limit Theorem)**.** Let $X_1, X_2, \ldots, X_n$ be independent, identically distributed random variables with $\mathbb{E}[X_i] = \mu$ and $\text{Var}(X_i) = \sigma^2 < \infty$. Define the sample mean:

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i \tag{2.10}$$

Then as $n \to \infty$:

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1) \tag{2.11}$$

Equivalently, for large n:

$$\bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right) \tag{2.12}$$

Interpretation: The sample mean of *any* distribution (with finite variance) becomes approximately normally distributed for large sample sizes. The approximation quality improves as n increases.

Implications for A/B Testing

The CLT has profound practical consequences:

1. **Universality:** Whether testing conversions (Bernoulli), revenue (right-skewed), or engagement time (exponential), sample means are approximately normal for sufficient n.

2. **Quantifiable Uncertainty:** The standard error $\text{SE}(\bar{X}) = \sigma/\sqrt{n}$ precisely quantifies estimation uncertainty.

3. **Confidence Intervals:** Normal approximation enables constructing confidence intervals: $\bar{X} \pm z_{\alpha/2} \cdot \text{SE}(\bar{X})$.

4. **Hypothesis Testing:** Test statistics become approximately normal, enabling p-value calculation.

Example 2.3 (CLT for Conversion Rates)**.** Users convert with probability $p = 0.05$. Individual conversions $X_i \sim \text{Bernoulli}(0.05)$ take only values 0 and 1—decidedly non-normal! The PMF is:

$$P(X_i = 0) = 0.95, \quad P(X_i = 1) = 0.05 \tag{2.13}$$

However, with $n = 1000$ users, the sample conversion rate $\hat{p} = \bar{X}_n$ is approximately:

$$\hat{p} \sim \mathcal{N}\left(0.05, \frac{0.05 \times 0.95}{1000}\right) = \mathcal{N}(0.05, 0.0000475) \tag{2.14}$$

Standard error: $\text{SE}(\hat{p}) = \sqrt{0.0000475} \approx 0.0069$ or about 0.69 percentage points. A 95% confidence interval:

$$0.05 \pm 1.96 \times 0.0069 = [0.0365, 0.0635] \tag{2.15}$$

How Large is "Large Enough"?

A practical question: when is the normal approximation accurate? Rules of thumb:

- **For proportions:** $np \geq 5$ and $n(1-p) \geq 5$
- **For symmetric distributions:** $n \geq 30$ often sufficient
- **For highly skewed distributions:** May need $n \geq 100$ or more

These are guidelines, not hard rules. Simulation can verify adequacy for specific cases.

Listing 2.2: Demonstrating the Central Limit Theorem

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm, bernoulli, expon, kstest

sns.set_style("whitegrid")
np.random.seed(42)

def demonstrate_clt(distribution, params, sample_sizes, n_simulations
    =10000):
    """
␣␣␣␣Demonstrate␣CLT␣for␣different␣distributions␣and␣sample␣sizes.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣distribution␣:␣str
␣␣␣␣␣␣␣␣'bernoulli',␣'exponential',␣or␣'uniform'
␣␣␣␣params␣:␣dict
␣␣␣␣␣␣␣␣Distribution␣parameters
␣␣␣␣sample_sizes␣:␣list
```

```
␣␣␣␣␣␣␣␣List␣of␣sample␣sizes␣to␣demonstrate
␣␣␣␣n_simulations␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣simulations␣per␣sample␣size
␣␣␣␣"""
    n_plots = len(sample_sizes)
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    axes = axes.flatten()

    # Generate data based on distribution
    if distribution == 'bernoulli':
        p = params['p']
        true_mean = p
        true_std = np.sqrt(p * (1-p))
        generate_sample = lambda size: bernoulli.rvs(p, size=size)
        title_prefix = f"Bernoulli(p={p})"
    elif distribution == 'exponential':
        scale = params['scale']
        true_mean = scale
        true_std = scale
        generate_sample = lambda size: expon.rvs(scale=scale, size=size
            )
        title_prefix = f"Exponential(  =1/{scale})"
    elif distribution == 'uniform':
        a, b = params['a'], params['b']
        true_mean = (a + b) / 2
        true_std = np.sqrt((b - a)**2 / 12)
        generate_sample = lambda size: np.random.uniform(a, b, size)
        title_prefix = f"Uniform({a},␣{b})"

    ks_results = []

    for idx, n in enumerate(sample_sizes):
        # Simulate sample means
        sample_means = np.array([
            generate_sample(n).mean()
            for _ in range(n_simulations)
        ])

        # Theoretical normal distribution for sample mean
        theoretical_mean = true_mean
        theoretical_std = true_std / np.sqrt(n)

        # Standardize for normality test
        standardized = (sample_means - theoretical_mean) /
            theoretical_std
        ks_stat, p_value = kstest(standardized, 'norm')
        ks_results.append((n, ks_stat, p_value))

        # Create theoretical normal curve
        x_range = np.linspace(
            theoretical_mean - 4*theoretical_std,
            theoretical_mean + 4*theoretical_std,
            1000
        )
        theoretical_pdf = norm.pdf(x_range, theoretical_mean,
            theoretical_std)

        # Plot
```

```
        ax = axes[idx]
        ax.hist(sample_means, bins=50, density=True, alpha=0.7,
                color='steelblue', label='Simulated sample means',
                   edgecolor='black')
        ax.plot(x_range, theoretical_pdf, 'r-', linewidth=2.5,
                label='Theoretical Normal', alpha=0.9)
        ax.axvline(true_mean, color='green', linestyle='--', linewidth
           =2,
                   label=f'True mean = {true_mean:.3f}')

        ax.set_title(f'{title_prefix}, n = {n}\nSE = {theoretical_std
           :.4f}, ' +
                     f'KS p-value = {p_value:.4f}',
                     fontsize=11, fontweight='bold')
        ax.set_xlabel('Sample Mean', fontsize=10)
        ax.set_ylabel('Density', fontsize=10)
        ax.legend(fontsize=9)
        ax.grid(alpha=0.3)

    plt.tight_layout()
    plt.savefig(f'clt_demonstration_{distribution}.pdf', dpi=300,
       bbox_inches='tight')
    plt.show()

    # Print normality test results
    print(f"\nKolmogorov-Smirnov test for normality ({title_prefix}):")
    print(f"{'n':>6} {'KS statistic':>15} {'p-value':>10} {'Assessment
       ':>20}")
    print("-" * 55)
    for n, ks_stat, p_val in ks_results:
        assessment = "Excellent fit" if p_val > 0.05 else "Poor fit"
        print(f"{n:>6} {ks_stat:>15.4f} {p_val:>10.4f} {assessment:>20}
           ")

# Demonstrate for different distributions
print("=" * 70)
print("CLT Demonstration: Bernoulli Distribution (conversion rates)")
print("=" * 70)
demonstrate_clt('bernoulli', {'p': 0.05}, [10, 50, 200, 1000])

print("\n" + "=" * 70)
print("CLT Demonstration: Exponential Distribution (session times)")
print("=" * 70)
demonstrate_clt('exponential', {'scale': 5}, [10, 50, 200, 1000])

print("\n" + "=" * 70)
print("CLT Demonstration: Uniform Distribution")
print("=" * 70)
demonstrate_clt('uniform', {'a': 0, 'b': 10}, [10, 50, 200, 1000])
```

2.3.4 Law of Large Numbers

Closely related to the CLT is the Law of Large Numbers, which formalizes the intuition that sample averages converge to population means.

Theorem 2.2 (Weak Law of Large Numbers)**.** Let $X_1, X_2, \ldots$ be independent, identically

distributed with $\mathbb{E}[X_i] = \mu$. Then for any $\epsilon > 0$:

$$P(|\bar{X}_n - \mu| > \epsilon) \to 0 \text{ as } n \to \infty \tag{2.16}$$

In words: the sample mean converges in probability to the population mean.

Distinction from CLT: The Law of Large Numbers tells us sample means get close to the truth. The CLT tells us how the sample mean is distributed around the truth, enabling quantification of uncertainty.

2.4 Key Probability Distributions for A/B Testing

Several distributions arise repeatedly in experimental analysis. Understanding their properties and relationships is essential for correct application.

2.4.1 Bernoulli and Binomial Distributions

Definition 2.5 (Bernoulli Distribution)**.** A random variable $X \sim \text{Bernoulli}(p)$ represents a single binary trial, taking value 1 (success) with probability p and 0 (failure) with probability $1 - p$.

$$P(X = 1) = p, \quad P(X = 0) = 1 - p \tag{2.17}$$

$$\mathbb{E}[X] = p \tag{2.18}$$

$$\text{Var}(X) = p(1 - p) \tag{2.19}$$

Note that variance is maximized at $p = 0.5$ (maximum uncertainty) and approaches 0 as $p \to 0$ or $p \to 1$ (near certainty).

Definition 2.6 (Binomial Distribution)**.** If $X_1, \ldots, X_n$ are independent Bernoulli(p) variables, their sum $Y = \sum_{i=1}^{n} X_i$ follows Binomial(n, p), representing the number of successes in n trials.

$$P(Y = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \ldots, n \tag{2.20}$$

$$\mathbb{E}[Y] = np \tag{2.21}$$

$$\text{Var}(Y) = np(1 - p) \tag{2.22}$$

Practical Application: In A/B testing:

- X_i = whether user i converts (Bernoulli)
- Y = total number of conversions (Binomial)
- $\hat{p} = Y/n$ = observed conversion rate

2.4.2 Normal (Gaussian) Distribution

The normal distribution is ubiquitous in A/B testing, justified by the CLT.

Definition 2.7 (Normal Distribution)**.** A random variable $X \sim \mathcal{N}(\mu, \sigma^2)$ has probability density function:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R} \tag{2.23}$$

Properties:

- $\mathbb{E}[X] = \mu$ (location parameter)
- $\text{Var}(X) = \sigma^2$ (scale parameter)
- Symmetric around μ
- $\approx 68\%$ of mass within $\mu \pm \sigma$, 95% within $\mu \pm 2\sigma$, 99.7% within $\mu \pm 3\sigma$

The **standard normal distribution** $Z \sim \mathcal{N}(0, 1)$ serves as reference. Any normal variable can be standardized:

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1) \tag{2.24}$$

Proposition 2.3 (Properties of Normal Distribution)**.**

1. **Linear transformation:** If $X \sim \mathcal{N}(\mu, \sigma^2)$ and $Y = aX + b$, then:

$$Y \sim \mathcal{N}(a\mu + b, a^2\sigma^2) \tag{2.25}$$

2. **Sum of independent normals:** If $X_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ independently, then:

$$\sum_{i=1}^{n} X_i \sim \mathcal{N}\left(\sum_{i=1}^{n} \mu_i, \sum_{i=1}^{n} \sigma_i^2\right) \tag{2.26}$$

3. **Difference of independent normals:** Crucial for A/B testing:

$$X_1 - X_2 \sim \mathcal{N}(\mu_1 - \mu_2, \sigma_1^2 + \sigma_2^2) \tag{2.27}$$

2.4.3 Student's t-Distribution

When population variance is unknown (the usual case) and must be estimated from data, the t-distribution provides more accurate inference than the normal distribution, especially for small samples.

Definition 2.8 (Student's t-Distribution)**.** If $Z \sim \mathcal{N}(0, 1)$ and $V \sim \chi^2_\nu$ independently, then:

$$T = \frac{Z}{\sqrt{V/\nu}} \sim t_\nu \tag{2.28}$$

where ν is the degrees of freedom parameter.

Key properties:

- Symmetric around 0, like the normal distribution

- Heavier tails than normal (more probability in extremes)
- As $\nu \to \infty$, $t_\nu \to \mathcal{N}(0,1)$
- For $\nu \geq 30$, practically indistinguishable from normal

When to use:

- Comparing means when variance is estimated from data
- Small to moderate sample sizes ($n < 30$ per group)
- Constructing confidence intervals for means
- Two-sample t-tests in A/B experiments

Example 2.4 (t-Distribution in Practice)**.** Comparing average order value between variants with $n_1 = 25$ and $n_2 = 25$:

- Degrees of freedom: $\nu = n_1 + n_2 - 2 = 48$
- Use t_{48} critical values, not $\mathcal{N}(0,1)$
- For 95% CI: $t_{0.025,48} = 2.011$ vs. $z_{0.025} = 1.96$

The difference is small but matters for proper uncertainty quantification.

With $n_1 = n_2 = 1000$, we'd have $\nu = 1998$, and $t_{0.025,1998} \approx 1.961$ — essentially identical to the normal quantile.

2.4.4 Beta Distribution

The Beta distribution is the conjugate prior for the Bernoulli/Binomial likelihood in Bayesian A/B testing (Chapter 7).

Definition 2.9 (Beta Distribution)**.** A random variable $X \sim \text{Beta}(\alpha, \beta)$ has support on $[0,1]$ with PDF:

$$f(x) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}, \quad 0 \leq x \leq 1 \tag{2.29}$$

Properties:

$$\mathbb{E}[X] = \frac{\alpha}{\alpha+\beta} \tag{2.30}$$

$$\text{Var}(X) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)} \tag{2.31}$$

$$\text{Mode} = \frac{\alpha-1}{\alpha+\beta-2} \quad (\text{for } \alpha, \beta > 1) \tag{2.32}$$

The Beta distribution is extremely flexible:

- $\text{Beta}(1,1) = \text{Uniform}(0,1)$ (non-informative prior)
- $\alpha = \beta$ gives symmetric distributions
- $\alpha < \beta$ skews left, $\alpha > \beta$ skews right

- Large $\alpha + \beta$ gives narrow distributions (strong prior beliefs)

Listing 2.3: Visualizing Key Distributions

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import binom, norm, t, beta

sns.set_style("whitegrid")
np.random.seed(42)

fig = plt.figure(figsize=(16, 10))

# 1. Binomial distributions
ax1 = plt.subplot(2, 3, 1)
n = 100
for p in [0.1, 0.3, 0.5]:
    x = np.arange(0, n+1)
    pmf = binom.pmf(x, n, p)
    ax1.plot(x, pmf, marker='o', markersize=2, label=f'p = {p}', alpha
        =0.7)
ax1.set_xlabel('Number of Successes', fontsize=11)
ax1.set_ylabel('Probability', fontsize=11)
ax1.set_title('Binomial(n=100, p) Distributions', fontsize=12,
    fontweight='bold')
ax1.legend()
ax1.grid(alpha=0.3)

# 2. Normal distributions
ax2 = plt.subplot(2, 3, 2)
x = np.linspace(-8, 8, 1000)
for mu, sigma in [(0, 1), (0, 2), (2, 1)]:
    pdf = norm.pdf(x, mu, sigma)
    ax2.plot(x, pdf, linewidth=2.5,
             label=f'mu = {mu}, sigma = {sigma}', alpha=0.7)
ax2.set_xlabel('x', fontsize=11)
ax2.set_ylabel('Density', fontsize=11)
ax2.set_title('Normal Distributions', fontsize=12, fontweight='bold')
ax2.legend()
ax2.grid(alpha=0.3)

# 3. t-distributions
ax3 = plt.subplot(2, 3, 3)
x = np.linspace(-4, 4, 1000)
ax3.plot(x, norm.pdf(x, 0, 1), linewidth=2.5,
         label='Normal(0,1)', color='black', linestyle='--')
for df in [1, 3, 10, 30]:
    pdf = t.pdf(x, df)
    ax3.plot(x, pdf, linewidth=2, label=f'df = {df}', alpha=0.7)
ax3.set_xlabel('x', fontsize=11)
ax3.set_ylabel('Density', fontsize=11)
ax3.set_title("Student's t-Distributions", fontsize=12, fontweight='
    bold')
ax3.legend()
ax3.grid(alpha=0.3)

# 4. Beta distributions
```

```
ax4 = plt.subplot(2, 3, 4)
x = np.linspace(0, 1, 1000)
for alpha, beta_param in [(1, 1), (2, 5), (5, 2), (10, 10)]:
    pdf = beta.pdf(x, alpha, beta_param)
    ax4.plot(x, pdf, linewidth=2.5,
             label=f'  =  {alpha},    =  {beta_param}', alpha=0.7)
ax4.set_xlabel('x', fontsize=11)
ax4.set_ylabel('Density', fontsize=11)
ax4.set_title('Beta Distributions', fontsize=12, fontweight='bold')
ax4.legend()
ax4.grid(alpha=0.3)

# 5. Normal approximation to Binomial
ax5 = plt.subplot(2, 3, 5)
n, p = 100, 0.3
x_binom = np.arange(0, n+1)
pmf = binom.pmf(x_binom, n, p)
ax5.bar(x_binom, pmf, alpha=0.6, label='Binomial(100, 0.3)', color='
    steelblue')

mu = n * p
sigma = np.sqrt(n * p * (1-p))
x_norm = np.linspace(0, n, 1000)
pdf_norm = norm.pdf(x_norm, mu, sigma)
ax5.plot(x_norm, pdf_norm, 'r-', linewidth=2.5,
         label=f'Normal({mu:.0f}, {sigma:.2f}  )', alpha=0.9)
ax5.set_xlabel('Number of Successes', fontsize=11)
ax5.set_ylabel('Probability/Density', fontsize=11)
ax5.set_title('Normal Approximation to Binomial', fontsize=12,
    fontweight='bold')
ax5.legend()
ax5.grid(alpha=0.3)

# 6. Comparison of confidence intervals: z vs t
ax6 = plt.subplot(2, 3, 6)
sample_sizes = np.arange(5, 101, 5)
z_critical = norm.ppf(0.975)
t_critical = [t.ppf(0.975, n-1) for n in sample_sizes]

ax6.plot(sample_sizes, t_critical, 'b-', linewidth=2.5,
         label='t critical value', marker='o', markersize=4)
ax6.axhline(z_critical, color='red', linestyle='--', linewidth=2.5,
            label=f'z critical value = {z_critical:.3f}')
ax6.set_xlabel('Sample Size', fontsize=11)
ax6.set_ylabel('Critical Value (two-tailed,    =0.05)', fontsize=11)
ax6.set_title('t vs. Normal Critical Values', fontsize=12, fontweight='
    bold')
ax6.legend()
ax6.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('distributions_visualization.pdf', dpi=300, bbox_inches='
    tight')
plt.show()

# Print practical insights
print("\n" + "="*70)
print("Practical Insights on Distribution Selection")
```

```
print("="*70)
print(f"\nt-distribution␣vs␣Normal:")
print(f"Sample␣size␣␣␣␣␣t-critical␣(95%␣CI)␣␣␣␣␣z-critical␣␣␣␣␣
    Difference")
print("-" * 70)
for n in [5, 10, 20, 30, 50, 100]:
    t_crit = t.ppf(0.975, n-1)
    z_crit = norm.ppf(0.975)
    diff = (t_crit - z_crit) / z_crit * 100
    print(f"n␣=␣{n:3d}␣␣␣␣␣␣␣␣␣{t_crit:.4f}␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣{z_crit:.4f}
        ␣␣␣␣␣␣␣␣{diff:+.2f}%")
```

2.5 Statistical Inference

Statistical inference is the process of drawing conclusions about populations from samples. The fundamental challenge: we observe sample data but want to make statements about unobserved population parameters.

2.5.1 Population vs. Sample

- **Population:** The complete set of all individuals/units of interest
  - Parameters (fixed, unknown): μ, σ, p
  - What we want to know about
- **Sample:** A subset of the population that we actually observe
  - Statistics (random, observed): $\bar{x}, s, \hat{p}$
  - What we use to learn about parameters

Example 2.5 (Population vs. Sample in A/B Testing).

- **Population:** All potential users who could ever see our website
- **Parameter:** True conversion rate p for each variant
- **Sample:** Users in our experiment (e.g., 10,000 users)
- **Statistic:** Observed conversion rate $\hat{p} = 0.078$
- **Goal:** Use $\hat{p}$ to infer p and compare variants

2.5.2 Sampling Distributions

A sampling distribution is the probability distribution of a statistic over repeated samples from the same population.

Definition 2.10 (Sampling Distribution). If we:

1. Draw a random sample of size n from a population
2. Calculate a statistic (e.g., sample mean $\bar{X}$)

3. Repeat this process infinitely many times

The distribution of all those statistics is the sampling distribution.

Key insight: Even though we observe only one sample, we can deduce properties of the sampling distribution using probability theory (especially the CLT). This enables quantifying uncertainty.

2.5.3 Standard Error

The standard error (SE) is the standard deviation of a sampling distribution—it quantifies how much a statistic varies across repeated samples.

Definition 2.11 (Standard Error)**.** For a statistic $\hat{\theta}$ with sampling distribution variance $\text{Var}(\hat{\theta})$:

$$\text{SE}(\hat{\theta}) = \sqrt{\text{Var}(\hat{\theta})} \tag{2.33}$$

Common standard errors:

- **Sample mean:** $\text{SE}(\bar{X}) = \sigma/\sqrt{n}$
- **Sample proportion:** $\text{SE}(\hat{p}) = \sqrt{p(1-p)/n}$
- **Difference in proportions:** $\text{SE}(\hat{p}_1 - \hat{p}_2) = \sqrt{\frac{p_1(1-p_1)}{n_1} + \frac{p_2(1-p_2)}{n_2}}$

In practice, we estimate SE by plugging in sample values for unknown parameters:

$$\widehat{\text{SE}}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \tag{2.34}$$

2.5.4 Confidence Intervals

Confidence intervals provide a range of plausible values for a parameter, quantifying estimation uncertainty.

Definition 2.12 (Confidence Interval)**.** A $(1-\alpha)\times 100\%$ confidence interval for parameter θ is a random interval $[L, U]$ constructed from sample data such that:

$$P(L \leq \theta \leq U) = 1 - \alpha \tag{2.35}$$

before we observe the data. Common choices: $\alpha = 0.05$ (95% CI) or $\alpha = 0.01$ (99% CI).

Correct interpretation: "If we repeated this experiment many times and constructed a 95% CI each time, approximately 95% of those intervals would contain the true parameter value."

Common misinterpretation: "There is a 95% probability the true parameter is in this specific interval." (This is a Bayesian interpretation, not frequentist!)

Confidence Interval for a Proportion

For large n, by the CLT:

$$\hat{p} \sim \mathcal{N}\left(p, \frac{p(1-p)}{n}\right) \tag{2.36}$$

A $(1-\alpha)$ confidence interval (Wald interval):

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \tag{2.37}$$

where $z_{\alpha/2} = \Phi^{-1}(1-\alpha/2)$ is the standard normal quantile. For $\alpha = 0.05$: $z_{0.025} = 1.96$.

Example 2.6 (CI for Conversion Rate)**.** Experiment results: 85 conversions from 1,000 users.

$$\hat{p} = 85/1000 = 0.085 \tag{2.38}$$

$$\widehat{\mathrm{SE}}(\hat{p}) = \sqrt{\frac{0.085 \times 0.915}{1000}} = 0.00882 \tag{2.39}$$

95% CI:

$$0.085 \pm 1.96 \times 0.00882 = [0.0677, 0.1023] \tag{2.40}$$

Interpretation: We are 95% confident the true conversion rate is between 6.77% and 10.23%.

Confidence Interval for Difference in Proportions

For comparing two independent proportions (the core of A/B testing):

$$(\hat{p}_2 - \hat{p}_1) \pm z_{\alpha/2}\sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_2(1-\hat{p}_2)}{n_2}} \tag{2.41}$$

If the CI contains 0, we cannot conclude a significant difference at level α. If the CI is entirely positive (or negative), we have evidence of a difference.

Listing 2.4: Computing Confidence Intervals

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

def proportion_ci(conversions, n, confidence=0.95, method='wald'):
    """
␣␣␣␣Calculate␣confidence␣interval␣for␣a␣proportion.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣conversions␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣successes
```

```
    n : int
        Total number of trials
    confidence : float
        Confidence level (default 0.95)
    method : str
        'wald' (default), 'wilson', or 'agresti-coull'

    Returns:
    --------
    dict with 'estimate', 'ci_lower', 'ci_upper', 'se'
    """
    p_hat = conversions / n
    alpha = 1 - confidence
    z = stats.norm.ppf(1 - alpha/2)

    if method == 'wald':
        # Standard Wald interval
        se = np.sqrt(p_hat * (1 - p_hat) / n)
        ci_lower = p_hat - z * se
        ci_upper = p_hat + z * se

    elif method == 'wilson':
        # Wilson score interval (better for small p or n)
        denominator = 1 + z**2/n
        center = (p_hat + z**2/(2*n)) / denominator
        margin = z * np.sqrt(p_hat*(1-p_hat)/n + z**2/(4*n**2)) /
            denominator
        ci_lower = center - margin
        ci_upper = center + margin
        se = np.sqrt(p_hat * (1 - p_hat) / n)
        p_hat = center  # Wilson estimate

    elif method == 'agresti-coull':
        # Agresti-Coull interval (add 2 successes and 2 failures)
        n_tilde = n + z**2
        p_tilde = (conversions + z**2/2) / n_tilde
        se = np.sqrt(p_tilde * (1 - p_tilde) / n_tilde)
        ci_lower = p_tilde - z * se
        ci_upper = p_tilde + z * se
        p_hat = p_tilde

    return {
        'estimate': p_hat,
        'ci_lower': max(0, ci_lower),  # Ensure in [0,1]
        'ci_upper': min(1, ci_upper),
        'se': se
    }


def proportion_diff_ci(conv1, n1, conv2, n2, confidence=0.95):
    """
    Calculate confidence interval for difference in proportions.

    Parameters:
    -----------
    conv1, conv2 : int
        Number of conversions in each group
    n1, n2 : int
```

```
        Sample sizes
    confidence : float
        Confidence level

    Returns:
    --------
    dict with difference estimates and CI
    """
    p1_hat = conv1 / n1
    p2_hat = conv2 / n2
    diff = p2_hat - p1_hat

    se = np.sqrt(p1_hat * (1 - p1_hat) / n1 +
                 p2_hat * (1 - p2_hat) / n2)

    alpha = 1 - confidence
    z = stats.norm.ppf(1 - alpha/2)

    ci_lower = diff - z * se
    ci_upper = diff + z * se

    # Relative lift
    relative_lift = (p2_hat - p1_hat) / p1_hat if p1_hat > 0 else np.
        nan

    return {
        'p1': p1_hat,
        'p2': p2_hat,
        'difference': diff,
        'relative_lift': relative_lift,
        'ci_lower': ci_lower,
        'ci_upper': ci_upper,
        'se': se,
        'significant': not (ci_lower <= 0 <= ci_upper)
    }


# Example: Compare CI methods
print("="*70)
print("Comparing Confidence Interval Methods")
print("="*70)

scenarios = [
    (50, 500, "Moderate p, large n"),
    (5, 50, "Small p, small n"),
    (250, 500, "p near 0.5, large n"),
    (2, 100, "Very small p")
]

for conversions, n, description in scenarios:
    print(f"\n{description}: {conversions}/{n} conversions")
    print("-" * 70)

    for method in ['wald', 'wilson', 'agresti-coull']:
        result = proportion_ci(conversions, n, method=method)
        width = result['ci_upper'] - result['ci_lower']
        print(f"{method:15s}: [{result['ci_lower']:.4f}, {result['
            ci_upper']:.4f}]  " +
```

```
                f"width = {width:.4f}")

# Example: A/B test comparison
print("\n" + "="*70)
print("A/B Test: Confidence Interval for Difference")
print("="*70)

control_conv, control_n = 80, 1000
treatment_conv, treatment_n = 95, 1000

result = proportion_diff_ci(control_conv, control_n,
                            treatment_conv, treatment_n)

print(f"\nControl:   {result['p1']:.2%} conversion rate ({control_conv}/{control_n})")
print(f"Treatment: {result['p2']:.2%} conversion rate ({treatment_conv}/{treatment_n})")
print(f"\nAbsolute difference: {result['difference']:.4f} ({result['difference']*100:.2f} pp)")
print(f"Relative lift:       {result['relative_lift']:.2%}")
print(f"\n95% CI for difference: [{result['ci_lower']:.4f}, {result['ci_upper']:.4f}]")
print(f"Significant at  =0.05: {result['significant']}")

# Visualize confidence intervals
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: CI comparison across methods
ax1 = axes[0]
methods = ['Wald', 'Wilson', 'Agresti-Coull']
estimates = []
lowers = []
uppers = []

for method in ['wald', 'wilson', 'agresti-coull']:
    ci = proportion_ci(50, 500, method=method)
    estimates.append(ci['estimate'])
    lowers.append(ci['ci_lower'])
    uppers.append(ci['ci_upper'])

y_pos = np.arange(len(methods))
errors = np.array([[est - low for est, low in zip(estimates, lowers)],
                   [up - est for up, est in zip(uppers, estimates)]])

ax1.errorbar(estimates, y_pos, xerr=errors, fmt='o', markersize=8,
    capsize=5, capthick=2)
ax1.set_yticks(y_pos)
ax1.set_yticklabels(methods)
ax1.set_xlabel('Conversion Rate', fontsize=12)
ax1.set_title('95% CI Methods Comparison (50/500 conversions)',
              fontsize=12, fontweight='bold')
ax1.grid(axis='x', alpha=0.3)

# Plot 2: A/B test visualization
ax2 = axes[1]
groups = ['Control', 'Treatment']
conversion_rates = [result['p1'], result['p2']]
ci_control = proportion_ci(control_conv, control_n)
```

```
ci_treatment = proportion_ci(treatment_conv, treatment_n)

errors = np.array([
    [ci_control['estimate'] - ci_control['ci_lower'],
     ci_treatment['estimate'] - ci_treatment['ci_lower']],
    [ci_control['ci_upper'] - ci_control['estimate'],
     ci_treatment['ci_upper'] - ci_treatment['estimate']]
])

x_pos = np.arange(len(groups))
ax2.bar(x_pos, conversion_rates, alpha=0.7, color=['steelblue', 'coral'])
ax2.errorbar(x_pos, conversion_rates, yerr=errors, fmt='none',
             ecolor='black', capsize=10, capthick=2)
ax2.set_xticks(x_pos)
ax2.set_xticklabels(groups, fontsize=12)
ax2.set_ylabel('Conversion Rate', fontsize=12)
ax2.set_title('A/B Test Comparison with 95% CIs', fontsize=12, fontweight='bold')
ax2.grid(axis='y', alpha=0.3)

# Add significance indicator
if result['significant']:
    ax2.text(0.5, max(conversion_rates)*1.1, 'Significant difference (p<0.05)',
             ha='center', fontsize=11, bbox=dict(boxstyle='round',
                facecolor='lightgreen'))
else:
    ax2.text(0.5, max(conversion_rates)*1.1, 'No significant difference ( p 0 .05)',
             ha='center', fontsize=11, bbox=dict(boxstyle='round',
                facecolor='lightyellow'))

plt.tight_layout()
plt.savefig('confidence_intervals_comparison.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

2.6 Hypothesis Testing Framework

Hypothesis testing provides a formal framework for making decisions under uncertainty. It's the workhorse methodology for A/B testing analysis.

2.6.1 The Logic of Hypothesis Testing

Hypothesis testing uses proof by contradiction:

1. Assume the null hypothesis (typically "no effect") is true
2. Calculate: "If null were true, how surprising is our observed data?"
3. If data is very surprising under the null, reject the null
4. Otherwise, fail to reject (we don't "accept" the null)

Definition 2.13 (Null and Alternative Hypotheses). • **Null hypothesis (H_0):** The default assumption, typically "no effect" or "no difference"

• **Alternative hypothesis (H_1 or H_A):** What we suspect or want to demonstrate In A/B testing:

$$H_0 : p_{\text{treatment}} = p_{\text{control}} \quad \text{(no difference)} \tag{2.42}$$
$$H_1 : p_{\text{treatment}} \neq p_{\text{control}} \quad \text{(two-sided)} \tag{2.43}$$
$$\text{or } H_1 : p_{\text{treatment}} > p_{\text{control}} \quad \text{(one-sided)} \tag{2.44}$$

2.6.2 Test Statistics and p-values

Definition 2.14 (Test Statistic). A test statistic is a function of sample data used to test hypotheses. It quantifies how far the data deviates from what we'd expect under H_0.

For comparing proportions:

$$z = \frac{(\hat{p}_2 - \hat{p}_1) - 0}{\text{SE}_0(\hat{p}_2 - \hat{p}_1)} \tag{2.45}$$

where SE_0 is computed under the null hypothesis of equal proportions.

Definition 2.15 (p-value). The p-value is the probability of observing a test statistic at least as extreme as what we observed, assuming H_0 is true:

$$p\text{-value} = P(\text{test statistic} \geq \text{observed value} \mid H_0 \text{ true}) \tag{2.46}$$

Interpretation:

• Small p-value (< 0.05): Data is surprising under H_0, reject H_0

• Large p-value (0.05): Data is consistent with H_0, fail to reject

Critical point: The p-value is NOT the probability that H_0 is true. It's the probability of seeing data this extreme if H_0 were true—a subtle but crucial distinction.

2.6.3 Significance Level and Decision Rule

Definition 2.16 (Significance Level). The significance level α is the threshold for rejecting H_0. Common choices:

• $\alpha = 0.05$ (5%): Standard in most applications

• $\alpha = 0.01$ (1%): More conservative, reduces false positives

• $\alpha = 0.10$ (10%): More liberal, increases power but more false positives

Decision rule: Reject H_0 if p-value $< \alpha$

Reality	**Fail to Reject H_0**	**Reject H_0**
H_0 true	Correct decision (1-α)	**Type I Error** (α)
H_1 true	**Type II Error** (β)	Correct decision (1-β)

Table 2.1: Type I and Type II errors in hypothesis testing

2.6.4 Type I and Type II Errors

Hypothesis testing involves two types of errors:

Definition 2.17 (Type I Error (False Positive))**.** Rejecting H_0 when it's actually true—declaring a difference when none exists.

$$P(\text{Type I Error}) = \alpha \tag{2.47}$$

In A/B testing: Launching a "winning" variant that's actually no better (or worse) than control.

Definition 2.18 (Type II Error (False Negative))**.** Failing to reject H_0 when H_1 is true—missing a real effect.

$$P(\text{Type II Error}) = \beta \tag{2.48}$$

In A/B testing: Abandoning a variant that would actually improve metrics.

Definition 2.19 (Statistical Power)**.** Power is the probability of correctly rejecting H_0 when H_1 is true:

$$\text{Power} = 1 - \beta = P(\text{reject } H_0 \mid H_1 \text{ true}) \tag{2.49}$$

Desired power is typically 0.80 (80%) or 0.90 (90%). Chapter 4 covers power analysis in depth.

Tradeoff: Decreasing α (being more conservative about declaring significance) increases β (reduces power) for fixed sample size. The only way to decrease both is to increase sample size.

2.6.5 Two-Sample Z-Test for Proportions

The workhorse test for A/B testing with binary outcomes.

Setup:

- Control: n_1 trials, x_1 successes, $\hat{p}_1 = x_1/n_1$
- Treatment: n_2 trials, x_2 successes, $\hat{p}_2 = x_2/n_2$

Hypotheses:

$$H_0 : p_1 = p_2 \tag{2.50}$$
$$H_1 : p_1 \neq p_2 \quad \text{(two-sided)} \tag{2.51}$$

Under H_0: Pool data to estimate common proportion:

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2} \tag{2.52}$$

Test statistic:

$$z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}} \tag{2.53}$$

Under H_0, $z \sim \mathcal{N}(0,1)$ approximately (by CLT).
p-value (two-sided):

$$p = 2 \cdot P(Z \geq |z|) = 2 \cdot (1 - \Phi(|z|)) \tag{2.54}$$

where Φ is the standard normal CDF.

Example 2.7 (Z-Test for A/B Experiment).

- Control: 1,200 users, 96 conversions, $\hat{p}_1 = 0.08$
- Treatment: 1,200 users, 120 conversions, $\hat{p}_2 = 0.10$

Pooled proportion:

$$\hat{p} = \frac{96 + 120}{1200 + 1200} = \frac{216}{2400} = 0.09 \tag{2.55}$$

Test statistic:

$$z = \frac{0.10 - 0.08}{\sqrt{0.09 \times 0.91 \times (1/1200 + 1/1200)}} = \frac{0.02}{0.01168} = 1.712 \tag{2.56}$$

p-value:

$$p = 2 \cdot P(Z \geq 1.712) = 2 \cdot (1 - 0.9566) = 0.0868 \tag{2.57}$$

Conclusion: With $p = 0.0868 > 0.05$, we fail to reject H_0 at the 5% level. Insufficient evidence to conclude treatment differs from control, despite the 2 percentage point observed difference.

Listing 2.5: Implementing Hypothesis Tests

```
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")

def z_test_proportions(conv1, n1, conv2, n2, alpha=0.05, alternative='two-sided'):
    """
␣␣␣␣Two-sample␣z-test␣for␣proportions.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣conv1,␣conv2␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣conversions␣in␣each␣group
␣␣␣␣n1,␣n2␣:␣int
␣␣␣␣␣␣␣␣Sample␣sizes
␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣Significance␣level
␣␣␣␣alternative␣:␣str
␣␣␣␣␣␣␣␣'two-sided',␣'greater',␣or␣'less'

```

```
    Returns:
    --------
    dict with test results
    """
    p1_hat = conv1 / n1
    p2_hat = conv2 / n2

    # Pooled proportion under H0
    p_pool = (conv1 + conv2) / (n1 + n2)

    # Standard error under H0
    se_null = np.sqrt(p_pool * (1 - p_pool) * (1/n1 + 1/n2))

    # Test statistic
    z_stat = (p2_hat - p1_hat) / se_null

    # p-value
    if alternative == 'two-sided':
        p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))
    elif alternative == 'greater':
        p_value = 1 - stats.norm.cdf(z_stat)
    elif alternative == 'less':
        p_value = stats.norm.cdf(z_stat)

    # Critical value
    if alternative == 'two-sided':
        z_critical = stats.norm.ppf(1 - alpha/2)
    else:
        z_critical = stats.norm.ppf(1 - alpha)

    # Decision
    reject_null = p_value < alpha

    # Effect size (relative lift)
    relative_lift = (p2_hat - p1_hat) / p1_hat if p1_hat > 0 else np.
        nan

    return {
        'p1': p1_hat,
        'p2': p2_hat,
        'p_pool': p_pool,
        'difference': p2_hat - p1_hat,
        'relative_lift': relative_lift,
        'z_statistic': z_stat,
        'p_value': p_value,
        'alpha': alpha,
        'z_critical': z_critical,
        'reject_null': reject_null,
        'significant': reject_null
    }


def visualize_hypothesis_test(result):
    """Visualize the hypothesis test result."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Plot 1: Conversion rates with error bars
    ax1 = axes[0]
```

```
    groups = ['Control', 'Treatment']
    rates = [result['p1'], result['p2']]
    colors = ['steelblue', 'coral']

    bars = ax1.bar(groups, rates, color=colors, alpha=0.7, edgecolor='
        black', linewidth=1.5)
    ax1.set_ylabel('Conversion Rate', fontsize=12)
    ax1.set_title('Conversion Rates Comparison', fontsize=13,
        fontweight='bold')
    ax1.grid(axis='y', alpha=0.3)

    # Add value labels
    for i, (bar, rate) in enumerate(zip(bars, rates)):
        height = bar.get_height()
        ax1.text(bar.get_x() + bar.get_width()/2., height,
                f'{rate:.2%}',
                ha='center', va='bottom', fontsize=11, fontweight='bold
                    ')

    # Add significance indicator
    y_max = max(rates) * 1.15
    if result['significant']:
        ax1.plot([0, 1], [y_max, y_max], 'k-', linewidth=2)
        ax1.text(0.5, y_max*1.02, f'p = {result["p_value"]:.4f} *',
                ha='center', fontsize=11, fontweight='bold',
                bbox=dict(boxstyle='round', facecolor='lightgreen',
                    alpha=0.7))
    else:
        ax1.text(0.5, y_max, f'p = {result["p_value"]:.4f}\n(not
            significant)',
                ha='center', fontsize=11,
                bbox=dict(boxstyle='round', facecolor='lightyellow',
                    alpha=0.7))

    # Plot 2: Standard normal distribution with test statistic
    ax2 = axes[1]
    x = np.linspace(-4, 4, 1000)
    y = stats.norm.pdf(x, 0, 1)

    ax2.plot(x, y, 'b-', linewidth=2, label='Null distribution N(0,1)')
    ax2.fill_between(x, 0, y, where=(x <= -result['z_critical']) | (x
        >= result['z_critical']),
                    alpha=0.3, color='red', label=f'Rejection region (
                        ={result["alpha"]})')

    # Mark observed z-statistic
    ax2.axvline(result['z_statistic'], color='darkred', linestyle='--',
                linewidth=2.5, label=f'Observed z = {result["z_statistic
                    "]:.3f}')

    ax2.set_xlabel('z-value', fontsize=12)
    ax2.set_ylabel('Density', fontsize=12)
    ax2.set_title('Hypothesis Test Visualization', fontsize=13,
        fontweight='bold')
    ax2.legend(fontsize=10)
    ax2.grid(alpha=0.3)

    plt.tight_layout()
```

```
    return fig


# Example: Run multiple tests
print("="*80)
print("Hypothesis Testing Examples")
print("="*80)

test_scenarios = [
    {
        'name': 'Significant effect (large sample)',
        'conv1': 80, 'n1': 1000,
        'conv2': 120, 'n2': 1000
    },
    {
        'name': 'Non-significant effect (large sample)',
        'conv1': 80, 'n1': 1000,
        'conv2': 95, 'n2': 1000
    },
    {
        'name': 'Significant effect (small sample)',
        'conv1': 8, 'n1': 50,
        'conv2': 20, 'n2': 50
    },
    {
        'name': 'Non-significant effect (small sample)',
        'conv1': 4, 'n1': 40,
        'conv2': 8, 'n2': 40
    }
]

for i, scenario in enumerate(test_scenarios, 1):
    print(f"\n{'='*80}")
    print(f"Scenario {i}: {scenario['name']}")
    print('='*80)

    result = z_test_proportions(
        scenario['conv1'], scenario['n1'],
        scenario['conv2'], scenario['n2']
    )

    print(f"\nControl:    {result['p1']:.2%} ({scenario['conv1']}/{
        scenario['n1']} conversions)")
    print(f"Treatment: {result['p2']:.2%} ({scenario['conv2']}/{
        scenario['n2']} conversions)")
    print(f"\nAbsolute difference: {result['difference']:.4f} ({result
        ['difference']*100:.2f} pp)")
    print(f"Relative lift:       {result['relative_lift']:.2%}")
    print(f"\nPooled proportion:   {result['p_pool']:.4f}")
    print(f"z-statistic:         {result['z_statistic']:.4f}")
    print(f"p-value:             {result['p_value']:.6f}")
    print(f"Critical value (  =0.05):    {result['z_critical']:.4f}")
    print(f"\nDecision: {'REJECT H0 - Significant difference' if result
        ['significant'] else 'FAIL TO REJECT H0 - No significant
        difference'}")

    # Visualize first scenario
    if i == 1:
```

```
        fig = visualize_hypothesis_test(result)
        plt.savefig(f'hypothesis_test_scenario_{i}.pdf', dpi=300,
            bbox_inches='tight')
        plt.show()

# Demonstrate power of test via simulation
print("\n" + "="*80)
print("Statistical Power Demonstration via Simulation")
print("="*80)

def simulate_power(p1, p2, n, alpha=0.05, n_sims=10000):
    """
    Simulate statistical power for given parameters.
    """
    significant_count = 0

    for _ in range(n_sims):
        # Generate data under H1 (p2 != p1)
        conv1 = np.random.binomial(n, p1)
        conv2 = np.random.binomial(n, p2)

        # Perform test
        result = z_test_proportions(conv1, n, conv2, n, alpha=alpha)
        if result['significant']:
            significant_count += 1

    return significant_count / n_sims

# Test different sample sizes
p_control = 0.08
p_treatment = 0.10  # 25% relative lift
sample_sizes = [100, 300, 500, 1000, 2000, 5000]

print(f"\nTrue conversion rates: Control = {p_control:.1%}, Treatment =
    {p_treatment:.1%}")
print(f"True relative lift: {(p_treatment - p_control)/p_control:.1%}")
print(f"\n{'Sample size (per group)':>25} {'Empirical power':>20}")
print("-" * 50)

powers = []
for n in sample_sizes:
    power = simulate_power(p_control, p_treatment, n)
    powers.append(power)
    print(f"{n:>25} {power:>20.3f}")

# Plot power curve
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(sample_sizes, powers, 'bo-', linewidth=2, markersize=8)
ax.axhline(0.8, color='red', linestyle='--', linewidth=2, label='Target
    power = 0.80')
ax.set_xlabel('Sample Size (per group)', fontsize=12)
ax.set_ylabel('Statistical Power', fontsize=12)
ax.set_title(f'Power Curve (p1={p_control:.1%}, p2={p_treatment:.1%},
     =0.05)',
             fontsize=13, fontweight='bold')
ax.grid(alpha=0.3)
ax.legend(fontsize=11)
plt.tight_layout()
```

```
plt.savefig('power_curve_simulation.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

2.7 Effect Size Measures

Statistical significance tells us whether an effect exists; effect size tells us whether it matters. This distinction is crucial for A/B testing—a statistically significant result may be practically irrelevant, especially with large samples.

2.7.1 Why Effect Size Matters

Consider two scenarios:

1. $n = 100,000$ per variant: 5.00% vs. 5.05% conversion, $p < 0.001$ (highly significant)
2. $n = 500$ per variant: 5.0% vs. 7.5% conversion, $p = 0.08$ (not significant)

In Scenario 1, the difference is statistically significant but tiny (0.05 percentage points). Implementation costs may exceed benefits.

In Scenario 2, the difference is large (2.5 percentage points, 50% relative lift) but not statistically significant due to small sample. More data might be worthwhile.

Key principle: Always report both statistical significance (p-value) and effect size. Make decisions considering both along with practical constraints.

2.7.2 Absolute vs. Relative Differences

Definition 2.20 (Absolute Difference)**.**

$$\text{Absolute difference} = p_2 - p_1 \tag{2.58}$$

Measured in the same units as the metric (e.g., percentage points for conversion rates).

Definition 2.21 (Relative Difference (Relative Lift))**.**

$$\text{Relative lift} = \frac{p_2 - p_1}{p_1} = \frac{p_2}{p_1} - 1 \tag{2.59}$$

Expressed as a percentage or decimal. Answers: "By what fraction did the metric change?"

Example 2.8 (Absolute vs. Relative)**.** 1. Control: 2%, Treatment: 3%

- Absolute difference: $3\% - 2\% = 1$ percentage point
- Relative lift: $(3\% - 2\%)/2\% = 0.50 = 50\%$

2. Control: 20%, Treatment: 21%

- Absolute difference: $21\% - 20\% = 1$ percentage point
- Relative lift: $(21\% - 20\%)/20\% = 0.05 = 5\%$

Same absolute difference, vastly different relative lifts. Which matters more depends on context.

2.7.3 Cohen's h for Proportions

Cohen's h standardizes the difference between proportions, enabling comparison across experiments with different baseline rates.

Definition 2.22 (Cohen's h)**.** For proportions p_1 and p_2:

$$h = 2(\arcsin\sqrt{p_2} - \arcsin\sqrt{p_1}) \tag{2.60}$$

where arcsin is the inverse sine function (arcsine).

Interpretation guidelines (Cohen, 1988):

- $|h| < 0.2$: Small effect
- $0.2 \leq |h| < 0.5$: Medium effect
- $|h| \geq 0.5$: Large effect

Advantage: Unlike relative lift, Cohen's h behaves well for proportions near 0 or 1.

2.7.4 Odds Ratio

The odds ratio is commonly used in medical research and logistic regression contexts.

Definition 2.23 (Odds and Odds Ratio)**.** The **odds** of an event with probability p is:

$$\text{Odds} = \frac{p}{1-p} \tag{2.61}$$

The **odds ratio** comparing two proportions:

$$\text{OR} = \frac{p_2/(1-p_2)}{p_1/(1-p_1)} = \frac{p_2(1-p_1)}{p_1(1-p_2)} \tag{2.62}$$

Interpretation:

- OR $= 1$: Equal odds (no effect)
- OR > 1: Higher odds in treatment
- OR < 1: Lower odds in treatment

Example 2.9 (Odds Ratio Calculation)**.** Control: 10% conversion, Treatment: 15% conversion

$$\text{Odds}_{\text{control}} = 0.10/0.90 = 0.111 \tag{2.63}$$
$$\text{Odds}_{\text{treatment}} = 0.15/0.85 = 0.176 \tag{2.64}$$
$$\text{OR} = 0.176/0.111 = 1.59 \tag{2.65}$$

Interpretation: The odds of conversion are 1.59 times higher in treatment—or 59% higher odds.

Log odds ratio: Often log(OR) is reported because it's symmetric around 0 and unbounded.

2.7.5 Cohen's d for Continuous Metrics

For continuous metrics (e.g., revenue, session duration), Cohen's d measures effect size in standard deviation units.

Definition 2.24 (Cohen's d)**.**

$$d = \frac{\bar{x}_2 - \bar{x}_1}{s_{\text{pooled}}} \tag{2.66}$$

where the pooled standard deviation is:

$$s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}} \tag{2.67}$$

Interpretation guidelines:

- $|d| < 0.2$: Small effect
- $0.2 \leq |d| < 0.5$: Small to medium effect
- $0.5 \leq |d| < 0.8$: Medium to large effect
- $|d| \geq 0.8$: Large effect

Example 2.10 (Cohen's d for Revenue)**.** Testing pricing change:

- Control: $\bar{x}_1 = \$45$, $s_1 = \$20$, $n_1 = 500$
- Treatment: $\bar{x}_2 = \$52$, $s_2 = \$22$, $n_2 = 500$

Pooled standard deviation:

$$s_{\text{pooled}} = \sqrt{\frac{499 \times 20^2 + 499 \times 22^2}{998}} = \sqrt{440.2} = 20.98 \tag{2.68}$$

Cohen's d:

$$d = \frac{52 - 45}{20.98} = \frac{7}{20.98} = 0.334 \tag{2.69}$$

This is a small to medium effect (between 0.2 and 0.5).

Listing 2.6: Computing Effect Sizes

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_style("whitegrid")

def compute_effect_sizes(conv1, n1, conv2, n2):
    """
␣␣␣␣Compute␣multiple␣effect␣size␣measures␣for␣proportion␣comparison.

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣with␣various␣effect␣size␣measures
␣␣␣␣"""
```

```
    p1 = conv1 / n1
    p2 = conv2 / n2

    # Absolute difference
    abs_diff = p2 - p1

    # Relative lift
    rel_lift = (p2 - p1) / p1 if p1 > 0 else np.nan

    # Cohen's h
    cohens_h = 2 * (np.arcsin(np.sqrt(p2)) - np.arcsin(np.sqrt(p1)))

    # Odds ratio
    odds1 = p1 / (1 - p1) if p1 < 1 else np.inf
    odds2 = p2 / (1 - p2) if p2 < 1 else np.inf
    odds_ratio = odds2 / odds1 if odds1 > 0 and np.isfinite(odds1) else
        np.nan
    log_odds_ratio = np.log(odds_ratio) if odds_ratio > 0 and np.
       isfinite(odds_ratio) else np.nan

    # Number needed to treat (NNT) - for proportions
    nnt = 1 / abs_diff if abs_diff != 0 else np.inf

    return {
        'p1': p1,
        'p2': p2,
        'absolute_diff': abs_diff,
        'relative_lift': rel_lift,
        'cohens_h': cohens_h,
        'odds_ratio': odds_ratio,
        'log_odds_ratio': log_odds_ratio,
        'nnt': nnt
    }


def cohens_d_continuous(mean1, std1, n1, mean2, std2, n2):
    """
␣␣␣␣Compute␣Cohen's␣d␣for␣continuous␣metrics.
␣␣␣␣"""
    # Pooled standard deviation
    pooled_std = np.sqrt(((n1-1)*std1**2 + (n2-1)*std2**2) / (n1 + n2 -
        2))

    # Cohen's d
    d = (mean2 - mean1) / pooled_std

    return {
        'mean1': mean1,
        'mean2': mean2,
        'pooled_std': pooled_std,
        'cohens_d': d,
        'interpretation': interpret_cohens_d(abs(d))
    }


def interpret_cohens_h(h):
    """Interpret␣Cohen's␣h␣effect␣size."""
    h_abs = abs(h)
```

```
    if h_abs < 0.2:
        return "Small"
    elif h_abs < 0.5:
        return "Medium"
    else:
        return "Large"


def interpret_cohens_d(d):
    """Interpret Cohen's d effect size."""
    d_abs = abs(d)
    if d_abs < 0.2:
        return "Negligible"
    elif d_abs < 0.5:
        return "Small"
    elif d_abs < 0.8:
        return "Medium"
    else:
        return "Large"


# Example calculations
print("="*80)
print("Effect Size Examples for Proportions")
print("="*80)

scenarios = [
    {
        'name': 'Low baseline, small absolute diff',
        'conv1': 20, 'n1': 1000,
        'conv2': 30, 'n2': 1000
    },
    {
        'name': 'Low baseline, large absolute diff',
        'conv1': 20, 'n1': 1000,
        'conv2': 60, 'n2': 1000
    },
    {
        'name': 'High baseline, small absolute diff',
        'conv1': 400, 'n1': 1000,
        'conv2': 410, 'n2': 1000
    },
    {
        'name': 'High baseline, large absolute diff',
        'conv1': 400, 'n1': 1000,
        'conv2': 500, 'n2': 1000
    }
]

for scenario in scenarios:
    print(f"\n{'-'*80}")
    print(f"{scenario['name']}")
    print('-'*80)

    es = compute_effect_sizes(
        scenario['conv1'], scenario['n1'],
        scenario['conv2'], scenario['n2']
    )
```

```
129
130     print(f"Control:␣␣␣{es['p1']:.2%}␣({scenario['conv1']}/{scenario['
            n1']})")
131     print(f"Treatment:␣{es['p2']:.2%}␣({scenario['conv2']}/{scenario['
            n2']})")
132     print(f"\nEffect␣sizes:")
133     print(f"␣␣Absolute␣difference:␣{es['absolute_diff']:.4f}␣({es['
            absolute_diff']*100:.2f}␣pp)")
134     print(f"␣␣Relative␣lift:␣␣␣␣␣␣␣{es['relative_lift']:.2%}")
135     print(f"␣␣Cohen's␣h:␣␣␣␣␣␣␣␣␣␣{es['cohens_h']:.4f}␣({
            interpret_cohens_h(es['cohens_h'])})")
136     print(f"␣␣Odds␣ratio:␣␣␣␣␣␣␣␣␣{es['odds_ratio']:.4f}")
137     print(f"␣␣Log␣odds␣ratio:␣␣␣␣␣␣{es['log_odds_ratio']:.4f}")
138     print(f"␣␣NNT:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣{es['nnt']:.1f}")
139
140 # Example for continuous metrics
141 print("\n" + "="*80)
142 print("Effect␣Size␣Examples␣for␣Continuous␣Metrics")
143 print("="*80)
144
145 continuous_scenarios = [
146     {
147         'name': 'Revenue␣per␣user',
148         'mean1': 45, 'std1': 20, 'n1': 500,
149         'mean2': 52, 'std2': 22, 'n2': 500,
150         'unit': '$'
151     },
152     {
153         'name': 'Session␣duration',
154         'mean1': 180, 'std1': 120, 'n1': 1000,
155         'mean2': 200, 'std2': 125, 'n2': 1000,
156         'unit': 'seconds'
157     }
158 ]
159
160 for scenario in continuous_scenarios:
161     print(f"\n{'-'*80}")
162     print(f"{scenario['name']}")
163     print('-'*80)
164
165     result = cohens_d_continuous(
166         scenario['mean1'], scenario['std1'], scenario['n1'],
167         scenario['mean2'], scenario['std2'], scenario['n2']
168     )
169
170     print(f"Control:␣␣␣{scenario['unit']}{result['mean1']:.2f}␣(SD␣=␣{
            scenario['std1']:.2f},␣n␣=␣{scenario['n1']})")
171     print(f"Treatment:␣{scenario['unit']}{result['mean2']:.2f}␣(SD␣=␣{
            scenario['std2']:.2f},␣n␣=␣{scenario['n2']})")
172     print(f"\nPooled␣SD:␣{scenario['unit']}{result['pooled_std']:.2f}")
173     print(f"Cohen's␣d:␣{result['cohens_d']:.4f}␣({result['
            interpretation']}␣effect)")
174
175 # Visualize effect size landscape
176 print("\n" + "="*80)
177 print("Visualizing␣Effect␣Size␣Landscape")
178 print("="*80)
179
```

```
# Create grid of p1 and p2 values
p1_values = np.linspace(0.01, 0.5, 50)
p2_values = np.linspace(0.01, 0.5, 50)
P1, P2 = np.meshgrid(p1_values, p2_values)

# Compute relative lift and Cohen's h for each combination
relative_lift = (P2 - P1) / P1
cohens_h_grid = 2 * (np.arcsin(np.sqrt(P2)) - np.arcsin(np.sqrt(P1)))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Relative lift heatmap
im1 = axes[0].contourf(P1, P2, relative_lift, levels=20, cmap='RdYlGn')
axes[0].plot([0, 0.5], [0, 0.5], 'k--', linewidth=2, label='No effect line')
axes[0].set_xlabel('Control Conversion Rate', fontsize=12)
axes[0].set_ylabel('Treatment Conversion Rate', fontsize=12)
axes[0].set_title('Relative Lift Landscape', fontsize=13, fontweight='bold')
plt.colorbar(im1, ax=axes[0], label='Relative Lift')
axes[0].legend()

# Plot 2: Cohen's h heatmap
im2 = axes[1].contourf(P1, P2, cohens_h_grid, levels=20, cmap='RdYlGn')
axes[1].plot([0, 0.5], [0, 0.5], 'k--', linewidth=2, label='No effect line')
# Add contour lines for small/medium/large thresholds
axes[1].contour(P1, P2, np.abs(cohens_h_grid), levels=[0.2, 0.5],
                colors='black', linewidths=2, linestyles=[':', '--'])
axes[1].set_xlabel('Control Conversion Rate', fontsize=12)
axes[1].set_ylabel('Treatment Conversion Rate', fontsize=12)
axes[1].set_title("Cohen's h Landscape", fontsize=13, fontweight='bold')
plt.colorbar(im2, ax=axes[1], label="Cohen's h")
axes[1].legend()

plt.tight_layout()
plt.savefig('effect_size_landscape.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

2.7.6 Practical Guidelines for Effect Size Reporting

When reporting A/B test results, always include:

1. **Sample sizes:** $n_1 = 1,000$, $n_2 = 1,000$
2. **Observed metrics:** Control = 8.0%, Treatment = 10.0%
3. **Absolute difference:** 2.0 percentage points [95% CI: 0.5, 3.5 pp]
4. **Relative lift:** 25.0% [95% CI: 6.3%, 43.8%]
5. **Statistical significance:** $p = 0.009$ (significant at $\alpha = 0.05$)
6. **Effect size:** Cohen's $h = 0.15$ (small effect)

7. **Business impact:** Expected +200 conversions/day, $10,000 additional revenue/-month

This comprehensive reporting enables stakeholders to judge both statistical and practical significance.

2.8 Frequentist vs. Bayesian Inference

Two philosophical frameworks underpin statistical inference. Understanding both perspectives enriches A/B testing practice.

2.8.1 Frequentist Framework

Core philosophy:

- Parameters are fixed but unknown constants
- Probability describes long-run frequencies of events
- Inference focuses on procedures with good repeated sampling properties
- Uncertainty is quantified through confidence intervals and p-values

Key concepts:

- **Confidence intervals:** "95% of such intervals contain the true parameter"
- **p-values:** "Probability of data this extreme if null is true"
- **Hypothesis testing:** Reject or fail to reject H_0 based on α

Strengths:

- Objective—no prior beliefs required
- Well-established theory and widespread acceptance
- Clear Type I and Type II error control
- Standard in regulatory contexts (FDA, etc.)

Limitations:

- Cannot make probability statements about parameters ("probability that $p > 0.05$")
- P-values are frequently misinterpreted
- Fixed sample size typically required for valid inference
- Doesn't naturally incorporate prior information

2.8.2 Bayesian Framework

Core philosophy:

- Parameters are random variables with probability distributions
- Probability describes degree of belief about unknowns
- Prior beliefs are updated with data via Bayes' theorem
- Inference produces posterior distributions for parameters

Theorem 2.3 (Bayes' Theorem)**.** For parameter θ and data D:

$$\underbrace{p(\theta|D)}_{\text{Posterior}} = \frac{\overbrace{p(D|\theta)}^{\text{Likelihood}} \times \overbrace{p(\theta)}^{\text{Prior}}}{\underbrace{p(D)}_{\text{Evidence}}} \tag{2.70}$$

where $p(D) = \int p(D|\theta)p(\theta)d\theta$ normalizes the posterior.
In words:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior} \tag{2.71}$$

Key concepts:

- **Prior distribution:** Encodes initial beliefs before seeing data
- **Likelihood:** How probable the data is under different parameter values
- **Posterior distribution:** Updated beliefs after observing data
- **Credible intervals:** "95% probability parameter is in this interval"

Strengths:

- Direct probability statements about parameters
- Natural incorporation of prior information
- Coherent framework for sequential testing
- Intuitive interpretation for decision-making

Limitations:

- Requires specifying prior distributions (though this can be an advantage)
- Computational complexity for complex models (mitigated by modern MCMC)
- Results depend on prior choice
- Less standard in regulatory contexts

2.8.3 When to Use Each Approach

Prefer frequentist when:

- Regulatory requirements demand it
- You want objectivity without prior specification
- You have fixed sample size and single analysis
- Standard null hypothesis testing suffices

Prefer Bayesian when:

- You have meaningful prior information to incorporate
- You want direct probability statements about parameters
- Sequential testing or early stopping is needed
- Decision-making requires posterior distributions

In practice: Many modern A/B testing platforms use Bayesian methods because they handle sequential testing naturally and provide intuitive outputs for business stakeholders. Chapter 7 covers Bayesian A/B testing comprehensively.

2.9 Summary

This chapter established the probabilistic and statistical foundations essential for rigorous A/B testing:

- **Probability theory** provides the mathematical framework for quantifying uncertainty. Key results include the Central Limit Theorem, which justifies normal approximations for sample means regardless of underlying distributions.
- **Key distributions**—Bernoulli, binomial, normal, t, beta, and chi-squared—model different aspects of experiments. Understanding their properties enables correct application.
- **Statistical inference** uses sample data to draw conclusions about populations. Point estimates provide best guesses; confidence intervals quantify uncertainty.
- **Hypothesis testing** offers a formal framework for decision-making. Understanding null hypotheses, p-values, Type I and Type II errors, and statistical power is essential for correct interpretation.
- **Effect sizes** measure practical significance, complementing statistical significance. Always report both—a large p-value with large effect size may warrant more data; a small p-value with tiny effect size may be practically irrelevant.
- **Frequentist and Bayesian frameworks** offer complementary perspectives. Frequentist methods dominate traditional practice; Bayesian methods offer advantages for sequential testing and direct probability statements.

These foundations enable the hypothesis testing procedures, sample size calculations, multiple testing corrections, and Bayesian methods developed in subsequent chapters. Mastery of these concepts distinguishes practitioners who merely run tests from those who design sound experiments and interpret results correctly.

2.10 Further Reading

- **?**: Comprehensive treatment of statistical inference with mathematical rigor
- **?**: Modern theoretical statistics with clear exposition
- **?**: Mathematical statistics emphasizing practical application
- **?**: Definitive text on Bayesian data analysis
- **?**: Classic reference on statistical power and effect sizes
- **?**: Permutation tests and resampling methods
- **?**: ASA statement on p-values and their interpretation

2.11 Exercises

2.11.1 Conceptual Questions

1. Explain the distinction between a parameter and a statistic. Why does this matter for A/B testing?
2. A colleague says: "We ran the experiment and got $p = 0.03$, so there's only a 3% chance the null hypothesis is true." What is wrong with this interpretation? Provide the correct interpretation.
3. Why does the Central Limit Theorem matter for A/B testing? What would be different if it didn't hold?
4. Explain the difference between a 95% confidence interval (frequentist) and a 95% credible interval (Bayesian). When would each be appropriate?
5. An experiment shows a statistically significant difference ($p < 0.001$) with Cohen's $h = 0.05$. What does this tell you? Should you implement the change?
6. Why might Type II error be more costly than Type I error in A/B testing? Conversely, when might Type I error be more concerning?

2.11.2 Probability and Distributions

1. Prove that for $X \sim \text{Bernoulli}(p)$: $\text{Var}(X) = p(1-p)$.
2. Show that the binomial distribution $\text{Binomial}(n, p)$ has maximum variance at $p = 0.5$ for fixed n.
3. For $X \sim \mathcal{N}(\mu, \sigma^2)$, derive the distribution of:

(a) $Y = aX + b$

(b) $Z = (X - \mu)/\sigma$

4. Simulate the CLT for an exponential distribution with $\lambda = 0.2$. Show convergence to normality for $n = 5, 20, 50, 200$. Use K-S test to assess normality.

5. A website has average session duration of 4 minutes, exponentially distributed. With 100 users, what is the approximate distribution of average session duration? Calculate a 95% confidence interval.

2.11.3 Hypothesis Testing

1. Conduct a two-sample z-test by hand (show all steps):

   - Control: 1,500 users, 120 conversions
   - Treatment: 1,500 users, 150 conversions
   - $\alpha = 0.05$

   Include: hypotheses, test statistic, p-value, decision, interpretation.

2. For the scenario above, construct a 95% confidence interval for the difference in proportions. Is it consistent with your hypothesis test conclusion?

3. Verify by simulation that Type I error rate is controlled at $\alpha = 0.05$ when testing $H_0 : p_1 = p_2 = 0.1$ with $n_1 = n_2 = 500$. Run 10,000 simulations.

4. Simulate power for detecting a difference from $p_1 = 0.08$ to $p_2 = 0.10$ with $\alpha = 0.05$. Plot power vs. sample size for $n = 100, 200, \ldots, 2000$ (per group).

5. Investigate the relationship between α, β, effect size, and sample size via simulation. Fix three and vary the fourth.

2.11.4 Confidence Intervals

1. Construct 90%, 95%, and 99% confidence intervals for conversion rate given 75 conversions from 1,000 users. How does interval width change with confidence level?

2. Implement and compare three CI methods for proportions: Wald, Wilson score, and Agresti-Coull. For which scenarios (n, p combinations) do they differ substantially?

3. Verify by simulation that 95% confidence intervals contain the true parameter approximately 95% of the time. Test for $p = 0.05, 0.1, 0.3, 0.5$ and $n = 50, 200, 1000$.

4. Two variants show: A ($145/2000 = 7.25\%$), B ($170/2000 = 8.50\%$). Construct:

   (a) 95% CI for each proportion

   (b) 95% CI for difference in proportions

   (c) 95% CI for relative lift

2.11.5 Effect Sizes

1. Calculate all effect size measures (absolute difference, relative lift, Cohen's h, odds ratio) for:

   (a) $p_1 = 0.05$, $p_2 = 0.07$

   (b) $p_1 = 0.20$, $p_2 = 0.22$

   (c) $p_1 = 0.40$, $p_2 = 0.50$

   Compare and interpret the measures across scenarios.

2. For continuous metrics, compute Cohen's d:

   - Control: $\bar{x}_1 = 100$, $s_1 = 30$, $n_1 = 250$
   - Treatment: $\bar{x}_2 = 110$, $s_2 = 32$, $n_2 = 250$

   Interpret the effect size. How many additional observations would be needed to detect this effect with 80% power at $\alpha = 0.05$?

3. Create a visualization showing the "landscape" of effect sizes: For $p_1 \in [0.01, 0.5]$ and $p_2 \in [0.01, 0.5]$, plot heatmaps of (a) relative lift, (b) Cohen's h, and (c) log odds ratio. Identify regions of small/medium/large effects.

4. A company runs 100 A/B tests per year. Each test has a true effect size of Cohen's $h = 0.1$ (small). With $n = 1000$ per variant and $\alpha = 0.05$, what fraction of tests will:

   (a) Correctly detect the effect (power)?

   (b) Fail to detect the effect (Type II error)?

   (c) Show a "significant" effect in the wrong direction?

   Verify via simulation.

2.11.6 Programming Projects

1. **A/B Test Analysis Package:** Implement a comprehensive Python class `ABTest` that:

   - Accepts control and treatment data
   - Computes all relevant statistics (proportions, SEs, CIs)
   - Performs hypothesis tests (z-test, chi-squared test, permutation test)
   - Calculates effect sizes
   - Generates publication-quality visualizations
   - Outputs formatted report with interpretation

   Include docstrings, type hints, unit tests, and example usage.

2. **Interactive Dashboard:** Create a web dashboard (using Streamlit, Dash, or Shiny) that:

- Allows users to input A/B test data
- Computes and displays all statistics
- Visualizes confidence intervals, power curves
- Provides interpretation and recommendations
- Includes sample size calculator

3. **Simulation Study:** Design and execute a comprehensive simulation investigating:

   - Coverage of different CI methods across parameter space
   - Power as a function of effect size, sample size, baseline rate
   - Impact of unequal sample sizes on test properties
   - Consequences of peeking (sequential testing without correction)

   Write a report summarizing findings with visualizations.

4. **Bayesian vs. Frequentist Comparison:** Implement both frameworks for A/B testing:

   - Frequentist: Confidence intervals and hypothesis tests
   - Bayesian: Posterior distributions with Beta-Binomial conjugacy

   Compare results across scenarios varying sample size, effect size, and prior specification. When do they agree/disagree?

Chapter 3

Experimental Design Principles

3.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand and apply the fundamental principles of experimental design
- Implement various randomization techniques for different experimental contexts
- Identify and mitigate common sources of bias and confounding
- Assess experimental validity across multiple dimensions
- Make informed design decisions for online A/B tests
- Implement randomization schemes and bias detection in Python

3.2 Introduction

Experimental design is the architecture underlying A/B testing. Poor design invalidates even the most sophisticated statistical analysis—garbage in, garbage out. Conversely, thoughtful design can dramatically increase the efficiency and reliability of experiments, enabling faster learning and better decisions.

This chapter explores the principles that separate rigorous experiments from flawed ones. We'll see how randomization protects against bias, how stratification increases power, how confounding variables mislead, and how validity considerations constrain generalization. These aren't merely theoretical concerns—they directly impact whether your experiment provides trustworthy guidance or leads you astray.

The history of experimental design offers sobering lessons. Fisher's agricultural experiments in the 1920s established randomization as essential. Medical trials in the 1940s demonstrated the necessity of control groups and blinding. The digital era has introduced new challenges: network effects, novelty effects, and implementation bugs that invalidate traditional assumptions. Modern A/B testing requires both classical wisdom and contemporary adaptations.

3.3 Fundamental Principles of Experimental Design

Four core principles underpin sound experimental design. Violate any of them, and your experiment's validity is threatened.

3.3.1 Randomization

Randomization is the cornerstone of causal inference. By randomly assigning units to treatment and control, we ensure that—in expectation—groups differ only in the treatment received, not in other characteristics that might affect outcomes.

Definition 3.1 (Randomization)**.** Random assignment is the process of allocating experimental units to treatment conditions using a random mechanism, ensuring each unit has a known, non-zero probability of assignment to each condition.

Why randomization matters:

1. **Eliminates selection bias:** Without randomization, systematic differences between groups confound treatment effects. Users who self-select into groups differ in unmeasured ways.

2. **Balances confounders:** Randomization balances both measured and *unmeasured* confounding variables across groups. This is crucial—we can't control for variables we don't know about or can't measure.

3. **Justifies statistical inference:** Probability theory underlying hypothesis tests and confidence intervals assumes random sampling or random assignment. Without randomization, these tools lack foundation.

4. **Enables causal interpretation:** Observational studies can establish association; randomized experiments establish causation. The difference is randomization.

Example 3.1 (Randomization vs. Self-Selection)**.** Consider testing a new premium subscription tier:

Flawed approach (self-selection): Offer the new tier to all users, compare adopters vs. non-adopters. Problem: Adopters likely differ systematically—higher engagement, more resources, different needs. Observed differences confound tier effect with user characteristics.

Correct approach (randomization): Randomly assign users to see new tier vs. old tier. Now groups are balanced in expectation. Observed differences are attributable to tier design, not pre-existing user differences.

Common randomization mistakes:

- **Alternating assignment:** "Every other user gets treatment" is NOT random. Predictability enables gaming; time-based patterns confound results.

- **Assignment by characteristics:** "Users in California get treatment" introduces geographic confounding.

- **Insufficient randomness:** Using low-quality random number generators or seeding incorrectly can introduce patterns.

- **Post-hoc selection:** Assigning users randomly but then filtering or selecting subsets for analysis reintroduces bias.

3.3.2 Control Groups

A control group provides the counterfactual: what would have happened without treatment? Without controls, we can't distinguish treatment effects from natural variation, seasonal trends, or external factors.

Definition 3.2 (Control Group)**.** The control group receives no treatment (or the current standard treatment), providing a baseline for comparison. The treatment group receives the intervention being tested.

Example 3.2 (The Necessity of Controls)**.** A company launches a new website design and observes a 10% increase in conversions over the next week. Success? Not necessarily:

- Was there a marketing campaign that week?
- Is it a seasonal peak (e.g., holiday shopping)?
- Did competitors change their offerings?
- Was there a temporary bug in the old design that was fixed?

Without a control group maintaining the old design, these confounds are inseparable from the design effect. With proper randomization and controls, we isolate the causal impact of the design change.

Types of controls:

- **No-treatment control:** Users receive no intervention. Appropriate when testing entirely new features.
- **Placebo control:** Users receive a neutral intervention that mimics treatment exposure without active ingredients. Less common in digital contexts but relevant for psychological effects.
- **Active control:** Users receive current standard treatment. Essential when testing improvements to existing features.
- **Multiple controls:** Sometimes multiple baseline conditions are needed to isolate specific mechanisms.

3.3.3 Blocking and Stratification

While randomization ensures balance in expectation, chance can produce imbalanced groups in any single experiment. Blocking and stratification improve balance on known covariates, increasing precision.

Definition 3.3 (Blocking)**.** Blocking divides experimental units into homogeneous groups (blocks) based on covariates, then randomizes separately within each block. This ensures balance on the blocking variables.

Definition 3.4 (Stratification)**.** Stratification is similar to blocking but typically refers to analysis rather than design. Units are divided into strata post-randomization, and stratum-specific effects are estimated.

When to use blocking:

- **Known important covariates:** If user type, geographic region, or device strongly affects outcomes, blocking ensures balance and increases power.
- **Small sample sizes:** With few units, chance imbalance is more likely. Blocking provides insurance.
- **Heterogeneous treatment effects:** If effects vary across subgroups, blocking enables subgroup-specific analysis.

Example 3.3 (Blocking in Practice)**.** Testing a mobile app redesign where behavior differs dramatically between iOS and Android:

Simple randomization: 50% iOS users to treatment, 50% Android users to treatment. By chance, treatment might get 52% iOS, control 48% iOS—introducing imbalance.

Blocked randomization: Within iOS users, randomize 50% to each group. Within Android users, randomize 50% to each group. Now platform is perfectly balanced.

If iOS users have 15% conversion rate and Android users 10%, blocking eliminates this source of variance, increasing power to detect the treatment effect.

Practical considerations:

- **Number of blocks:** Too many small blocks reduce efficiency. Aim for substantial units per block.
- **Blocking variables:** Choose strong predictors of outcomes. Blocking on weak predictors adds complexity without benefit.
- **Crossover:** Can block on multiple variables (e.g., platform $\times$ country), but beware exponentially growing strata.

3.3.4 Blinding

Blinding conceals treatment assignment from participants, experimenters, or both, preventing knowledge of assignment from influencing behavior or assessment.

Definition 3.5 (Blinding)**.**

- **Single-blind:** Participants don't know their treatment assignment
- **Double-blind:** Neither participants nor experimenters know assignment
- **Triple-blind:** Participants, experimenters, and analysts all remain blind

Why blinding matters:

1. **Placebo effects:** Participants' knowledge of treatment can affect outcomes independent of treatment's direct effects.
2. **Experimenter bias:** Researchers who know assignment may unconsciously influence participants or assess outcomes differently.
3. **Demand characteristics:** Participants may alter behavior to confirm experimenters' hypotheses.

4. **Differential attrition:** If users realize they're in control, they may drop out at different rates than treatment.

Example 3.4 (Blinding in Digital Experiments)**.** In A/B testing online, blinding is often implicit:

Naturally blinded: Users typically don't know they're in an experiment or which group they're in. A new button color appears to some users; they don't realize others see something different.

Not blinded: If treatment is a new feature prominently labeled "BETA" or "NEW!", users know they're in treatment. Their behavior may reflect curiosity, novelty-seeking, or frustration with bugs rather than the feature's inherent value.

Experimenter blinding: Data analysts should ideally remain blind to which variant is A vs. B until analysis completion, preventing p-hacking or selective reporting.

Limitations in online testing:

- Complete blinding is often impossible—users see different interfaces
- Product teams must know which variant is which to iterate
- The goal: minimize systematic bias from knowledge of assignment

3.4 Randomization Techniques

Different contexts demand different randomization approaches. We examine four primary techniques with implementation details.

3.4.1 Simple Randomization

Simple randomization assigns each unit independently to treatment or control with fixed probability, typically 50%.

Advantages:

- Conceptually simple
- Easy to implement
- Provably unbiased
- No memory or coordination needed

Disadvantages:

- Can produce imbalanced group sizes
- No guarantee of balance on covariates
- Variance in group sizes reduces power

When to use: Large experiments (thousands of units) where chance imbalance is negligible.

Listing 3.1: Simple Randomization Implementation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from typing import List, Dict, Tuple

sns.set_style("whitegrid")
np.random.seed(42)

def simple_randomization(n_units: int, p_treatment: float = 0.5,
                         seed: int = None) -> np.ndarray:
    """
    Simple randomization: each unit independently assigned.

    Parameters:
    -----------
    n_units : int
        Number of units to randomize
    p_treatment : float
        Probability of treatment assignment (default 0.5)
    seed : int
        Random seed for reproducibility

    Returns:
    --------
    np.ndarray : Binary array (0=control, 1=treatment)
    """
    if seed is not None:
        np.random.seed(seed)

    assignments = np.random.binomial(1, p_treatment, n_units)

    return assignments


def evaluate_randomization(assignments: np.ndarray,
                           covariates: pd.DataFrame = None,
                           covariate_names: List[str] = None) -> Dict:
    """
    Evaluate quality of randomization.

    Parameters:
    -----------
    assignments : np.ndarray
        Treatment assignments (0/1)
    covariates : pd.DataFrame, optional
        Covariates to check balance on
    covariate_names : List[str], optional
        Names of covariates to check

    Returns:
    --------
    dict : Evaluation metrics
    """
    n_total = len(assignments)
    n_treatment = assignments.sum()
    n_control = n_total - n_treatment
```

```
58
59      results = {
60          'n_total': n_total,
61          'n_treatment': n_treatment,
62          'n_control': n_control,
63          'prop_treatment': n_treatment / n_total,
64          'balance_score': abs(n_treatment - n_control) / n_total
65      }
66
67      # Check covariate balance if provided
68      if covariates is not None:
69          from scipy import stats as sp_stats
70          balance_tests = {}
71
72          for col in covariate_names or covariates.columns:
73              treatment_vals = covariates.loc[assignments == 1, col]
74              control_vals = covariates.loc[assignments == 0, col]
75
76              # t-test for continuous, chi-square for categorical
77              if covariates[col].dtype in [np.float64, np.int64]:
78                  t_stat, p_val = sp_stats.ttest_ind(treatment_vals,
                        control_vals)
79                  balance_tests[col] = {
80                      'mean_treatment': treatment_vals.mean(),
81                      'mean_control': control_vals.mean(),
82                      'diff': treatment_vals.mean() - control_vals.mean()
                            ,
83                      'p_value': p_val
84                  }
85
86          results['covariate_balance'] = balance_tests
87
88      return results
89
90
91  # Example: Simple randomization with evaluation
92  print("="*80)
93  print("Simple␣Randomization␣Example")
94  print("="*80)
95
96  n_users = 1000
97  assignments = simple_randomization(n_users, p_treatment=0.5, seed=42)
98
99  # Create synthetic covariates
100 np.random.seed(42)
101 covariates = pd.DataFrame({
102     'age': np.random.normal(35, 10, n_users),
103     'prior_purchases': np.random.poisson(3, n_users),
104     'engagement_score': np.random.normal(50, 15, n_users)
105 })
106
107 # Evaluate
108 results = evaluate_randomization(assignments, covariates)
109
110 print(f"\nTotal␣units:␣{results['n_total']}")
111 print(f"Treatment:␣{results['n_treatment']}␣({results['prop_treatment
        ']:.2%})")
112 print(f"Control:␣{results['n_control']}␣({1-results['prop_treatment
```

```
    ']:.2%})")
print(f"Balance score: {results['balance_score']:.4f}")

if 'covariate_balance' in results:
    print("\nCovariate Balance:")
    print("-" * 80)
    print(f"{'Covariate':<20} {'Treatment Mean':<15} {'Control Mean':<15} {'Difference':<12} {'p-value':<10}")
    print("-" * 80)
    for cov, stats in results['covariate_balance'].items():
        print(f"{cov:<20} {stats['mean_treatment']:<15.2f} {stats['mean_control']:<15.2f} "
              f"{stats['diff']:<12.2f} {stats['p_value']:<10.4f}")

# Simulate multiple randomizations to show variability
n_sims = 1000
treatment_props = []
for i in range(n_sims):
    sim_assign = simple_randomization(1000, p_treatment=0.5, seed=i)
    treatment_props.append(sim_assign.sum() / 1000)

# Visualize
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribution of treatment proportions
axes[0].hist(treatment_props, bins=30, edgecolor='black', alpha=0.7, color='steelblue')
axes[0].axvline(0.5, color='red', linestyle='--', linewidth=2, label='Target (50%)')
axes[0].axvline(np.mean(treatment_props), color='green', linestyle='-',
                linewidth=2, label=f'Mean ({np.mean(treatment_props):.3f})')
axes[0].set_xlabel('Treatment Proportion', fontsize=12)
axes[0].set_ylabel('Frequency', fontsize=12)
axes[0].set_title(f'Distribution of Treatment Proportions\n({n_sims} simulations, n=1000)',
                  fontsize=12, fontweight='bold')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Covariate balance check for one randomization
treatment_mask = assignments == 1
axes[1].scatter(covariates.loc[treatment_mask, 'age'],
                covariates.loc[treatment_mask, 'engagement_score'],
                alpha=0.5, label='Treatment', color='coral', s=20)
axes[1].scatter(covariates.loc[~treatment_mask, 'age'],
                covariates.loc[~treatment_mask, 'engagement_score'],
                alpha=0.5, label='Control', color='steelblue', s=20)
axes[1].set_xlabel('Age', fontsize=12)
axes[1].set_ylabel('Engagement Score', fontsize=12)
axes[1].set_title('Covariate Balance: Age vs Engagement', fontsize=12, fontweight='bold')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('simple_randomization.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

3.4.2 Block Randomization

Block randomization ensures exactly balanced group sizes by randomizing in small blocks rather than one unit at a time.

Mechanism:

1. Define block size (e.g., 4 or 6)
2. Within each block, assign equal numbers to treatment and control
3. Randomize the order within block

Advantages:

- Guarantees balanced group sizes
- Protects against temporal trends
- Reduces variance in treatment effect estimates

Disadvantages:

- If block size is known and assignments observable, late assignments in a block are predictable
- Requires coordination/memory across units
- Slightly more complex implementation

Listing 3.2: Block Randomization Implementation

```
def block_randomization(n_units: int, block_size: int = 4,
                        p_treatment: float = 0.5, seed: int = None) ->
                           np.ndarray:
    """
␣␣␣␣Block␣randomization:␣guarantees␣balance␣within␣blocks.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣n_units␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣units␣to␣randomize
␣␣␣␣block_size␣:␣int
␣␣␣␣␣␣␣␣Size␣of␣each␣block␣(should␣be␣even␣for␣50-50␣split)
␣␣␣␣p_treatment␣:␣float
␣␣␣␣␣␣␣␣Proportion␣assigned␣to␣treatment␣within␣each␣block
␣␣␣␣seed␣:␣int
␣␣␣␣␣␣␣␣Random␣seed␣for␣reproducibility

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣np.ndarray␣:␣Binary␣array␣(0=control,␣1=treatment)
␣␣␣␣"""
    if seed is not None:
        np.random.seed(seed)

    # Calculate number of treatment assignments per block
    n_treatment_per_block = int(block_size * p_treatment)

```

```
    # Create full blocks
    n_full_blocks = n_units // block_size
    assignments = []

    for _ in range(n_full_blocks):
        # Create block with correct proportions
        block = np.array([1] * n_treatment_per_block +
                         [0] * (block_size - n_treatment_per_block))
        # Shuffle within block
        np.random.shuffle(block)
        assignments.extend(block)

    # Handle remaining units
    remainder = n_units % block_size
    if remainder > 0:
        n_treatment_remainder = int(remainder * p_treatment)
        remainder_block = np.array([1] * n_treatment_remainder +
                                   [0] * (remainder -
                                      n_treatment_remainder))
        np.random.shuffle(remainder_block)
        assignments.extend(remainder_block)

    return np.array(assignments)


# Example: Compare simple vs block randomization
print("\n" + "="*80)
print("Block Randomization vs Simple Randomization")
print("="*80)

n_users = 1000
n_sims = 1000

simple_balances = []
block_balances = []

for i in range(n_sims):
    # Simple randomization
    simple_assign = simple_randomization(n_users, seed=i)
    simple_balances.append(abs(simple_assign.sum() - n_users/2))

    # Block randomization
    block_assign = block_randomization(n_users, block_size=10, seed=i)
    block_balances.append(abs(block_assign.sum() - n_users/2))

print(f"\nBalance (deviation from 50-50 split):")
print(f"{'Method':<20} {'Mean':<12} {'Std Dev':<12} {'Max':<12}")
print("-" * 60)
print(f"{'Simple':<20} {np.mean(simple_balances):<12.2f} "
      f"{np.std(simple_balances):<12.2f} {np.max(simple_balances):<12.0
        f}")
print(f"{'Block (size=10)':<20} {np.mean(block_balances):<12.2f} "
      f"{np.std(block_balances):<12.2f} {np.max(block_balances):<12.0f}
        ")

# Visualize comparison
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

```

```
axes[0].hist(simple_balances, bins=30, alpha=0.6, label='Simple',
             color='steelblue', edgecolor='black')
axes[0].hist(block_balances, bins=30, alpha=0.6, label='Block (size=10)',
             color='coral', edgecolor='black')
axes[0].set_xlabel('Absolute Deviation from Perfect Balance', fontsize=12)
axes[0].set_ylabel('Frequency', fontsize=12)
axes[0].set_title('Randomization Balance Comparison', fontsize=12, fontweight='bold')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Cumulative assignments over time
np.random.seed(42)
simple_cumsum = np.cumsum(simple_randomization(1000, seed=42))
block_cumsum = np.cumsum(block_randomization(1000, block_size=10, seed=42))
perfect = np.arange(1, 1001) * 0.5

axes[1].plot(simple_cumsum, label='Simple randomization', linewidth=2, alpha=0.7)
axes[1].plot(block_cumsum, label='Block randomization', linewidth=2, alpha=0.7)
axes[1].plot(perfect, 'k--', label='Perfect balance', linewidth=2)
axes[1].set_xlabel('User Number', fontsize=12)
axes[1].set_ylabel('Cumulative Treatment Assignments', fontsize=12)
axes[1].set_title('Balance Over Time', fontsize=12, fontweight='bold')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('block_randomization_comparison.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

3.4.3 Stratified Randomization

Stratified randomization divides units into strata based on important covariates, then randomizes separately within each stratum.

Difference from blocking: Blocking typically refers to small, homogeneous groups. Stratification involves larger groups defined by key characteristics (e.g., country, user segment).

Advantages:

- Guarantees balance on stratifying variables
- Increases power by reducing between-stratum variance
- Enables stratified analysis
- Protects against chance imbalances on important covariates

Disadvantages:

- Requires knowing stratifying variables before randomization

- Can create many small strata if stratifying by multiple variables
- Complicates analysis (must account for stratification)

Listing 3.3: Stratified Randomization Implementation

```
def stratified_randomization(strata: np.ndarray, p_treatment: float =
    0.5,
                             block_size: int = None, seed: int = None)
                                -> np.ndarray:
    """
␣␣␣␣Stratified␣randomization:␣randomize␣within␣strata.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣strata␣:␣np.ndarray
␣␣␣␣␣␣␣␣Array␣indicating␣stratum␣membership␣for␣each␣unit
␣␣␣␣p_treatment␣:␣float
␣␣␣␣␣␣␣␣Proportion␣assigned␣to␣treatment␣within␣each␣stratum
␣␣␣␣block_size␣:␣int,␣optional
␣␣␣␣␣␣␣␣If␣provided,␣use␣block␣randomization␣within␣strata
␣␣␣␣seed␣:␣int
␣␣␣␣␣␣␣␣Random␣seed␣for␣reproducibility

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣np.ndarray␣:␣Binary␣array␣(0=control,␣1=treatment)
␣␣␣␣"""
    if seed is not None:
        np.random.seed(seed)

    n_units = len(strata)
    assignments = np.zeros(n_units, dtype=int)

    # Get unique strata
    unique_strata = np.unique(strata)

    for stratum in unique_strata:
        stratum_mask = strata == stratum
        stratum_size = stratum_mask.sum()

        # Randomize within stratum
        if block_size is not None:
            stratum_assignments = block_randomization(
                stratum_size, block_size, p_treatment
            )
        else:
            stratum_assignments = simple_randomization(
                stratum_size, p_treatment
            )

        assignments[stratum_mask] = stratum_assignments

    return assignments


def analyze_stratified_experiment(outcomes: np.ndarray,
                                  assignments: np.ndarray,
```

```
                                    strata: np.ndarray) -> Dict:
    """
    Analyze stratified experiment with proper variance estimation.

    Parameters:
    -----------
    outcomes : np.ndarray
        Outcome values
    assignments : np.ndarray
        Treatment assignments (0/1)
    strata : np.ndarray
        Stratum indicators

    Returns:
    --------
    dict : Analysis results including stratum-specific effects
    """
    from scipy import stats as sp_stats

    results = {
        'strata_effects': {},
        'overall_effect': None
    }

    unique_strata = np.unique(strata)
    n_strata = len(unique_strata)

    stratum_effects = []
    stratum_vars = []
    stratum_weights = []

    for stratum in unique_strata:
        stratum_mask = strata == stratum
        stratum_outcomes = outcomes[stratum_mask]
        stratum_assignments = assignments[stratum_mask]

        # Calculate stratum-specific effect
        treatment_outcomes = stratum_outcomes[stratum_assignments == 1]
        control_outcomes = stratum_outcomes[stratum_assignments == 0]

        if len(treatment_outcomes) > 0 and len(control_outcomes) > 0:
            effect = treatment_outcomes.mean() - control_outcomes.mean
                ()

            # Variance of difference
            var_treatment = treatment_outcomes.var() / len(
                treatment_outcomes)
            var_control = control_outcomes.var() / len(control_outcomes
                )
            var_effect = var_treatment + var_control

            stratum_effects.append(effect)
            stratum_vars.append(var_effect)
            stratum_weights.append(stratum_mask.sum())

            results['strata_effects'][stratum] = {
                'effect': effect,
                'se': np.sqrt(var_effect),
```

```
                'n': stratum_mask.sum(),
                'n_treatment': (stratum_assignments == 1).sum(),
                'n_control': (stratum_assignments == 0).sum(),
                'mean_treatment': treatment_outcomes.mean(),
                'mean_control': control_outcomes.mean()
            }

    # Calculate overall weighted effect
    stratum_effects = np.array(stratum_effects)
    stratum_vars = np.array(stratum_vars)
    stratum_weights = np.array(stratum_weights) / sum(stratum_weights)

    overall_effect = np.sum(stratum_weights * stratum_effects)
    overall_var = np.sum(stratum_weights**2 * stratum_vars)
    overall_se = np.sqrt(overall_var)

    # Test statistic
    z_stat = overall_effect / overall_se
    p_value = 2 * (1 - sp_stats.norm.cdf(abs(z_stat)))

    results['overall_effect'] = {
        'effect': overall_effect,
        'se': overall_se,
        'z_statistic': z_stat,
        'p_value': p_value,
        'ci_lower': overall_effect - 1.96 * overall_se,
        'ci_upper': overall_effect + 1.96 * overall_se
    }

    return results


# Example: Stratified randomization with heterogeneous effects
print("\n" + "="*80)
print("Stratified Randomization Example")
print("="*80)

n_users = 2000
np.random.seed(42)

# Create strata (e.g., user segments)
segments = np.random.choice(['New', 'Casual', 'Power'], n_users,
                            p=[0.3, 0.5, 0.2])

# Stratified randomization
assignments = stratified_randomization(segments, p_treatment=0.5, seed
    =42)

# Simulate outcomes with heterogeneous treatment effects
baseline_conversions = {
    'New': 0.05,
    'Casual': 0.10,
    'Power': 0.20
}
treatment_effects = {
    'New': 0.02,      # 40% relative lift for new users
    'Casual': 0.01,   # 10% relative lift for casual
    'Power': 0.01     # 5% relative lift for power users
```

```
}

outcomes = np.zeros(n_users)
for i in range(n_users):
    segment = segments[i]
    baseline = baseline_conversions[segment]
    effect = treatment_effects[segment] if assignments[i] == 1 else 0
    outcomes[i] = np.random.binomial(1, baseline + effect)

# Analyze
results = analyze_stratified_experiment(outcomes, assignments, segments
    )

print("\nStratum-Specific Effects:")
print("-" * 90)
print(f"{'Stratum':<12} {'Treatment Mean':<15} {'Control Mean':<15} {'
    Effect':<12} {'SE':<10} {'n':<8}")
print("-" * 90)
for stratum, stats in results['strata_effects'].items():
    print(f"{stratum:<12} {stats['mean_treatment']:<15.4f} {stats['
        mean_control']:<15.4f} "
          f"{stats['effect']:<12.4f} {stats['se']:<10.4f} {stats['n
              ']:<8}")

print("\nOverall Effect (Weighted):")
print("-" * 90)
overall = results['overall_effect']
print(f"Effect: {overall['effect']:.4f}")
print(f"SE: {overall['se']:.4f}")
print(f"95% CI: [{overall['ci_lower']:.4f}, {overall['ci_upper']:.4f}]"
    )
print(f"z-statistic: {overall['z_statistic']:.4f}")
print(f"p-value: {overall['p_value']:.6f}")

# Visualize stratum-specific effects
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Effect sizes by stratum
strata_names = list(results['strata_effects'].keys())
effects = [results['strata_effects'][s]['effect'] for s in strata_names
    ]
ses = [results['strata_effects'][s]['se'] for s in strata_names]

axes[0].barh(strata_names, effects, xerr=[1.96*se for se in ses],
             alpha=0.7, color='steelblue', capsize=5)
axes[0].axvline(0, color='red', linestyle='--', linewidth=2)
axes[0].axvline(overall['effect'], color='green', linestyle='-',
    linewidth=2,
                label=f"Overall: {overall['effect']:.4f}")
axes[0].set_xlabel('Treatment Effect', fontsize=12)
axes[0].set_ylabel('Stratum', fontsize=12)
axes[0].set_title('Stratum-Specific Treatment Effects', fontsize=12,
    fontweight='bold')
axes[0].legend()
axes[0].grid(alpha=0.3)

# Conversion rates by group and stratum
x = np.arange(len(strata_names))
```

```
width = 0.35
control_means = [results['strata_effects'][s]['mean_control'] for s in
    strata_names]
treatment_means = [results['strata_effects'][s]['mean_treatment'] for s
    in strata_names]

axes[1].bar(x - width/2, control_means, width, label='Control',
            alpha=0.7, color='steelblue')
axes[1].bar(x + width/2, treatment_means, width, label='Treatment',
            alpha=0.7, color='coral')
axes[1].set_xlabel('Stratum', fontsize=12)
axes[1].set_ylabel('Conversion␣Rate', fontsize=12)
axes[1].set_title('Conversion␣Rates␣by␣Stratum␣and␣Group', fontsize=12,
    fontweight='bold')
axes[1].set_xticks(x)
axes[1].set_xticklabels(strata_names)
axes[1].legend()
axes[1].grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('stratified_randomization.pdf', dpi=300, bbox_inches='tight
    ')
plt.show()
```

3.4.4 Cluster Randomization

Cluster randomization assigns groups of units rather than individual units. Essential when treatment naturally applies to groups or when individual-level randomization is infeasible.

When to use:

- Treatment inherently group-level (e.g., teacher training affects entire classrooms)
- Individual contamination likely (users in same household interact)
- Technical constraints prevent individual randomization
- Network effects mean individual outcomes are not independent

Key challenge: Outcomes within clusters are correlated, inflating standard errors. Effective sample size is reduced.

Definition 3.6 (Intracluster Correlation (ICC))**.** The ICC measures correlation of outcomes within clusters:

$$\rho = \frac{\sigma^2_{\text{between}}}{\sigma^2_{\text{between}} + \sigma^2_{\text{within}}} \tag{3.1}$$

where $\sigma^2_{\text{between}}$ is variance between cluster means and σ^2_{within} is variance within clusters.

Design effect: Cluster randomization increases required sample size by the design effect:

$$\text{DE} = 1 + (m - 1)\rho \tag{3.2}$$

where m is average cluster size and ρ is ICC. Higher ICC or larger clusters require proportionally more total units.

Example 3.5 (Cluster Randomization Penalty)**.** Testing a feature where users in same household interact:

- ICC = 0.2 (moderate correlation within households)
- Average household size = 3
- Design effect: $1 + (3 - 1) \times 0.2 = 1.4$

Must collect 40% more households (or equivalently, more total users) compared to individual randomization to achieve same power.

Listing 3.4: Cluster Randomization Implementation

```
def cluster_randomization(cluster_ids: np.ndarray, p_treatment: float =
    0.5,
                          seed: int = None) -> np.ndarray:
    """
␣␣␣␣Cluster␣randomization:␣randomize␣at␣cluster␣level.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣cluster_ids␣:␣np.ndarray
␣␣␣␣␣␣␣␣Array␣indicating␣cluster␣membership␣for␣each␣unit
␣␣␣␣p_treatment␣:␣float
␣␣␣␣␣␣␣␣Proportion␣of␣clusters␣assigned␣to␣treatment
␣␣␣␣seed␣:␣int
␣␣␣␣␣␣␣␣Random␣seed␣for␣reproducibility

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣np.ndarray␣:␣Binary␣array␣(0=control,␣1=treatment)
␣␣␣␣"""
    if seed is not None:
        np.random.seed(seed)

    # Get unique clusters
    unique_clusters = np.unique(cluster_ids)
    n_clusters = len(unique_clusters)

    # Randomize clusters
    n_treatment_clusters = int(n_clusters * p_treatment)
    treatment_clusters = np.random.choice(unique_clusters,
       n_treatment_clusters,
                                          replace=False)

    # Assign all units in treatment clusters
    assignments = np.isin(cluster_ids, treatment_clusters).astype(int)

    return assignments


def calculate_icc(outcomes: np.ndarray, cluster_ids: np.ndarray) ->
   float:
    """
␣␣␣␣Calculate␣intracluster␣correlation␣coefficient.

␣␣␣␣Parameters:
```

```
    ----------
    outcomes : np.ndarray
        Outcome values
    cluster_ids : np.ndarray
        Cluster identifiers

    Returns:
    --------
    float : ICC
    """
    unique_clusters = np.unique(cluster_ids)
    n_clusters = len(unique_clusters)

    # Grand mean
    grand_mean = outcomes.mean()

    # Calculate between and within variance
    cluster_means = []
    cluster_sizes = []
    within_ss = 0

    for cluster in unique_clusters:
        cluster_mask = cluster_ids == cluster
        cluster_outcomes = outcomes[cluster_mask]
        cluster_mean = cluster_outcomes.mean()
        cluster_size = len(cluster_outcomes)

        cluster_means.append(cluster_mean)
        cluster_sizes.append(cluster_size)

        # Within-cluster sum of squares
        within_ss += np.sum((cluster_outcomes - cluster_mean)**2)

    # Between-cluster sum of squares
    cluster_means = np.array(cluster_means)
    cluster_sizes = np.array(cluster_sizes)
    between_ss = np.sum(cluster_sizes * (cluster_means - grand_mean)
        **2)

    # Mean squares
    df_between = n_clusters - 1
    df_within = len(outcomes) - n_clusters
    ms_between = between_ss / df_between
    ms_within = within_ss / df_within

    # ICC calculation
    mean_cluster_size = np.mean(cluster_sizes)
    icc = (ms_between - ms_within) / (ms_between + (mean_cluster_size -
        1) * ms_within)
    icc = max(0, icc)  # ICC should be non-negative

    return icc


def analyze_cluster_experiment(outcomes: np.ndarray,
                               assignments: np.ndarray,
                               cluster_ids: np.ndarray) -> Dict:
    """
```

```
    Analyze cluster-randomized experiment with cluster-robust SEs.

    Parameters:
    -----------
    outcomes : np.ndarray
        Outcome values
    assignments : np.ndarray
        Treatment assignments (0/1)
    cluster_ids : np.ndarray
        Cluster identifiers

    Returns:
    --------
    dict : Analysis results accounting for clustering
    """
    from scipy import stats as sp_stats

    # Calculate ICC
    icc = calculate_icc(outcomes, cluster_ids)

    # Get cluster-level summaries
    unique_clusters = np.unique(cluster_ids)
    cluster_means = []
    cluster_assignments = []
    cluster_sizes = []

    for cluster in unique_clusters:
        cluster_mask = cluster_ids == cluster
        cluster_means.append(outcomes[cluster_mask].mean())
        cluster_assignments.append(assignments[cluster_mask][0])  # All
            same
        cluster_sizes.append(cluster_mask.sum())

    cluster_means = np.array(cluster_means)
    cluster_assignments = np.array(cluster_assignments)
    cluster_sizes = np.array(cluster_sizes)

    # Cluster-level analysis
    treatment_cluster_means = cluster_means[cluster_assignments == 1]
    control_cluster_means = cluster_means[cluster_assignments == 0]

    effect = treatment_cluster_means.mean() - control_cluster_means.
        mean()

    # Cluster-robust standard error
    var_treatment = treatment_cluster_means.var() / len(
        treatment_cluster_means)
    var_control = control_cluster_means.var() / len(
        control_cluster_means)
    se_cluster = np.sqrt(var_treatment + var_control)

    # Naive individual-level SE (for comparison)
    treatment_outcomes = outcomes[assignments == 1]
    control_outcomes = outcomes[assignments == 0]
    var_treatment_naive = treatment_outcomes.var() / len(
        treatment_outcomes)
    var_control_naive = control_outcomes.var() / len(control_outcomes)
    se_naive = np.sqrt(var_treatment_naive + var_control_naive)
```

```
151
152     # Design effect
153     mean_cluster_size = np.mean(cluster_sizes)
154     design_effect = 1 + (mean_cluster_size - 1) * icc
155
156     # Test statistic
157     z_stat = effect / se_cluster
158     p_value = 2 * (1 - sp_stats.norm.cdf(abs(z_stat)))
159
160     return {
161         'effect': effect,
162         'se_cluster_robust': se_cluster,
163         'se_naive': se_naive,
164         'inflation_factor': se_cluster / se_naive,
165         'icc': icc,
166         'design_effect': design_effect,
167         'mean_cluster_size': mean_cluster_size,
168         'n_clusters': len(unique_clusters),
169         'n_treatment_clusters': (cluster_assignments == 1).sum(),
170         'n_control_clusters': (cluster_assignments == 0).sum(),
171         'z_statistic': z_stat,
172         'p_value': p_value,
173         'ci_lower': effect - 1.96 * se_cluster,
174         'ci_upper': effect + 1.96 * se_cluster
175     }
176
177
178 # Example: Cluster randomization
179 print("\n" + "="*80)
180 print("Cluster␣Randomization␣Example")
181 print("="*80)
182
183 n_clusters = 200
184 cluster_sizes = np.random.poisson(10, n_clusters) + 1  # Vary cluster
        sizes
185 n_users = cluster_sizes.sum()
186
187 # Create cluster IDs
188 cluster_ids = np.repeat(np.arange(n_clusters), cluster_sizes)
189
190 # Randomize at cluster level
191 np.random.seed(42)
192 assignments = cluster_randomization(cluster_ids, p_treatment=0.5, seed
        =42)
193
194 # Simulate outcomes with ICC
195 icc_true = 0.3
196 treatment_effect = 0.05
197
198 # Generate cluster-level random effects
199 cluster_effects = np.random.normal(0, np.sqrt(icc_true), n_clusters)
200
201 # Generate outcomes
202 baseline_prob = 0.10
203 outcomes = np.zeros(n_users)
204
205 for i in range(n_users):
206     cluster_id = cluster_ids[i]
```

```
    cluster_effect = cluster_effects[cluster_id]
    treatment_contrib = treatment_effect if assignments[i] == 1 else 0

    # Probability for this user
    prob = baseline_prob + cluster_effect + treatment_contrib
    prob = np.clip(prob, 0, 1)
    outcomes[i] = np.random.binomial(1, prob)

# Analyze
results = analyze_cluster_experiment(outcomes, assignments, cluster_ids)

print(f"\nCluster Design:")
print(f"  Number of clusters: {results['n_clusters']}")
print(f"  Treatment clusters: {results['n_treatment_clusters']}")
print(f"  Control clusters: {results['n_control_clusters']}")
print(f"  Mean cluster size: {results['mean_cluster_size']:.1f}")
print(f"  Total units: {n_users}")

print(f"\nIntracluster Correlation:")
print(f"  ICC: {results['icc']:.4f} (true: {icc_true:.4f})")
print(f"  Design effect: {results['design_effect']:.4f}")

print(f"\nTreatment Effect:")
print(f"  Effect: {results['effect']:.4f} (true: {treatment_effect:.4f})")
print(f"  SE (cluster-robust): {results['se_cluster_robust']:.4f}")
print(f"  SE (naive): {results['se_naive']:.4f}")
print(f"  Inflation factor: {results['inflation_factor']:.4f}")
print(f"  95% CI: [{results['ci_lower']:.4f}, {results['ci_upper']:.4f}]")
print(f"  z-statistic: {results['z_statistic']:.4f}")
print(f"  p-value: {results['p_value']:.6f}")

# Visualize clustering effect
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribution of cluster means
cluster_means_by_group = {
    'Control': [],
    'Treatment': []
}
for cluster in np.unique(cluster_ids):
    cluster_mask = cluster_ids == cluster
    cluster_mean = outcomes[cluster_mask].mean()
    cluster_assignment = assignments[cluster_mask][0]

    if cluster_assignment == 1:
        cluster_means_by_group['Treatment'].append(cluster_mean)
    else:
        cluster_means_by_group['Control'].append(cluster_mean)

axes[0].hist(cluster_means_by_group['Control'], bins=20, alpha=0.6,
             label='Control clusters', color='steelblue', edgecolor='black')
axes[0].hist(cluster_means_by_group['Treatment'], bins=20, alpha=0.6,
             label='Treatment clusters', color='coral', edgecolor='black')
```

```
axes[0].set_xlabel('Cluster Mean Outcome', fontsize=12)
axes[0].set_ylabel('Frequency', fontsize=12)
axes[0].set_title('Distribution of Cluster Means', fontsize=12,
    fontweight='bold')
axes[0].legend()
axes[0].grid(alpha=0.3)

# SE inflation as function of ICC
icc_range = np.linspace(0, 0.5, 50)
cluster_size = results['mean_cluster_size']
inflation = np.sqrt(1 + (cluster_size - 1) * icc_range)

axes[1].plot(icc_range, inflation, linewidth=2.5, color='steelblue')
axes[1].axvline(results['icc'], color='red', linestyle='--', linewidth
    =2,
                label=f"Observed ICC = {results['icc']:.3f}")
axes[1].axhline(results['inflation_factor'], color='green', linestyle='
    --', linewidth=2,
                label=f"Inflation = {results['inflation_factor']:.3f}")
axes[1].set_xlabel('Intracluster Correlation (ICC)', fontsize=12)
axes[1].set_ylabel('SE Inflation Factor', fontsize=12)
axes[1].set_title(f'SE Inflation (cluster size = {cluster_size:.0f})',
                  fontsize=12, fontweight='bold')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('cluster_randomization.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

3.5 Confounding Variables and Bias

Even with randomization, various biases can threaten experimental validity. Recognizing and addressing them is crucial.

3.5.1 Selection Bias

Selection bias occurs when treatment and control groups differ systematically due to the selection mechanism.

Common sources:

1. **Self-selection:** Users choose their group rather than being randomly assigned
2. **Non-random attrition:** Differential dropout between groups
3. **Survivorship bias:** Analyzing only users who complete the experiment
4. **Sample restriction:** Post-randomization filtering creates imbalance

Example 3.6 (Selection Bias in Practice)**.** Testing a premium feature:

Biased approach: "Offer to all users, compare those who adopt vs. don't adopt"
Adopters likely have:

- Higher income / willingness to pay

- Different needs
- More engagement
- Different demographics

Observed differences confound adoption decision with feature effect.

Correct approach: Randomly offer feature to 50%, don't offer to other 50% (regardless of interest). Compare outcomes between groups.

3.5.2 Survivorship Bias

Survivorship bias arises when analysis excludes units that drop out, especially if dropout is differential across groups.

Example 3.7 (Survivorship Bias)**.** Testing a complex new interface:

- Treatment (new interface): 30% of users abandon immediately due to confusion
- Control (old interface): 10% abandon
- Analysis of remaining users shows treatment has higher engagement

Conclusion: New interface is better?

Problem: You're comparing the 70% who persevered through treatment (highly engaged, patient users) with 90% of control users (more representative). The selection creates bias.

Solution: Analyze all randomized users using intention-to-treat principle, not just those who completed.

3.5.3 Simpson's Paradox

Simpson's paradox occurs when a trend appears in different groups but reverses when groups are combined—or vice versa.

Example 3.8 (Simpson's Paradox in A/B Testing)**.** Testing checkout redesign across mobile and desktop:

Platform	Control Conv.	Treatment Conv.	Winner
Mobile	5% (100/2000)	6% (120/2000)	Treatment
Desktop	15% (1500/10000)	16% (1600/10000)	Treatment
Combined	**13.3%** (1600/12000)	**14.3%** (1720/12000)	Treatment

Treatment wins on both platforms and overall—straightforward.

Now consider a different allocation:

Platform	Control Conv.	Treatment Conv.	Winner
Mobile	5% (50/1000)	6% (180/3000)	Treatment
Desktop	15% (1650/11000)	14% (1260/9000)	Control
Combined	**14.2%** (1700/12000)	**12.0%** (1440/12000)	**Control**

Treatment wins on mobile but loses on desktop and overall! The reversal occurs because:

- Treatment got more mobile users (low conversion)
- Control got more desktop users (high conversion)
- Platform composition confounds treatment effect

Resolution: Proper randomization prevents such confounding by balancing group composition. Post-stratification or regression adjustment can account for it in analysis.

Listing 3.5: Detecting Simpson's Paradox

```
def detect_simpsons_paradox(outcomes: np.ndarray,
                            assignments: np.ndarray,
                            strata: np.ndarray) -> Dict:
    """
␣␣␣␣Detect␣Simpson's␣paradox␣in␣stratified␣data.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣outcomes␣:␣np.ndarray
␣␣␣␣␣␣␣␣Binary␣outcomes
␣␣␣␣assignments␣:␣np.ndarray
␣␣␣␣␣␣␣␣Treatment␣assignments␣(0/1)
␣␣␣␣strata␣:␣np.ndarray
␣␣␣␣␣␣␣␣Stratum␣indicators

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Analysis␣showing␣overall␣and␣stratum-specific␣effects
␣␣␣␣"""
    results = {
        'overall': {},
        'strata': {},
        'paradox_detected': False
    }

    # Overall effect
    overall_treatment_rate = outcomes[assignments == 1].mean()
    overall_control_rate = outcomes[assignments == 0].mean()
    overall_effect = overall_treatment_rate - overall_control_rate

    results['overall'] = {
        'treatment_rate': overall_treatment_rate,
        'control_rate': overall_control_rate,
        'effect': overall_effect,
        'treatment_wins': overall_effect > 0
    }

    # Stratum-specific effects
    unique_strata = np.unique(strata)
    stratum_treatment_wins = []

    for stratum in unique_strata:
        stratum_mask = strata == stratum
        stratum_outcomes = outcomes[stratum_mask]
        stratum_assignments = assignments[stratum_mask]

```

```
        treatment_rate = stratum_outcomes[stratum_assignments == 1].
            mean()
        control_rate = stratum_outcomes[stratum_assignments == 0].mean
            ()
        effect = treatment_rate - control_rate

        results['strata'][stratum] = {
            'treatment_rate': treatment_rate,
            'control_rate': control_rate,
            'effect': effect,
            'treatment_wins': effect > 0,
            'n_treatment': (stratum_assignments == 1).sum(),
            'n_control': (stratum_assignments == 0).sum()
        }

        stratum_treatment_wins.append(effect > 0)

    # Check for paradox
    all_strata_agree = len(set(stratum_treatment_wins)) == 1
    overall_agrees = results['overall']['treatment_wins'] ==
        stratum_treatment_wins[0]

    if all_strata_agree and not overall_agrees:
        results['paradox_detected'] = True
        results['paradox_type'] = 'reversal'
    elif not all_strata_agree:
        results['paradox_detected'] = True
        results['paradox_type'] = 'inconsistent'

    return results


# Example: Simpson's Paradox simulation
print("\n" + "="*80)
print("Simpson's Paradox Detection")
print("="*80)

# Scenario 1: No paradox
np.random.seed(42)
n_mobile = 4000
n_desktop = 20000

platform = np.array(['Mobile'] * n_mobile + ['Desktop'] * n_desktop)
assignments_balanced = simple_randomization(len(platform), seed=42)

# Generate outcomes (treatment helps on both platforms)
outcomes_no_paradox = np.zeros(len(platform))
for i in range(len(platform)):
    if platform[i] == 'Mobile':
        base_rate = 0.05
        treatment_lift = 0.01
    else:
        base_rate = 0.15
        treatment_lift = 0.01

    prob = base_rate + (treatment_lift if assignments_balanced[i] == 1
        else 0)
    outcomes_no_paradox[i] = np.random.binomial(1, prob)
```

```

results_no_paradox = detect_simpsons_paradox(outcomes_no_paradox,
                                             assignments_balanced,
                                                platform)

print("\nScenario 1: Balanced Randomization (No Paradox)")
print("-" * 80)
print(f"Overall: Treatment = {results_no_paradox['overall']['
    treatment_rate']:.4f}, "
      f"Control = {results_no_paradox['overall']['control_rate']:.4f}, "
      f"Winner = {'Treatment' if results_no_paradox['overall']['
          treatment_wins'] else 'Control'}")

for stratum, stats in results_no_paradox['strata'].items():
    print(f"{stratum}: Treatment = {stats['treatment_rate']:.4f}, "
          f"Control = {stats['control_rate']:.4f}, "
          f"Winner = {'Treatment' if stats['treatment_wins'] else '
              Control'}")

print(f"Simpson's Paradox: {results_no_paradox['paradox_detected']}")

# Scenario 2: Simpson's Paradox due to imbalanced allocation
# Artificially create imbalance
assignments_imbalanced = np.zeros(len(platform), dtype=int)
# More treatment in mobile (low conversion)
assignments_imbalanced[platform == 'Mobile'] = np.random.binomial(
    1, 0.75, n_mobile
)
# Less treatment in desktop (high conversion)
assignments_imbalanced[platform == 'Desktop'] = np.random.binomial(
    1, 0.25, n_desktop
)

outcomes_paradox = outcomes_no_paradox.copy()  # Same true effects

results_paradox = detect_simpsons_paradox(outcomes_paradox,
                                          assignments_imbalanced,
                                             platform)

print("\nScenario 2: Imbalanced Allocation (Potential Paradox)")
print("-" * 80)
print(f"Overall: Treatment = {results_paradox['overall']['
    treatment_rate']:.4f}, "
      f"Control = {results_paradox['overall']['control_rate']:.4f}, "
      f"Winner = {'Treatment' if results_paradox['overall']['
          treatment_wins'] else 'Control'}")

for stratum, stats in results_paradox['strata'].items():
    print(f"{stratum}: Treatment = {stats['treatment_rate']:.4f}, "
          f"Control = {stats['control_rate']:.4f}, "
          f"Winner = {'Treatment' if stats['treatment_wins'] else '
              Control'}, "
          f"n_treat = {stats['n_treatment']}, n_ctrl = {stats['
              n_control']}")

print(f"Simpson's Paradox: {results_paradox['paradox_detected']}")
if results_paradox['paradox_detected']:
```

```
    print(f"Type: {results_paradox['paradox_type']}")

# Visualize
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scenario 1
strata_names = list(results_no_paradox['strata'].keys())
x = np.arange(len(strata_names) + 1)
treatment_rates = [results_no_paradox['overall']['treatment_rate']] + \
                  [results_no_paradox['strata'][s]['treatment_rate']
                     for s in strata_names]
control_rates = [results_no_paradox['overall']['control_rate']] + \
                [results_no_paradox['strata'][s]['control_rate'] for s
                   in strata_names]
labels = ['Overall'] + strata_names

width = 0.35
axes[0].bar(x - width/2, control_rates, width, label='Control',
            alpha=0.7, color='steelblue')
axes[0].bar(x + width/2, treatment_rates, width, label='Treatment',
            alpha=0.7, color='coral')
axes[0].set_ylabel('Conversion Rate', fontsize=12)
axes[0].set_title('Balanced Allocation (No Paradox)', fontsize=12,
   fontweight='bold')
axes[0].set_xticks(x)
axes[0].set_xticklabels(labels, rotation=15)
axes[0].legend()
axes[0].grid(alpha=0.3, axis='y')

# Scenario 2
treatment_rates_p = [results_paradox['overall']['treatment_rate']] + \
                    [results_paradox['strata'][s]['treatment_rate'] for
                        s in strata_names]
control_rates_p = [results_paradox['overall']['control_rate']] + \
                  [results_paradox['strata'][s]['control_rate'] for s
                     in strata_names]

axes[1].bar(x - width/2, control_rates_p, width, label='Control',
            alpha=0.7, color='steelblue')
axes[1].bar(x + width/2, treatment_rates_p, width, label='Treatment',
            alpha=0.7, color='coral')
axes[1].set_ylabel('Conversion Rate', fontsize=12)
axes[1].set_title("Imbalanced Allocation (Simpson's Paradox)",
                  fontsize=12, fontweight='bold')
axes[1].set_xticks(x)
axes[1].set_xticklabels(labels, rotation=15)
axes[1].legend()
axes[1].grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('simpsons_paradox.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

3.5.4 Network Effects

Network effects occur when one user's treatment affects another user's outcomes, violating the Stable Unit Treatment Value Assumption (SUTVA).

Definition 3.7 (SUTVA)**.** The Stable Unit Treatment Value Assumption states that:

1. Each unit has only one potential outcome under each treatment (no hidden variations)
2. One unit's treatment does not affect another's outcome (no interference)

Common network effects in digital experiments:

- **Direct interaction:** Social features where treated users interact with control users (messaging, sharing, tagging)
- **Marketplace effects:** Two-sided platforms where treatment affecting buyers impacts sellers, and vice versa
- **Capacity constraints:** Limited resources (e.g., customer support) where treatment increasing demand affects all users
- **Competitive dynamics:** Treatment improving one seller's visibility reduces others' (zero-sum outcomes)

Consequences of network effects:

- Biased treatment effect estimates (usually toward zero)
- Invalid confidence intervals
- Spillover effects confound results
- May underestimate global impact

Solutions:

1. **Cluster randomization:** Randomize social networks, households, or geographic regions rather than individuals
2. **Geo-based experiments:** Randomize by city or region when network effects are geographically bounded
3. **Temporal rollouts:** Full rollout with before/after comparison (weaker causal inference but avoids contamination)
4. **Model-based adjustment:** Estimate and adjust for spillover effects (advanced, requires modeling assumptions)

3.6 Validity Considerations

Experimental validity assesses whether an experiment tests what it purports to test and whether conclusions generalize appropriately.

3.6.1 Internal Validity

Internal validity asks: Does the experiment actually measure the causal effect of treatment on outcomes, or are results confounded?

Threats to internal validity:

1. **Selection bias:** Non-random assignment or differential attrition
2. **History effects:** External events occurring during experiment affecting one group more than another
3. **Maturation effects:** Natural changes over time (learning, fatigue) affecting groups differently
4. **Instrumentation:** Measurement changes during experiment (e.g., bug fixes, metric definition changes)
5. **Testing effects:** Awareness of being tested affects behavior (Hawthorne effect)
6. **Regression to the mean:** Extreme values naturally regress toward average on retesting

Ensuring internal validity:

- Proper randomization
- Adequate sample size
- Blinding where feasible
- Consistent measurement
- Intention-to-treat analysis
- Pre-registration of hypotheses

3.6.2 External Validity

External validity asks: Do results generalize beyond the specific experimental context to other populations, settings, and times?

Threats to external validity:

1. **Sample non-representativeness:** Experimental sample differs from target population
2. **Setting artificiality:** Experimental conditions differ from real-world deployment
3. **Temporal specificity:** Results hold only at specific times (seasonality, trends)
4. **Treatment variation:** Experimental treatment differs from how it would be deployed at scale

Example 3.9 (External Validity Challenges)**.** Testing a new mobile app design:

- **Sample:** Only iOS users, English language, US residents
- **Setting:** Short 1-week test during holiday season
- **Treatment:** Prototype with potential bugs, incomplete features

Questions:

- Will effects hold for Android users? Non-English speakers? Other countries?
- Are holiday patterns representative of typical usage?
- Will polished version perform differently than prototype?

These questions concern external validity—whether specific experimental findings generalize.

Improving external validity:

- Representative sampling
- Realistic treatment implementation
- Sufficient duration to capture normal patterns
- Multiple experiments across contexts
- Replication studies

3.6.3 Construct Validity

Construct validity asks: Do we measure the theoretical construct we intend to measure?

Common construct validity issues:

1. **Metric doesn't capture intent:** Using clicks as proxy for user satisfaction
2. **Metric captures unintended constructs:** Revenue increase might reflect price increase, not value increase
3. **Missing important dimensions:** Focusing on engagement while ignoring user well-being
4. **Confounded metrics:** Single metric conflates multiple constructs

Example 3.10 (Construct Validity)**.** Goal: Improve user satisfaction with search results

Weak construct validity:

- Metric: Number of search result clicks
- Problem: More clicks might indicate *worse* results (users can't find answer, keep clicking)

Better construct validity:

- Metrics: Time to first click (faster is better), probability of return to search (lower is better), explicit satisfaction ratings
- These more directly measure the intended construct (satisfaction)

3.6.4 Statistical Conclusion Validity

Statistical conclusion validity asks: Are statistical inferences correct? Do we properly quantify uncertainty?

Threats to statistical conclusion validity:

1. **Low statistical power:** Insufficient sample size to detect real effects
2. **Violated assumptions:** Using tests that assume normality, independence, etc. when assumptions don't hold
3. **Fishing and p-hacking:** Multiple testing, selective reporting, post-hoc hypotheses
4. **Improper error estimation:** Ignoring clustering, using wrong standard errors
5. **Unreliable measurement:** High measurement error inflates variance, reduces power

3.7 Practical Design Decisions

Translating experimental design principles into actual A/B tests requires numerous practical decisions.

3.7.1 Unit of Randomization

The unit of randomization is the entity assigned to treatment or control. Common choices:

- **User:** Most common; tracks individual across sessions
- **Session:** Each visit randomized independently (rarely appropriate)
- **Device:** When user identity is unavailable or unreliable
- **Household/Account:** When multiple users share accounts
- **Geographic unit:** City, region, or country for market-level interventions
- **Temporal unit:** Time periods (week, day) for platform-wide changes

Considerations:

1. **Treatment application level:** Unit must be at least as coarse as treatment application
2. **Outcome measurement level:** Ideally aligned with measurement
3. **Network effects:** Cluster at level that contains interference
4. **Sample size:** More granular units provide larger samples but risk contamination

3.7.2 Temporal Considerations

Time affects experiments in multiple ways:

1. **Duration:**
    - Too short: Miss delayed effects, affected by day-of-week patterns
    - Too long: External events confound results, delayed decisions
    - Typical: 1-4 weeks, capturing full weekly cycles
2. **Seasonality:**
    - Holiday periods atypical
    - Back-to-school, tax season, etc. for relevant domains
    - Consider running in representative periods
3. **Novelty effects:**
    - Initial curiosity inflates treatment metrics
    - May fade after days/weeks
    - Distinguish short-term from long-term effects
4. **Learning effects:**
    - Users adapt to new interfaces
    - Performance may improve over time
    - Consider learning curves in duration decisions

3.7.3 Geographic Considerations

Geographic variation affects experimental design:

1. **Regulatory constraints:** Some features unavailable in certain regions
2. **Market maturity:** Effects may differ between mature and emerging markets
3. **Cultural differences:** UI conventions, color meanings, interaction patterns vary
4. **Infrastructure:** Network speed, device types affect experience
5. **Time zones:** Usage patterns differ; ensure balanced sampling across times

Approaches:

- **Stratify by region:** Ensure balance across key markets
- **Run region-specific tests:** Estimate heterogeneous effects
- **Stage rollouts geographically:** Test in one market before expanding

Platform	Issues	Considerations
Web	Session continuity, cookie deletion	Use server-side randomization
Mobile apps	App store delays, version fragmentation	Staged rollouts, feature flags
Desktop software	Infrequent updates, offline usage	Long experiment duration
APIs	Rate limiting, caching	Cache invalidation, proper logging
Email	Spam filters, client rendering	Check deliverability, test across clients

3.7.4 Platform-Specific Issues

Different platforms present unique challenges:

3.8 Summary

Experimental design principles form the foundation for trustworthy A/B testing:

- **Randomization** eliminates selection bias and enables causal inference. It's non-negotiable for valid experiments.
- **Control groups** provide the counterfactual, distinguishing treatment effects from background variation.
- **Blocking and stratification** improve precision by ensuring balance on important covariates.
- **Different randomization techniques**—simple, block, stratified, cluster—suit different contexts. Choose based on sample size, covariate importance, and interference patterns.
- **Biases**—selection, survivorship, confounding—threaten validity even with randomization. Recognize and mitigate them through design and analysis choices.
- **Validity** encompasses internal (causal correctness), external (generalizability), construct (measurement appropriateness), and statistical conclusion (inference correctness) dimensions. Strong experiments excel on all four.
- **Practical decisions**—unit of randomization, duration, geographic scope, platform specifics—require balancing statistical ideals with operational constraints.

Sound experimental design precedes and enables sound statistical analysis. Invest time in design; it pays dividends in reliable insights and confident decisions.

3.9 Further Reading

- **?**: Fisher's foundational work on experimental design and randomization
- **?**: Comprehensive modern treatment of experimental design
- **?**: Causal inference framework for experimental and observational studies
- **?**: Practical guide to online controlled experiments
- **?**: Detailed treatment of cluster randomized trials
- **?**: Network interference in experiments

3.10 Exercises

3.10.1 Conceptual Questions

1. Explain why randomization is essential for causal inference. What assumptions does randomization satisfy that observational studies typically violate?

2. A colleague proposes assigning even-numbered user IDs to treatment and odd-numbered to control. What's wrong with this approach?

3. When would you prefer stratified randomization over simple randomization? What are the costs and benefits?

4. Describe three scenarios where cluster randomization is necessary despite its statistical efficiency loss.

5. Explain Simpson's paradox in your own words. Provide an original example from a domain you're familiar with.

6. Distinguish between internal and external validity. Can an experiment have high internal validity but low external validity? Provide an example.

3.10.2 Design Exercises

1. Design an A/B test for a two-sided marketplace (e.g., Airbnb, Uber) where treatment affects one side but you care about both sides' outcomes. What design challenges arise? How would you address them?

2. You're testing a social feature where users can tag friends. Describe the network effects you'd expect. How would you design the randomization to account for these?

3. Design a long-duration experiment (6+ months) for a subscription service. What temporal factors would you consider? How would you handle seasonality?

4. Compare designs for testing the same feature on web, iOS app, and Android app. What platform-specific challenges arise? Would you run separate experiments or one combined experiment?

5. You want to test a pricing change but worry about fairness (charging different prices to different users). Design an experiment that balances statistical validity with ethical concerns.

3.10.3 Implementation Exercises

1. Implement a hash-based randomization function that:

   - Takes user ID and experiment name as input
   - Returns deterministic assignment (same user always gets same variant)
   - Supports arbitrary split ratios
   - Allows easy "salting" for independent experiments

2. Create a stratified randomization system that:
    - Accepts multiple stratifying variables
    - Handles continuous and categorical variables
    - Ensures balance within strata
    - Provides diagnostic plots and balance tests
3. Write a bias detection tool that:
    - Checks for selection bias via covariate balance tests
    - Detects potential Simpson's paradox across strata
    - Identifies temporal trends suggesting history effects
    - Generates report with visualizations and recommendations
4. Implement cluster randomization with ICC estimation:
    - Randomize at cluster level
    - Estimate ICC from pilot data
    - Calculate design effect and required sample size
    - Perform cluster-robust inference

3.10.4 Analysis Exercises

1. Given experimental data with known selection bias (provided dataset), use multiple methods to detect and quantify the bias. How would you correct for it in analysis?
2. Simulate a scenario with network effects. Compare results from individual randomization (ignoring interference) vs. cluster randomization. Quantify the bias introduced by ignoring network effects.
3. Create synthetic data exhibiting Simpson's paradox. Show reversal of treatment effect sign when aggregating vs. stratifying. Explain which analysis is correct and why.
4. Analyze a multi-platform experiment (web + mobile) where treatment was randomly assigned but platform composition differs between groups. Demonstrate stratified analysis accounting for this imbalance.
5. Given survival data (time-to-conversion) with differential attrition, compare intention-to-treat analysis vs. per-protocol analysis vs. survivor analysis. Discuss when each is appropriate.

3.10.5 Simulation Projects

1. **Randomization comparison:** Simulate 10,000 experiments comparing simple, block, and stratified randomization. Compare:
    - Group size balance

- Covariate balance
- Power to detect effects
- Type I error rates

Write report with visualizations and recommendations.

2. **Network effects impact:** Simulate social network with varying levels of connectivity. Compare:
   - Individual randomization (ignoring network)
   - Cluster randomization by network communities
   - Bias as function of network density and treatment effect strength
3. **Validity threats:** Implement experiments with various validity threats:
   - Selection bias (non-random attrition)
   - History effects (external shock during experiment)
   - Instrumentation (metric definition change mid-experiment)
   - Regression to mean (selecting extreme performers)

   Quantify how each threat affects conclusions. Demonstrate mitigation strategies.
4. **Optimal stratification:** Simulate experiments with multiple potential stratifying variables. Investigate:
   - How many strata provide optimal power?
   - Which variables are worth stratifying on?
   - Trade-off between stratification benefit and complexity

Chapter 4

Sample Size Calculation and Statistical Power

4.1 Learning Objectives

After completing this chapter, readers will be able to:

- Calculate required sample sizes for different types of A/B tests
- Understand the relationship between sample size, effect size, and statistical power
- Apply power analysis to determine experiment feasibility
- Create and interpret power curves
- Account for practical constraints in sample size determination
- Implement sample size calculators in Python and R
- Make informed decisions about minimum detectable effects

4.2 Introduction

"How many users do we need?" is perhaps the most common question in A/B testing. Too few, and we waste time running inconclusive experiments. Too many, and we delay decisions unnecessarily or expose users to potentially inferior variants longer than needed.

Sample size calculation transforms this practical question into a rigorous statistical framework. By specifying what we want to detect and how confident we need to be, we determine exactly how much data is required. This chapter develops the theory and practice of sample size determination, from fundamental formulas to real-world constraints that complicate the textbook approach.

4.3 Power Analysis Fundamentals

Statistical power is the probability of detecting an effect when it truly exists. Understanding power is essential for designing effective experiments.

4.3.1 Definition of Statistical Power

Definition 4.1 (Statistical Power)**.** Statistical power is the probability of correctly rejecting the null hypothesis when the alternative hypothesis is true:

$$\text{Power} = 1 - \beta = P(\text{Reject } H_0 \mid H_1 \text{ true}) \tag{4.1}$$

where β is the probability of a Type II error (false negative).

Power answers the question: "If there really is an effect of size δ, what's the probability our experiment will detect it?"

Example 4.1 (Interpreting Power)**.** Power = 0.80 means:

- If the true treatment effect is δ, we have an 80% chance of obtaining $p < \alpha$
- Equivalently, 20% chance of missing the effect (Type II error)
- If we ran 100 such experiments where an effect exists, 80 would correctly declare significance

4.3.2 Factors Affecting Power

Four interrelated factors determine statistical power:

1. **Effect Size (δ)**: Larger effects are easier to detect
   - Power increases with δ
   - Detecting 10% lift requires less data than 1% lift
2. **Sample Size (n)**: More data increases power
   - Power increases with $\sqrt{n}$
   - Doubling power requires roughly 4× sample size
3. **Significance Level (α)**: Higher α increases power
   - Relaxing from $\alpha = 0.01$ to 0.05 increases power
   - Tradeoff: more false positives
4. **Variance (σ^2)**: Lower variance increases power
   - Variance reduction techniques boost power without more data
   - Covered in Chapter 8

Theorem 4.1 (Power Relationships)**.** For comparing two proportions with equal sample sizes:

$$\text{Power} = \Phi\left(\frac{\delta\sqrt{n}}{2\sqrt{p(1-p)}} - z_{1-\alpha/2}\right) \tag{4.2}$$

where $p = (p_1 + p_2)/2$, $\delta = p_2 - p_1$, and Φ is the standard normal CDF.
Key insights:

- Power increases with effect size δ
- Power increases with $\sqrt{n}$ (not linearly!)
- Power decreases as α becomes more stringent
- Power depends on baseline rate p (highest at $p = 0.5$)

4.3.3 The Type I and Type II Error Tradeoff

Sample size determination involves balancing two types of errors:

Reality	**Fail to Reject H_0**	**Reject H_0**
H_0 true	Correct decision (1-α)	**Type I Error** (α)
H_1 true	**Type II Error** (β)	Correct decision (1-β)

Table 4.1: Type I and Type II errors in hypothesis testing

Conventional choices:

- $\alpha = 0.05$: Willing to accept 5% false positive rate
- Power = 0.80: Willing to accept 20% false negative rate

Why power = 0.80 is standard: This represents a 4:1 tradeoff between Type II and Type I errors ($\beta/\alpha = 0.20/0.05 = 4$). In most contexts, missing a real effect is considered 4× worse than claiming a spurious one.

When to use higher power:

- High implementation costs (difficult to change later)
- Regulatory requirements
- Mission-critical features
- One-shot opportunities

4.3.4 Power Curves and Interpretation

Power curves visualize how power depends on various parameters, providing intuition about experimental design tradeoffs.

Listing 4.1: Power Curves for Proportions

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from typing import Tuple, List

sns.set_style("whitegrid")

def calculate_power_proportions(p1: float, p2: float, n: int,
                                alpha: float = 0.05,
                                alternative: str = 'two-sided') ->
                                   float:
```

```
    """
␣␣␣␣Calculate␣statistical␣power␣for␣two-sample␣proportion␣test.

␣␣␣␣Parameters:
␣␣␣␣----------
␣␣␣␣p1␣:␣float
␣␣␣␣␣␣␣␣Control␣proportion
␣␣␣␣p2␣:␣float
␣␣␣␣␣␣␣␣Treatment␣proportion
␣␣␣␣n␣:␣int
␣␣␣␣␣␣␣␣Sample␣size␣per␣group
␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣Significance␣level
␣␣␣␣alternative␣:␣str
␣␣␣␣␣␣␣␣'two-sided'␣or␣'one-sided'

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣float␣:␣Statistical␣power
␣␣␣␣"""
    # Effect size
    delta = p2 - p1

    # Pooled proportion under H0
    p_bar = (p1 + p2) / 2

    # Standard error under H0
    se_null = np.sqrt(2 * p_bar * (1 - p_bar) / n)

    # Standard error under H1
    se_alt = np.sqrt((p1 * (1 - p1) + p2 * (1 - p2)) / n)

    # Critical value
    if alternative == 'two-sided':
        z_crit = stats.norm.ppf(1 - alpha/2)
    else:
        z_crit = stats.norm.ppf(1 - alpha)

    # Non-centrality parameter
    ncp = delta / se_alt

    # Power calculation
    if alternative == 'two-sided':
        power = 1 - stats.norm.cdf(z_crit - ncp) + stats.norm.cdf(-
            z_crit - ncp)
    else:
        power = 1 - stats.norm.cdf(z_crit - ncp)

    return power


def plot_power_curves(p1: float = 0.10, alpha: float = 0.05,
                      save_path: str = None):
    """
␣␣␣␣Generate␣comprehensive␣power␣curve␣visualizations.

␣␣␣␣Parameters:
␣␣␣␣----------
```

```
    p1 : float
        Baseline conversion rate
    alpha : float
        Significance level
    save_path : str
        Optional path to save figure
    """
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # 1. Power vs Sample Size for different effect sizes
    ax1 = axes[0, 0]
    sample_sizes = np.arange(100, 10001, 100)
    relative_lifts = [0.05, 0.10, 0.20, 0.30]

    for lift in relative_lifts:
        p2 = p1 * (1 + lift)
        powers = [calculate_power_proportions(p1, p2, n, alpha) for n
            in sample_sizes]
        ax1.plot(sample_sizes, powers, linewidth=2.5,
                label=f'{lift:.0%} relative lift', alpha=0.8)

    ax1.axhline(0.8, color='red', linestyle='--', linewidth=2,
               label='Target power = 0.80')
    ax1.set_xlabel('Sample Size (per group)', fontsize=12)
    ax1.set_ylabel('Statistical Power', fontsize=12)
    ax1.set_title(f'Power vs Sample Size (baseline = {p1:.1%},  = {
        alpha})',
                  fontsize=13, fontweight='bold')
    ax1.legend(fontsize=10)
    ax1.grid(alpha=0.3)
    ax1.set_ylim([0, 1])

    # 2. Power vs Effect Size for different sample sizes
    ax2 = axes[0, 1]
    relative_lifts = np.linspace(0.01, 0.50, 100)
    sample_sizes_to_plot = [500, 1000, 2000, 5000]

    for n in sample_sizes_to_plot:
        powers = []
        for lift in relative_lifts:
            p2 = p1 * (1 + lift)
            power = calculate_power_proportions(p1, p2, n, alpha)
            powers.append(power)
        ax2.plot(relative_lifts * 100, powers, linewidth=2.5,
                label=f'n = {n:,}', alpha=0.8)

    ax2.axhline(0.8, color='red', linestyle='--', linewidth=2,
               label='Target power = 0.80')
    ax2.set_xlabel('Relative Lift (%)', fontsize=12)
    ax2.set_ylabel('Statistical Power', fontsize=12)
    ax2.set_title(f'Power vs Effect Size (baseline = {p1:.1%},  = {
        alpha})',
                  fontsize=13, fontweight='bold')
    ax2.legend(fontsize=10)
    ax2.grid(alpha=0.3)
    ax2.set_ylim([0, 1])

    # 3. Power vs Baseline Rate for fixed relative lift
```

```
    ax3 = axes[1, 0]
    baselines = np.linspace(0.01, 0.50, 100)
    relative_lift = 0.10
    sample_sizes_to_plot = [1000, 2000, 5000, 10000]

    for n in sample_sizes_to_plot:
        powers = []
        for p1_var in baselines:
            p2 = p1_var * (1 + relative_lift)
            power = calculate_power_proportions(p1_var, p2, n, alpha)
            powers.append(power)
        ax3.plot(baselines * 100, powers, linewidth=2.5,
                 label=f'n = {n:,}', alpha=0.8)

    ax3.axhline(0.8, color='red', linestyle='--', linewidth=2,
                label='Target power = 0.80')
    ax3.set_xlabel('Baseline Conversion Rate (%)', fontsize=12)
    ax3.set_ylabel('Statistical Power', fontsize=12)
    ax3.set_title(f'Power vs Baseline Rate ({relative_lift:.0%} relative lift,  = {alpha})',
                  fontsize=13, fontweight='bold')
    ax3.legend(fontsize=10)
    ax3.grid(alpha=0.3)
    ax3.set_ylim([0, 1])

    # 4. Required Sample Size vs Effect Size
    ax4 = axes[1, 1]
    relative_lifts = np.linspace(0.05, 0.50, 50)
    target_powers = [0.70, 0.80, 0.90]

    for target_power in target_powers:
        required_n = []
        for lift in relative_lifts:
            p2 = p1 * (1 + lift)
            # Binary search for required n
            n_low, n_high = 10, 100000
            while n_high - n_low > 1:
                n_mid = (n_low + n_high) // 2
                power = calculate_power_proportions(p1, p2, n_mid,
                    alpha)
                if power < target_power:
                    n_low = n_mid
                else:
                    n_high = n_mid
            required_n.append(n_high)

        ax4.plot(relative_lifts * 100, required_n, linewidth=2.5,
                 label=f'Power = {target_power:.0%}', alpha=0.8)

    ax4.set_xlabel('Relative Lift (%)', fontsize=12)
    ax4.set_ylabel('Required Sample Size (per group)', fontsize=12)
    ax4.set_title(f'Sample Size Requirements (baseline = {p1:.1%},  = {alpha})',
                  fontsize=13, fontweight='bold')
    ax4.legend(fontsize=10)
    ax4.grid(alpha=0.3)
    ax4.set_yscale('log')

```

```
    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()

    # Print key insights
    print("="*80)
    print("Power Analysis Insights")
    print("="*80)

    # Calculate some specific examples
    print(f"\nBaseline conversion rate: {p1:.1%}")
    print(f"Significance level:  =  {alpha}")
    print("\nSample size required for 80% power:\n")

    for lift in [0.05, 0.10, 0.20]:
        p2 = p1 * (1 + lift)
        # Binary search
        n_low, n_high = 10, 100000
        while n_high - n_low > 1:
            n_mid = (n_low + n_high) // 2
            power = calculate_power_proportions(p1, p2, n_mid, alpha)
            if power < 0.80:
                n_low = n_mid
            else:
                n_high = n_mid

        print(f"  {lift:>5.0%} lift ({p1:.1%}    {p2:.1%}): {n_high
            :>6,} users/group")

# Generate power curves
print("Generating power curve visualizations...\n")
plot_power_curves(p1=0.10, alpha=0.05, save_path='power_curves.pdf')
```

4.4 Sample Size Formulas

Sample size formulas translate desired power, significance level, and effect size into required data volume. We derive formulas for the most common metrics.

4.4.1 Sample Size for Proportions (Conversion Rates)

The most common A/B test compares conversion rates between two groups.

Two-Sample Test for Proportions

Test: $H_0 : p_1 = p_2$ vs. $H_1 : p_1 \neq p_2$

Assumptions:

- Two independent samples
- Equal sample sizes $n_1 = n_2 = n$
- Two-sided test

Theorem 4.2 (Sample Size for Proportions)**.** The required sample size per group is:

$$n = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \cdot [p_1(1-p_1) + p_2(1-p_2)]}{(p_2 - p_1)^2} \tag{4.3}$$

where:

- $z_{1-\alpha/2}$ is the $(1-\alpha/2)$ quantile of standard normal (e.g., 1.96 for $\alpha = 0.05$)
- $z_{1-\beta}$ is the $(1-\beta)$ quantile of standard normal (e.g., 0.84 for power $= 0.80$)
- p_1, p_2 are the control and treatment proportions

Derivation (sketch). Under H_0, the test statistic follows:

$$Z_0 = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\bar{p}(1-\bar{p})(1/n + 1/n)}} \sim \mathcal{N}(0,1) \tag{4.4}$$

Under H_1 with true difference $\delta = p_2 - p_1$:

$$Z_1 = \frac{\hat{p}_2 - \hat{p}_1 - \delta}{\sqrt{p_1(1-p_1)/n + p_2(1-p_2)/n}} \sim \mathcal{N}(0,1) \tag{4.5}$$

For power $1-\beta$, we require:

$$P(|Z_0| > z_{1-\alpha/2} \mid H_1) = 1 - \beta \tag{4.6}$$

This occurs when:

$$\frac{\delta}{\sqrt{(p_1(1-p_1) + p_2(1-p_2))/n}} = z_{1-\alpha/2} + z_{1-\beta} \tag{4.7}$$

Solving for n yields the formula above. □

Example 4.2 (Sample Size Calculation)**.** An e-commerce site wants to test a new checkout flow:

- Current conversion rate: $p_1 = 0.10$ (10%)
- Minimum detectable effect: 10% relative lift $\rightarrow p_2 = 0.11$ (11%)
- Significance level: $\alpha = 0.05$
- Desired power: $1 - \beta = 0.80$

$$n = \frac{(1.96 + 0.84)^2 \cdot [0.10(0.90) + 0.11(0.89)]}{(0.11 - 0.10)^2} \tag{4.8}$$

$$= \frac{7.84 \cdot [0.09 + 0.0979]}{0.0001} \tag{4.9}$$

$$= \frac{7.84 \cdot 0.1879}{0.0001} \tag{4.10}$$

$$= 14,732 \tag{4.11}$$

Required: 14,732 users per group (29,464 total).
At 1,000 users/day, this experiment takes 30 days.

Listing 4.2: Sample Size Calculator for Proportions

```
def sample_size_proportions(p1: float, p2: float,
                            alpha: float = 0.05,
                            power: float = 0.80,
                            ratio: float = 1.0) -> dict:
    """
␣␣␣␣Calculate␣required␣sample␣size␣for␣two-proportion␣test.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣p1␣:␣float
␣␣␣␣␣␣␣␣Control␣proportion
␣␣␣␣p2␣:␣float
␣␣␣␣␣␣␣␣Treatment␣proportion
␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣Significance␣level␣(default␣0.05)
␣␣␣␣power␣:␣float
␣␣␣␣␣␣␣␣Statistical␣power␣(default␣0.80)
␣␣␣␣ratio␣:␣float
␣␣␣␣␣␣␣␣Ratio␣n2/n1␣(default␣1.0␣for␣equal␣allocation)

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Sample␣sizes␣and␣related␣statistics
␣␣␣␣"""
    from scipy import stats

    # Critical values
    z_alpha = stats.norm.ppf(1 - alpha/2)  # Two-sided
    z_beta = stats.norm.ppf(power)

    # Effect size
    delta = p2 - p1
    if delta == 0:
        raise ValueError("p1␣and␣p2␣must␣be␣different")

    # Sample size calculation
    if ratio == 1.0:
        # Equal allocation (more efficient)
        numerator = (z_alpha + z_beta)**2 * (p1*(1-p1) + p2*(1-p2))
        denominator = delta**2
        n1 = numerator / denominator
        n2 = n1
    else:
        # Unequal allocation
        p_avg = (p1 + ratio * p2) / (1 + ratio)
        numerator = (z_alpha * np.sqrt((1+ratio)*p_avg*(1-p_avg)) +
                     z_beta * np.sqrt(p1*(1-p1) + ratio*p2*(1-p2)))**2
        denominator = ratio * delta**2
        n1 = numerator / denominator
        n2 = ratio * n1

    # Round up
    n1 = int(np.ceil(n1))
    n2 = int(np.ceil(n2))

    # Calculate actual power with these sample sizes
    se = np.sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)
```

```
    ncp = delta / se
    actual_power = 1 - stats.norm.cdf(z_alpha - ncp) + stats.norm.cdf(-z_alpha - ncp)

    return {
        'n1': n1,
        'n2': n2,
        'n_total': n1 + n2,
        'p1': p1,
        'p2': p2,
        'absolute_difference': delta,
        'relative_lift': delta / p1,
        'alpha': alpha,
        'requested_power': power,
        'actual_power': actual_power,
        'z_alpha': z_alpha,
        'z_beta': z_beta
    }


def sample_size_table(p1: float, relative_lifts: List[float],
                      alpha: float = 0.05, power: float = 0.80):
    """
    Generate table of sample sizes for different effect sizes.

    Parameters:
    -----------
    p1 : float
        Baseline conversion rate
    relative_lifts : List[float]
        List of relative lifts to evaluate
    alpha : float
        Significance level
    power : float
        Desired power
    """
    print("="*90)
    print(f"Sample Size Requirements (baseline = {p1:.1%},  = {alpha}, power = {power:.0%})")
    print("="*90)
    print(f"{'Relative Lift':<15} {'Absolute':<12} {'p2':<10} "
          f"{'n per group':<15} {'Total n':<12} {'Duration (days)':>15}")
    print("-"*90)

    for lift in relative_lifts:
        p2 = p1 * (1 + lift)
        result = sample_size_proportions(p1, p2, alpha, power)

        # Assume 1000 users per day per group
        duration = np.ceil(result['n1'] / 1000)

        print(f"{lift:>14.1%} {result['absolute_difference']:>11.4f} "
              f"{p2:>9.3f} {result['n1']:>14,} {result['n_total']:>11,} "
              f"{duration:>14.0f}")

    print("="*90)
```

```
    print("Assumes 1,000 users per day per group")
    print()


# Example usage
print("\n" + "="*80)
print("Sample Size Calculations for A/B Testing")
print("="*80 + "\n")

# Example 1: E-commerce checkout
print("Example 1: E-commerce Checkout Optimization")
print("-"*80)
result = sample_size_proportions(p1=0.10, p2=0.11, alpha=0.05, power
    =0.80)
print(f"Baseline conversion: {result['p1']:.1%}")
print(f"Target conversion:   {result['p2']:.1%}")
print(f"Relative lift:       {result['relative_lift']:.1%}")
print(f"Required n per group: {result['n1']:,}")
print(f"Total required n:    {result['n_total']:,}")
print(f"Actual power:        {result['actual_power']:.3f}")
print()

# Example 2: Generate table
print("Example 2: Sample Size Table for Different Effect Sizes")
print("-"*80)
sample_size_table(p1=0.10,
                  relative_lifts=[0.05, 0.10, 0.15, 0.20, 0.25, 0.30],
                  alpha=0.05,
                  power=0.80)

# Example 3: Compare different power levels
print("Example 3: Impact of Power Requirements")
print("-"*80)
p1, p2 = 0.10, 0.12
print(f"Baseline: {p1:.1%}, Target: {p2:.1%} ({(p2-p1)/p1:.0%} lift)\n"
    )
print(f"{'Power':<10} {'n per group':<15} {'Total n':<12}")
print("-"*40)
for power in [0.70, 0.75, 0.80, 0.85, 0.90]:
    result = sample_size_proportions(p1, p2, power=power)
    print(f"{power:<10.0%} {result['n1']:<14,} {result['n_total']:<11,}
        ")
```

4.4.2 Sample Size for Means (Continuous Metrics)

When testing continuous outcomes like revenue, session duration, or engagement scores, we use formulas for comparing means.

Theorem 4.3 (Sample Size for Means)**.** For testing $H_0 : \mu_1 = \mu_2$ vs. $H_1 : \mu_1 \neq \mu_2$ with equal sample sizes:

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{(\mu_2 - \mu_1)^2} \tag{4.12}$$

where:

- σ^2 is the common variance (assumed equal in both groups)

- μ_1, μ_2 are the control and treatment means

For unequal variances (Welch's t-test):

$$n_1 = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2(\sigma_1^2 + \sigma_2^2/r)}{(\mu_2 - \mu_1)^2} \tag{4.13}$$

where $r = n_2/n_1$ is the allocation ratio.

Cohen's d effect size:
Often we specify effect size in standardized units:

$$d = \frac{\mu_2 - \mu_1}{\sigma} \tag{4.14}$$

Then:

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2}{d^2} \tag{4.15}$$

Cohen's conventions:

- Small effect: $d = 0.2$
- Medium effect: $d = 0.5$
- Large effect: $d = 0.8$

Example 4.3 (Sample Size for Revenue)**.** Testing pricing change:

- Current average revenue per user: \$50
- Standard deviation: \$30
- Minimum detectable increase: \$5 (10%)
- $\alpha = 0.05$, power $= 0.80$

$$n = \frac{2(1.96 + 0.84)^2 \cdot 30^2}{5^2} \tag{4.16}$$

$$= \frac{2 \cdot 7.84 \cdot 900}{25} \tag{4.17}$$

$$= \frac{14,112}{25} \tag{4.18}$$

$$= 565 \tag{4.19}$$

Required: 565 users per group (1,130 total).
Cohen's d: $d = 5/30 = 0.167$ (small effect).

Listing 4.3: Sample Size Calculator for Means

```
def sample_size_means(mu1: float, mu2: float, sigma: float,
                      alpha: float = 0.05,
                      power: float = 0.80,
                      ratio: float = 1.0) -> dict:
    """
```

```
    Calculate required sample size for two-sample t-test.

    Parameters:
    -----------
    mu1 : float
        Control mean
    mu2 : float
        Treatment mean
    sigma : float
        Common standard deviation
    alpha : float
        Significance level
    power : float
        Statistical power
    ratio : float
        Ratio n2/n1

    Returns:
    --------
    dict : Sample sizes and effect size measures
    """
    from scipy import stats

    # Critical values
    z_alpha = stats.norm.ppf(1 - alpha/2)
    z_beta = stats.norm.ppf(power)

    # Effect size
    delta = mu2 - mu1
    if delta == 0:
        raise ValueError("mu1 and mu2 must be different")

    cohens_d = delta / sigma

    # Sample size
    if ratio == 1.0:
        n1 = 2 * (z_alpha + z_beta)**2 * sigma**2 / delta**2
        n2 = n1
    else:
        n1 = (1 + 1/ratio) * (z_alpha + z_beta)**2 * sigma**2 / delta
            **2
        n2 = ratio * n1

    n1 = int(np.ceil(n1))
    n2 = int(np.ceil(n2))

    # Actual power
    se = sigma * np.sqrt(1/n1 + 1/n2)
    ncp = delta / se
    actual_power = 1 - stats.norm.cdf(z_alpha - ncp) + stats.norm.cdf(-
        z_alpha - ncp)

    return {
        'n1': n1,
        'n2': n2,
        'n_total': n1 + n2,
        'mu1': mu1,
        'mu2': mu2,
```

```
        'sigma': sigma,
        'absolute_difference': delta,
        'relative_difference': delta / mu1,
        'cohens_d': cohens_d,
        'effect_size_interpretation': interpret_cohens_d(abs(cohens_d)),
        'alpha': alpha,
        'requested_power': power,
        'actual_power': actual_power
    }


def interpret_cohens_d(d: float) -> str:
    """Interpret Cohen's d effect size."""
    if d < 0.2:
        return "Negligible"
    elif d < 0.5:
        return "Small"
    elif d < 0.8:
        return "Medium"
    else:
        return "Large"


# Example usage
print("\n" + "="*80)
print("Sample Size for Continuous Metrics")
print("="*80 + "\n")

# Example 1: Revenue per user
print("Example 1: Revenue Per User")
print("-"*80)
result = sample_size_means(mu1=50, mu2=55, sigma=30, alpha=0.05, power=0.80)
print(f"Control mean:     ${result['mu1']:.2f}")
print(f"Treatment mean:   ${result['mu2']:.2f}")
print(f"Standard dev:     ${result['sigma']:.2f}")
print(f"Absolute diff:    ${result['absolute_difference']:.2f}")
print(f"Relative diff:    {result['relative_difference']:.1%}")
print(f"Cohen's d:        {result['cohens_d']:.3f} ({result['effect_size_interpretation']})")
print(f"Required n/group: {result['n1']:,}")
print(f"Total required n: {result['n_total']:,}")
print(f"Actual power:     {result['actual_power']:.3f}")
print()

# Example 2: Session duration
print("Example 2: Session Duration (seconds)")
print("-"*80)
result = sample_size_means(mu1=180, mu2=200, sigma=120, alpha=0.05, power=0.80)
print(f"Control mean:     {result['mu1']:.0f}s")
print(f"Treatment mean:   {result['mu2']:.0f}s")
print(f"Increase:         {result['absolute_difference']:.0f}s ({result['relative_difference']:.1%})")
print(f"Cohen's d:        {result['cohens_d']:.3f} ({result['effect_size_interpretation']})")
print(f"Required n/group: {result['n1']:,}")
```

```
print()

# Example 3: Effect size table
print("Example 3: Sample Size by Effect Size (sigma = 30)")
print("-"*80)
print(f"{'Cohen\\'s d':<12} {'Interpretation':<15} {'Delta mu':<10} "
      f"{'n per group':<15} {'Total n':<12}")
print("-"*70)

for d in [0.1, 0.2, 0.3, 0.5, 0.8]:
    delta = d * 30  # sigma = 30
    result = sample_size_means(mu1=100, mu2=100+delta, sigma=30)
    print(f"{d:<12.1f} {result['effect_size_interpretation']:<15} "
          f"{delta:<10.1f} {result['n1']:<14,} {result['n_total']:<11,}
             ")
```

4.4.3 Sample Size for Survival Data

For time-to-event outcomes (e.g., time to purchase, churn time), sample size depends on the expected number of events rather than just sample size.

Theorem 4.4 (Sample Size for Survival Analysis)**.** For comparing two survival curves using the log-rank test:

$$n = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2}{p_1 p_2 (\log(HR))^2} \tag{4.20}$$

where:

- HR is the hazard ratio (treatment/control)
- p_1, p_2 are proportions in each group
- The formula gives total number of *events* needed

Total sample size depends on event rate:

$$N = \frac{n}{\text{event rate}} \tag{4.21}$$

4.4.4 Sample Size for Ratio Metrics

Ratio metrics (e.g., revenue per user, clicks per session) require special handling due to their non-normal distributions.

Delta method approximation:

For ratio $R = X/Y$ where X and Y are correlated:

$$\text{Var}(R) \approx \frac{1}{\mu_Y^2}\left[\text{Var}(X) + R^2\text{Var}(Y) - 2R\text{Cov}(X,Y)\right] \tag{4.22}$$

Sample size calculation uses this variance in the standard means formula, but often requires pilot data to estimate covariances.

Practical approach: Bootstrap or simulate from pilot data to estimate variance, then use means formula.

4.5 Practical Considerations

Textbook formulas provide starting points, but real-world experiments require additional considerations.

4.5.1 Minimum Detectable Effect (MDE)

The minimum detectable effect is the smallest effect size your experiment can reliably detect given sample size, power, and significance level constraints.

Definition 4.2 (Minimum Detectable Effect)**.** Given fixed n, α, and power, the MDE is the effect size δ satisfying:

$$\text{Power}(\delta, n, \alpha) = 1 - \beta \tag{4.23}$$

Why MDE matters:

- Experiments can only detect effects $\geq$ MDE reliably
- Smaller effects may exist but will appear non-significant
- MDE should align with minimum practically important difference

Example 4.4 (MDE Calculation)**.** Given:

- Available sample: 5,000 users per group
- Baseline conversion: 10%
- $\alpha = 0.05$, power $= 0.80$

Solve for MDE by inverting sample size formula:

$$\delta = \sqrt{\frac{(z_{1-\alpha/2} + z_{1-\beta})^2 [p_1(1-p_1) + p_2(1-p_2)]}{n}} \tag{4.24}$$

With $p_2 = p_1 + \delta$ and $n = 5000$, we solve iteratively to find MDE ≈ 0.0165 (1.65 percentage points), corresponding to 16.5% relative lift.

Interpretation: This experiment can reliably detect effects of 16.5% or larger, but will likely miss smaller effects.

Listing 4.4: MDE Calculator

```
def calculate_mde(p1: float, n: int,
                  alpha: float = 0.05,
                  power: float = 0.80) -> dict:
    """
␣␣␣␣Calculate␣minimum␣detectable␣effect␣given␣sample␣size.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣p1␣:␣float
␣␣␣␣␣␣␣␣Baseline␣conversion␣rate
␣␣␣␣n␣:␣int
␣␣␣␣␣␣␣␣Sample␣size␣per␣group
␣␣␣␣alpha␣:␣float
```

```
␣␣␣␣␣␣␣␣Significance␣level
␣␣␣␣power␣:␣float
␣␣␣␣␣␣␣␣Desired␣power

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣MDE␣statistics
␣␣␣␣"""
    from scipy import stats

    z_alpha = stats.norm.ppf(1 - alpha/2)
    z_beta = stats.norm.ppf(power)

    # Binary search for MDE
    delta_low, delta_high = 0.0001, 1.0
    epsilon = 0.0001

    while delta_high - delta_low > epsilon:
        delta_mid = (delta_low + delta_high) / 2
        p2 = p1 + delta_mid

        # Calculate power for this effect size
        actual_power = calculate_power_proportions(p1, p2, n, alpha)

        if actual_power < power:
            delta_low = delta_mid
        else:
            delta_high = delta_mid

    mde_absolute = delta_high
    mde_relative = mde_absolute / p1

    return {
        'p1': p1,
        'n_per_group': n,
        'mde_absolute': mde_absolute,
        'mde_relative': mde_relative,
        'p2': p1 + mde_absolute,
        'alpha': alpha,
        'power': power
    }


def mde_sensitivity_analysis(p1: float, sample_sizes: List[int],
                             alpha: float = 0.05, power: float = 0.80):
    """
␣␣␣␣Analyze␣how␣MDE␣changes␣with␣sample␣size.
␣␣␣␣"""
    print("="*80)
    print(f"MDE␣Sensitivity␣Analysis␣(baseline␣=␣{p1:.1%},␣  ␣=␣{alpha
        },␣power␣=␣{power:.0%})")
    print("="*80)
    print(f"{'n␣per␣group':<15}␣{'MDE␣(abs)':<12}␣{'MDE␣(rel)':<12}␣"
          f"{'p2':<10}␣{'Feasibility':<20}")
    print("-"*80)

    for n in sample_sizes:
        result = calculate_mde(p1, n, alpha, power)
```

```
        feasibility = assess_mde_feasibility(result['mde_relative'])

        print(f"{n:<14,} {result['mde_absolute']:<11.4f} "
              f"{result['mde_relative']:<11.1%} {result['p2']:<9.3f} {
                  feasibility:<20}")

    print("="*80 + "\n")


def assess_mde_feasibility(mde_relative: float) -> str:
    """Assess whether MDE is practically feasible."""
    if mde_relative < 0.05:
        return "Very ambitious"
    elif mde_relative < 0.10:
        return "Ambitious"
    elif mde_relative < 0.20:
        return "Moderate"
    else:
        return "Conservative"


# Example usage
print("\n" + "="*80)
print("Minimum Detectable Effect (MDE) Analysis")
print("="*80 + "\n")

# Example 1: What can we detect with available traffic?
print("Example 1: MDE for Given Sample Size")
print("-"*80)
result = calculate_mde(p1=0.10, n=5000, alpha=0.05, power=0.80)
print(f"Baseline conversion:   {result['p1']:.1%}")
print(f"Sample size per group: {result['n_per_group']:,}")
print(f"MDE (absolute):        {result['mde_absolute']:.4f} ({result['
    mde_absolute']*100:.2f} pp)")
print(f"MDE (relative):        {result['mde_relative']:.1%}")
print(f"Detectable p2:         {result['p2']:.3f}")
print()

# Example 2: Sensitivity analysis
print("Example 2: How MDE Changes with Sample Size")
print("-"*80)
mde_sensitivity_analysis(
    p1=0.10,
    sample_sizes=[1000, 2000, 5000, 10000, 20000, 50000],
    alpha=0.05,
    power=0.80
)

# Visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: MDE vs sample size
sample_sizes = np.logspace(2.5, 5, 50)
mdes_abs = []
mdes_rel = []

for n in sample_sizes:
    n_int = int(n)
```

```
    result = calculate_mde(p1=0.10, n=n_int, alpha=0.05, power=0.80)
    mdes_abs.append(result['mde_absolute'])
    mdes_rel.append(result['mde_relative'])

axes[0].plot(sample_sizes, np.array(mdes_abs)*100, linewidth=2.5, color='steelblue')
axes[0].set_xlabel('Sample Size (per group)', fontsize=12)
axes[0].set_ylabel('MDE (percentage points)', fontsize=12)
axes[0].set_title('Minimum Detectable Effect vs Sample Size\n(baseline = 10%,  =0.05, power=0.80)',
                  fontsize=12, fontweight='bold')
axes[0].set_xscale('log')
axes[0].grid(alpha=0.3)

axes[1].plot(sample_sizes, np.array(mdes_rel)*100, linewidth=2.5, color='coral')
axes[1].set_xlabel('Sample Size (per group)', fontsize=12)
axes[1].set_ylabel('MDE (relative lift %)', fontsize=12)
axes[1].set_title('Relative MDE vs Sample Size\n(baseline = 10%, =0.05, power=0.80)',
                  fontsize=12, fontweight='bold')
axes[1].set_xscale('log')
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('mde_analysis.pdf', dpi=300, bbox_inches='tight')
plt.show()
```

4.5.2 Business Significance vs. Statistical Significance

Statistical significance ($p < 0.05$) doesn't imply business significance. Sample size planning must consider minimum *practically important* differences.

Framework for determining MDE:

1. **Business value:** What's the smallest improvement worth implementing?
   - Consider implementation costs
   - Account for risks
   - Factor in opportunity costs
2. **Statistical feasibility:** Can we detect that difference with available traffic?
   - Calculate required sample size
   - Estimate experiment duration
   - Assess if timeline is acceptable
3. **Reconciliation:** If desired MDE requires impractical sample size:
   - Increase traffic allocation
   - Run experiment longer
   - Accept lower power
   - Reconsider if test is worthwhile

Example 4.5 (Business vs. Statistical Significance)**.** E-commerce company testing checkout redesign:

Business analysis:

- Current conversion: 10%
- Average order value: \$100
- Monthly users: 100,000
- Implementation cost: \$50,000

Break-even calculation:

- Need \$50,000 / (\$100 × 100,000/month) = 5% conversion lift to break even in 1 month
- Absolute lift: 0.5 percentage points
- This becomes minimum practically important difference

Sample size for 0.5pp lift:

- $n = 14,732$ per group (from earlier calculation)
- At 50,000 users/month per group: 9 days
- Feasible!

If implementation cost were \$500,000, break-even requires 50% lift—likely infeasible.

4.5.3 Cost-Benefit Analysis of Sample Sizes

Larger samples increase power but have costs: longer experiments, more users exposed to potentially inferior variants, delayed decisions.

Cost factors:

- **Opportunity cost:** Delayed rollout of winning variant
- **Exposure cost:** Users experiencing losing variant
- **Resource cost:** Engineering time, infrastructure

Benefit factors:

- **Reduced Type II error:** Lower risk of missing beneficial changes
- **More precise estimates:** Better understanding of effect magnitude
- **Subgroup analysis:** Sufficient power for heterogeneous effects

Optimal sample size balances these factors. Standard power = 0.80 represents conventional wisdom, but context-specific optimization may justify different choices.

4.5.4 Duration Estimation

Sample size determines experiment duration given traffic volume.

$$\text{Duration (days)} = \frac{n_{\text{total}}}{\text{users per day} \times \text{allocation fraction}} \tag{4.25}$$

Considerations:

- **Day-of-week effects:** Run full weekly cycles
- **Seasonality:** Avoid atypical periods
- **Minimum duration:** At least 1 week to capture patterns
- **Maximum duration:** Product changes, external events limit feasibility

Example 4.6 (Duration Calculation)**.** Required sample: 20,000 per group (40,000 total)

Scenario 1: High traffic site

- Daily users: 50,000
- 50/50 split: 25,000 per group per day
- Duration: 20,000 / 25,000 = 0.8 days
- **Decision:** Run for 1 week to capture weekly patterns

Scenario 2: Medium traffic site

- Daily users: 2,000
- 50/50 split: 1,000 per group per day
- Duration: 20,000 / 1,000 = 20 days
- **Decision:** Acceptable, run for 3 weeks (21 days)

Scenario 3: Low traffic site

- Daily users: 200
- 50/50 split: 100 per group per day
- Duration: 20,000 / 100 = 200 days
- **Decision:** Infeasible! Consider:
  - Accept larger MDE (smaller sample)
  - Lower power requirement
  - Don't run experiment (too uncertain)

4.6 Advanced Topics

4.6.1 Unequal Sample Sizes

Sometimes unequal allocation is optimal or necessary.

Reasons for unequal allocation:

- Limited traffic to treatment variant
- Risk mitigation (keep most users on proven control)
- Cost asymmetry (treatment is expensive)

Theorem 4.5 (Optimal Allocation Ratio)**.** For fixed total sample size $N = n_1 + n_2$, power is maximized when:

$$\frac{n_2}{n_1} = \frac{\sigma_2}{\sigma_1} \tag{4.26}$$

For proportions with similar rates, equal allocation (1:1) is nearly optimal.

For highly unequal allocation, efficiency loss is:

$$\text{Efficiency} = \frac{4r}{(1+r)^2} \tag{4.27}$$

where $r = n_2/n_1$.

Example 4.7 (Allocation Efficiency)**.**

- 1:1 (50/50): Efficiency = 1.00 (optimal)
- 2:1 (67/33): Efficiency = 0.89 (11% loss)
- 3:1 (75/25): Efficiency = 0.75 (25% loss)
- 9:1 (90/10): Efficiency = 0.36 (64% loss!)

Practical recommendation: Avoid allocation more extreme than 2:1 unless necessary. Efficiency loss outweighs most benefits.

4.6.2 Sequential Testing Implications

Classical sample size formulas assume:

- Fixed sample size determined in advance
- Single analysis at experiment conclusion

Sequential testing (peeking at results, stopping early) inflates Type I error rate. Naïve peeking can increase α from 0.05 to 0.30!

Solutions:

- **Alpha spending:** Adjust significance thresholds for multiple looks
- **Sequential probability ratio test (SPRT):** Optimal sequential procedure
- **Group sequential design:** Pre-specified analysis points with adjusted α

Sample sizes for sequential testing are **not** determined by classical formulas. Chapter 8 covers sequential methods.

4.6.3 Multiple Metrics Considerations

Testing multiple metrics simultaneously requires adjustment for multiple comparisons.

Approaches:

1. **Primary metric only:** Design for power on single most important metric
   - Simple, clear decision rule
   - Risk: Miss effects on other metrics
2. **Family-wise error rate (FWER):** Control probability of any false positive
   - Bonferroni: Increase sample size by factor of $\sqrt{m}$ for m metrics
   - Very conservative
3. **False discovery rate (FDR):** Control expected proportion of false discoveries
   - More powerful than FWER
   - Appropriate when some false positives acceptable
4. **Guardrail metrics:** Primary metric at $\alpha = 0.05$, secondary metrics at $\alpha = 0.10$ or 0.20
   - Practical compromise
   - Flags potential issues without requiring huge samples

Practical recommendation: Design for adequate power on primary metric. Use secondary metrics as diagnostics with less stringent thresholds. See Chapter 6 for comprehensive treatment.

4.7 Implementation Tools

4.7.1 Python Implementation

We've developed comprehensive functions. Here's a unified calculator:

Listing 4.5: Comprehensive Sample Size Calculator

```
class ABTestCalculator:
    """
␣␣␣␣Comprehensive␣A/B␣test␣sample␣size␣and␣power␣calculator.
␣␣␣␣"""

    def __init__(self, alpha: float = 0.05, power: float = 0.80):
        """
␣␣␣␣␣␣␣␣Initialize␣calculator␣with␣default␣parameters.

␣␣␣␣␣␣␣␣Parameters:
␣␣␣␣␣␣␣␣-----------
␣␣␣␣␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣␣␣␣␣Significance␣level␣(default␣0.05)
␣␣␣␣␣␣␣␣power␣:␣float
␣␣␣␣␣␣␣␣␣␣␣␣Statistical␣power␣(default␣0.80)
␣␣␣␣␣␣␣␣"""
```

```
        self.alpha = alpha
        self.power = power

    def sample_size_proportions(self, p1: float, p2: float,
                                ratio: float = 1.0) -> dict:
        """Calculate sample size for proportion test."""
        return sample_size_proportions(p1, p2, self.alpha, self.power,
            ratio)

    def sample_size_means(self, mu1: float, mu2: float, sigma: float,
                          ratio: float = 1.0) -> dict:
        """Calculate sample size for means test."""
        return sample_size_means(mu1, mu2, sigma, self.alpha, self.
            power, ratio)

    def calculate_power(self, p1: float, p2: float, n: int) -> float:
        """Calculate power for given parameters."""
        return calculate_power_proportions(p1, p2, n, self.alpha)

    def calculate_mde(self, p1: float, n: int) -> dict:
        """Calculate minimum detectable effect."""
        return calculate_mde(p1, n, self.alpha, self.power)

    def recommend_sample_size(self, p1: float, desired_lift: float,
                              traffic_per_day: int,
                              max_duration_days: int = 30) -> dict:
        """
        Provide sample size recommendation with practical constraints.

        Parameters:
        -----------
        p1 : float
            Baseline conversion rate
        desired_lift : float
            Desired relative lift to detect
        traffic_per_day : int
            Available users per day
        max_duration_days : int
            Maximum acceptable experiment duration

        Returns:
        --------
        dict : Recommendation with feasibility assessment
        """
        p2 = p1 * (1 + desired_lift)

        # Calculate required sample size
        result = self.sample_size_proportions(p1, p2)

        # Calculate duration
        users_per_group_per_day = traffic_per_day / 2
        duration_days = np.ceil(result['n1'] / users_per_group_per_day)

        # Feasibility assessment
        feasible = duration_days <= max_duration_days

        # If not feasible, calculate achievable MDE
        if not feasible:
```

```
            max_n = int(users_per_group_per_day * max_duration_days)
            achievable_mde = self.calculate_mde(p1, max_n)
        else:
            achievable_mde = None

        return {
            'required_n_per_group': result['n1'],
            'required_total_n': result['n_total'],
            'duration_days': int(duration_days),
            'duration_weeks': int(np.ceil(duration_days / 7)),
            'feasible': feasible,
            'achievable_mde': achievable_mde,
            'recommendation': self._generate_recommendation(
                feasible, duration_days, max_duration_days,
                    achievable_mde
            ),
            'details': result
        }

    def _generate_recommendation(self, feasible: bool, duration: float,
                                 max_duration: float, achievable_mde:
                                    dict) -> str:
        """Generate human-readable recommendation."""
        if feasible:
            return (f"Experiment is FEASIBLE. Run for {int(np.ceil(
                duration/7))} weeks "
                    f"({int(duration)} days) to achieve desired power.")
        else:
            if achievable_mde:
                return (f"Experiment NOT FEASIBLE within {max_duration}
                    days. "
                        f"Can only detect {achievable_mde['mde_relative
                            ']:.1%} lift "
                        f"with available traffic. Consider: (1)
                            Accepting larger MDE, "
                        f"(2) Running longer, (3) Increasing traffic
                            allocation, "
                        f"(4) Not running experiment.")
            else:
                return "Experiment NOT FEASIBLE. Insufficient traffic."


# Example: Comprehensive analysis
print("\n" + "="*80)
print("Comprehensive A/B Test Sample Size Analysis")
print("="*80 + "\n")

# Initialize calculator
calc = ABTestCalculator(alpha=0.05, power=0.80)

# Scenario: E-commerce checkout optimization
print("Scenario: E-commerce Checkout Redesign")
print("-"*80)
print("Current conversion rate: 10%")
print("Desired lift to detect: 10% (relative)")
print("Daily traffic: 10,000 users")
print("Maximum acceptable duration: 30 days")
print()
```

```

recommendation = calc.recommend_sample_size(
    p1=0.10,
    desired_lift=0.10,
    traffic_per_day=10000,
    max_duration_days=30
)

print("ANALYSIS␣RESULTS:")
print("-"*80)
print(f"Required␣sample␣size:␣{recommendation['required_n_per_group
    ']:,}␣per␣group")
print(f"Total␣required:␣{recommendation['required_total_n']:,}␣users")
print(f"Estimated␣duration:␣{recommendation['duration_days']}␣days␣({
    recommendation['duration_weeks']}␣weeks)")
print(f"Feasibility:␣{'  ␣FEASIBLE'␣if␣recommendation['feasible']␣else
    ␣'  ␣NOT␣FEASIBLE'}")
print()
print("RECOMMENDATION:")
print("-"*80)
print(recommendation['recommendation'])
print()

# Additional scenarios
print("\nComparing␣Different␣Traffic␣Levels:")
print("="*80)

for daily_traffic in [1000, 5000, 10000, 50000]:
    rec = calc.recommend_sample_size(
        p1=0.10,
        desired_lift=0.10,
        traffic_per_day=daily_traffic,
        max_duration_days=30
    )

    print(f"\nDaily␣traffic:␣{daily_traffic:,}")
    print(f"␣␣Duration:␣{rec['duration_days']}␣days")
    print(f"␣␣Feasible:␣{'Yes'␣if␣rec['feasible']␣else␣'No'}")
    if not rec['feasible'] and rec['achievable_mde']:
        print(f"␣␣Achievable␣MDE:␣{rec['achievable_mde']['mde_relative
            ']:.1%}")
```

4.7.2 R Implementation

R provides excellent power analysis tools through the `pwr` package.

Listing 4.6: Sample Size Calculation in R

```
# Install and load required packages
install.packages("pwr")
library(pwr)

# 1. Sample size for two proportions
# Testing 10% vs 11% conversion
p1 <- 0.10
p2 <- 0.11

```

```
# Calculate effect size (Cohen's h)
h <- ES.h(p1, p2)
cat(sprintf("Cohen's h: %.4f\n", h))

# Calculate required sample size
power_result <- pwr.2p.test(
  h = h,
  sig.level = 0.05,
  power = 0.80,
  alternative = "two.sided"
)

print(power_result)
cat(sprintf("\nRequired n per group: %.0f\n", ceiling(power_result$n)))
cat(sprintf("Total n: %.0f\n", ceiling(power_result$n * 2)))

# 2. Sample size for means
# Testing $50 vs $55 with SD = $30
mu1 <- 50
mu2 <- 55
sigma <- 30

# Cohen's d
d <- (mu2 - mu1) / sigma
cat(sprintf("\nCohen's d: %.4f\n", d))

# Calculate required sample size
power_result_means <- pwr.t.test(
  d = d,
  sig.level = 0.05,
  power = 0.80,
  type = "two.sample",
  alternative = "two.sided"
)

print(power_result_means)

# 3. Power curve
p1_vals <- seq(0.05, 0.30, 0.05)
lift <- 0.10
sample_sizes <- c(500, 1000, 2000, 5000)

power_matrix <- matrix(NA, nrow = length(p1_vals), ncol = length(sample
    _sizes))
colnames(power_matrix) <- paste0("n=", sample_sizes)
rownames(power_matrix) <- paste0("p1=", p1_vals)

for (i in seq_along(p1_vals)) {
  p1 <- p1_vals[i]
  p2 <- p1 * (1 + lift)
  h <- ES.h(p1, p2)

  for (j in seq_along(sample_sizes)) {
    n <- sample_sizes[j]
    result <- pwr.2p.test(
      h = h,
      n = n,
      sig.level = 0.05,
```

```r
      alternative = "two.sided"
    )
    power_matrix[i, j] <- result$power
  }
}

print(round(power_matrix, 3))

# 4. Plot power curve
plot_power_curve <- function(p1, lift_range, n, alpha = 0.05) {
  powers <- numeric(length(lift_range))

  for (i in seq_along(lift_range)) {
    p2 <- p1 * (1 + lift_range[i])
    h <- ES.h(p1, p2)
    result <- pwr.2p.test(h = h, n = n, sig.level = alpha)
    powers[i] <- result$power
  }

  plot(lift_range * 100, powers,
       type = "l", lwd = 2,
       xlab = "Relative Lift (%)",
       ylab = "Statistical Power",
       main = sprintf("Power Curve (baseline = %.1f%%, n = %d)", p1 *
          100, n))
  abline(h = 0.80, col = "red", lty = 2)
  grid()
}

# Generate power curve
lift_range <- seq(0.05, 0.50, 0.01)
plot_power_curve(p1 = 0.10, lift_range = lift_range, n = 5000)
```

4.8 Summary

Sample size determination transforms experimental planning from guesswork to rigorous calculation:

- **Power analysis** connects sample size, effect size, significance level, and power. Specifying any three determines the fourth.
- **Standard conventions** ($\alpha = 0.05$, power $= 0.80$) represent reasonable defaults but should be adjusted for context.
- **Sample size formulas** differ by metric type. Proportions and means have closed-form solutions; other metrics require simulation or approximation.
- **Minimum detectable effect** (MDE) defines the smallest effect reliably detectable. MDE must be both statistically feasible and business-relevant.
- **Practical constraints**—traffic volume, acceptable duration, implementation costs—often dominate pure statistical considerations.
- **Advanced topics**—unequal allocation, sequential testing, multiple metrics—require specialized methods beyond classical formulas.

- **Implementation tools** in Python and R enable rapid calculation and sensitivity analysis.

Proper sample size calculation ensures experiments are neither underpowered (wasting time and resources) nor overpowered (unnecessarily delaying decisions). Master these techniques to design efficient, informative experiments.

4.9 Further Reading

- **?**: Definitive reference on statistical power analysis
- **?**: Sample size methods for clinical research
- **?**: Practical guidance for online experiments
- **?**: Documentation for R's pwr package

4.10 Exercises

4.10.1 Conceptual Questions

1. Explain the relationship between Type I error, Type II error, power, and significance level. How are they connected?
2. Why is power = 0.80 a common standard? Under what circumstances would you use power = 0.90? Power = 0.70?
3. A colleague says: "Let's just run the test for a month and see what happens." What's wrong with this approach?
4. Explain why doubling sample size doesn't double statistical power. What's the actual relationship?
5. Describe three scenarios where unequal allocation (e.g., 75/25 split) might be justified despite efficiency loss.
6. What's the difference between minimum detectable effect and minimum practically important difference? Why do both matter?

4.10.2 Calculation Exercises

1. Calculate required sample size for testing:
   - Baseline conversion: 5%
   - Target conversion: 6% (20% relative lift)
   - $\alpha = 0.05$, power = 0.80

   Show all work. How long would this experiment take with 10,000 users/day?
2. Repeat previous calculation for power = 0.90. By what factor does required sample size increase?

3. Given 3,000 users per group available, baseline conversion 15%, calculate the minimum detectable effect (MDE) for $\alpha = 0.05$, power = 0.80.

4. Calculate required sample size for continuous metric:

   - Control mean: \$75, SD: \$40
   - Target mean: \$85 (13.3% increase)
   - $\alpha = 0.05$, power = 0.80

5. Compare sample size requirements for detecting 10% relative lift at baseline rates of 2%, 10%, 30%, and 50%. What pattern emerges?

4.10.3 Programming Exercises

1. Implement a comprehensive power analysis function that:

   - Accepts multiple input formats (absolute difference, relative lift, Cohen's h)
   - Supports both one-sided and two-sided tests
   - Handles unequal allocation ratios
   - Provides clear warnings about assumptions
   - Includes input validation

2. Create an interactive sample size calculator using Streamlit or Shiny that:

   - Takes baseline metrics and desired lift as input
   - Shows power curves dynamically
   - Estimates experiment duration given traffic
   - Provides feasibility recommendations
   - Exports results as PDF report

3. Write a function to perform sensitivity analysis:

   - How does power change if true effect is 50% smaller/larger than assumed?
   - How does power change if variance is 20% higher than expected?
   - Generate visualization showing sensitivity regions

4. Implement simulation-based power calculation for:

   - Ratio metrics (revenue per user)
   - Count metrics (purchases per customer)
   - Mixture distributions (users who convert vs. don't convert)

   Compare simulation results to closed-form approximations.

4.10.4 Applied Exercises

1. Design experiments for these scenarios with full justification:

   Scenario A: SaaS company testing onboarding flow

   - Current trial-to-paid conversion: 8%
   - 500 new trials per week
   - Implementation cost: $100,000
   - Average customer lifetime value: $5,000

   Scenario B: News website testing article layout

   - Current average time on page: 2.5 minutes (SD: 3.0 minutes)
   - 1 million pageviews per day
   - Easy to implement and reverse

   Scenario C: Mobile app testing premium feature

   - Current premium conversion: 2%
   - 10,000 daily active users
   - Major engineering effort ($500K)

   For each: Specify desired lift, calculate sample size, estimate duration, assess feasibility, provide recommendation.

2. A stakeholder insists on detecting 2% relative lift with 95% power. Your analysis shows this requires 100,000 users per group but only 50,000 total are available. Prepare a memo explaining:

   - Why their requirement is infeasible
   - What can be achieved with available sample
   - Alternative approaches (sequential testing, focus on larger effects, etc.)
   - Recommended path forward

3. Review a past A/B test that failed to reach significance. Using actual baseline metrics and observed difference:

   - Calculate what power the experiment actually had
   - Determine what sample size would have been needed for 80% power
   - Assess whether observed effect was below MDE
   - Recommend whether to rerun with larger sample or abandon

4.10.5 Advanced Projects

1. **Sequential testing sample size:** Implement group sequential design with:
   - O'Brien-Fleming spending function
   - 3-5 planned interim analyses
   - Calculate required sample size accounting for multiple looks
   - Compare to fixed sample size
2. **Multiple metrics:** Design experiment testing 1 primary and 4 secondary metrics:
   - Determine sample size for each metric individually
   - Apply Bonferroni, Holm, and FDR corrections
   - Compare required sample sizes
   - Recommend practical approach
3. **Variance reduction impact:** Simulate experiments with and without CUPED variance reduction:
   - Generate correlated pre/post data
   - Compare power with/without covariate adjustment
   - Quantify sample size savings
   - Visualize power gains
4. **Bayesian sample size determination:** Implement Bayesian approach:
   - Specify prior distribution for effect size
   - Calculate probability of success for various sample sizes
   - Compare to frequentist power analysis
   - Discuss advantages and limitations

Chapter 5

Hypothesis Testing for A/B Tests

5.1 Learning Objectives

After completing this chapter, readers will be able to:

- Conduct z-tests and t-tests for A/B testing scenarios
- Interpret p-values correctly and avoid common misinterpretations
- Apply chi-squared tests for categorical outcomes
- Use permutation tests as non-parametric alternatives
- Understand assumptions underlying different test procedures
- Implement hypothesis tests in Python and R

5.2 The Framework of Hypothesis Testing

Hypothesis testing provides a formal procedure for making decisions under uncertainty. While we introduced the concept earlier, this chapter provides rigorous treatment of test procedures for A/B testing.

5.2.1 Structure of a Hypothesis Test

1. **State hypotheses:**
    - Null hypothesis H_0: No effect or no difference
    - Alternative hypothesis H_1: An effect exists
2. **Choose significance level:** α (typically 0.05 or 0.01)
3. **Calculate test statistic:** Function of data measuring evidence against H_0
4. **Determine p-value or critical region:**
    - P-value: Probability of observing data this extreme under H_0
    - Critical region: Values of test statistic leading to rejection

5. **Make decision:**
   - Reject H_0 if p-value $< \alpha$ (or test statistic in critical region)
   - Fail to reject H_0 otherwise
6. **Interpret in context:** Translate statistical conclusion to practical meaning

5.2.2 One-Sided vs. Two-Sided Tests

Two-sided test:

$$H_0 : \mu_1 = \mu_2 \quad \text{vs.} \quad H_1 : \mu_1 \neq \mu_2 \tag{5.1}$$

Tests for any difference (positive or negative). Most common in A/B testing.

One-sided test:

$$H_0 : \mu_1 \geq \mu_2 \quad \text{vs.} \quad H_1 : \mu_1 < \mu_2 \tag{5.2}$$

Tests specifically for improvement (or degradation). More powerful but only detects effects in specified direction.

When to use one-sided:

- Only care about improvements (would not implement if treatment is worse)
- Regulatory or business constraints dictate directionality
- Preregistered directional hypothesis

Caution: One-sided tests are often misused. Default to two-sided unless strong justification exists.

5.3 Z-Test for Proportions

The z-test compares proportions between two groups using normal approximation.

5.3.1 Two-Sample Z-Test

For control proportion $\hat{p}_1 = x_1/n_1$ and treatment proportion $\hat{p}_2 = x_2/n_2$:

$$H_0 : p_1 = p_2 \quad \text{vs.} \quad H_1 : p_1 \neq p_2 \tag{5.3}$$

Under H_0, we use the pooled proportion:

$$\hat{p}_{\text{pooled}} = \frac{x_1 + x_2}{n_1 + n_2} \tag{5.4}$$

Test statistic:

$$Z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\hat{p}_{\text{pooled}}(1 - \hat{p}_{\text{pooled}})(1/n_1 + 1/n_2)}} \tag{5.5}$$

Under H_0: $Z \sim \mathcal{N}(0, 1)$ approximately.

P-value (two-sided): $p = 2(1 - \Phi(|Z|))$ where Φ is the standard normal CDF.

Decision rule: Reject H_0 if $|Z| > z_{\alpha/2}$ (equivalently, if $p < \alpha$).

Example 5.1 (Conversion Rate Test)**.**

$$\text{Control: } n_1 = 2000, x_1 = 160, \hat{p}_1 = 0.080$$
$$\text{Treatment: } n_2 = 2000, x_2 = 190, \hat{p}_2 = 0.095$$

Pooled proportion:

$$\hat{p}_{\text{pooled}} = \frac{160 + 190}{2000 + 2000} = \frac{350}{4000} = 0.0875 \tag{5.6}$$

Standard error:

$$SE = \sqrt{0.0875 \times 0.9125 \times (1/2000 + 1/2000)} = 0.00894 \tag{5.7}$$

Test statistic:

$$Z = \frac{0.095 - 0.080}{0.00894} = 1.68 \tag{5.8}$$

P-value: $p = 2(1 - \Phi(1.68)) = 2(1 - 0.9535) = 0.093$

At $\alpha = 0.05$, we fail to reject H_0. The observed difference is not statistically significant.

Listing 5.1: Z-Test Implementation

```python
import numpy as np
from scipy.stats import norm

def z_test_proportions(x1, n1, x2, n2, alpha=0.05, two_sided=True):
    """
    Perform z-test for comparing two proportions.

    Parameters:
    -----------
    x1, x2 : int
        Number of successes in each group
    n1, n2 : int
        Sample sizes
    alpha : float
        Significance level
    two_sided : bool
        Perform two-sided test (default True)

    Returns:
    --------
    dict : Test results including statistic, p-value, decision
    """
    # Sample proportions
    p1_hat = x1 / n1
    p2_hat = x2 / n2

    # Pooled proportion under null hypothesis
    p_pooled = (x1 + x2) / (n1 + n2)

    # Standard error under null
    se = np.sqrt(p_pooled * (1 - p_pooled) * (1/n1 + 1/n2))

    # Test statistic
```

```
    z_stat = (p2_hat - p1_hat) / se

    # P-value
    if two_sided:
        p_value = 2 * (1 - norm.cdf(abs(z_stat)))
    else:
        p_value = 1 - norm.cdf(z_stat)

    # Decision
    reject_null = p_value < alpha

    # Confidence interval (using unpooled variance)
    se_unpooled = np.sqrt(p1_hat*(1-p1_hat)/n1 + p2_hat*(1-p2_hat)/n2)
    z_crit = norm.ppf(1 - alpha/2) if two_sided else norm.ppf(1 - alpha
        )
    ci_lower = (p2_hat - p1_hat) - z_crit * se_unpooled
    ci_upper = (p2_hat - p1_hat) + z_crit * se_unpooled

    return {
        'p1_hat': p1_hat,
        'p2_hat': p2_hat,
        'difference': p2_hat - p1_hat,
        'z_statistic': z_stat,
        'p_value': p_value,
        'alpha': alpha,
        'reject_null': reject_null,
        'ci_lower': ci_lower,
        'ci_upper': ci_upper
    }

# Example
result = z_test_proportions(x1=160, n1=2000, x2=190, n2=2000)

print(f"Control conversion: {result['p1_hat']:.3f}")
print(f"Treatment conversion: {result['p2_hat']:.3f}")
print(f"Difference: {result['difference']:.3f}")
print(f"Z-statistic: {result['z_statistic']:.3f}")
print(f"P-value: {result['p_value']:.4f}")
print(f"Reject null: {result['reject_null']}")
print(f"95% CI: [{result['ci_lower']:.4f}, {result['ci_upper']:.4f}]")
```

5.3.2 Assumptions and Validity

The z-test relies on:

1. **Independence:** Observations are independent within and between groups
2. **Random sampling:** Groups formed by randomization
3. **Large samples:** Normal approximation valid

   Rule of thumb: $n_i p_i > 5$ and $n_i(1 - p_i) > 5$ for $i = 1, 2$.

When assumptions are violated, consider:

- Exact tests (Fisher's exact test) for small samples

- Permutation tests (robust to distributional assumptions)
- Adjustments for clustering or correlation

5.4 T-Test for Means

When comparing continuous outcomes with unknown variance, the t-test is appropriate.

5.4.1 Two-Sample T-Test

For means $\bar{X}_1$ and $\bar{X}_2$ from groups with variances s_1^2 and s_2^2:

$$H_0 : \mu_1 = \mu_2 \quad \text{vs.} \quad H_1 : \mu_1 \neq \mu_2 \tag{5.9}$$

Equal variances (pooled t-test):
Pooled variance:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2} \tag{5.10}$$

Test statistic:

$$t = \frac{\bar{X}_2 - \bar{X}_1}{s_p\sqrt{1/n_1 + 1/n_2}} \tag{5.11}$$

Distribution under H_0: $t \sim t_{n_1+n_2-2}$
Unequal variances (Welch's t-test):
Test statistic:

$$t = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{s_1^2/n_1 + s_2^2/n_2}} \tag{5.12}$$

Degrees of freedom (Welch-Satterthwaite):

$$\nu = \frac{(s_1^2/n_1 + s_2^2/n_2)^2}{\frac{(s_1^2/n_1)^2}{n_1-1} + \frac{(s_2^2/n_2)^2}{n_2-1}} \tag{5.13}$$

When to use which:

- Welch's t-test is more robust and generally recommended
- Use pooled when variances are truly equal (rare in practice)

Example 5.2 (Revenue Per User).

$$\text{Control: } n_1 = 500, \bar{X}_1 = 45.2, s_1 = 28.3$$
$$\text{Treatment: } n_2 = 500, \bar{X}_2 = 48.7, s_2 = 31.1$$

Using Welch's t-test:
Standard error:

$$SE = \sqrt{\frac{28.3^2}{500} + \frac{31.1^2}{500}} = \sqrt{1.603 + 1.934} = 1.88 \tag{5.14}$$

Test statistic:

$$t = \frac{48.7 - 45.2}{1.88} = 1.86 \tag{5.15}$$

Degrees of freedom ≈ 982.
For t_{982}, $t \approx \mathcal{N}(0, 1)$, so $p \approx 0.063$.
Not significant at $\alpha = 0.05$.

Listing 5.2: T-Test in R

```r
# Simulate A/B test data
set.seed(123)
control <- rnorm(500, mean = 45.2, sd = 28.3)
treatment <- rnorm(500, mean = 48.7, sd = 31.1)

# Welch's t-test (default in R)
test_result <- t.test(treatment, control, var.equal = FALSE)

print(test_result)

# Extract key components
cat("\nTest statistic:", test_result$statistic, "\n")
cat("P-value:", test_result$p.value, "\n")
cat("95% CI:", test_result$conf.int, "\n")

# Pooled t-test (for comparison)
test_pooled <- t.test(treatment, control, var.equal = TRUE)
cat("\nPooled t-test p-value:", test_pooled$p.value, "\n")
```

5.5 Understanding P-Values

Definition 5.1 (P-Value)**.** The p-value is the probability, under the null hypothesis, of observing a test statistic at least as extreme as the one actually observed.

5.5.1 Correct Interpretation

A p-value of 0.03 means:

- ✓If H_0 were true, we would observe data this extreme in 3% of experiments
- ✓The data are somewhat inconsistent with H_0

5.5.2 Common Misinterpretations

1. **WRONG:** "The probability that H_0 is true is 0.03"

   P-values are NOT the probability of hypotheses. They assume H_0 is true.

2. **WRONG:** "The probability that the result occurred by chance is 0.03"

   Everything has some probability; this confuses conditional probabilities.

3. **WRONG:** "A p-value of 0.049 is meaningfully different from 0.051"

   The $\alpha = 0.05$ threshold is arbitrary. Treat p-values as continuous measures of evidence, not binary outcomes.

4. **WRONG:** "A non-significant result means no effect exists"

   Absence of evidence is not evidence of absence. May lack power to detect small effects.

5.5.3 Statistical Significance vs. Practical Significance

Statistical significance ($p < \alpha$) does not imply practical importance.

Example 5.3 (Large Sample Paradox)**.** With $n = 100{,}000$ per variant, a conversion rate increase from 10.00% to 10.05% (0.05 percentage points) may be highly statistically significant ($p < 0.001$) but economically negligible.

Conversely, with $n = 100$, a 5 percentage point increase may not reach significance despite clear practical importance.

Recommendation: Report effect sizes and confidence intervals alongside p-values. Make decisions based on practical significance, informed by statistical uncertainty.

5.6 Chi-Squared Test for Categorical Data

The chi-squared test assesses association between categorical variables.

5.6.1 Pearson's Chi-Squared Test

For 2×2 contingency table:

	Converted	**Not Converted**	**Total**
Control	O_{11}	O_{12}	n_1
Treatment	O_{21}	O_{22}	n_2
Total	c_1	c_2	n

Expected counts under independence:

$$E_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{column } j \text{ total})}{n} \tag{5.16}$$

Test statistic:

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \tag{5.17}$$

Under H_0 of independence: $\chi^2 \sim \chi^2_{(r-1)(c-1)}$ where r and c are number of rows and columns.

For 2×2 tables: $df = 1$.

Proposition 5.1 (Equivalence of Z-Test and Chi-Squared Test)**.** For 2×2 tables, $\chi^2 = Z^2$ where Z is the z-test statistic for proportions. The tests are equivalent for two-sided alternatives.

5.6.2 Fisher's Exact Test

When sample sizes are small or expected cell counts < 5, use Fisher's exact test instead of chi-squared.

Fisher's test calculates the exact probability of the observed table and all more extreme tables under H_0, without relying on asymptotics.

Listing 5.3: Chi-Squared and Fisher's Exact Tests

```
from scipy.stats import chi2_contingency, fisher_exact
import numpy as np

# Contingency table
# Rows: Control, Treatment
# Columns: Converted, Not Converted
table = np.array([[160, 1840],
                  [190, 1810]])

# Chi-squared test
chi2_stat, p_chi2, dof, expected = chi2_contingency(table)

print("Chi-Squared Test:")
print(f"  Chi-squared statistic: {chi2_stat:.3f}")
print(f"  P-value: {p_chi2:.4f}")
print(f"  Degrees of freedom: {dof}")
print("\nExpected frequencies:")
print(expected)

# Fisher's exact test
oddsratio, p_fisher = fisher_exact(table)

print("\nFisher's Exact Test:")
print(f"  Odds ratio: {oddsratio:.3f}")
print(f"  P-value: {p_fisher:.4f}")

# Verify equivalence with z-test
from scipy.stats import norm

x1, n1 = table[0, 0], table[0].sum()
x2, n2 = table[1, 0], table[1].sum()

p1 = x1 / n1
p2 = x2 / n2
p_pooled = (x1 + x2) / (n1 + n2)

z = (p2 - p1) / np.sqrt(p_pooled * (1 - p_pooled) * (1/n1 + 1/n2))
p_z = 2 * (1 - norm.cdf(abs(z)))

print(f"\nZ-test p-value: {p_z:.4f}")
print(f"z^2 = {z**2:.3f}, chi2 = {chi2_stat:.3f}")
print(f"Difference: {abs(z**2 - chi2_stat):.6f}")
```

5.7 Permutation Tests

Permutation tests offer a non-parametric alternative requiring minimal distributional assumptions.

5.7.1 Permutation Test Procedure

1. Calculate observed test statistic T_{obs} (e.g., difference in means)

2. Under H_0 of no effect, treatment labels are exchangeable

3. Generate permutation distribution:
    - Randomly shuffle treatment assignments
    - Calculate test statistic for permuted data
    - Repeat many times (e.g., 10,000)
4. P-value: proportion of permutations where $|T| \geq |T_{\text{obs}}|$

Advantages:

- Exact finite-sample validity under randomization
- No distributional assumptions
- Works for complex test statistics

Disadvantages:

- Computationally intensive
- Less familiar to stakeholders

Listing 5.4: Permutation Test Implementation

```
import numpy as np
import matplotlib.pyplot as plt

def permutation_test(group1, group2, n_permutations=10000,
                     statistic='mean_diff', seed=None):
    """
␣␣␣␣Perform␣permutation␣test␣for␣two␣groups.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣group1,␣group2␣:␣array-like
␣␣␣␣␣␣␣␣Data␣for␣each␣group
␣␣␣␣n_permutations␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣permutations
␣␣␣␣statistic␣:␣str
␣␣␣␣␣␣␣␣Test␣statistic␣('mean_diff',␣'median_diff',␣'t_stat')
␣␣␣␣seed␣:␣int
␣␣␣␣␣␣␣␣Random␣seed

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Test␣results
␣␣␣␣"""
    if seed is not None:
        np.random.seed(seed)

    group1 = np.array(group1)
    group2 = np.array(group2)
    combined = np.concatenate([group1, group2])
    n1 = len(group1)

    # Define test statistic function
    def calc_statistic(data1, data2):
```

```
        if statistic == 'mean_diff':
            return np.mean(data2) - np.mean(data1)
        elif statistic == 'median_diff':
            return np.median(data2) - np.median(data1)
        elif statistic == 't_stat':
            from scipy.stats import ttest_ind
            return ttest_ind(data2, data1, equal_var=False).statistic

    # Observed test statistic
    obs_stat = calc_statistic(group1, group2)

    # Permutation distribution
    perm_stats = []
    for _ in range(n_permutations):
        # Shuffle combined data
        permuted = np.random.permutation(combined)
        perm_group1 = permuted[:n1]
        perm_group2 = permuted[n1:]
        perm_stat = calc_statistic(perm_group1, perm_group2)
        perm_stats.append(perm_stat)

    perm_stats = np.array(perm_stats)

    # P-value (two-sided)
    p_value = np.mean(np.abs(perm_stats) >= np.abs(obs_stat))

    return {
        'observed_statistic': obs_stat,
        'permutation_statistics': perm_stats,
        'p_value': p_value,
        'n_permutations': n_permutations
    }

# Example with simulated data
np.random.seed(42)
control = np.random.exponential(scale=50, size=500)
treatment = np.random.exponential(scale=55, size=500)

result = permutation_test(control, treatment, n_permutations=10000)

print(f"Observed difference in means: {result['observed_statistic']:.3f}")
print(f"P-value: {result['p_value']:.4f}")

# Visualize permutation distribution
plt.figure(figsize=(10, 6))
plt.hist(result['permutation_statistics'], bins=50, density=True,
         alpha=0.7, label='Permutation distribution')
plt.axvline(result['observed_statistic'], color='red',
            linestyle='--', linewidth=2, label='Observed statistic')
plt.axvline(-result['observed_statistic'], color='red',
            linestyle='--', linewidth=2)
plt.xlabel('Test Statistic (Difference in Means)')
plt.ylabel('Density')
plt.title('Permutation Test: Null Distribution')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

```
plt.savefig('permutation_distribution.pdf')
plt.show()
```

5.8 Summary

Hypothesis testing provides a rigorous framework for making decisions from experimental data:

- Z-tests compare proportions using normal approximation
- T-tests compare means with unknown variances
- P-values measure consistency of data with null hypothesis
- Statistical significance differs from practical significance
- Chi-squared tests assess categorical associations
- Permutation tests offer non-parametric alternatives

Proper application requires understanding assumptions, limitations, and correct interpretation. The next chapter addresses challenges arising from multiple comparisons.

5.9 Further Reading

- **?**: Theoretical foundations
- **?**: Comprehensive treatment of permutation tests
- **?**: ASA statement on p-values

5.10 Exercises

1. **Test Implementation**
   (a) Implement a function performing z-test, t-test, chi-squared test, and Fisher's exact test. Compare results for various scenarios.
   (b) Create visualizations showing p-value distributions under null and alternative hypotheses.
2. **P-Value Interpretation**
   (a) Explain why "probability null hypothesis is true" is incorrect interpretation.
   (b) Simulate experiments under null hypothesis. What proportion have $p < 0.05$?
3. **Power and Sample Size**
   (a) For given effect size, simulate experiments with various sample sizes. Plot empirical power vs. theoretical power.

(b) Demonstrate how increasing sample size can make trivial effects statistically significant.

4. **Permutation Tests**

   (a) Compare permutation test p-values to parametric test p-values for: normal data, exponential data, heavy-tailed data.

   (b) Implement a permutation test for ratio of means instead of difference. When might this be useful?

5. **Real Data Analysis**

   (a) Analyze a real A/B test dataset using multiple test procedures. Compare conclusions.

   (b) Investigate how outliers affect different test procedures.

Chapter 6

Multiple Testing and False Discovery Rate

6.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand why multiple testing inflates false positive rates
- Apply family-wise error rate (FWER) correction methods
- Use false discovery rate (FDR) control procedures
- Choose appropriate correction methods for different scenarios
- Handle multiple metrics and variants in A/B testing

6.2 The Multiple Testing Problem

When conducting many hypothesis tests simultaneously, the probability of at least one false positive increases dramatically—even when all null hypotheses are true.

6.2.1 Probability of False Positives

For a single test at level $\alpha = 0.05$:

$$P(\text{Type I error}) = 0.05 \tag{6.1}$$

For m independent tests, all with true null hypotheses:

$$P(\text{at least one Type I error}) = 1 - (1 - \alpha)^m \tag{6.2}$$

Example 6.1 (Inflation of False Positive Rate)**.**

Tests	**FWER** ($\alpha = 0.05$)	**FWER** ($\alpha = 0.01$)
1	0.050	0.010
5	0.226	0.049
10	0.401	0.096
20	0.642	0.182
50	0.923	0.395
100	0.994	0.634

With 20 tests, probability of at least one false positive exceeds 64%!

6.2.2 Sources of Multiplicity in A/B Testing

1. **Multiple Metrics:** Testing conversion, revenue, engagement simultaneously
2. **Multiple Variants:** A/B/C/D tests with $\binom{k}{2}$ pairwise comparisons
3. **Multiple Subgroups:** Analyzing mobile vs. desktop, new vs. returning users
4. **Multiple Time Points:** Continuous monitoring, interim analyses
5. **Multiple Experiments:** Running many tests in parallel

6.3 Family-Wise Error Rate (FWER)

Definition 6.1 (Family-Wise Error Rate)**.** The FWER is the probability of making at least one Type I error among all tests:

$$\text{FWER} = P(\text{Reject at least one true } H_0) \tag{6.3}$$

FWER control ensures strong protection against false positives but may reduce power.

6.3.1 Bonferroni Correction

The Bonferroni method is the simplest FWER control procedure.

Theorem 6.1 (Bonferroni Correction)**.** For m tests, reject null hypothesis i if $p_i < \alpha/m$. This guarantees FWER $\leq \alpha$ regardless of test dependence.

Proof: By Boole's inequality:

$$\text{FWER} = P\left(\bigcup_{i=1}^{m}\{p_i < \alpha/m\}\right) \leq \sum_{i=1}^{m} P(p_i < \alpha/m) = m \cdot \frac{\alpha}{m} = \alpha \tag{6.4}$$

Example 6.2 (Bonferroni in Practice)**.** Testing 5 metrics at overall level $\alpha = 0.05$:
Bonferroni threshold: $\alpha^* = 0.05/5 = 0.01$
Reject individual tests only if $p < 0.01$.
Observed p-values: 0.008, 0.025, 0.045, 0.003, 0.120
Reject: Tests 1 and 4 (p-values 0.008 and 0.003)
Without correction, would reject tests 1, 2, 3, and 4.

Advantages:

- Simple to implement
- Valid under arbitrary dependence
- Conservative (strong Type I error control)

Disadvantages:

- Often too conservative
- Reduces power substantially
- Treats all tests equally (no prioritization)

6.3.2 Holm-Bonferroni Method

The Holm-Bonferroni procedure is uniformly more powerful than Bonferroni while maintaining FWER control.

Algorithm 2 Holm-Bonferroni Procedure

Order p-values: $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$
for $i = 1$ to m **do**
  if $p_{(i)} \leq \alpha/(m - i + 1)$ **then**
    Reject $H_{(i)}$
  else
    Stop; retain $H_{(i)}, H_{(i+1)}, \ldots, H_{(m)}$
  end if
end for

Example 6.3 (Holm-Bonferroni Application)**.** Five p-values: 0.008, 0.025, 0.045, 0.003, 0.120

Ordered: $p_{(1)} = 0.003, p_{(2)} = 0.008, p_{(3)} = 0.025, p_{(4)} = 0.045, p_{(5)} = 0.120$
Thresholds ($\alpha = 0.05$):

$$\text{Test 1: } p_{(1)} = 0.003 \leq 0.05/5 = 0.010 \quad \checkmark \text{ Reject} \tag{6.5}$$
$$\text{Test 2: } p_{(2)} = 0.008 \leq 0.05/4 = 0.0125 \quad \checkmark \text{ Reject} \tag{6.6}$$
$$\text{Test 3: } p_{(3)} = 0.025 > 0.05/3 = 0.0167 \quad \times \text{ Stop} \tag{6.7}$$

Reject tests 1 and 2; retain 3, 4, 5.
Bonferroni would reject only tests 1 and 4, missing test 2.

Listing 6.1: Bonferroni and Holm-Bonferroni Implementation

```
import numpy as np

def bonferroni_correction(p_values, alpha=0.05):
    """
␣␣␣␣Apply␣Bonferroni␣correction.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣p_values␣:␣array-like
␣␣␣␣␣␣␣␣Observed␣p-values
␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣Family-wise␣error␣rate

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Rejection␣decisions␣and␣adjusted␣p-values
␣␣␣␣"""
    p_values = np.array(p_values)
    m = len(p_values)

    # Adjusted significance level
    alpha_bonf = alpha / m

    # Adjusted p-values (capped at 1)
```

```
    adjusted_p = np.minimum(p_values * m, 1.0)

    # Rejection decisions
    reject = p_values < alpha_bonf

    return {
        'reject': reject,
        'adjusted_p_values': adjusted_p,
        'threshold': alpha_bonf,
        'num_rejected': np.sum(reject)
    }

def holm_bonferroni(p_values, alpha=0.05):
    """
    Apply Holm-Bonferroni procedure.

    Parameters:
    -----------
    p_values : array-like
        Observed p-values
    alpha : float
        Family-wise error rate

    Returns:
    --------
    dict : Rejection decisions and adjusted p-values
    """
    p_values = np.array(p_values)
    m = len(p_values)

    # Sort p-values and track original indices
    sorted_indices = np.argsort(p_values)
    sorted_p = p_values[sorted_indices]

    # Apply Holm procedure
    reject_sorted = np.zeros(m, dtype=bool)
    for i in range(m):
        if sorted_p[i] <= alpha / (m - i):
            reject_sorted[i] = True
        else:
            break  # Stop at first non-rejection

    # Map back to original order
    reject = np.zeros(m, dtype=bool)
    reject[sorted_indices] = reject_sorted

    # Adjusted p-values
    adjusted_p = np.zeros(m)
    for i in range(m):
        adjusted_p[sorted_indices[i]] = min(
            max(sorted_p[:i+1] * (m - np.arange(i+1))),
            1.0
        )

    return {
        'reject': reject,
        'adjusted_p_values': adjusted_p,
        'num_rejected': np.sum(reject)
```

```
    }

# Example
p_values = [0.008, 0.025, 0.045, 0.003, 0.120]

bonf_result = bonferroni_correction(p_values)
holm_result = holm_bonferroni(p_values)

print("Original p-values:", p_values)
print("\nBonferroni:")
print("  Threshold:", bonf_result['threshold'])
print("  Reject:", bonf_result['reject'])
print("  Adjusted p-values:", bonf_result['adjusted_p_values'])

print("\nHolm-Bonferroni:")
print("  Reject:", holm_result['reject'])
print("  Adjusted p-values:", holm_result['adjusted_p_values'])
```

6.3.3 Šidák Correction

For independent tests, the Šidák correction is less conservative than Bonferroni.

$$\alpha_{\text{Šidák}} = 1 - (1 - \alpha)^{1/m} \tag{6.8}$$

This achieves exact FWER $= \alpha$ under independence.
For $\alpha = 0.05$ and $m = 10$:

$$\text{Bonferroni: } \alpha^* = 0.05/10 = 0.005 \tag{6.9}$$

$$\text{Šidák: } \alpha^* = 1 - (1 - 0.05)^{1/10} \approx 0.00512 \tag{6.10}$$

Slightly more powerful but requires independence assumption.

6.4 False Discovery Rate (FDR)

FWER control is conservative when testing many hypotheses. The false discovery rate offers a less stringent alternative.

Definition 6.2 (False Discovery Rate)**.** The FDR is the expected proportion of false positives among rejected hypotheses:

$$\text{FDR} = \mathbb{E}\left[\frac{\text{False Positives}}{\text{Total Rejections}}\right] \tag{6.11}$$

where the ratio is defined as 0 if no hypotheses are rejected.

FDR control is particularly useful when:

- Testing many hypotheses (e.g., $m > 20$)
- Some false positives are acceptable
- Power is important (exploratory analysis)

6.4.1 Benjamini-Hochberg Procedure

The Benjamini-Hochberg (BH) procedure controls FDR at level α.

Algorithm 3 Benjamini-Hochberg Procedure

Order p-values: $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$
Find largest k such that $p_{(k)} \leq \frac{k}{m}\alpha$
Reject hypotheses $H_{(1)}, \ldots, H_{(k)}$

Theorem 6.2 (BH FDR Control)**.** Under independence or positive dependence, the BH procedure controls FDR at level α.

Example 6.4 (Benjamini-Hochberg Application)**.** Ten p-values at $\alpha = 0.10$: 0.001, 0.008, 0.039, 0.041, 0.042, 0.060, 0.074, 0.205, 0.212, 0.216

Rank	P-value	Threshold	Decision
1	0.001	$1/10 \times 0.10 = 0.010$	Reject
2	0.008	$2/10 \times 0.10 = 0.020$	Reject
3	0.039	$3/10 \times 0.10 = 0.030$	**Fail**
4	0.041	$4/10 \times 0.10 = 0.040$	**Fail**

Largest k with $p_{(k)} \leq k/m \times \alpha$: $k = 2$
Reject tests 1 and 2.
With Bonferroni ($\alpha^* = 0.01$): reject only test 1.

Listing 6.2: FDR Control in R

```r
# Benjamini-Hochberg procedure
p_values <- c(0.001, 0.008, 0.039, 0.041, 0.042,
              0.060, 0.074, 0.205, 0.212, 0.216)

# Using built-in p.adjust
bh_adjusted <- p.adjust(p_values, method = "BH")
bonf_adjusted <- p.adjust(p_values, method = "bonferroni")
holm_adjusted <- p.adjust(p_values, method = "holm")

# Compare methods
comparison <- data.frame(
  Original = p_values,
  BH = bh_adjusted,
  Bonferroni = bonf_adjusted,
  Holm = holm_adjusted
)

print(comparison)

# Rejection decisions at alpha = 0.10
alpha <- 0.10
cat("\nRejections at alpha =", alpha, ":\n")
cat("  BH:", sum(bh_adjusted < alpha), "\n")
cat("  Bonferroni:", sum(bonf_adjusted < alpha), "\n")
cat("  Holm:", sum(holm_adjusted < alpha), "\n")
```

6.4.2 Choosing Between FWER and FDR

Consideration	FWER	FDR
Number of tests	Few ($m < 20$)	Many ($m > 20$)
False positive cost	High	Moderate
Power priority	Lower	Higher
Setting	Confirmatory	Exploratory
Common use	Primary analysis	Screening

6.5 Multiple Testing Strategies for A/B Testing

6.5.1 Hierarchical Testing

Designate metrics hierarchically:

1. **Primary Metric:** Decision-making metric (no correction needed)
2. **Secondary Metrics:** Supporting evidence (apply correction)
3. **Guardrail Metrics:** Monitor for harm (separate testing)

Example:

- Primary: Conversion rate
- Secondary: Revenue per user, engagement time
- Guardrails: Page load time, error rate

Test primary metric at $\alpha = 0.05$. If significant, test secondary metrics with Bonferroni or FDR correction.

6.5.2 Pre-specification and Registration

Pre-specify:

- Which metrics will be tested
- Primary vs. secondary distinction
- Multiple testing correction method
- Subgroups of interest

This prevents:

- Post-hoc hypothesis generation
- P-hacking and data dredging
- Cherry-picking significant results

6.5.3 Handling Multiple Variants

For k variants, there are $\binom{k}{2}$ pairwise comparisons.

Strategies:

1. **Control Only:** Compare each treatment to control (most common)

   Number of tests: $k - 1$

   Less correction needed than all pairwise comparisons

2. **Dunnett's Test:** Specialized procedure for comparing multiple treatments to control

   More powerful than Bonferroni for this specific structure

3. **All Pairwise:** Compare all pairs

   Number of tests: $\binom{k}{2}$

   Requires substantial correction

4. **Best Treatment:** Simply select best-performing variant

   No hypothesis testing—exploratory approach

6.6 Practical Recommendations

1. **Limit Metrics:** Test only truly important metrics
2. **Hierarchical Structure:** Use primary/secondary framework
3. **Pre-register:** Specify hypotheses before seeing data
4. **Choose Correction Wisely:**
   - Few tests, high stakes: FWER (Bonferroni/Holm)
   - Many tests, exploratory: FDR (Benjamini-Hochberg)
5. **Report All Tests:** Document all tests conducted, not just significant ones
6. **Distinguish Confirmatory and Exploratory:** Use different standards
7. **Consider Effect Sizes:** Don't rely solely on p-values

6.7 Summary

Multiple testing is pervasive in A/B testing and requires careful attention:

- Without correction, false positive rates increase dramatically
- FWER methods (Bonferroni, Holm) provide strong error control
- FDR methods (Benjamini-Hochberg) offer more power for many tests
- Hierarchical testing and pre-specification prevent p-hacking

- Method choice depends on number of tests and acceptable error rates

Proper multiple testing control maintains experimental integrity while preserving statistical power.

6.8 Further Reading

- **?**: Original FDR paper
- **?**: Comprehensive multiple testing treatment
- **?**: Practical applications

6.9 Exercises

1. **Error Rate Calculations**
   (a) Derive the formula for FWER with m independent tests.
   (b) Simulate 10,000 experiments each testing 20 true nulls. Verify empirical FWER matches theory.
2. **Method Comparisons**
   (a) Compare power of Bonferroni, Holm, and BH for $m = 5, 10, 20, 50$ tests. Plot results.
   (b) Under what conditions does FDR control provide substantially more power than FWER?
3. **A/B Testing Applications**
   (a) Design a multiple testing strategy for an A/B/C/D test with 5 metrics.
   (b) Analyze a dataset with 15 metrics. Compare conclusions under different correction methods.
4. **Simulation Studies**
   (a) Simulate mixture of null and alternative hypotheses. Evaluate FWER and FDR empirically.
   (b) Investigate robustness of BH procedure to positive and negative dependence among tests.

Chapter 7

Bayesian Approaches to A/B Testing

7.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply Bayesian inference to A/B testing
- Construct and interpret posterior distributions
- Use conjugate priors for analytical solutions
- Calculate probability of superiority and expected loss
- Implement Bayesian A/B tests in Python and R
- Understand advantages and limitations of Bayesian approaches

7.2 Bayesian vs. Frequentist Paradigms

The Bayesian framework treats parameters as random variables and updates beliefs using observed data.

7.2.1 Key Philosophical Differences

Aspect	Frequentist	Bayesian
Parameters	Fixed constants	Random variables
Probability	Long-run frequency	Degree of belief
Inference	Confidence intervals	Credible intervals
Prior info	Not incorporated	Explicitly modeled
Interpretation	Indirect (via procedures)	Direct (probability)
Sequential testing	Problematic	Natural

7.2.2 Bayes' Theorem for A/B Testing

$$p(\theta|D) = \frac{p(D|\theta) \cdot p(\theta)}{p(D)} \propto p(D|\theta) \cdot p(\theta) \tag{7.1}$$

where:

- θ: Parameter of interest (e.g., conversion rate)
- D: Observed data
- $p(\theta)$: Prior distribution (before seeing data)
- $p(D|\theta)$: Likelihood (probability of data given parameter)
- $p(\theta|D)$: Posterior distribution (after seeing data)

The posterior combines prior beliefs with observed evidence.

7.3 Conjugate Priors for Common Distributions

Conjugate priors yield posterior distributions in the same family, enabling analytical solutions.

7.3.1 Beta-Binomial Model for Conversion Rates

For binary outcomes (conversions), the natural model is:

Likelihood:

$$X|p \sim \text{Binomial}(n, p) \tag{7.2}$$

Prior:

$$p \sim \text{Beta}(\alpha, \beta) \tag{7.3}$$

The Beta distribution has PDF:

$$f(p; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1}, \quad p \in (0, 1) \tag{7.4}$$

Properties:

- $\mathbb{E}[p] = \frac{\alpha}{\alpha+\beta}$
- $\text{Mode}[p] = \frac{\alpha-1}{\alpha+\beta-2}$ for $\alpha, \beta > 1$
- $\text{Var}(p) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$

Posterior:

Given x conversions from n trials:

Theorem 7.1 (Beta-Binomial Conjugacy)**.** If $p \sim \text{Beta}(\alpha, \beta)$ and $X|p \sim \text{Binomial}(n, p)$, then:

$$p|X = x \sim \text{Beta}(\alpha + x, \beta + n - x) \tag{7.5}$$

Interpretation:

- Prior $\text{Beta}(\alpha, \beta)$ represents $\alpha - 1$ prior successes and $\beta - 1$ prior failures
- Posterior adds observed data: x successes and $n - x$ failures
- Total: $\alpha + x - 1$ successes, $\beta + n - x - 1$ failures

Example 7.1 (Uninformative Prior)**.** Use Beta$(1, 1)$ (uniform on $[0, 1]$) when no prior information exists.

After observing 80 conversions from 1000 users:

$$p|D \sim \text{Beta}(1 + 80, 1 + 920) = \text{Beta}(81, 921) \tag{7.6}$$

Posterior mean: $\mathbb{E}[p|D] = \frac{81}{81+921} = 0.0808$
95% credible interval: $[0.0649, 0.0992]$ (using beta quantiles)

Listing 7.1: Bayesian Inference for Conversion Rate

```python
import numpy as np
from scipy.stats import beta
import matplotlib.pyplot as plt

def bayesian_conversion_inference(conversions, trials,
                                  prior_alpha=1, prior_beta=1):
    """
␣␣␣␣Bayesian␣inference␣for␣conversion␣rate␣using␣Beta-Binomial␣model.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣conversions␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣conversions
␣␣␣␣trials␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣trials
␣␣␣␣prior_alpha,␣prior_beta␣:␣float
␣␣␣␣␣␣␣␣Beta␣prior␣parameters

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Posterior␣statistics
␣␣␣␣"""
    # Posterior parameters
    post_alpha = prior_alpha + conversions
    post_beta = prior_beta + trials - conversions

    # Posterior statistics
    post_mean = post_alpha / (post_alpha + post_beta)
    post_mode = (post_alpha - 1) / (post_alpha + post_beta - 2) \
                if post_alpha > 1 and post_beta > 1 else None
    post_var = (post_alpha * post_beta) / \
               ((post_alpha + post_beta)**2 * (post_alpha + post_beta +
                   1))

    # Credible intervals
    ci_95 = beta.ppf([0.025, 0.975], post_alpha, post_beta)
    ci_90 = beta.ppf([0.05, 0.95], post_alpha, post_beta)

    return {
        'posterior_alpha': post_alpha,
        'posterior_beta': post_beta,
        'posterior_mean': post_mean,
        'posterior_mode': post_mode,
        'posterior_std': np.sqrt(post_var),
        'ci_95': ci_95,
        'ci_90': ci_90,
```

```
        'observed_rate': conversions / trials
    }

# Example
result = bayesian_conversion_inference(conversions=80, trials=1000)

print("Posterior Beta({}, {})".format(result['posterior_alpha'],
                                      result['posterior_beta']))
print(f"Posterior mean: {result['posterior_mean']:.4f}")
print(f"Posterior std: {result['posterior_std']:.4f}")
print(f"95% Credible Interval: [{result['ci_95'][0]:.4f}, "
      f"{result['ci_95'][1]:.4f}]")

# Visualize posterior
p_values = np.linspace(0.05, 0.12, 1000)
posterior_pdf = beta.pdf(p_values, result['posterior_alpha'],
                         result['posterior_beta'])

plt.figure(figsize=(10, 6))
plt.plot(p_values, posterior_pdf, linewidth=2, label='Posterior')
plt.axvline(result['posterior_mean'], color='red', linestyle='--',
            label=f"Mean = {result['posterior_mean']:.4f}")
plt.fill_between(p_values, 0, posterior_pdf,
                 where=(p_values >= result['ci_95'][0]) &
                       (p_values <= result['ci_95'][1]),
                 alpha=0.3, label='95% Credible Interval')
plt.xlabel('Conversion Rate')
plt.ylabel('Density')
plt.title('Posterior Distribution for Conversion Rate')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('bayesian_posterior.pdf')
plt.show()
```

7.3.2 Normal-Normal Model for Continuous Metrics

For continuous outcomes with known variance:

Likelihood:

$$Y_i|\mu \sim \mathcal{N}(\mu, \sigma^2) \tag{7.7}$$

Prior:

$$\mu \sim \mathcal{N}(\mu_0, \tau^2) \tag{7.8}$$

Posterior:

Given data $\mathbf{y} = (y_1, \ldots, y_n)$ with sample mean $\bar{y}$:

Theorem 7.2 (Normal-Normal Conjugacy)**.**

$$\mu|\mathbf{y} \sim \mathcal{N}\left(\frac{\frac{\mu_0}{\tau^2} + \frac{n\bar{y}}{\sigma^2}}{\frac{1}{\tau^2} + \frac{n}{\sigma^2}}, \left(\frac{1}{\tau^2} + \frac{n}{\sigma^2}\right)^{-1}\right) \tag{7.9}$$

The posterior mean is a precision-weighted average of prior mean and sample mean.

7.4 Bayesian A/B Testing

For comparing two variants, we need the posterior distribution of the difference or ratio.

7.4.1 Comparing Two Conversion Rates

Given:

$$p_A \sim \text{Beta}(\alpha_A, \beta_A) \tag{7.10}$$
$$p_B \sim \text{Beta}(\alpha_B, \beta_B) \tag{7.11}$$

We want to know:

$$P(p_B > p_A|D) = \int_0^1 \int_0^{p_B} f(p_A|D) f(p_B|D)\, dp_A\, dp_B \tag{7.12}$$

No closed form, but easily computed via:

1. **Numerical integration**
2. **Monte Carlo simulation:**
   - Sample $p_A^{(1)}, \ldots, p_A^{(S)}$ from $\text{Beta}(\alpha_A + x_A, \beta_A + n_A - x_A)$
   - Sample $p_B^{(1)}, \ldots, p_B^{(S)}$ from $\text{Beta}(\alpha_B + x_B, \beta_B + n_B - x_B)$
   - Estimate $P(p_B > p_A|D) \approx \frac{1}{S} \sum_{s=1}^{S} \mathbb{I}(p_B^{(s)} > p_A^{(s)})$

Example 7.2 (Bayesian A/B Test)**.**

$$\text{Variant A: 80 conversions from 1000 users} \tag{7.13}$$
$$\text{Variant B: 95 conversions from 1000 users} \tag{7.14}$$

Using uniform priors $\text{Beta}(1, 1)$:

$$p_A|D \sim \text{Beta}(81, 921) \tag{7.15}$$
$$p_B|D \sim \text{Beta}(96, 906) \tag{7.16}$$

Via Monte Carlo: $P(p_B > p_A|D) \approx 0.92$

Interpretation: There is a 92% probability that variant B has higher conversion rate than A.

Listing 7.2: Bayesian A/B Test in R

```
# Bayesian A/B test comparison
bayesian_ab_test <- function(x_a, n_a, x_b, n_b,
                             prior_alpha = 1, prior_beta = 1,
                             n_simulations = 100000) {
  # Posterior parameters
  post_alpha_a <- prior_alpha + x_a
  post_beta_a <- prior_beta + n_a - x_a

  post_alpha_b <- prior_alpha + x_b
  post_beta_b <- prior_beta + n_b - x_b

```

```r
  # Monte Carlo simulation
  p_a_samples <- rbeta(n_simulations, post_alpha_a, post_beta_a)
  p_b_samples <- rbeta(n_simulations, post_alpha_b, post_beta_b)

  # Probability B > A
  prob_b_better <- mean(p_b_samples > p_a_samples)

  # Expected lift
  lift <- p_b_samples / p_a_samples - 1
  expected_lift <- mean(lift)
  lift_ci <- quantile(lift, c(0.025, 0.975))

  # Difference distribution
  diff <- p_b_samples - p_a_samples
  diff_mean <- mean(diff)
  diff_ci <- quantile(diff, c(0.025, 0.975))

  list(
    prob_b_better = prob_b_better,
    expected_lift = expected_lift,
    lift_ci = lift_ci,
    difference_mean = diff_mean,
    difference_ci = diff_ci,
    posterior_a = list(alpha = post_alpha_a, beta = post_beta_a),
    posterior_b = list(alpha = post_alpha_b, beta = post_beta_b)
  )
}

# Example
result <- bayesian_ab_test(x_a = 80, n_a = 1000, x_b = 95, n_b = 1000)

cat("Probability B > A:", result$prob_b_better, "\n")
cat("Expected relative lift:", result$expected_lift, "\n")
cat("95% CI for lift:", result$lift_ci, "\n")
cat("Mean difference:", result$difference_mean, "\n")
cat("95% CI for difference:", result$difference_ci, "\n")
```

7.4.2 Expected Loss

Instead of binary decisions, we can quantify the expected loss from choosing the wrong variant.

Definition 7.1 (Expected Loss)**.** The expected loss from choosing variant A is:

$$\mathcal{L}(A) = \mathbb{E}[\max(0, p_B - p_A)|D] \tag{7.17}$$

Similarly for choosing B:

$$\mathcal{L}(B) = \mathbb{E}[\max(0, p_A - p_B)|D] \tag{7.18}$$

Decision rule: Choose the variant with smaller expected loss.

Interpretation: $\mathcal{L}(A)$ is the expected opportunity cost (in terms of conversion rate) if we choose A but B is actually better.

Listing 7.3: Expected Loss Calculation

```python
def expected_loss(x_a, n_a, x_b, n_b, prior_alpha=1, prior_beta=1,
                  n_simulations=100000):
    """
    Calculate expected loss for both variants.

    Returns:
    --------
    dict : Expected losses and recommendation
    """
    # Posterior parameters
    post_alpha_a = prior_alpha + x_a
    post_beta_a = prior_beta + n_a - x_a
    post_alpha_b = prior_alpha + x_b
    post_beta_b = prior_beta + n_b - x_b

    # Simulate from posteriors
    p_a = np.random.beta(post_alpha_a, post_beta_a, n_simulations)
    p_b = np.random.beta(post_alpha_b, post_beta_b, n_simulations)

    # Expected loss calculations
    loss_a = np.mean(np.maximum(0, p_b - p_a))
    loss_b = np.mean(np.maximum(0, p_a - p_b))

    # Relative expected loss
    rel_loss_a = loss_a / np.mean(p_a)
    rel_loss_b = loss_b / np.mean(p_b)

    recommendation = 'A' if loss_a < loss_b else 'B'

    return {
        'loss_a': loss_a,
        'loss_b': loss_b,
        'relative_loss_a': rel_loss_a,
        'relative_loss_b': rel_loss_b,
        'recommendation': recommendation,
        'loss_ratio': min(loss_a, loss_b) / max(loss_a, loss_b)
    }

# Example
result = expected_loss(x_a=80, n_a=1000, x_b=95, n_b=1000)

print(f"Expected loss choosing A: {result['loss_a']:.6f}")
print(f"Expected loss choosing B: {result['loss_b']:.6f}")
print(f"Relative loss A: {result['relative_loss_a']:.2%}")
print(f"Relative loss B: {result['relative_loss_b']:.2%}")
print(f"Recommendation: Variant {result['recommendation']}")
```

7.5 Prior Selection

Prior choice affects posterior, especially with limited data.

7.5.1 Types of Priors

1. **Uninformative (Flat) Priors**

   Beta$(1, 1)$ (uniform) or Jeffrey's prior Beta$(0.5, 0.5)$

Lets data dominate; minimal assumptions

2. **Weakly Informative Priors**
   Beta$(2, 20)$ for expected conversion around 10%
   Provides gentle regularization without strong assumptions
3. **Informative Priors**
   Based on historical data or expert knowledge
   Example: Beta$(50, 450)$ from previous experiments showing 10% conversion
4. **Empirical Bayes**
   Use data itself to estimate prior (controversial but practical)

7.5.2 Prior Sensitivity Analysis

Always check how conclusions depend on prior choice.

Example 7.3 (Prior Sensitivity)**.** Test with 10 conversions from 100 users:

Prior	Post. Mean	Post. Std	95% CI
Beta$(1, 1)$	0.108	0.031	[0.058, 0.175]
Beta$(5, 45)$	0.106	0.026	[0.063, 0.162]
Beta$(10, 90)$	0.105	0.023	[0.066, 0.154]
Beta$(20, 180)$	0.103	0.020	[0.069, 0.145]

With small data, stronger priors pull estimates toward prior mean.

7.6 Advantages of Bayesian A/B Testing

1. **Direct Probability Statements**
   "95% probability that B is better" vs. "reject null with p = 0.03"
   More intuitive for stakeholders
2. **Natural Sequential Testing**
   Can stop anytime without inflating error rates
   No need for α-spending functions
3. **Incorporation of Prior Information**
   Leverage historical data and expert knowledge
   Particularly valuable for rare events
4. **Coherent Decision Framework**
   Expected loss provides principled stopping rules
   Balances statistical and business considerations
5. **Full Uncertainty Quantification**
   Entire posterior distribution available
   Can compute any quantity of interest

7.7 Limitations and Challenges

1. **Prior Dependence**

   Results depend on prior, especially with little data

   Requires prior specification and sensitivity analysis

2. **Computational Complexity**

   Complex models may require MCMC or variational inference

   Not always analytical like conjugate examples

3. **Interpretation Challenges**

   "Probability B > A" requires careful framing

   Can be misinterpreted as certainty

4. **Less Familiar**

   Stakeholders more familiar with p-values and confidence intervals

   Requires education and buy-in

5. **No Error Rate Guarantees**

   Unlike frequentist methods, no Type I/II error control

   Different philosophical framework for errors

7.8 Bayesian vs. Frequentist: When to Use Which

Scenario	Frequentist	Bayesian
Fixed sample size	✓	✓
Sequential/adaptive		✓
Prior information available		✓
Regulatory requirement	✓	
Simple communication	✓	
Probability statements needed		✓
Multiple metrics/variants	✓	✓

Recommendation: Both frameworks are valuable. Choose based on:

- Organizational culture and stakeholder preferences
- Regulatory requirements
- Prior information availability
- Sequential testing needs
- Computational resources

Many organizations use hybrid approaches: frequentist for primary analysis, Bayesian for exploration.

7.9 Summary

Bayesian A/B testing offers a powerful alternative to frequentist methods:

- Treats parameters as random variables with probability distributions
- Updates beliefs via Bayes' theorem
- Conjugate priors enable analytical solutions
- Provides direct probability statements
- Handles sequential testing naturally
- Requires prior specification and sensitivity analysis

The choice between Bayesian and frequentist approaches depends on context, but understanding both enriches the practitioner's toolkit.

7.10 Further Reading

- **?**: Comprehensive Bayesian data analysis
- **?**: Bayesian analysis for psychology (accessible)
- **?**: Bayesian A/B testing for practitioners

7.11 Exercises

1. **Beta-Binomial Calculations**
   (a) Derive the posterior for Beta-Binomial model from first principles.
   (b) Show that Beta$(1, 1)$ is the uniform distribution on $[0, 1]$.
2. **Bayesian A/B Test**
   (a) Implement complete Bayesian A/B test including posterior visualization, probability calculations, and expected loss.
   (b) Compare Bayesian and frequentist conclusions for various sample sizes.
3. **Prior Sensitivity**
   (a) Conduct sensitivity analysis varying prior from uninformative to strongly informative.
   (b) At what sample size do priors matter little?
4. **Sequential Testing**
   (a) Simulate sequential Bayesian testing with stopping rule based on probability threshold.
   (b) Compare to frequentist sequential testing in terms of sample size and error rates.

Chapter 8

Advanced Experimental Designs

8.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and analyze factorial experiments testing multiple factors simultaneously
- Implement multi-armed bandit algorithms for dynamic traffic allocation
- Conduct sequential testing with appropriate statistical controls
- Handle network effects and interference in experimental settings
- Apply switchback designs for marketplace and platform experiments
- Choose appropriate advanced designs for specific business contexts

8.2 Factorial Designs

Factorial designs test multiple factors simultaneously, providing efficiency gains and enabling the detection of interaction effects that would be missed in separate experiments.

8.2.1 Full Factorial Designs

A 2^k factorial design tests k binary factors with all 2^k combinations.

Example 8.1 (2×2 Factorial Design)**.** Testing email subject line (A vs. B) and send time (morning vs. evening):

	Morning	**Evening**
Subject A	Cell 1 (n=250)	Cell 2 (n=250)
Subject B	Cell 3 (n=250)	Cell 4 (n=250)

With 1,000 total users, we can estimate:

- Main effect of subject line: Effect of A vs. B averaged across times
- Main effect of send time: Effect of morning vs. evening averaged across subjects
- Interaction: Does subject line effect depend on time of day?

8.2.2 Statistical Model for Factorial Experiments

For a two-factor design:

$$Y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk} \tag{8.1}$$

where:

- μ: Grand mean
- α_i: Main effect of factor A (level i)
- β_j: Main effect of factor B (level j)
- $(\alpha\beta)_{ij}$: Interaction effect
- ϵ_{ijk}: Random error, $\epsilon_{ijk} \sim \mathcal{N}(0, \sigma^2)$

Main Effects:

$$\text{Effect of A} = \bar{Y}_{\text{A high}} - \bar{Y}_{\text{A low}} \tag{8.2}$$
$$\text{Effect of B} = \bar{Y}_{\text{B high}} - \bar{Y}_{\text{B low}} \tag{8.3}$$

Interaction Effect:

$$\text{Interaction} = (\bar{Y}_{11} - \bar{Y}_{10}) - (\bar{Y}_{01} - \bar{Y}_{00}) \tag{8.4}$$

An interaction exists when the effect of one factor depends on the level of another factor.

Theorem 8.1 (Efficiency of Factorial Designs)**.** A 2^k factorial design with n observations per cell provides the same precision for estimating main effects as k separate experiments each with $2^{k-1} \times n$ observations, while also estimating all interaction effects.

Listing 8.1: Comprehensive Factorial Design Analysis

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from typing import Dict, Tuple

def simulate_factorial_experiment(
    n_per_cell: int,
    main_effect_A: float,
    main_effect_B: float,
    interaction: float,
    baseline: float = 0.10,
    seed: int = None
) -> pd.DataFrame:
    """
␣␣␣␣Simulate␣a␣2x2␣factorial␣experiment␣with␣binary␣outcome.

```

```
    Parameters:
    ----------
    n_per_cell : int
        Sample size per cell
    main_effect_A : float
        Main effect of factor A (additive on probability scale)
    main_effect_B : float
        Main effect of factor B
    interaction : float
        Interaction effect (additional effect when both A and B are present)
    baseline : float
        Baseline conversion rate (A=0, B=0)
    seed : int
        Random seed

    Returns:
    --------
    pd.DataFrame : Simulated experiment data
    """
    if seed is not None:
        np.random.seed(seed)

    # Define cell probabilities
    # Cell (0,0): baseline
    # Cell (1,0): baseline + main_effect_A
    # Cell (0,1): baseline + main_effect_B
    # Cell (1,1): baseline + main_effect_A + main_effect_B + interaction

    probabilities = {
        (0, 0): baseline,
        (1, 0): baseline + main_effect_A,
        (0, 1): baseline + main_effect_B,
        (1, 1): baseline + main_effect_A + main_effect_B + interaction
    }

    data_list = []

    for factor_A in [0, 1]:
        for factor_B in [0, 1]:
            p = probabilities[(factor_A, factor_B)]
            conversions = np.random.binomial(1, p, n_per_cell)

            cell_data = pd.DataFrame({
                'factor_A': factor_A,
                'factor_B': factor_B,
                'conversion': conversions,
                'user_id': range(
                    (factor_A * 2 + factor_B) * n_per_cell,
                    (factor_A * 2 + factor_B + 1) * n_per_cell
                )
            })
            data_list.append(cell_data)

    data = pd.concat(data_list, ignore_index=True)

    # Add categorical labels
```

```
    data['factor_A_label'] = data['factor_A'].map({0: 'A0', 1: 'A1'})
    data['factor_B_label'] = data['factor_B'].map({0: 'B0', 1: 'B1'})

    return data

def analyze_factorial_experiment(data: pd.DataFrame) -> Dict:
    """
    Comprehensive analysis of 2x2 factorial experiment.

    Parameters:
    -----------
    data : pd.DataFrame
        Must contain columns: factor_A, factor_B, conversion

    Returns:
    --------
    dict : Analysis results including effects, tests, and diagnostics
    """
    # Cell means
    cell_means = data.groupby(['factor_A', 'factor_B'])['conversion'].
        agg([
        'mean', 'count', 'std'
    ]).reset_index()

    # Extract cell means
    y00 = cell_means[(cell_means['factor_A']==0) &
                     (cell_means['factor_B']==0)]['mean'].values[0]
    y10 = cell_means[(cell_means['factor_A']==1) &
                     (cell_means['factor_B']==0)]['mean'].values[0]
    y01 = cell_means[(cell_means['factor_A']==0) &
                     (cell_means['factor_B']==1)]['mean'].values[0]
    y11 = cell_means[(cell_means['factor_A']==1) &
                     (cell_means['factor_B']==1)]['mean'].values[0]

    # Calculate effects
    main_effect_A = ((y10 + y11) / 2) - ((y00 + y01) / 2)
    main_effect_B = ((y01 + y11) / 2) - ((y00 + y10) / 2)
    interaction_effect = (y11 - y10) - (y01 - y00)

    # ANOVA using statsmodels
    formula = 'conversion ~ C(factor_A) * C(factor_B)'
    model = ols(formula, data=data).fit()
    anova_table = sm.stats.anova_lm(model, typ=2)

    # Effect sizes (Cohen's d for binary factors)
    overall_std = data['conversion'].std()
    cohens_d_A = main_effect_A / overall_std
    cohens_d_B = main_effect_B / overall_std
    cohens_d_interaction = interaction_effect / overall_std

    # Confidence intervals for effects (using bootstrap)
    n_bootstrap = 1000
    bootstrap_effects = {
        'main_A': [],
        'main_B': [],
        'interaction': []
    }

```

```
    np.random.seed(42)
    for _ in range(n_bootstrap):
        boot_sample = data.sample(n=len(data), replace=True)
        boot_means = boot_sample.groupby(
            ['factor_A', 'factor_B']
        )['conversion'].mean()

        b_y00 = boot_means[(0, 0)]
        b_y10 = boot_means[(1, 0)]
        b_y01 = boot_means[(0, 1)]
        b_y11 = boot_means[(1, 1)]

        bootstrap_effects['main_A'].append(
            ((b_y10 + b_y11) / 2) - ((b_y00 + b_y01) / 2)
        )
        bootstrap_effects['main_B'].append(
            ((b_y01 + b_y11) / 2) - ((b_y00 + b_y10) / 2)
        )
        bootstrap_effects['interaction'].append(
            (b_y11 - b_y10) - (b_y01 - b_y00)
        )

    ci_main_A = np.percentile(bootstrap_effects['main_A'], [2.5, 97.5])
    ci_main_B = np.percentile(bootstrap_effects['main_B'], [2.5, 97.5])
    ci_interaction = np.percentile(bootstrap_effects['interaction'],
        [2.5, 97.5])

    return {
        'cell_means': {
            'A0_B0': y00,
            'A1_B0': y10,
            'A0_B1': y01,
            'A1_B1': y11
        },
        'effects': {
            'main_effect_A': main_effect_A,
            'main_effect_B': main_effect_B,
            'interaction': interaction_effect
        },
        'effect_sizes': {
            'cohens_d_A': cohens_d_A,
            'cohens_d_B': cohens_d_B,
            'cohens_d_interaction': cohens_d_interaction
        },
        'confidence_intervals': {
            'main_A_ci': ci_main_A,
            'main_B_ci': ci_main_B,
            'interaction_ci': ci_interaction
        },
        'anova_table': anova_table,
        'model': model
    }

def visualize_factorial_experiment(data: pd.DataFrame, results: Dict):
    """Create comprehensive visualizations for factorial experiment."""
    fig, axes = plt.subplots(2, 2, figsize=(14, 12))

    # 1. Cell means with error bars
```

```
    ax = axes[0, 0]
    cell_stats = data.groupby(['factor_A_label', 'factor_B_label'])[
        'conversion'
    ].agg(['mean', 'sem'])

    x_pos = np.arange(4)
    cell_labels = ['A0_B0', 'A1_B0', 'A0_B1', 'A1_B1']
    means = [results['cell_means'][label] for label in cell_labels]

    # Calculate SEM for error bars
    sems = []
    for a in [0, 1]:
        for b in [0, 1]:
            subset = data[(data['factor_A']==a) & (data['factor_B']==b)
                ]
            sems.append(subset['conversion'].sem())

    ax.bar(x_pos, means, yerr=[1.96*s for s in sems],
           capsize=5, alpha=0.7, color=['C0', 'C1', 'C0', 'C1'])
    ax.set_xticks(x_pos)
    ax.set_xticklabels(cell_labels)
    ax.set_ylabel('Conversion Rate')
    ax.set_title('Cell Means with 95% CI')
    ax.grid(axis='y', alpha=0.3)

    # 2. Interaction plot
    ax = axes[0, 1]
    for b_val, b_label in [(0, 'B0'), (1, 'B1')]:
        subset = data[data['factor_B'] == b_val]
        means_by_A = subset.groupby('factor_A')['conversion'].mean()
        ax.plot([0, 1], means_by_A.values, marker='o',
                markersize=10, linewidth=2, label=b_label)

    ax.set_xticks([0, 1])
    ax.set_xticklabels(['A0', 'A1'])
    ax.set_xlabel('Factor A')
    ax.set_ylabel('Conversion Rate')
    ax.set_title('Interaction Plot')
    ax.legend(title='Factor B')
    ax.grid(alpha=0.3)

    # 3. Main effects visualization
    ax = axes[1, 0]
    effects = results['effects']
    effect_names = ['Main Effect A', 'Main Effect B', 'Interaction']
    effect_values = [
        effects['main_effect_A'],
        effects['main_effect_B'],
        effects['interaction']
    ]

    cis = [
        results['confidence_intervals']['main_A_ci'],
        results['confidence_intervals']['main_B_ci'],
        results['confidence_intervals']['interaction_ci']
    ]

    y_pos = np.arange(len(effect_names))
```

```
    colors = ['green' if ci[0] > 0 or ci[1] < 0 else 'gray'
              for ci in cis]

    ax.barh(y_pos, effect_values, color=colors, alpha=0.7)

    for i, (val, ci) in enumerate(zip(effect_values, cis)):
        ax.plot([ci[0], ci[1]], [i, i], 'k-', linewidth=2)
        ax.plot([ci[0], ci[0]], [i-0.1, i+0.1], 'k-', linewidth=2)
        ax.plot([ci[1], ci[1]], [i-0.1, i+0.1], 'k-', linewidth=2)

    ax.axvline(x=0, color='red', linestyle='--', linewidth=1)
    ax.set_yticks(y_pos)
    ax.set_yticklabels(effect_names)
    ax.set_xlabel('Effect Size (Absolute)')
    ax.set_title('Effects with 95% Bootstrap CI')
    ax.grid(axis='x', alpha=0.3)

    # 4. Heatmap of cell means
    ax = axes[1, 1]
    heatmap_data = np.array([
        [results['cell_means']['A0_B0'], results['cell_means']['A0_B1']],
        [results['cell_means']['A1_B0'], results['cell_means']['A1_B1']]
    ])

    im = ax.imshow(heatmap_data, cmap='RdYlGn', aspect='auto')
    ax.set_xticks([0, 1])
    ax.set_yticks([0, 1])
    ax.set_xticklabels(['B0', 'B1'])
    ax.set_yticklabels(['A0', 'A1'])
    ax.set_xlabel('Factor B')
    ax.set_ylabel('Factor A')
    ax.set_title('Heatmap of Conversion Rates')

    # Add text annotations
    for i in range(2):
        for j in range(2):
            text = ax.text(j, i, f'{heatmap_data[i, j]:.3f}',
                           ha="center", va="center", color="black",
                           fontsize=14, fontweight='bold')

    plt.colorbar(im, ax=ax)

    plt.tight_layout()
    return fig

# Example: Simulate and analyze factorial experiment
np.random.seed(42)
data = simulate_factorial_experiment(
    n_per_cell=500,
    main_effect_A=0.02,     # Subject B increases conversion by 2pp
    main_effect_B=0.015,    # Evening increases conversion by 1.5pp
    interaction=0.01,       # Additional 1pp when both are combined
    baseline=0.10
)

results = analyze_factorial_experiment(data)
```

```
304
305 print("=== Factorial Experiment Analysis ===\n")
306 print("Cell Means:")
307 for cell, mean in results['cell_means'].items():
308     print(f"  {cell}: {mean:.4f}")
309
310 print("\nEstimated Effects:")
311 print(f"  Main Effect A: {results['effects']['main_effect_A']:.4f}")
312 print(f"  Main Effect B: {results['effects']['main_effect_B']:.4f}")
313 print(f"  Interaction:   {results['effects']['interaction']:.4f}")
314
315 print("\nEffect Sizes (Cohen's d):")
316 print(f"  Main Effect A: {results['effect_sizes']['cohens_d_A']:.3f}")
317 print(f"  Main Effect B: {results['effect_sizes']['cohens_d_B']:.3f}")
318 print(f"  Interaction:   {results['effect_sizes']['cohens_d_interaction
    ']:.3f}")
319
320 print("\n95% Confidence Intervals:")
321 print(f"  Main A: [{results['confidence_intervals']['main_A_ci'][0]:.4f
    }, "
322       f"{results['confidence_intervals']['main_A_ci'][1]:.4f}]")
323 print(f"  Main B: [{results['confidence_intervals']['main_B_ci'][0]:.4f
    }, "
324       f"{results['confidence_intervals']['main_B_ci'][1]:.4f}]")
325 print(f"  Interaction: [{results['confidence_intervals']['interaction_
    ci'][0]:.4f}, "
326       f"{results['confidence_intervals']['interaction_ci'][1]:.4f}]")
327
328 print("\nANOVA Table:")
329 print(results['anova_table'])
330
331 # Create visualizations
332 fig = visualize_factorial_experiment(data, results)
333 plt.savefig('factorial_analysis.pdf', bbox_inches='tight')
334 plt.show()
```

8.2.3 Fractional Factorial Designs

When testing many factors, full factorial designs become impractical. A 2^k design requires 2^k experimental conditions.

Problem: Testing 6 factors requires $2^6 = 64$ cells. With 100 users per cell, need 6,400 users.

Solution: Fractional factorial designs test a carefully chosen subset of conditions.

Example 8.2 (Half-Fraction Design)**.** A 2^{5-1} (read: "two to the five minus one") design tests 5 factors using only $2^4 = 16$ conditions instead of 32.

We sacrifice the ability to estimate certain high-order interactions (which are often negligible) to gain efficiency.

Confounding: In fractional designs, some effects cannot be separately estimated (they are "aliased").

For 2^{5-1} with defining relation $I = ABCDE$:

- Main effect A is aliased with 4-way interaction BCDE

- Two-way interaction AB is aliased with CDE

Resolution: Describes which effects are confounded:

- Resolution III: Main effects clear of each other but aliased with 2-way interactions
- Resolution IV: Main effects clear; 2-way interactions aliased with other 2-way interactions
- Resolution V: Main effects and 2-way interactions clear

Practical Recommendation: Use Resolution IV or V designs when possible. Assume high-order interactions are negligible.

8.3 Multi-Armed Bandits

Multi-armed bandits (MAB) dynamically allocate traffic to variants, balancing exploration (learning about variants) and exploitation (assigning users to better-performing variants).

8.3.1 The Exploration-Exploitation Tradeoff

Traditional A/B test: Fixed allocation (e.g., 50/50) throughout experiment.

Multi-armed bandit: Adaptive allocation that shifts traffic toward better variants as data accumulates.

Regret: Cumulative opportunity cost from suboptimal assignments:

$$R(T) = \sum_{t=1}^{T} (\mu^* - \mu_{A_t}) \tag{8.5}$$

where:

- T: Total number of users
- μ^*: True mean of best variant
- A_t: Variant assigned to user t
- μ_{A_t}: True mean of assigned variant

Goal: Minimize cumulative regret while maintaining valid inference.

8.3.2 ϵ-Greedy Algorithm

Simplest bandit algorithm: with probability ϵ, explore randomly; otherwise, exploit current best.

Algorithm 4 ϵ-Greedy Bandit

Initialize: $N_i = 0$, $\hat{\mu}_i = 0$ for all variants $i = 1, \ldots, K$
for each user $t = 1, 2, \ldots, T$ **do**
  Generate $u \sim \text{Uniform}(0, 1)$
  if $u < \epsilon$ **then**
    Select random variant A_t (explore)
  else
    Select $A_t = \arg\max_i \hat{\mu}_i$ (exploit)
  end if
  Observe reward r_t
  Update: $N_{A_t} \leftarrow N_{A_t} + 1$
  Update: $\hat{\mu}_{A_t} \leftarrow \frac{(N_{A_t}-1)\hat{\mu}_{A_t}+r_t}{N_{A_t}}$
end for

Parameter Selection:

- Fixed $\epsilon \in [0.05, 0.20]$: Simple but continues exploring indefinitely
- Decaying $\epsilon_t = \min(1, \epsilon_0/t)$: Reduces exploration over time

8.3.3 Upper Confidence Bound (UCB) Algorithm

UCB addresses ϵ-greedy's limitation by explicitly balancing exploitation and exploration using confidence intervals.

Algorithm 5 UCB1 Algorithm

Initialize: $N_i = 0$, $\hat{\mu}_i = 0$ for all variants i
Play each variant once to initialize
for each user $t = K + 1, K + 2, \ldots, T$ **do**
  Select variant:

$$A_t = \arg\max_i \left[\hat{\mu}_i + \sqrt{\frac{2 \ln t}{N_i}} \right]$$

  Observe reward r_t
  Update N_{A_t} and $\hat{\mu}_{A_t}$
end for

The UCB term $\sqrt{\frac{2 \ln t}{N_i}}$ represents uncertainty: variants with fewer observations get bonus exploration.

Theorem (Auer et al., 2002): UCB1 achieves logarithmic regret:

$$R(T) \leq 8 \sum_{i:\mu_i<\mu^*} \frac{\ln T}{\Delta_i} + \left(1 + \frac{\pi^2}{3}\right) \sum_{i=1}^{K} \Delta_i \tag{8.6}$$

where $\Delta_i = \mu^* - \mu_i$ is the gap between optimal and variant i.

8.3.4 Thompson Sampling (Bayesian Bandit)

Thompson sampling maintains posterior distributions over variant success rates and samples from them to make allocation decisions.

For Bernoulli rewards (conversion rates):

Algorithm 6 Thompson Sampling for Bernoulli Bandits

Initialize: $\alpha_i = 1, \beta_i = 1$ for all variants i (Beta prior)
for each user $t = 1, 2, \ldots, T$ **do**
  for each variant $i = 1, \ldots, K$ **do**
    Sample $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$
  end for
  Select variant $A_t = \arg\max_i \theta_i$
  Observe reward $r_t \in \{0, 1\}$
  Update: $\alpha_{A_t} \leftarrow \alpha_{A_t} + r_t$
  Update: $\beta_{A_t} \leftarrow \beta_{A_t} + (1 - r_t)$
end for

Intuition: Variants with higher uncertainty (wider posteriors) have higher probability of being sampled, naturally balancing exploration and exploitation.

Listing 8.2: Comprehensive Bandit Implementations and Comparison

```
import numpy as np
import matplotlib.pyplot as plt
from typing import List, Tuple
from abc import ABC, abstractmethod

class Bandit(ABC):
    """Abstract base class for bandit algorithms."""

    def __init__(self, n_arms: int):
        self.n_arms = n_arms
        self.counts = np.zeros(n_arms)
        self.values = np.zeros(n_arms)
        self.total_reward = 0

    @abstractmethod
    def select_arm(self, t: int) -> int:
        """Select an arm to pull."""
        pass

    def update(self, arm: int, reward: float):
        """Update statistics after observing reward."""
        self.counts[arm] += 1
        self.values[arm] += (reward - self.values[arm]) / self.counts[
            arm]
        self.total_reward += reward

class EpsilonGreedy(Bandit):
    """Epsilon-greedy bandit."""

    def __init__(self, n_arms: int, epsilon: float = 0.1,
                 decay: bool = False):
        super().__init__(n_arms)
```

```
        self.epsilon = epsilon
        self.decay = decay

    def select_arm(self, t: int) -> int:
        epsilon = self.epsilon / t if self.decay else self.epsilon

        if np.random.random() < epsilon:
            return np.random.randint(0, self.n_arms)
        else:
            return np.argmax(self.values)

class UCB1(Bandit):
    """UCB1 algorithm."""

    def __init__(self, n_arms: int):
        super().__init__(n_arms)

    def select_arm(self, t: int) -> int:
        # Initially pull each arm once
        for arm in range(self.n_arms):
            if self.counts[arm] == 0:
                return arm

        # Calculate UCB for each arm
        ucb_values = self.values + np.sqrt(
            2 * np.log(t) / self.counts
        )
        return np.argmax(ucb_values)

class ThompsonSampling(Bandit):
    """Thompson Sampling with Beta-Bernoulli model."""

    def __init__(self, n_arms: int):
        super().__init__(n_arms)
        self.alpha = np.ones(n_arms)  # Successes + 1 (prior)
        self.beta = np.ones(n_arms)   # Failures + 1 (prior)

    def select_arm(self, t: int) -> int:
        # Sample from posterior for each arm
        samples = np.random.beta(self.alpha, self.beta)
        return np.argmax(samples)

    def update(self, arm: int, reward: float):
        super().update(arm, reward)
        self.alpha[arm] += reward
        self.beta[arm] += (1 - reward)

def simulate_bandit_experiment(
    bandit: Bandit,
    true_rates: List[float],
    n_trials: int
) -> dict:
    """
    Simulate bandit experiment.

    Parameters:
    -----------
    bandit : Bandit
```

```
        Bandit algorithm instance
    true_rates : list
        True conversion rate for each arm
    n_trials : int
        Number of users

    Returns:
    --------
    dict : Simulation results
    """
    n_arms = len(true_rates)
    true_rates = np.array(true_rates)
    optimal_rate = np.max(true_rates)
    optimal_arm = np.argmax(true_rates)

    # Track metrics over time
    arm_selections = np.zeros((n_trials, n_arms))
    cumulative_reward = np.zeros(n_trials)
    cumulative_regret = np.zeros(n_trials)
    optimal_selections = np.zeros(n_trials)

    for t in range(n_trials):
        # Select arm
        arm = bandit.select_arm(t + 1)
        arm_selections[t, arm] = 1

        # Observe reward
        reward = np.random.binomial(1, true_rates[arm])

        # Update bandit
        bandit.update(arm, reward)

        # Track metrics
        regret = optimal_rate - true_rates[arm]

        cumulative_reward[t] = bandit.total_reward
        cumulative_regret[t] = (
            cumulative_regret[t-1] + regret if t > 0 else regret
        )
        optimal_selections[t] = (arm == optimal_arm)

    # Cumulative arm selections
    cumulative_selections = np.cumsum(arm_selections, axis=0)

    return {
        'arm_selections': arm_selections,
        'cumulative_selections': cumulative_selections,
        'final_proportions': cumulative_selections[-1] / n_trials,
        'cumulative_reward': cumulative_reward,
        'cumulative_regret': cumulative_regret,
        'optimal_selection_rate': np.cumsum(optimal_selections) /
                                  np.arange(1, n_trials + 1),
        'final_values': bandit.values
    }

def compare_bandit_algorithms(
    true_rates: List[float],
    n_trials: int = 10000,
```

```
    n_simulations: int = 100
):
    """
    Compare different bandit algorithms through simulation.

    Parameters:
    -----------
    true_rates : list
        True conversion rate for each variant
    n_trials : int
        Number of trials per simulation
    n_simulations : int
        Number of simulation replications
    """
    n_arms = len(true_rates)

    algorithms = {
        'epsilon-greedy (0.1)': lambda: EpsilonGreedy(n_arms, 0.1),
        'epsilon-greedy (decay)': lambda: EpsilonGreedy(n_arms, 1.0,
                                                        decay=True),
        'UCB1': lambda: UCB1(n_arms),
        'Thompson Sampling': lambda: ThompsonSampling(n_arms)
    }

    results = {name: [] for name in algorithms.keys()}

    for sim in range(n_simulations):
        np.random.seed(sim)

        for name, create_bandit in algorithms.items():
            bandit = create_bandit()
            result = simulate_bandit_experiment(bandit, true_rates, n_
                trials)
            results[name].append(result)

    # Aggregate results
    aggregated = {}
    for name in algorithms.keys():
        regrets = [r['cumulative_regret'] for r in results[name]]
        optimal_rates = [r['optimal_selection_rate'] for r in results[
            name]]

        aggregated[name] = {
            'mean_regret': np.mean(regrets, axis=0),
            'std_regret': np.std(regrets, axis=0),
            'mean_optimal_rate': np.mean(optimal_rates, axis=0)
        }

    # Visualization
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # 1. Cumulative regret
    ax = axes[0, 0]
    for name, data in aggregated.items():
        ax.plot(data['mean_regret'], label=name, linewidth=2)
        ax.fill_between(
            range(n_trials),
            data['mean_regret'] - data['std_regret'],
```

```
            data['mean_regret'] + data['std_regret'],
            alpha=0.2
        )
    ax.set_xlabel('Trial')
    ax.set_ylabel('Cumulative Regret')
    ax.set_title('Cumulative Regret Over Time')
    ax.legend()
    ax.grid(alpha=0.3)

    # 2. Optimal arm selection rate
    ax = axes[0, 1]
    for name, data in aggregated.items():
        ax.plot(data['mean_optimal_rate'], label=name, linewidth=2)
    ax.set_xlabel('Trial')
    ax.set_ylabel('Proportion Selecting Optimal Arm')
    ax.set_title('Convergence to Optimal Arm')
    ax.legend()
    ax.grid(alpha=0.3)
    ax.set_ylim([0, 1])

    # 3. Final regret comparison
    ax = axes[1, 0]
    final_regrets = [data['mean_regret'][-1] for data in aggregated.
        values()]
    final_stds = [data['std_regret'][-1] for data in aggregated.values
        ()]

    x_pos = np.arange(len(algorithms))
    ax.bar(x_pos, final_regrets, yerr=final_stds, capsize=5, alpha=0.7)
    ax.set_xticks(x_pos)
    ax.set_xticklabels(algorithms.keys(), rotation=15, ha='right')
    ax.set_ylabel('Final Cumulative Regret')
    ax.set_title(f'Final Regret after {n_trials} Trials')
    ax.grid(axis='y', alpha=0.3)

    # 4. Arm selection distribution (Thompson Sampling)
    ax = axes[1, 1]
    ts_results = results['Thompson Sampling']
    final_props = np.array([r['final_proportions'] for r in ts_results
        ])
    mean_props = np.mean(final_props, axis=0)

    x_pos = np.arange(n_arms)
    ax.bar(x_pos, mean_props, alpha=0.7)
    ax.axhline(y=1/n_arms, color='red', linestyle='--',
               label='Equal allocation')
    ax.set_xticks(x_pos)
    ax.set_xticklabels([f'Arm {i}\n({rate:.1%})'
                        for i, rate in enumerate(true_rates)])
    ax.set_ylabel('Proportion of Selections')
    ax.set_title('Thompson Sampling: Final Allocation')
    ax.legend()
    ax.grid(axis='y', alpha=0.3)

    plt.tight_layout()
    return fig, aggregated

# Example comparison
```

```
true_rates = [0.08, 0.10, 0.12]
fig, results = compare_bandit_algorithms(
    true_rates,
    n_trials=10000,
    n_simulations=100
)

print("=== Bandit Algorithm Comparison ===")
print(f"True rates: {true_rates}")
print(f"Optimal arm: Arm {np.argmax(true_rates)} ({max(true_rates):.1%})\n")

for name, data in results.items():
    print(f"{name}:")
    print(f"  Final regret: {data['mean_regret'][-1]:.1f} "
          f"(± {data['std_regret'][-1]:.1f})")
    print(f"  Final optimal rate: {data['mean_optimal_rate'][-1]:.3f}\n")

plt.savefig('bandit_comparison.pdf', bbox_inches='tight')
plt.show()
```

8.3.5 When to Use Bandits vs. Traditional A/B Tests

Use bandits when:

- **Optimizing during the test is valuable:** E.g., content recommendation, ad selection
- **Cost of poor experiences is high:** Want to minimize exposure to inferior variants
- **Rapid learning is possible:** Quick feedback enables adaptation
- **Continuous optimization desired:** Not a one-time decision
- **Many variants:** Efficient at eliminating poor options

Use traditional A/B when:

- **Clean inference is critical:** Need p-values, confidence intervals for decision making
- **Regulatory requirements:** Medical devices, financial products
- **Long-term effects matter:** User habituation, retention effects
- **Stakeholder communication:** Fixed allocation easier to explain
- **Implementation is permanent:** Making a one-time product decision

Hybrid Approach: Use bandits for traffic allocation during experiment, then analyze using causal inference methods (e.g., inverse propensity weighting) for final decision.

8.4 Sequential Testing

Sequential testing allows stopping experiments early when sufficient evidence accumulates, potentially saving time and resources.

8.4.1 The Peeking Problem

Naive peeking: Continuously checking p-values and stopping when $p < 0.05$ inflates Type I error far above 5%.

Example 8.3 (Inflation from Peeking)**.** Simulation with no true effect (H_0 true):

- Fixed sample (n=1000): Type I error = 5.0%
- Check every 100 users, stop if $p < 0.05$: Type I error = 29%
- Check every 50 users: Type I error = 38%

Problem: Multiple testing without correction.
Solution: Group sequential methods with adjusted boundaries.

8.4.2 Group Sequential Testing

Pre-specify:

- Information fractions: e.g., analyze at 25%, 50%, 75%, 100% of planned sample
- Spending function: how to allocate Type I error across looks
- Stopping boundaries: thresholds for stopping

O'Brien-Fleming Boundaries

Conservative early stopping; save most α for final analysis.

For K equally spaced looks at information fractions $1/K, 2/K, \ldots, K/K$:

$$c_k = c\sqrt{\frac{K}{k}} \tag{8.7}$$

where $c \approx 2$ for $\alpha = 0.05$ (determined numerically to control FWER).

Early boundaries are stringent:

- At 25% information: $c_1 = c\sqrt{4} = 2c \approx 4$ (need $|Z| > 4$)
- At 50% information: $c_2 = c\sqrt{2} \approx 2.83$
- At 100% information: $c_K = c \approx 2$

Advantage: Preserves power well; if don't stop early, final test nearly as powerful as fixed design.

Pocock Boundaries

Equal thresholds at all looks: $c_1 = c_2 = \cdots = c_K$.

For $\alpha = 0.05$ and $K = 5$ looks: $c \approx 2.41$.

More aggressive early stopping than O'Brien-Fleming.

Disadvantage: If no early stop, final test less powerful due to α spent early.

Listing 8.3: Group Sequential Testing Implementation

```
import numpy as np
from scipy import stats
from typing import List, Tuple
import matplotlib.pyplot as plt

class GroupSequentialTest:
    """
␣␣␣␣Implement␣group␣sequential␣testing␣with␣O'Brien-Fleming␣or␣Pocock␣
    boundaries.
␣␣␣␣"""

    def __init__(self,
                 max_n: int,
                 n_looks: int,
                 alpha: float = 0.05,
                 beta: float = 0.20,
                 boundary_type: str = 'obf'):
        """
␣␣␣␣␣␣␣␣Parameters:
␣␣␣␣␣␣␣␣-----------
␣␣␣␣␣␣␣␣max_n␣:␣int
␣␣␣␣␣␣␣␣␣␣␣␣Maximum␣sample␣size␣per␣group
␣␣␣␣␣␣␣␣n_looks␣:␣int
␣␣␣␣␣␣␣␣␣␣␣␣Number␣of␣interim␣analyses␣(including␣final)
␣␣␣␣␣␣␣␣alpha␣:␣float
␣␣␣␣␣␣␣␣␣␣␣␣Type␣I␣error␣rate
␣␣␣␣␣␣␣␣beta␣:␣float
␣␣␣␣␣␣␣␣␣␣␣␣Type␣II␣error␣rate␣(1␣-␣power)
␣␣␣␣␣␣␣␣boundary_type␣:␣str
␣␣␣␣␣␣␣␣␣␣␣␣'obf'␣(O'Brien-Fleming)␣or␣'pocock'
␣␣␣␣␣␣␣␣"""
        self.max_n = max_n
        self.n_looks = n_looks
        self.alpha = alpha
        self.beta = beta
        self.boundary_type = boundary_type

        # Information fractions
        self.info_fracs = np.array([
            (k+1) / n_looks for k in range(n_looks)
        ])

        # Calculate boundaries
        self.boundaries = self._calculate_boundaries()

    def _calculate_boundaries(self) -> np.ndarray:
        """Calculate␣stopping␣boundaries."""
        if self.boundary_type == 'obf':
            # O'Brien-Fleming: conservative early, lenient late
```

```
            # Use Lan-DeMets spending function approximation
            z_alpha = stats.norm.ppf(1 - self.alpha/2)
            boundaries = z_alpha * np.sqrt(1 / self.info_fracs)

        elif self.boundary_type == 'pocock':
            # Pocock: equal boundaries
            # Approximate for 2-sided test
            if self.n_looks == 1:
                c = stats.norm.ppf(1 - self.alpha/2)
            elif self.n_looks == 2:
                c = 2.178
            elif self.n_looks == 3:
                c = 2.289
            elif self.n_looks == 4:
                c = 2.361
            elif self.n_looks == 5:
                c = 2.413
            else:
                # Approximation for larger K
                c = stats.norm.ppf(1 - self.alpha/(2*self.n_looks))

            boundaries = np.ones(self.n_looks) * c
        else:
            raise ValueError("boundary_type must be 'obf' or 'pocock'")

        return boundaries

    def analyze_look(self,
                     n_current: int,
                     control_conversions: int,
                     treatment_conversions: int,
                     control_n: int,
                     treatment_n: int) -> dict:
        """
        Analyze data at a specific look.

        Returns:
        --------
        dict with keys:
            - 'z_stat': Test statistic
            - 'boundary': Critical value for this look
            - 'stop_efficacy': Stop for efficacy
            - 'stop_futility': Stop for futility
            - 'continue': Continue to next look
        """
        # Calculate test statistic
        p1 = control_conversions / control_n
        p2 = treatment_conversions / treatment_n
        p_pooled = (control_conversions + treatment_conversions) / (
            control_n + treatment_n
        )

        se = np.sqrt(p_pooled * (1 - p_pooled) * (1/control_n + 1/
            treatment_n))
        z_stat = (p2 - p1) / se

        # Determine which look we're at
        look_idx = int(np.round(n_current / self.max_n * self.n_looks))
```

```
            - 1
        look_idx = np.clip(look_idx, 0, self.n_looks - 1)

        boundary = self.boundaries[look_idx]

        # Stopping decisions
        stop_efficacy = abs(z_stat) >= boundary

        # Simple futility boundary (can be more sophisticated)
        futility_boundary = 0.5
        stop_futility = abs(z_stat) < futility_boundary and look_idx >=
            1

        return {
            'look': look_idx + 1,
            'n_current': n_current,
            'z_stat': z_stat,
            'boundary': boundary,
            'stop_efficacy': stop_efficacy,
            'stop_futility': stop_futility,
            'continue': not (stop_efficacy or stop_futility),
            'p1': p1,
            'p2': p2
        }

    def visualize_boundaries(self):
        """Visualize stopping boundaries."""
        fig, ax = plt.subplots(figsize=(10, 6))

        # Plot boundaries
        looks = np.arange(1, self.n_looks + 1)
        ax.plot(looks, self.boundaries, 'ro-', linewidth=2,
                markersize=10, label='Efficacy boundary')
        ax.plot(looks, -self.boundaries, 'ro-', linewidth=2, markersize
            =10)

        # Add futility boundary
        ax.axhline(y=0.5, color='orange', linestyle='--',
                   label='Futility boundary')
        ax.axhline(y=-0.5, color='orange', linestyle='--')
        ax.axhline(y=0, color='gray', linestyle='-', alpha=0.3)

        # Styling
        ax.set_xlabel('Analysis Number', fontsize=12)
        ax.set_ylabel('Z-statistic Threshold', fontsize=12)
        ax.set_title(f'{self.boundary_type.upper()} Boundaries '
                     f'(K={self.n_looks},  ={self.alpha})', fontsize
                        =14)
        ax.set_xticks(looks)
        ax.grid(alpha=0.3)
        ax.legend(fontsize=11)

        # Add information fractions
        for i, (look, frac) in enumerate(zip(looks, self.info_fracs)):
            ax.text(look, self.boundaries[i] + 0.3,
                    f'{frac:.1%}', ha='center', fontsize=10)

        plt.tight_layout()
```

```
        return fig

def simulate_sequential_experiment(
    true_effect: float,
    baseline_rate: float,
    max_n_per_group: int,
    n_looks: int,
    boundary_type: str,
    n_simulations: int = 1000
) -> dict:
    """
␣␣␣␣Simulate␣group␣sequential␣experiments.

␣␣␣␣Returns␣type␣I␣error,␣power,␣and␣average␣sample␣size.
␣␣␣␣"""
    gst = GroupSequentialTest(
        max_n=max_n_per_group,
        n_looks=n_looks,
        boundary_type=boundary_type
    )

    stopped_for_efficacy = 0
    stopped_for_futility = 0
    total_n = []

    p1 = baseline_rate
    p2 = baseline_rate + true_effect

    for sim in range(n_simulations):
        np.random.seed(sim)

        stopped = False

        for look in range(1, n_looks + 1):
            n_current = int(max_n_per_group * look / n_looks)

            # Generate data
            control = np.random.binomial(1, p1, n_current)
            treatment = np.random.binomial(1, p2, n_current)

            result = gst.analyze_look(
                n_current=n_current,
                control_conversions=control.sum(),
                treatment_conversions=treatment.sum(),
                control_n=n_current,
                treatment_n=n_current
            )

            if result['stop_efficacy']:
                stopped_for_efficacy += 1
                total_n.append(n_current * 2)
                stopped = True
                break
            elif result['stop_futility']:
                stopped_for_futility += 1
                total_n.append(n_current * 2)
                stopped = True
                break
```

```
218
219         if not stopped:
220             total_n.append(max_n_per_group * 2)
221
222     return {
223         'type_i_error' if true_effect == 0 else 'power':
224             stopped_for_efficacy / n_simulations,
225         'stopped_for_futility_rate': stopped_for_futility / n_
               simulations,
226         'average_sample_size': np.mean(total_n),
227         'median_sample_size': np.median(total_n),
228         'sample_size_reduction': 1 - np.mean(total_n) / (max_n_per_
               group * 2)
229     }
230
231 # Example usage
232 print("===␣Group␣Sequential␣Testing␣===\n")
233
234 # Visualize boundaries
235 gst_obf = GroupSequentialTest(max_n=5000, n_looks=5, boundary_type='obf
       ')
236 gst_pocock = GroupSequentialTest(max_n=5000, n_looks=5, boundary_type='
       pocock')
237
238 print("O'Brien-Fleming␣Boundaries:")
239 print(gst_obf.boundaries)
240 print("\nPocock␣Boundaries:")
241 print(gst_pocock.boundaries)
242
243 fig_obf = gst_obf.visualize_boundaries()
244 plt.savefig('obf_boundaries.pdf')
245
246 fig_pocock = gst_pocock.visualize_boundaries()
247 plt.savefig('pocock_boundaries.pdf')
248
249 # Simulate to check Type I error control
250 print("\n===␣Simulation␣Results␣(1000␣replications)␣===")
251 print("\nUnder␣null␣(true_effect␣=␣0):")
252
253 for boundary_type in ['obf', 'pocock']:
254     result = simulate_sequential_experiment(
255         true_effect=0.0,
256         baseline_rate=0.10,
257         max_n_per_group=5000,
258         n_looks=5,
259         boundary_type=boundary_type,
260         n_simulations=1000
261     )
262
263     print(f"\n{boundary_type.upper()}:")
264     print(f"␣␣Type␣I␣error:␣{result['type_i_error']:.3f}␣"
265           f"(target:␣0.05)")
266     print(f"␣␣Avg␣sample␣size:␣{result['average_sample_size']:.0f}␣"
267           f"({result['sample_size_reduction']:.1%}␣reduction)")
268
269 print("\nUnder␣alternative␣(true_effect␣=␣0.02):")
270
271 for boundary_type in ['obf', 'pocock']:
```

```
    result = simulate_sequential_experiment(
        true_effect=0.02,
        baseline_rate=0.10,
        max_n_per_group=5000,
        n_looks=5,
        boundary_type=boundary_type,
        n_simulations=1000
    )

    print(f"\n{boundary_type.upper()}:")
    print(f"  Power: {result['power']:.3f}")
    print(f"  Avg sample size: {result['average_sample_size']:.0f} "
          f"({result['sample_size_reduction']:.1%} reduction)")
```

8.5 Network Effects and Interference

When users interact, standard A/B testing assumptions break down. One user's treatment can affect another user's outcome.

8.5.1 Types of Interference

1. **Direct Network Effects:** User A's treatment directly affects User B

   Example: Social network feature where treated users send more friend requests, affecting control users

2. **Marketplace Effects:** Two-sided platforms with supply-demand dynamics

   Example: Ride-sharing app where increasing driver supply in treatment affects wait times for all users

3. **Competition Effects:** Users compete for limited resources

   Example: Bidding platform where treated users' behavior affects prices for everyone

Consequence: Treatment effect estimates are biased. May underestimate (positive spillover) or overestimate (negative spillover) true effects.

8.5.2 SUTVA Violation

The Stable Unit Treatment Value Assumption (SUTVA) requires:

1. No interference: User i's outcome depends only on their own treatment
2. Consistency: Treatment is well-defined and consistently applied

Network effects violate SUTVA.

8.5.3 Mitigation Strategies

1. Cluster Randomization

Randomize groups of connected users together.

Example: For social network experiment, randomize entire friend groups or geographic regions.

Trade-off: Reduces interference but decreases effective sample size.

2. Ego Network Randomization

Randomize both focal user and their immediate neighbors to same treatment.
Advantage: Reduces spillover for randomized pairs.
Limitation: Difficult with overlapping networks.

3. Temporal Switchbacks

Alternate entire system between control and treatment over time periods.

Example 8.4 (Switchback Design)**.** Week 1: Monday (Control), Tuesday (Treatment), Wednesday (Control), ...
Week 2: Monday (Treatment), Tuesday (Control), ...
Compare outcomes between control and treatment periods.

Analysis: Must account for temporal correlation and carryover effects.

8.6 Switchback Designs

Switchback (crossover) designs are particularly useful for marketplace and platform experiments.

8.6.1 Structure

Alternate between control and treatment conditions over time, typically:

- Geographic units (cities, regions)
- Time periods (hours, days, weeks)

City	Week 1	Week 2	Week 3	Week 4
New York	Control	Treatment	Control	Treatment
San Francisco	Treatment	Control	Treatment	Control
Chicago	Control	Treatment	Control	Treatment
Boston	Treatment	Control	Treatment	Control

8.6.2 Analysis

Use mixed effects model or difference-in-differences:

$$Y_{it} = \alpha_i + \beta T_{it} + \gamma_t + \epsilon_{it} \tag{8.8}$$

where:

- α_i: Unit fixed effect
- T_{it}: Treatment indicator
- γ_t: Time fixed effect
- β: Treatment effect (estimand)

Key Assumption: No carryover effects, or sufficient washout period between switches.

8.6.3 Practical Considerations

- **Switch frequency:** Balance temporal correlation vs. statistical power
- **Washout periods:** Allow for treatment effects to dissipate
- **Temporal trends:** Control for day-of-week effects, seasonality
- **Synchronization:** Coordinate switches across units

8.7 Summary

Advanced experimental designs extend A/B testing capabilities:

- **Factorial designs** efficiently test multiple factors and detect interactions
- **Multi-armed bandits** optimize allocation during experiments, minimizing regret
- **Sequential testing** allows early stopping while controlling error rates
- **Network-aware designs** handle interference when users interact
- **Switchback designs** address marketplace and platform challenges

Each design involves trade-offs between efficiency, complexity, and validity. Choose designs based on:

- Business objectives (optimization vs. inference)
- User interaction structure (independent vs. networked)
- Resource constraints (sample size, time)
- Stakeholder requirements (interpretability, regulatory)

8.8 Further Reading

- **?**: Comprehensive treatment of factorial designs
- **?**: Theoretical foundations of multi-armed bandits
- **?**: Group sequential methods for clinical trials
- **?**: Design and analysis with network effects
- **?**: Cluster randomized trials

8.9 Exercises

8.9.1 Conceptual Questions

1. Explain when a 2^3 full factorial design would be preferable to three separate A/B tests, and when separate tests might be better.

2. Describe a scenario where Thompson Sampling would be preferable to a traditional A/B test, and explain your reasoning.

3. Why does "peeking" at A/B test results inflate Type I error? How do group sequential methods address this?

4. Give an example of a product feature that would violate SUTVA due to network effects. Propose an experimental design to test this feature.

8.9.2 Calculation Exercises

1. For a 2^4 factorial design with 200 observations per cell:

   (a) How many total observations are needed?

   (b) How many parameters can be estimated (main effects + interactions)?

   (c) What would be the effective sample size for estimating each main effect?

2. Calculate the UCB1 selection criterion for three arms with:

   - Arm 1: $\hat{\mu}_1 = 0.10$, $N_1 = 100$
   - Arm 2: $\hat{\mu}_2 = 0.12$, $N_2 = 50$
   - Arm 3: $\hat{\mu}_3 = 0.11$, $N_3 = 150$

   At time $t = 301$, which arm should be selected?

3. For O'Brien-Fleming boundaries with $K = 4$ looks and $\alpha = 0.05$, if the first look yields $Z = 2.5$, should you stop? What if the second look yields $Z = 2.8$?

8.9.3 Programming Exercises

1. Implement a complete factorial experiment simulator:

   - Generate data for 2^k designs with specified effects
   - Analyze using ANOVA
   - Create interaction plots
   - Calculate power for detecting interactions

2. Compare ϵ-greedy, UCB1, and Thompson Sampling:

   - Simulate 10 arms with various true rates
   - Run 10,000 trials for each algorithm
   - Plot cumulative regret

- Analyze convergence properties

3. Implement group sequential testing:

   - Create O'Brien-Fleming and Pocock boundaries
   - Simulate experiments under null and alternative
   - Verify Type I error control
   - Measure average sample size savings

8.9.4 Applied Exercises

1. **E-commerce Factorial Design:** Design a 2^3 experiment testing:

   - Product image size (small vs. large)
   - Price display (with vs. without discount)
   - Reviews prominence (top vs. bottom)

   Specify sample size, analysis plan, and expected interactions.

2. **Content Recommendation Bandit:** A news app wants to optimize article headlines dynamically:

   - 5 headline variants per article
   - Goal: maximize click-through rate
   - New headlines added continuously

   Recommend a bandit algorithm and justify your choice.

3. **Marketplace Experiment:** A ride-sharing platform wants to test dynamic pricing:

   - Treatment affects all riders and drivers in treated regions
   - Spillover between nearby regions
   - Time-varying demand

   Design an experiment accounting for interference. Specify randomization unit, analysis method, and duration.

4. **Sequential Testing Strategy:** Your company typically runs A/B tests for 2 weeks with 10,000 users per variant. Design a sequential testing procedure that could reduce average duration by 30% while maintaining $\alpha = 0.05$ and power = 80%.

8.9.5 Advanced Projects

1. **Bandit with Delayed Feedback:** Extend Thompson Sampling to handle rewards observed with delay (e.g., 7-day retention). Simulate and compare to standard Thompson Sampling.

2. **Network Interference Simulation:** Create a social network graph (1000 nodes) and simulate an experiment with spillover effects. Compare:

   - Individual randomization (biased)
   - Cluster randomization (less power)
   - Ego-network randomization

   Measure bias and power for each approach.

3. **Adaptive Factorial Design:** Implement a factorial design that drops unpromising factor levels based on interim data, while controlling error rates.

Chapter 9

Technical Implementation

9.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design robust experimentation infrastructure
- Implement randomization and assignment systems
- Build reliable logging and data collection pipelines
- Integrate A/B testing into applications
- Debug common implementation issues

9.2 Experimentation Infrastructure

A robust A/B testing system requires careful technical implementation.

9.2.1 Core Components

1. **Randomization Service**: Assigns users to variants
2. **Configuration Management**: Stores experiment definitions
3. **Logging System**: Records assignments and events
4. **Analysis Pipeline**: Computes metrics and statistical tests
5. **Monitoring Dashboard**: Tracks experiment health

9.2.2 Architecture Patterns

Client-side randomization:

- Randomization happens in browser/app
- Lightweight, no server dependency
- Risk of manipulation

Server-side randomization:

- Centralized assignment service
- More secure and consistent
- Requires infrastructure

Hybrid approach:

- Server assigns, client implements
- Balance of benefits

9.3 Randomization Implementation

9.3.1 Hash-Based Randomization

Use deterministic hashing for consistent assignments across sessions.

Listing 9.1: Hash-Based User Assignment

```
import hashlib

def assign_variant(user_id, experiment_id, num_variants=2,
                   traffic_allocation=1.0, salt=''):
    """
␣␣␣␣Assign␣user␣to␣variant␣using␣deterministic␣hashing.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣user_id␣:␣str
␣␣␣␣␣␣␣␣Unique␣user␣identifier
␣␣␣␣experiment_id␣:␣str
␣␣␣␣␣␣␣␣Unique␣experiment␣identifier
␣␣␣␣num_variants␣:␣int
␣␣␣␣␣␣␣␣Number␣of␣variants
␣␣␣␣traffic_allocation␣:␣float
␣␣␣␣␣␣␣␣Fraction␣of␣users␣to␣include␣(0.0-1.0)
␣␣␣␣salt␣:␣str
␣␣␣␣␣␣␣␣Random␣salt␣for␣security

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣variant␣:␣int␣or␣None
␣␣␣␣␣␣␣␣Assigned␣variant␣(0␣to␣num_variants-1)␣or␣None␣if␣not␣allocated
␣␣␣␣"""
    # Create hash input
    hash_input = f"{user_id}:{experiment_id}:{salt}".encode('utf-8')

    # Generate hash
    hash_output = hashlib.md5(hash_input).hexdigest()

    # Convert to integer and normalize to [0, 1]
    hash_int = int(hash_output, 16)
    hash_fraction = (hash_int % 10000) / 10000.0

```

```
    # Check traffic allocation
    if hash_fraction >= traffic_allocation:
        return None  # User not in experiment

    # Assign to variant
    variant_fraction = hash_fraction / traffic_allocation
    variant = int(variant_fraction * num_variants)

    return variant

# Example usage
user_id = "user_12345"
experiment_id = "button_color_test"

# Consistent assignment
for _ in range(3):
    variant = assign_variant(user_id, experiment_id, num_variants=2)
    print(f"User {user_id} assigned to variant: {variant}")

# Different experiments give different assignments
for exp_id in ["exp_1", "exp_2", "exp_3"]:
    variant = assign_variant(user_id, exp_id)
    print(f"Experiment {exp_id}: variant {variant}")
```

Key properties:

- Deterministic: same user always gets same variant
- Independent: different experiments independent
- Uniform: variants equally likely
- Secure: hard to game with salt

9.3.2 Handling Edge Cases

1. **New vs. returning users:** Ensure consistent assignment
2. **Logged out users:** Use cookies with careful handling
3. **Multi-device users:** Decision: device-level or user-level?
4. **Bots and crawlers:** Filter before assignment
5. **Internal traffic:** Exclude or separate analysis

9.4 Configuration Management

Store experiment definitions in a centralized system.

Listing 9.2: Experiment Configuration Schema

```
import json
from datetime import datetime

class ExperimentConfig:
```

```
    """Experiment configuration manager."""

    def __init__(self, config_path):
        with open(config_path, 'r') as f:
            self.config = json.load(f)

    def get_experiment(self, experiment_id):
        """Retrieve experiment configuration."""
        return self.config.get('experiments', {}).get(experiment_id)

    def is_active(self, experiment_id):
        """Check if experiment is currently active."""
        exp = self.get_experiment(experiment_id)
        if not exp:
            return False

        now = datetime.now()
        start = datetime.fromisoformat(exp.get('start_date'))
        end = datetime.fromisoformat(exp.get('end_date'))

        return start <= now <= end and exp.get('status') == 'active'

# Example configuration file (JSON)
example_config = {
    "experiments": {
        "checkout_button_test": {
            "name": "Checkout Button Color Test",
            "description": "Testing red vs blue checkout button",
            "owner": "product_team@example.com",
            "start_date": "2025-01-01T00:00:00",
            "end_date": "2025-01-31T23:59:59",
            "status": "active",
            "variants": [
                {"id": 0, "name": "control", "allocation": 0.5},
                {"id": 1, "name": "treatment", "allocation": 0.5}
            ],
            "metrics": {
                "primary": "conversion_rate",
                "secondary": ["revenue_per_user", "bounce_rate"],
                "guardrails": ["page_load_time", "error_rate"]
            },
            "targeting": {
                "countries": ["US", "CA", "UK"],
                "platforms": ["web", "mobile_web"]
            }
        }
    }
}
```

9.5 Logging and Data Collection

9.5.1 Event Schema

Design a comprehensive event schema capturing all necessary data.

Listing 9.3: Event Logging System

```
import uuid
from datetime import datetime
import json

class ExperimentLogger:
    """Log experiment assignments and events."""

    def __init__(self, storage_backend):
        self.storage = storage_backend

    def log_assignment(self, user_id, experiment_id, variant,
                       timestamp=None, metadata=None):
        """Log user assignment to experiment variant."""
        event = {
            'event_id': str(uuid.uuid4()),
            'event_type': 'assignment',
            'timestamp': timestamp or datetime.utcnow().isoformat(),
            'user_id': user_id,
            'experiment_id': experiment_id,
            'variant': variant,
            'metadata': metadata or {}
        }

        self.storage.write(event)
        return event

    def log_conversion(self, user_id, experiment_id, conversion_type,
                       value=None, timestamp=None, metadata=None):
        """Log conversion event."""
        event = {
            'event_id': str(uuid.uuid4()),
            'event_type': 'conversion',
            'timestamp': timestamp or datetime.utcnow().isoformat(),
            'user_id': user_id,
            'experiment_id': experiment_id,
            'conversion_type': conversion_type,
            'value': value,
            'metadata': metadata or {}
        }

        self.storage.write(event)
        return event

# Example: In-memory storage for demonstration
class InMemoryStorage:
    def __init__(self):
        self.events = []

    def write(self, event):
        self.events.append(event)

    def query(self, event_type=None, experiment_id=None):
        results = self.events
        if event_type:
            results = [e for e in results if e['event_type'] == event_
               type]
        if experiment_id:
```

```
            results = [e for e in results
                       if e.get('experiment_id') == experiment_id]
        return results

# Usage
storage = InMemoryStorage()
logger = ExperimentLogger(storage)

# Log assignment
logger.log_assignment('user_001', 'exp_123', variant=1,
                      metadata={'device': 'mobile'})

# Log conversion
logger.log_conversion('user_001', 'exp_123', 'purchase',
                      value=49.99, metadata={'product_id': 'SKU_789'})

# Query events
assignments = storage.query(event_type='assignment',
                            experiment_id='exp_123')
print(f"Found {len(assignments)} assignment events")
```

9.5.2 Data Quality Checks

Implement automated checks:

1. **Sample Ratio Mismatch (SRM):** Verify allocation matches expectations
2. **Assignment Consistency:** Users don't switch variants
3. **Timestamp Validity:** Events in reasonable order
4. **Data Completeness:** No missing critical fields
5. **Metric Sanity:** Values within expected ranges

Listing 9.4: Data Quality Validation

```
def validate_experiment_data(assignments_df, events_df):
    """
    Run data quality checks on experiment data.

    Returns:
    --------
    dict : Validation results with issues flagged
    """
    issues = []

    # Check 1: Sample Ratio Mismatch
    variant_counts = assignments_df['variant'].value_counts()
    expected_ratio = 1.0 / len(variant_counts)
    actual_ratios = variant_counts / len(assignments_df)

    from scipy.stats import chisquare
    expected = [len(assignments_df) * expected_ratio] * len(variant_
        counts)
    chi2, p_value = chisquare(variant_counts.values, expected)
```

```
19
20      if p_value < 0.01:
21          issues.append({
22              'type': 'SRM',
23              'severity': 'high',
24              'message': f'Sample ratio mismatch detected (p={p_value:.4f
                    })',
25              'details': dict(variant_counts)
26          })
27
28      # Check 2: Multiple assignments
29      multi_assigned = assignments_df.groupby('user_id')['variant'].
            nunique()
30      multi_assigned_users = (multi_assigned > 1).sum()
31
32      if multi_assigned_users > 0:
33          issues.append({
34              'type': 'consistency',
35              'severity': 'high',
36              'message': f'{multi_assigned_users} users assigned to
                    multiple variants',
37          })
38
39      # Check 3: Events before assignment
40      merged = events_df.merge(assignments_df, on='user_id', how='left')
41      merged['assignment_timestamp'] = pd.to_datetime(merged['assignment_
            timestamp'])
42      merged['event_timestamp'] = pd.to_datetime(merged['event_timestamp'
            ])
43
44      early_events = (merged['event_timestamp'] <
45                      merged['assignment_timestamp']).sum()
46
47      if early_events > 0:
48          issues.append({
49              'type': 'timing',
50              'severity': 'medium',
51              'message': f'{early_events} events occurred before
                    assignment'
52          })
53
54      return {
55          'valid': len(issues) == 0,
56          'issues': issues,
57          'checks_run': ['SRM', 'consistency', 'timing']
58      }
```

9.6 Integration Patterns

9.6.1 Feature Flags Integration

Combine experimentation with feature flag systems.

Listing 9.5: Feature Flag with A/B Testing

```
1 class FeatureFlagService:
```

```
    """Combined feature flags and A/B testing."""

    def __init__(self, experiment_service):
        self.experiment_service = experiment_service
        self.permanent_flags = {}

    def is_enabled(self, flag_name, user_id, default=False):
        """
        Check if feature is enabled for user.

        Supports:
        - Boolean flags (on/off for all users)
        - Percentage rollouts
        - A/B test variants
        """
        # Check permanent flags first
        if flag_name in self.permanent_flags:
            return self.permanent_flags[flag_name]

        # Check if it's an A/B test
        if self.experiment_service.is_experiment(flag_name):
            variant = self.experiment_service.get_variant(flag_name,
                user_id)
            return variant == 1  # Treat variant 1 as "enabled"

        return default

    def get_variant(self, experiment_name, user_id):
        """Get experiment variant for user."""
        return self.experiment_service.get_variant(experiment_name,
            user_id)
```

9.6.2 Metrics Collection

Integrate with existing analytics systems.

1. **Passive tracking:** Automatic event capture
2. **Active instrumentation:** Explicit metric logging
3. **Computed metrics:** Derived from raw events
4. **Real-time aggregation:** Pre-compute for dashboards

9.7 Common Implementation Pitfalls

9.7.1 Assignment Bugs

- **Issue:** Inconsistent assignments across sessions
- **Cause:** Not using stable user IDs
- **Fix:** Hash-based deterministic assignment

9.7.2 Survivorship Bias

- **Issue:** Only analyzing users who complete flows
- **Cause:** Filtering out non-converters
- **Fix:** Intent-to-treat analysis

9.7.3 Logging Failures

- **Issue:** Missing assignment or event data
- **Cause:** Client-side logging dropped
- **Fix:** Server-side logging, retry logic, monitoring

9.8 Summary

Robust implementation is critical for trustworthy A/B testing:

- Use deterministic hash-based randomization
- Maintain centralized configuration management
- Implement comprehensive logging with quality checks
- Integrate with feature flags and analytics
- Monitor for common implementation issues

Proper infrastructure enables scaling experimentation across the organization.

9.9 Further Reading

- **?**: Implementation best practices
- **?**: Experiment system architecture
- Open source platforms: Optimizely, GrowthBook, Unleash

9.10 Exercises

1. Implement complete hash-based randomization with traffic allocation and salting
2. Build logging system with SRM detection and automated alerts
3. Design experiment configuration schema supporting complex targeting
4. Create feature flag service integrating A/B tests and gradual rollouts

Chapter 10

Data Analysis and Result Interpretation

10.1 Learning Objectives

After completing this chapter, readers will be able to:

- Conduct end-to-end analysis of A/B test results
- Handle missing data and outliers appropriately
- Perform variance reduction techniques
- Interpret results for stakeholders
- Make sound business decisions from statistical evidence

10.2 Analysis Workflow

A systematic approach ensures thorough and accurate analysis.

10.2.1 Pre-Analysis Checks

Before analyzing results, verify data quality:

1. **Sample Ratio Mismatch:** Check allocation matches design
2. **Assignment Consistency:** Verify users don't switch variants
3. **Novelty Effects:** Examine temporal patterns
4. **External Validity Threats:** Check for anomalous events
5. **Data Completeness:** Assess missing data patterns

Listing 10.1: Pre-Analysis Data Quality Checks

```
import pandas as pd
import numpy as np
from scipy.stats import chisquare
import matplotlib.pyplot as plt

def preanalysis_checks(data):
    """
␣␣␣␣Run␣comprehensive␣pre-analysis␣checks.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣data␣:␣DataFrame
␣␣␣␣␣␣␣␣Must␣contain:␣user_id,␣variant,␣timestamp,␣conversion

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣dict␣:␣Check␣results␣and␣warnings
␣␣␣␣"""
    warnings = []

    # 1. Sample Ratio Mismatch
    variant_counts = data['variant'].value_counts()
    expected = [len(data) / len(variant_counts)] * len(variant_counts)
    chi2, srm_p_value = chisquare(variant_counts.values, expected)

    if srm_p_value < 0.01:
        warnings.append(f"SRM␣detected:␣p-value␣=␣{srm_p_value:.4f}")

    # 2. Temporal patterns
    data['date'] = pd.to_datetime(data['timestamp']).dt.date
    daily_data = data.groupby(['date', 'variant'])['conversion'].mean()
        .unstack()

    # Check for systematic time trends
    daily_variance = daily_data.var()
    if (daily_variance > 0.01).any():
        warnings.append("High␣temporal␣variance␣detected")

    # 3. Multiple assignments
    multi_assigned = data.groupby('user_id')['variant'].nunique()
    n_multi = (multi_assigned > 1).sum()

    if n_multi > 0:
        warnings.append(f"{n_multi}␣users␣with␣multiple␣variant␣
            assignments")

    # 4. Missing data
    missing_pct = data.isnull().sum() / len(data) * 100
    if (missing_pct > 5).any():
        warnings.append(f"High␣missing␣data:␣{dict(missing_pct[missing_
            pct␣>␣5])}")

    return {
        'srm_p_value': srm_p_value,
        'variant_counts': dict(variant_counts),
        'warnings': warnings,
        'passed': len(warnings) == 0
```

```
    }

# Example usage
# data = pd.read_csv('experiment_data.csv')
# checks = preanalysis_checks(data)
# print(f"Pre-analysis checks: {'PASSED' if checks['passed'] else '
   WARNINGS'}")
# for warning in checks['warnings']:
#     print(f"  - {warning}")
```

10.2.2 Exploratory Data Analysis

Visualize and summarize data before formal testing.

Listing 10.2: Exploratory Analysis Dashboard

```
def create_eda_dashboard(data):
    """Create exploratory analysis visualizations."""
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # 1. Overall conversion rates
    conversion_by_variant = data.groupby('variant')['conversion'].agg([
        ('mean', 'mean'),
        ('std', 'std'),
        ('count', 'count')
    ])

    axes[0, 0].bar(conversion_by_variant.index,
                   conversion_by_variant['mean'],
                   yerr=conversion_by_variant['std'] /
                        np.sqrt(conversion_by_variant['count']))
    axes[0, 0].set_xlabel('Variant')
    axes[0, 0].set_ylabel('Conversion Rate')
    axes[0, 0].set_title('Conversion Rate by Variant')
    axes[0, 0].grid(alpha=0.3)

    # 2. Time series
    data['date'] = pd.to_datetime(data['timestamp']).dt.date
    daily_cr = data.groupby(['date', 'variant'])['conversion'].mean().
       unstack()

    for variant in daily_cr.columns:
        axes[0, 1].plot(daily_cr.index, daily_cr[variant],
                        marker='o', label=f'Variant {variant}')

    axes[0, 1].set_xlabel('Date')
    axes[0, 1].set_ylabel('Daily Conversion Rate')
    axes[0, 1].set_title('Conversion Rate Over Time')
    axes[0, 1].legend()
    axes[0, 1].grid(alpha=0.3)
    plt.setp(axes[0, 1].xaxis.get_majorticklabels(), rotation=45)

    # 3. Distribution of continuous metric (if available)
    if 'value' in data.columns:
        for variant in data['variant'].unique():
            variant_data = data[data['variant'] == variant]['value'].
               dropna()
```

```
            axes[1, 0].hist(variant_data, bins=50, alpha=0.5,
                            label=f'Variant {variant}')

        axes[1, 0].set_xlabel('Value')
        axes[1, 0].set_ylabel('Frequency')
        axes[1, 0].set_title('Distribution of Values by Variant')
        axes[1, 0].legend()
        axes[1, 0].grid(alpha=0.3)

    # 4. Sample size accumulation
    data_sorted = data.sort_values('timestamp')
    data_sorted['cumulative'] = range(1, len(data_sorted) + 1)

    for variant in data['variant'].unique():
        variant_data = data_sorted[data_sorted['variant'] == variant]
        axes[1, 1].plot(variant_data['timestamp'],
                        variant_data['cumulative'],
                        label=f'Variant {variant}')

    axes[1, 1].set_xlabel('Time')
    axes[1, 1].set_ylabel('Cumulative Sample Size')
    axes[1, 1].set_title('Sample Size Accumulation')
    axes[1, 1].legend()
    axes[1, 1].grid(alpha=0.3)

    plt.tight_layout()
    return fig
```

10.3 Handling Data Issues

10.3.1 Missing Data

Missing completely at random (MCAR): Missing mechanism independent of data
Missing at random (MAR): Missing depends on observed data
Missing not at random (MNAR): Missing depends on unobserved data
Strategies:

1. **Complete case analysis:** Drop missing (if MCAR and limited missingness)
2. **Imputation:** Fill with mean, median, or model predictions
3. **Multiple imputation:** Account for imputation uncertainty
4. **Sensitivity analysis:** Test robustness to missing data assumptions

10.3.2 Outliers

Extreme values can bias estimates, especially for means.
Detection methods:

- Visual inspection (box plots, scatter plots)
- Statistical rules (e.g., beyond 3 standard deviations)

- Domain knowledge (impossible or implausible values)

Handling approaches:

1. **Winsorization:** Cap at percentiles (e.g., 1st and 99th)
2. **Transformation:** Log transform to reduce skewness
3. **Robust statistics:** Use median instead of mean
4. **Separate analysis:** Analyze with and without outliers

Listing 10.3: Outlier Treatment

```
def winsorize_data(data, column, lower_percentile=1, upper_percentile
    =99):
    """
␣␣␣␣Winsorize␣data␣by␣capping␣at␣specified␣percentiles.

␣␣␣␣Parameters:
␣␣␣␣-----------
␣␣␣␣data␣:␣DataFrame
␣␣␣␣␣␣␣␣Data␣to␣winsorize
␣␣␣␣column␣:␣str
␣␣␣␣␣␣␣␣Column␣to␣winsorize
␣␣␣␣lower_percentile,␣upper_percentile␣:␣float
␣␣␣␣␣␣␣␣Percentile␣thresholds

␣␣␣␣Returns:
␣␣␣␣--------
␣␣␣␣DataFrame␣:␣Data␣with␣winsorized␣column
␣␣␣␣"""
    lower_bound = np.percentile(data[column], lower_percentile)
    upper_bound = np.percentile(data[column], upper_percentile)

    data[f'{column}_winsorized'] = data[column].clip(
        lower=lower_bound,
        upper=upper_bound
    )

    n_capped = ((data[column] < lower_bound) |
                (data[column] > upper_bound)).sum()

    print(f"Winsorized␣{n_capped}␣values␣({n_capped/len(data)*100:.2f
        }%)")
    print(f"␣␣Bounds:␣[{lower_bound:.2f},␣{upper_bound:.2f}]")

    return data

# Compare analyses
def compare_outlier_treatment(data, metric_col):
    """Compare␣results␣with␣different␣outlier␣treatments."""
    results = {}

    # Original data
    for variant in data['variant'].unique():
        variant_data = data[data['variant'] == variant][metric_col]
        results[f'variant_{variant}_original'] = {
```

```
            'mean': variant_data.mean(),
            'median': variant_data.median(),
            'std': variant_data.std()
        }

    # Winsorized data
    data_winsorized = winsorize_data(data.copy(), metric_col)
    for variant in data['variant'].unique():
        variant_data = data_winsorized[
            data_winsorized['variant'] == variant
        ][f'{metric_col}_winsorized']
        results[f'variant_{variant}_winsorized'] = {
            'mean': variant_data.mean(),
            'median': variant_data.median(),
            'std': variant_data.std()
        }

    return pd.DataFrame(results).T
```

10.4 Variance Reduction Techniques

Reducing variance increases power without additional sample size.

10.4.1 CUPED (Controlled-Experiment Using Pre-Experiment Data)

Use pre-experiment covariates to reduce variance.

$$Y_{\text{adjusted}} = Y - \theta(X - \mathbb{E}[X]) \tag{10.1}$$

where X is pre-experiment covariate and $\theta = \frac{\text{Cov}(Y,X)}{\text{Var}(X)}$.

Variance reduction:

$$\text{Var}(Y_{\text{adjusted}}) = \text{Var}(Y)(1 - \rho^2) \tag{10.2}$$

where $\rho = \text{Cor}(Y, X)$.

Listing 10.4: CUPED Implementation

```
cuped_adjustment <- function(y, x) {
  # y: outcome metric
  # x: pre-experiment covariate

  # Estimate theta
  theta <- cov(y, x) / var(x)

  # Adjust outcome
  y_adjusted <- y - theta * (x - mean(x))

  # Calculate variance reduction
  variance_reduction <- 1 - (var(y_adjusted) / var(y))

  list(
    y_adjusted = y_adjusted,
    theta = theta,
```

```
    variance_reduction = variance_reduction,
    original_var = var(y),
    adjusted_var = var(y_adjusted)
  )
}

# Example
set.seed(123)
n <- 1000

# Pre-experiment metric (e.g., baseline conversion)
x_pre <- rbinom(n, 1, 0.10)

# Experiment outcome (correlated with baseline)
y <- rbinom(n, 1, 0.10 + 0.2 * x_pre)

# Apply CUPED
result <- cuped_adjustment(y, x_pre)

cat("Variance reduction:", result$variance_reduction, "\n")
cat("Original variance:", result$original_var, "\n")
cat("Adjusted variance:", result$adjusted_var, "\n")
```

10.4.2 Stratification

Analyze within strata and combine results.

Benefits:

- Increases precision
- Ensures balance on stratification variables
- Enables subgroup analysis

10.5 Result Interpretation

10.5.1 Statistical vs. Practical Significance

Report both:

1. **Statistical significance:** P-value and confidence interval
2. **Effect size:** Absolute and relative differences
3. **Business impact:** Expected revenue/user impact
4. **Confidence interval:** Range of plausible effects

Example 10.1 (Interpretation Template)**.** "The treatment increased conversion rate from 8.0% to 8.5%, a relative increase of 6.25% (95% CI: [2%, 11%], p = 0.01). This translates to an estimated 500 additional conversions per month and $25,000 in additional revenue (95% CI: [$10,000, $40,000])."

10.5.2 Communicating Uncertainty

Always quantify uncertainty:

- Report confidence intervals, not just point estimates
- Visualize uncertainty with error bars
- Discuss what we cannot conclude
- Consider robustness under different assumptions

10.6 Decision Making Framework

10.6.1 Quantitative Criteria

1. **Statistical significance:** $p < 0.05$ (or chosen α)
2. **Practical significance:** Effect exceeds minimum meaningful threshold
3. **Guardrail metrics:** No degradation on critical metrics
4. **Cost-benefit:** Expected value exceeds implementation cost

10.6.2 Qualitative Factors

- **Strategic alignment:** Fits product vision
- **User experience:** Beyond measured metrics
- **Technical debt:** Long-term maintainability
- **Competitive dynamics:** Market positioning
- **Brand considerations:** Perception and values

10.6.3 Decision Matrix

Statistical	Practical	Decision	Action
Significant	Significant	Ship	Implement
Significant	Not significant	Hold	More investigation
Not significant	Significant	Iterate	Refine and retest
Not significant	Not significant	Kill	Abandon

10.7 Summary

Thorough analysis and interpretation are critical:

- Conduct pre-analysis quality checks
- Handle missing data and outliers appropriately

- Apply variance reduction techniques
- Report both statistical and practical significance
- Communicate uncertainty clearly
- Make decisions combining quantitative and qualitative factors

Good analysis transforms data into actionable insights.

10.8 Further Reading

- **?**: CUPED methodology
- **?**: Interpretation best practices
- **?**: Missing data analysis

10.9 Exercises

1. Implement complete analysis pipeline including quality checks and variance reduction
2. Compare power with and without CUPED for varying correlations
3. Create stakeholder report template with visualizations
4. Build decision framework incorporating business constraints

Chapter 11

Case Studies and Real-World Applications

11.1 Learning Objectives

After completing this chapter, readers will be able to:

- Learn from real-world A/B testing successes and failures with detailed statistical analysis
- Understand common pitfalls through case examples with actual data
- Apply quantitative frameworks to similar business problems
- Recognize industry-specific considerations and adapt methods accordingly
- Conduct post-mortem analysis of experiments using provided templates

11.2 Case Study 1: Google Search Results and Page Load Time

11.2.1 Context and Business Problem

In 2009, Google tested whether displaying more search results per page (20 or 30 instead of 10) would improve user engagement. The hypothesis seemed reasonable: fewer clicks to pagination should improve user experience.

11.2.2 Experimental Design

Design: A/B/C test with three variants

- Control: 10 results per page (baseline)
- Variant A: 20 results per page
- Variant B: 30 results per page

Randomization: User-level randomization, 33.3% allocation each
Duration: 3 weeks
Sample size: Millions of users (Google traffic scale)
Metrics:

- Primary: Searches per user session
- Secondary: Page load time, click-through rate, user satisfaction

11.2.3 Results and Analysis

Metric	10 Results	20 Results	30 Results
Page load time (ms)	400	650	900
Load time increase	–	+63%	+125%
Searches per user	5.2	4.9	4.6
Relative change	–	-5.8%	-11.5%
Satisfaction score	7.8/10	7.5/10	7.2/10

Statistical significance: All differences highly significant ($p < 0.001$) due to massive sample size.

Correlation analysis: Strong negative correlation between page load time and searches per user:

$$\text{Elasticity} = \frac{\Delta\text{Searches}}{\Delta\text{Load time}} \approx -1.0 \tag{11.1}$$

A 100ms increase in load time corresponded to approximately 1% decrease in searches.

11.2.4 Deep Dive: Causal Mechanism

Why did more results decrease engagement?

1. **Performance degradation:** Rendering more results increased page weight and computation
2. **Perceived value:** Users valued speed over choice
3. **Cognitive overload:** More options paradoxically made decision harder
4. **User behavior:** Most users (>90%) found results in first 10 regardless

11.2.5 Statistical Analysis Framework

Listing 11.1: Analyzing Load Time vs. Engagement Trade-offs

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

# Simulated data based on Google's published results
```

```
np.random.seed(42)

def simulate_google_experiment(n_users=100000):
    """Simulate Google search results experiment."""

    # Allocate users to variants
    variants = np.random.choice(['10_results', '20_results', '30_
        results'],
                                size=n_users)

    # Simulate page load times (milliseconds)
    load_times = {
        '10_results': np.random.normal(400, 50),
        '20_results': np.random.normal(650, 70),
        '30_results': np.random.normal(900, 90)
    }

    # Simulate searches per user (Poisson with rate dependent on load
        time)
    # Model: slower load time -> lower engagement
    base_searches = {
        '10_results': 5.2,
        '20_results': 4.9,
        '30_results': 4.6
    }

    data = []
    for i, variant in enumerate(variants):
        load_time = load_times[variant]

        # Add user-level noise
        lambda_param = base_searches[variant] * np.random.lognormal(0,
            0.3)
        searches = np.random.poisson(lambda_param)

        # Satisfaction (0-10 scale, correlated with searches and load
            time)
        satisfaction = np.clip(
            7.8 - (load_time - 400) / 200 + np.random.normal(0, 1),
            0, 10
        )

        data.append({
            'user_id': i,
            'variant': variant,
            'load_time_ms': load_time,
            'searches': searches,
            'satisfaction': satisfaction
        })

    return pd.DataFrame(data)

# Generate data
data = simulate_google_experiment(n_users=100000)

# Aggregate statistics by variant
summary = data.groupby('variant').agg({
    'load_time_ms': ['mean', 'std'],
```

```
    'searches': ['mean', 'std'],
    'satisfaction': ['mean', 'std']
}).round(2)

print("=== Experiment Summary ===")
print(summary)

# Statistical tests
variants = ['10_results', '20_results', '30_results']

print("\n=== Pairwise Comparisons (Two-sample t-tests) ===")
for i in range(len(variants)):
    for j in range(i+1, len(variants)):
        v1, v2 = variants[i], variants[j]

        searches_1 = data[data['variant']==v1]['searches']
        searches_2 = data[data['variant']==v2]['searches']

        t_stat, p_value = stats.ttest_ind(searches_1, searches_2)

        diff = searches_2.mean() - searches_1.mean()
        rel_diff = diff / searches_1.mean() * 100

        print(f"\n{v1} vs {v2}:")
        print(f"  Mean difference: {diff:.3f} searches ({rel_diff:+.1f
            }%)")
        print(f"  t-statistic: {t_stat:.2f}")
        print(f"  p-value: {p_value:.2e}")

# Regression: searches ~ load_time
from sklearn.linear_model import LinearRegression

X = data[['load_time_ms']].values
y = data['searches'].values

model = LinearRegression()
model.fit(X, y)

print("\n=== Regression: Searches ~ Load Time ===")
print(f"Slope: {model.coef_[0]:.6f} searches per ms")
print(f"Interpretation: Each 100ms increase -> "
      f"{model.coef_[0] * 100:.2f} fewer searches")

# Visualization
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Load time by variant
ax = axes[0, 0]
data.boxplot(column='load_time_ms', by='variant', ax=ax)
ax.set_xlabel('Variant')
ax.set_ylabel('Page Load Time (ms)')
ax.set_title('Load Time Distribution by Variant')
plt.sca(ax)
plt.xticks(range(1, 4), ['10 results', '20 results', '30 results'])

# 2. Searches by variant
ax = axes[0, 1]
```

```
variant_means = data.groupby('variant')['searches'].agg(['mean', 'sem'])
variant_order = ['10_results', '20_results', '30_results']
means = [variant_means.loc[v, 'mean'] for v in variant_order]
sems = [variant_means.loc[v, 'sem'] for v in variant_order]

x = np.arange(3)
ax.bar(x, means, yerr=[1.96*s for s in sems], capsize=5, alpha=0.7)
ax.set_xticks(x)
ax.set_xticklabels(['10 results', '20 results', '30 results'])
ax.set_ylabel('Searches per User')
ax.set_title('Mean Searches by Variant (95% CI)')
ax.grid(axis='y', alpha=0.3)

# 3. Scatter: Load time vs searches
ax = axes[1, 0]
for variant, color in zip(variant_order, ['C0', 'C1', 'C2']):
    subset = data[data['variant']==variant].sample(1000)
    ax.scatter(subset['load_time_ms'], subset['searches'],
               alpha=0.3, s=20, label=variant.replace('_', ' '), color=
                   color)

# Add regression line
X_line = np.linspace(300, 1000, 100).reshape(-1, 1)
y_line = model.predict(X_line)
ax.plot(X_line, y_line, 'r--', linewidth=2, label='Regression fit')

ax.set_xlabel('Page Load Time (ms)')
ax.set_ylabel('Searches per User')
ax.set_title('Load Time vs. Engagement')
ax.legend()
ax.grid(alpha=0.3)

# 4. Satisfaction scores
ax = axes[1, 1]
data.boxplot(column='satisfaction', by='variant', ax=ax)
ax.set_xlabel('Variant')
ax.set_ylabel('Satisfaction Score (0-10)')
ax.set_title('User Satisfaction by Variant')
plt.sca(ax)
plt.xticks(range(1, 4), ['10 results', '20 results', '30 results'])

plt.tight_layout()
plt.savefig('google_case_study.pdf', bbox_inches='tight')
plt.show()

# Calculate business impact
print("\n=== Business Impact Estimation ===")
baseline_searches = data[data['variant']=='10_results']['searches'].
    mean()

for variant in ['20_results', '30_results']:
    variant_searches = data[data['variant']==variant]['searches'].mean
        ()
    loss = (variant_searches - baseline_searches) / baseline_searches *
         100

    # At Google scale (billions of searches/day)
```

```
    daily_searches_billions = 8.5  # Approximate 2009 volume
    annual_loss_millions = daily_searches_billions * 1000 * loss / 100
        * 365

    print(f"\n{variant}:")
    print(f"  Relative loss: {loss:.1f}%")
    print(f"  Estimated annual search loss: {annual_loss_millions:.0f}M
        searches")
    print(f"  Decision: DO NOT SHIP")
```

11.2.6 Lessons Learned

1. **Performance is a feature:** Page load time directly impacts user behavior at all scales, but especially at massive scale
2. **Guardrail metrics:** Always monitor technical performance metrics alongside engagement metrics
3. **User psychology:** More choice isn't always better; cognitive load and perceived value matter
4. **Scale amplification:** At Google's traffic volume, small performance differences have billion-search impacts
5. **Regression analysis:** Quantifying relationship between technical metrics and business outcomes enables forecasting

Actionable framework:

- Measure page load time as standard guardrail
- Set performance budgets (e.g., < 500ms) before testing
- Estimate elasticity of engagement with respect to performance
- Calculate ROI of performance optimization

11.3 Case Study 2: E-commerce Checkout Optimization

11.3.1 Context

An e-commerce company with $500M annual revenue hypothesized that simplifying checkout from 5 steps to 3 steps would increase conversion.

11.3.2 Experimental Design

Hypothesis: Reducing friction in checkout process will increase completion rate.
Design: A/B test

- Control: 5-step checkout (address, shipping, payment, review, confirm)

- Treatment: 3-step checkout (combined address/shipping, payment/review, confirm)

Randomization: User-level, 50/50 split
Sample size: 50,000 users per variant (calculated for 80% power to detect 5% relative lift)
Duration: 14 days (to capture two full weeks including weekends)
Metrics:

- Primary: Checkout completion rate (among initiated)
- Secondary: Overall conversion rate, revenue per user, abandonment points
- Guardrails: Customer support contacts, return rate

11.3.3 Overall Results

Metric	Control	Treatment	Lift
Checkout initiation	12.0%	12.1%	+0.8%
Checkout completion	68.0%	71.5%	+5.1%**
Overall conversion	8.16%	8.65%	+6.0%**
Revenue per user	$4.25	$4.50	+5.9%**
Avg order value	$52.08	$52.02	-0.1%
Support contacts	2.1%	2.3%	+9.5%

** Statistically significant at $\alpha = 0.05$

11.3.4 Segmentation Analysis

Critical insight came from examining heterogeneous treatment effects:
By customer type:

Segment	Control	Treatment	Lift	Size
New users	6.5%	7.8%	+20.0%**	60%
Returning users	9.2%	9.0%	-2.2%	40%

By device:

Device	Control	Treatment	Lift	Size
Mobile	5.8%	6.9%	+19.0%**	55%
Desktop	11.2%	11.0%	-1.8%	45%

Listing 11.2: Heterogeneous Treatment Effect Analysis

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

def simulate_checkout_experiment(n_users=100000):
```

```
    """Simulate e-commerce checkout experiment with segments."""

    np.random.seed(42)

    # User characteristics
    is_new = np.random.binomial(1, 0.60, n_users)  # 60% new users
    is_mobile = np.random.binomial(1, 0.55, n_users)  # 55% mobile

    # Treatment assignment
    treatment = np.random.binomial(1, 0.5, n_users)

    # Baseline conversion probabilities
    baseline_conv = np.where(
        is_new,
        np.where(is_mobile, 0.058, 0.070),  # New users: mobile vs desktop
        np.where(is_mobile, 0.085, 0.112)   # Returning: mobile vs desktop
    )

    # Treatment effects (heterogeneous)
    treatment_effect = np.where(
        is_new,
        np.where(is_mobile, 0.011, -0.001),  # New users benefit, esp mobile
        np.where(is_mobile, 0.005, -0.002)   # Returning slightly worse
    )

    # Realized conversion
    conv_prob = baseline_conv + treatment * treatment_effect
    converted = np.random.binomial(1, conv_prob)

    # Revenue (conditional on conversion)
    revenue = converted * np.random.lognormal(np.log(50), 0.6)

    data = pd.DataFrame({
        'user_id': range(n_users),
        'treatment': treatment,
        'is_new': is_new,
        'is_mobile': is_mobile,
        'converted': converted,
        'revenue': revenue
    })

    # Add labels
    data['variant'] = data['treatment'].map({0: 'Control', 1: 'Treatment'})
    data['user_type'] = data['is_new'].map({1: 'New', 0: 'Returning'})
    data['device'] = data['is_mobile'].map({1: 'Mobile', 0: 'Desktop'})

    return data

# Generate data
data = simulate_checkout_experiment(n_users=100000)

# Overall analysis
print("=== Overall Results ===")
overall = data.groupby('variant').agg({
```

```
    'converted': ['sum', 'mean', 'sem'],
    'revenue': ['sum', 'mean']
})

control_rate = overall.loc['Control', ('converted', 'mean')]
treatment_rate = overall.loc['Treatment', ('converted', 'mean')]
lift = (treatment_rate - control_rate) / control_rate * 100

print(f"Control conversion rate: {control_rate:.4f}")
print(f"Treatment conversion rate: {treatment_rate:.4f}")
print(f"Relative lift: {lift:+.1f}%")

# Statistical test
control_conv = data[data['treatment']==0]['converted']
treatment_conv = data[data['treatment']==1]['converted']

z_stat = (treatment_conv.mean() - control_conv.mean()) / np.sqrt(
    control_conv.var() / len(control_conv) +
    treatment_conv.var() / len(treatment_conv)
)
p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))

print(f"\nZ-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")

# Segmented analysis
print("\n=== Heterogeneous Treatment Effects ===")

for segment_var, segment_name in [('user_type', 'User Type'),
                                  ('device', 'Device')]:
    print(f"\nBy {segment_name}:")

    segment_results = data.groupby([segment_var, 'variant']).agg({
        'converted': ['mean', 'count']
    }).reset_index()

    # Pivot for easier comparison
    pivot = segment_results.pivot(
        index=segment_var,
        columns='variant',
        values=('converted', 'mean')
    )

    for segment in pivot.index:
        control_rate = pivot.loc[segment, ('Control')]
        treatment_rate = pivot.loc[segment, ('Treatment')]
        lift = (treatment_rate - control_rate) / control_rate * 100

        # Test within segment
        seg_data = data[data[segment_var]==segment]
        seg_control = seg_data[seg_data['treatment']==0]['converted']
        seg_treatment = seg_data[seg_data['treatment']==1]['converted']

        _, p = stats.ttest_ind(seg_treatment, seg_control)
        sig = '**' if p < 0.05 else '  '

        print(f"  {segment}: {control_rate:.4f} -> {treatment_rate:.4f}
            {sig}")
```

```
                f"({lift:+.1f}%){sig}")

# Visualization
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Overall conversion rates
ax = axes[0, 0]
overall_data = data.groupby('variant')['converted'].agg(['mean', 'sem'])
x = np.arange(2)
ax.bar(x, overall_data['mean'], yerr=1.96*overall_data['sem'],
       capsize=5, alpha=0.7, color=['C0', 'C1'])
ax.set_xticks(x)
ax.set_xticklabels(['Control', 'Treatment'])
ax.set_ylabel('Conversion Rate')
ax.set_title('Overall Conversion Rate (95% CI)')
ax.set_ylim([0, 0.10])
for i, (idx, row) in enumerate(overall_data.iterrows()):
    ax.text(i, row['mean'] + 0.003, f"{row['mean']:.3f}",
            ha='center', fontweight='bold')
ax.grid(axis='y', alpha=0.3)

# 2. By user type
ax = axes[0, 1]
user_type_data = data.groupby(['user_type', 'variant'])['converted'].mean().unstack()
user_type_data.plot(kind='bar', ax=ax, color=['C0', 'C1'], alpha=0.7)
ax.set_ylabel('Conversion Rate')
ax.set_title('Conversion Rate by User Type')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
ax.legend(title='Variant')
ax.grid(axis='y', alpha=0.3)

# 3. By device
ax = axes[1, 0]
device_data = data.groupby(['device', 'variant'])['converted'].mean().unstack()
device_data.plot(kind='bar', ax=ax, color=['C0', 'C1'], alpha=0.7)
ax.set_ylabel('Conversion Rate')
ax.set_title('Conversion Rate by Device')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
ax.legend(title='Variant')
ax.grid(axis='y', alpha=0.3)

# 4. Treatment effect by segment
ax = axes[1, 1]

segments = []
effects = []

for user_type in ['New', 'Returning']:
    for device in ['Mobile', 'Desktop']:
        seg_data = data[(data['user_type']==user_type) &
                        (data['device']==device)]

        control_mean = seg_data[seg_data['treatment']==0]['converted'].mean()
        treatment_mean = seg_data[seg_data['treatment']==1]['converted'
```

```
            ].mean()

        effect = (treatment_mean - control_mean) / control_mean * 100

        label = f"{user_type}\n{device}"
        segments.append(label)
        effects.append(effect)

colors = ['green' if e > 0 else 'red' for e in effects]
x_pos = np.arange(len(segments))

ax.barh(x_pos, effects, color=colors, alpha=0.7)
ax.axvline(x=0, color='black', linestyle='-', linewidth=0.8)
ax.set_yticks(x_pos)
ax.set_yticklabels(segments)
ax.set_xlabel('Relative Lift (%)')
ax.set_title('Treatment Effect by Segment')
ax.grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.savefig('checkout_segmentation.pdf', bbox_inches='tight')
plt.show()

# Revenue impact calculation
print("\n=== Business Impact ===")
annual_visitors = 10_000_000  # 10M annual visitors
current_revenue = 500_000_000  # $500M annually

baseline_conv = control_rate
new_conv = treatment_rate
revenue_per_conversion = data[data['converted']==1]['revenue'].mean()

incremental_conversions = annual_visitors * (new_conv - baseline_conv)
incremental_revenue = incremental_conversions * revenue_per_conversion

print(f"Baseline conversions: {annual_visitors * baseline_conv:,.0f}")
print(f"New conversions: {annual_visitors * new_conv:,.0f}")
print(f"Incremental conversions: {incremental_conversions:,.0f}")
print(f"Avg revenue per conversion: ${revenue_per_conversion:.2f}")
print(f"Incremental annual revenue: ${incremental_revenue:,.0f}")
print(f"Relative revenue increase: {incremental_revenue/current_revenue
    *100:.2f}%")

# Recommendation
print("\n=== Recommendation ===")
print("SHIP TO ALL USERS")
print("Rationale:")
print("- Statistically significant overall lift")
print("- Strong benefit for strategically important segments (new,
    mobile)")
print("- Small negative effect on returning/desktop acceptable")
print("- Projected $30M+ incremental annual revenue")
print("- Support contact increase minimal and manageable")
```

11.3.5 Decision Framework

The team faced a trade-off: overall benefit with segment-specific harm.

Decision criteria:

1. **Overall impact:** +6.0% conversion, highly significant
2. **Strategic alignment:** New users and mobile are growth priorities
3. **Magnitude:** +20% for new/mobile vs. -2% for returning/desktop
4. **Business value:** $30M+ projected incremental annual revenue
5. **Reversibility:** Can iterate if returning user experience degrades further

Decision: Ship to all users, monitor returning user metrics closely.

11.3.6 Post-Launch Monitoring

After launch, team implemented:

- Weekly cohort analysis of returning users
- Customer feedback analysis (surveys, support tickets)
- Funnel tracking for abandonment points
- Quarterly re-testing to validate continued benefit

11.3.7 Lessons Learned

1. **Segment analysis is critical:** Averages hide important heterogeneity
2. **Strategic weighting:** Not all segments are equal; weight by business priorities
3. **Trade-offs are normal:** Few changes benefit everyone universally
4. **Quantify business impact:** Translate statistical findings to revenue/profit
5. **Monitor post-launch:** Continue learning after shipping

11.4 Case Study 3: Netflix Thumbnail Personalization

11.4.1 Context

Netflix hypothesized that personalizing video thumbnails based on user viewing history and preferences would increase engagement (click-through rate and watch time).

11.4.2 Technical Challenge

Consistency requirement: Users see the same videos multiple times (homepage, search, recommendations). Showing different thumbnails each time would be confusing.

Solution: Deterministic assignment—hash(user_id, video_id) → thumbnail_variant
This ensures:

- Same user always sees same thumbnail for given video
- Different users see different thumbnails (personalization)
- Randomization for experimentation

11.4.3 Multi-Stage Experimental Approach

Netflix used an iterative experimental strategy:

Stage 1: Validate thumbnail selection matters

- Test: Random thumbnails vs. editorial picks
- Result: Editorial picks had 30% higher engagement
- Conclusion: Thumbnail selection is important lever

Stage 2: Test personalization hypothesis

- Test: Editorial picks (best average) vs. personalized selection
- Personalization: Select thumbnail based on user's genre preferences and viewing history
- Result: Personalized thumbnails 20% higher engagement than editorial
- Conclusion: Personalization works

Stage 3: Optimize personalization model

- Test: Multiple ML models for personalization
  - Collaborative filtering
  - Content-based filtering
  - Neural network (deep learning)
  - Ensemble
- Result: Neural network approach best (additional 5% lift)
- Conclusion: Ship neural network model

11.4.4 Experimental Design Details (Stage 2)

Randomization: Video-level stratified randomization

- For each video, select N candidate thumbnails (N=3-10)
- Randomly assign users to personalization (treatment) vs. best-on-average (control)
- Within treatment: use model to select best thumbnail for each user

Metrics:

- Primary: Click-through rate (CTR) - play rate when video is visible
- Secondary: Watch time, completion rate, satisfaction rating
- Guardrails: Customer support, cancellations

Duration: 4 weeks (to account for learning effects)
Sample: 10M users, 5M control / 5M treatment

11.4.5 Results

Metric	Control	Treatment	Lift
CTR (overall)	3.2%	3.84%	+20.0%**
Watch time (hrs/user/week)	12.5	13.1	+4.8%**
Completion rate	68%	70%	+2.9%**
Satisfaction (1-5)	4.1	4.2	+2.4%**

** $p < 0.001$

Heterogeneity by video type:
Personalization worked especially well for:

- Ensemble casts (different thumbnails feature different actors)
- Genre-spanning content (show different aspects to different users)
- Foreign language content (different cultural framing)

11.4.6 Implementation Considerations

1. **Infrastructure:** Real-time personalization requires low-latency model serving
2. **Quality control:** Filter inappropriate thumbnails (spoilers, misleading images)
3. **Fallback logic:** When model fails, default to editorial pick
4. **Cold start:** New users see editorial picks until preferences learned
5. **Monitoring:** Track model performance, A/A tests for bias

11.4.7 Lessons Learned

1. **Iterative experimentation:** Build incrementally, validating each assumption
2. **Technical-statistical integration:** Implementation details (hashing, persistence) affect experimental validity
3. **Personalization value:** Context-specific optimization often outperforms one-size-fits-all
4. **Heterogeneous effects:** Personalization benefits vary by content type
5. **Production readiness:** Consider operational constraints before launch

11.5 Case Study 4: Failed Experiment—Subscription Paywall

Not all experiments succeed. Learning from failures is as valuable as from successes.

11.5.1 Context

A digital media platform with freemium model (free articles with monthly limit, premium unlimited access) wanted to increase paid subscriptions.

Hypothesis: More aggressive paywall messaging and earlier paywall triggers would increase conversion to paid subscriptions.

11.5.2 Design

- Control: Paywall after 5 free articles/month, soft messaging
- Treatment: Paywall after 3 free articles/month, aggressive CTA

Primary metric: Subscription conversion rate (
Duration: 3 weeks
Sample: 200,000 users per variant

11.5.3 Results: The Good and The Bad

Metric	Control	Treatment	Abs. Diff	Rel. Lift
Positive:				
Subscription conversion	1.20%	1.38%	+0.18pp	+15.0%**
Negative:				
Bounce rate	35%	49%	+14pp	+40.0%**
Pages per user	8.2	5.1	-3.1	-37.8%**
Return rate (7-day)	42%	31%	-11pp	-26.2%**
Net effect:				
Net new subscribers	1200	1140	-60	-5.0%

** $p < 0.001$

11.5.4 What Went Wrong

1. **Metric misalignment:** Optimized subscription conversion rate (local metric) at expense of overall user base (global metric)
2. **Missing guardrails:** Didn't adequately monitor engagement and retention metrics
3. **Time horizon mismatch:** Immediate conversions vs. long-term user value
4. **User experience:** Aggressive tactics damaged brand perception

Root cause analysis:
The aggressive paywall converted users faster but drove away many more users who would have converted later:

- Fewer users reached the paywall (40% bounce increase)
- Lower return rate meant fewer conversion opportunities
- Net effect: slightly fewer total conversions despite higher rate

11.5.5 Correct Analysis

Listing 11.3: Analyzing Funnel Effects

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def analyze_paywall_funnel():
    """Analyze the full funnel, not just conversion rate."""

    # Monthly visitors
    monthly_visitors = 100000

    # Control funnel
    control_funnel = {
        'visitors': monthly_visitors,
        'bounce_rate': 0.35,
        'engaged': None,  # calculated
        'reach_paywall_rate': 0.15,  # % of engaged users
        'paywall_conversion': 0.012,
        'new_subscribers': None  # calculated
    }

    control_funnel['engaged'] = control_funnel['visitors'] * (
        1 - control_funnel['bounce_rate']
    )
    control_funnel['reach_paywall'] = control_funnel['engaged'] * (
        control_funnel['reach_paywall_rate']
    )
    control_funnel['new_subscribers'] = control_funnel['reach_paywall']
        * (
        control_funnel['paywall_conversion']
    )

    # Treatment funnel
    treatment_funnel = {
        'visitors': monthly_visitors,
        'bounce_rate': 0.49,
        'engaged': None,
        'reach_paywall_rate': 0.20,  # Higher due to earlier trigger
        'paywall_conversion': 0.0138,  # Higher rate
        'new_subscribers': None
    }

    treatment_funnel['engaged'] = treatment_funnel['visitors'] * (
        1 - treatment_funnel['bounce_rate']
    )
    treatment_funnel['reach_paywall'] = treatment_funnel['engaged'] * (
        treatment_funnel['reach_paywall_rate']
    )
    treatment_funnel['new_subscribers'] = treatment_funnel['reach_
       paywall'] * (
        treatment_funnel['paywall_conversion']
    )

    # Print analysis
    print("=== Funnel Analysis ===\n")
```

```
53
54      print("Control:")
55      print(f"  1. Visitors: {control_funnel['visitors']:,.0f}")
56      print(f"  2. Engaged (not bounced): {control_funnel['engaged']:,.0f}
            "
57            f"({(1-control_funnel['bounce_rate'])*100:.1f}%)")
58      print(f"  3. Reached paywall: {control_funnel['reach_paywall']:,.0f}
            "
59            f"({control_funnel['reach_paywall_rate']*100:.1f}%)")
60      print(f"  4. NEW SUBSCRIBERS: {control_funnel['new_subscribers
            ']:,.0f} "
61            f"({control_funnel['paywall_conversion']*100:.2f}% conversion
                )")
62
63      print("\nTreatment:")
64      print(f"  1. Visitors: {treatment_funnel['visitors']:,.0f}")
65      print(f"  2. Engaged (not bounced): {treatment_funnel['engaged
            ']:,.0f} "
66            f"({(1-treatment_funnel['bounce_rate'])*100:.1f}%)")
67      print(f"  3. Reached paywall: {treatment_funnel['reach_paywall
            ']:,.0f} "
68            f"({treatment_funnel['reach_paywall_rate']*100:.1f}%)")
69      print(f"  4. NEW SUBSCRIBERS: {treatment_funnel['new_subscribers
            ']:,.0f} "
70            f"({treatment_funnel['paywall_conversion']*100:.2f}%
                conversion)")
71
72      print("\n=== Key Insight ===")
73      subscriber_diff = (treatment_funnel['new_subscribers'] -
74                         control_funnel['new_subscribers'])
75      rel_diff = subscriber_diff / control_funnel['new_subscribers'] *
            100
76
77      print(f"Net subscriber change: {subscriber_diff:,.0f} ({rel_diff
            :+.1f}%)")
78      print("\nProblem: Higher conversion rate, but fewer total
            subscribers!")
79      print("Reason: Aggressive paywall drove away more users than it
            converted.")
80
81      # Visualization
82      fig, axes = plt.subplots(1, 2, figsize=(14, 6))
83
84      # Funnel visualization
85      ax = axes[0]
86
87      stages = ['Visitors', 'Engaged', 'Reached\nPaywall', 'Subscribed']
88      control_values = [
89          control_funnel['visitors'],
90          control_funnel['engaged'],
91          control_funnel['reach_paywall'],
92          control_funnel['new_subscribers']
93      ]
94      treatment_values = [
95          treatment_funnel['visitors'],
96          treatment_funnel['engaged'],
97          treatment_funnel['reach_paywall'],
98          treatment_funnel['new_subscribers']
```

```
    ]

    x = np.arange(len(stages))
    width = 0.35

    ax.bar(x - width/2, control_values, width, label='Control',
           alpha=0.8, color='C0')
    ax.bar(x + width/2, treatment_values, width, label='Treatment',
           alpha=0.8, color='C1')

    ax.set_ylabel('Users')
    ax.set_title('Subscription Funnel Comparison')
    ax.set_xticks(x)
    ax.set_xticklabels(stages)
    ax.legend()
    ax.grid(axis='y', alpha=0.3)
    ax.set_yscale('log')

    # Highlight the problem
    for i, (c_val, t_val) in enumerate(zip(control_values, treatment_
        values)):
        if i == len(stages) - 1:  # Final stage
            if t_val < c_val:
                ax.text(i, max(c_val, t_val) * 1.5, 'Problem!',
                        ha='center', color='red', fontweight='bold',
                        fontsize=12)

    # Conversion rates vs. absolute numbers
    ax = axes[1]

    metrics = ['Conversion\nRate', 'Total\nSubscribers']
    control_rel = [100, 100]  # Baseline
    treatment_rel = [
        (treatment_funnel['paywall_conversion'] /
         control_funnel['paywall_conversion']) * 100,
        (treatment_funnel['new_subscribers'] /
         control_funnel['new_subscribers']) * 100
    ]

    x = np.arange(len(metrics))
    width = 0.35

    ax.bar(x - width/2, control_rel, width, label='Control (baseline)',
           alpha=0.8, color='C0')
    bars = ax.bar(x + width/2, treatment_rel, width, label='Treatment',
                  alpha=0.8, color='C1')

    # Color bars by good/bad
    bars[0].set_color('green')  # Higher rate is green
    bars[1].set_color('red')    # Fewer subscribers is red

    ax.axhline(y=100, color='black', linestyle='--', alpha=0.5)
    ax.set_ylabel('Relative to Control (%)')
    ax.set_title('The Optimization Paradox')
    ax.set_xticks(x)
    ax.set_xticklabels(metrics)
    ax.legend()
    ax.grid(axis='y', alpha=0.3)
```

```
    # Add value labels
    for i, (val, bar) in enumerate(zip(treatment_rel, bars)):
        ax.text(bar.get_x() + bar.get_width()/2, val + 2,
                f'{val:.0f}%', ha='center', fontweight='bold')

    plt.tight_layout()
    plt.savefig('paywall_failure_analysis.pdf', bbox_inches='tight')
    plt.show()

analyze_paywall_funnel()
```

11.5.6 Lessons Learned

1. **Holistic metrics:** Measure the entire funnel, not isolated steps
2. **Guardrail importance:** Protect critical health metrics (engagement, retention)
3. **Local vs. global optimization:** Improving one metric can harm overall goal
4. **Customer lifetime value:** Consider long-term impact beyond immediate conversion
5. **Document failures:** Failed experiments provide valuable learning

Corrective actions:

- Revised success criteria to include engagement and retention
- Established mandatory guardrail metrics for all experiments
- Implemented funnel analysis as standard practice
- Created decision framework weighting multiple objectives

11.6 Common Anti-Patterns and How to Avoid Them

11.6.1 Anti-Pattern 1: P-Hacking

Definition: Manipulating analysis until finding statistical significance.

Manifestations:

- Testing many metrics without correction
- Stopping experiment early when results look good
- Trying different segmentations until finding significance
- Removing "outliers" selectively

Example: A team ran an email subject line test. Primary metric (open rate) showed no effect. They then tested click rate, conversion rate, revenue per user, time to conversion, and found one significant result (time to conversion, $p = 0.04$) and declared success.

Why it's wrong: Testing 5 metrics with $\alpha = 0.05$ gives $1 - (0.95)^5 = 22.6\%$ false positive rate.

Prevention:

- Pre-register hypotheses and analysis plan
- Apply Bonferroni or FDR correction for multiple metrics
- Use sequential testing methods if monitoring continuously
- Document all analyses, including non-significant results

11.6.2 Anti-Pattern 2: HiPPO (Highest Paid Person's Opinion)

Definition: Overriding experimental evidence based on executive intuition or seniority.

Example: An e-commerce A/B test showed that removing a product feature decreased conversion by 8% ($p < 0.001$). VP insisted on shipping anyway because "customers told me they wanted this."

Why it's problematic: Undermines experimentation culture, leads to poor decisions, demoralizes data teams.

Prevention:

- Establish decision-making framework upfront
- Educate leadership on statistical methodology
- Create "escape valves" for overriding data (with documentation)
- Balance quantitative data with qualitative insights systematically
- Foster culture where challenging authority with data is encouraged

11.6.3 Anti-Pattern 3: Analysis Paralysis

Definition: Running experiments indefinitely without making decisions.

Example: Team ran checkout test for 8 weeks (planned duration: 2 weeks). Results were conclusive after 2 weeks, but team kept running "to be sure," delaying feature launch.

Prevention:

- Set clear success criteria before starting
- Establish decision deadlines
- Accept that uncertainty is inherent
- Define minimum confidence levels
- Implement decision frameworks (when to ship, when to iterate, when to kill)

11.6.4 Anti-Pattern 4: Novelty Effect Ignorance

Definition: Ignoring that users may respond differently initially than long-term.

Example: Redesigned navigation initially showed +15% engagement. After 4 weeks, effect degraded to +2

Prevention:

- Run experiments long enough to capture habituation (typically 2-4 weeks minimum)
- Analyze early vs. late periods separately
- Use cohort analysis to distinguish novelty from sustained effects
- For major changes, maintain long-term holdout groups

11.7 Industry-Specific Considerations

11.7.1 E-commerce

Unique challenges:

- High-value, low-frequency transactions (variance)
- Seasonality (holidays, sales events)
- Multi-device journeys (attribution)
- Returning customer effects

Best practices:

- Use pre-experiment covariates (CUPED) to reduce variance
- Stratify by customer type (new/returning) and device
- Run experiments across seasonal periods or control for calendar effects
- Monitor cart abandonment as key guardrail
- Track long-term metrics (LTV, repeat purchase rate)

11.7.2 Social Media

Unique challenges:

- Network effects and interference
- Viral content creates non-independence
- User-generated content variability
- Engagement vs. time spent vs. well-being trade-offs

Best practices:

- Use cluster randomization (geo, time-based)
- Account for network structure in analysis
- Monitor both engagement and well-being metrics
- Use switchback designs for platform-level changes
- Be cautious of metric gaming (e.g., optimizing for clicks vs. value)

11.7.3 SaaS/B2B

Unique challenges:

- Long sales cycles
- Account-level vs. user-level randomization
- Low sample sizes (enterprise customers)
- Contractual obligations limiting experimentation

Best practices:

- Use hierarchical models for account/user nesting
- Focus on leading indicators (trial engagement, feature adoption)
- Stratify by account size/value
- Supplement with qualitative research (customer interviews)
- Run longer experiments to capture full conversion cycle

11.7.4 Healthcare/EdTech

Unique challenges:

- Ethical constraints on randomization
- Long-term outcomes more important than short-term
- Regulatory compliance (HIPAA, FERPA, IRB)
- Equity and access considerations

Best practices:

- Prioritize equipoise (clinical uncertainty) for ethical randomization
- Use stepped-wedge designs (everyone gets treatment eventually)
- Implement robust informed consent processes
- Monitor equity metrics across demographic groups
- Focus on validated outcome measures
- Consider long-term follow-up studies

11.8 Building an Experimentation Culture: Meta-Lessons

Beyond individual experiments, successful organizations build systematic experimentation capabilities:

11.8.1 Infrastructure and Tooling

- **Experimentation platform:** Centralized system for randomization, tracking, analysis
- **Metric pipeline:** Automated computation of key metrics with proper variance estimation
- **Analysis tools:** Self-service dashboards with correct statistical methods
- **Experiment registry:** Database of all experiments for meta-analysis

11.8.2 Process and Governance

- **Hypothesis template:** Structured approach to defining experiments
- **Review process:** Statistical review before launch
- **Decision framework:** Clear criteria for ship/no-ship/iterate
- **Documentation:** Consistent recording of design, results, decisions

11.8.3 Culture and Education

- **Leadership buy-in:** Executives champion data-driven decisions
- **Training programs:** Educate PMs, engineers, analysts on methodology
- **Celebrate learning:** Reward well-designed experiments, not just "wins"
- **Transparent communication:** Share results widely, including failures

11.9 Summary

Real-world case studies provide invaluable lessons:

- **Google:** Technical performance directly impacts user behavior; always monitor guardrails
- **E-commerce:** Segment analysis reveals heterogeneous effects; weight by strategic priorities
- **Netflix:** Iterative experimentation validates assumptions incrementally; personalization adds value
- **Paywall failure:** Local optimization can harm global objectives; measure full funnel

- **Anti-patterns:** Avoid p-hacking, HiPPO, analysis paralysis; establish processes to prevent
- **Industry context:** Adapt methods to specific challenges (network effects, seasonality, regulations)

Learning from both successes and failures builds experimentation expertise. The most mature organizations view experiments as learning opportunities, documenting and sharing insights to improve continuously.

11.10 Further Reading

- **?**: Extensive case studies from Microsoft, Amazon, Google, with detailed analysis
- **?**: Exploration-exploitation trade-offs in practice
- Company engineering blogs:
  - Netflix TechBlog: experimentation at scale
  - Airbnb Engineering: experiment analysis frameworks
  - Booking.com: 1,000+ experiments per year
  - Spotify Research: sequential testing and bandits

11.11 Exercises

11.11.1 Case Study Analysis

1. **Post-Mortem Template:** Create a structured template for analyzing experiments. Include sections for: hypothesis, design, implementation, results, decision, lessons learned. Apply to a recent experiment from your organization.
2. **Replication:** Reproduce the Google case study analysis using the provided code. Modify parameters to explore sensitivity: What if load time increased by 50ms instead of 500ms? What if engagement elasticity were -0.5 instead of -1.0?
3. **Segment Discovery:** Using the e-commerce checkout code, add a third segment dimension (e.g., product category, geographic region). Analyze three-way interactions. When do you stop slicing data?

11.11.2 Business Impact Calculation

1. For your organization:
   - Define key business metrics (revenue, LTV, retention)
   - Map experiment metrics to business outcomes
   - Create calculator estimating business impact from experimental results
   - Include confidence intervals and break-even analysis
2. **Paywall Re-design:** The failed paywall experiment. Design a better version that balances conversion rate and engagement. Specify success criteria and guardrails.

11.11.3 Anti-Pattern Detection

1. **Audit:** Review 5 recent experiments from your organization. Identify any anti-patterns (p-hacking, missing guardrails, insufficient duration, etc.). Propose corrective actions.

2. **Process Design:** Design a review checklist to prevent common anti-patterns. Include statistical, technical, and business considerations. Specify who reviews, when, and with what authority.

11.11.4 Industry Application

1. Choose your industry and design an experiment addressing industry-specific challenges:

   - E-commerce: Multi-device attribution problem
   - Social media: Network effects mitigation
   - SaaS: Long-sales cycle with leading indicators
   - Healthcare: Ethical randomization with stepped-wedge

   Specify design, analysis plan, and decision criteria.

11.11.5 Meta-Analysis Project

1. **Experiment Database:** Create a structured database of past experiments with fields: hypothesis, design, sample size, duration, primary metric, result, decision. Analyze:

   - What proportion of experiments are "successful"?
   - Do success rates differ by experiment type, team, or domain?
   - What factors predict successful experiments?
   - Are sample size and duration adequate on average?

Chapter 12

Ethics and Best Practices

12.1 Learning Objectives

After completing this chapter, readers will be able to:

- Navigate ethical considerations in human subjects experimentation
- Apply principles of informed consent and transparency
- Build organizational experimentation culture
- Establish governance frameworks
- Follow industry best practices

12.2 Ethical Foundations

A/B testing involves human subjects and carries ethical responsibilities.

12.2.1 Core Ethical Principles

1. **Beneficence:** Maximize benefits, minimize harms
2. **Non-maleficence:** Do no harm
3. **Autonomy:** Respect individual agency
4. **Justice:** Ensure fair distribution of benefits and burdens
5. **Transparency:** Be open about practices

12.2.2 The Belmont Report

While focused on medical research, the Belmont Report's principles apply to digital experimentation:

- **Respect for persons:** Treat individuals as autonomous agents
- **Beneficence:** Maximize possible benefits and minimize possible harms
- **Justice:** Fair distribution of research benefits and burdens

12.3 Informed Consent

12.3.1 Consent in Digital Experiments

Unlike medical trials, digital A/B tests typically operate under:

General consent: Users agree to terms of service that may include experimentation

Challenges:

- Users rarely read terms of service
- Consent may not be truly informed
- Ongoing vs. one-time consent
- Right to opt out

12.3.2 When Explicit Consent is Required

- Testing potentially harmful interventions
- Collecting sensitive personal data beyond normal use
- Significantly changing core functionality
- Research published academically
- Regulated industries (healthcare, finance, education)

12.3.3 Balancing Innovation and Consent

Example 12.1 (Acceptable Implicit Consent)**.** Testing button colors, layout variations, recommendation algorithms—changes within normal product optimization scope.

Example 12.2 (Requiring Explicit Consent)**.** Testing emotional manipulation, significantly degraded experiences, collection of biometric data.

12.4 Ethical Guardrails

12.4.1 Minimal Risk Standard

Experiments should pose no more than minimal risk—risks not greater than encountered in daily life or routine tests.

Risk assessment questions:

1. Could this experiment cause physical, psychological, or economic harm?
2. Are vulnerable populations disproportionately affected?
3. Is there potential for manipulation or deception?
4. Could this damage user trust or brand reputation?
5. Are there privacy or data security concerns?

12.4.2 Experimentation Review Board

Large organizations should establish review processes:

Composition:

- Product leaders
- Legal/compliance representatives
- Ethics officer or designated ethicist
- Data scientists
- Privacy/security experts

Responsibilities:

- Review high-risk experiments
- Establish ethical guidelines
- Provide guidance on edge cases
- Monitor incidents and update policies

12.4.3 Automated Guardrails

Implement technical controls:

Listing 12.1: Automated Ethical Guardrails

```
class ExperimentEthicsChecker:
    """Automated␣ethics␣and␣safety␣checks."""

    def __init__(self):
        self.high_risk_keywords = [
            'children', 'minors', 'emotion', 'manipulation',
            'addiction', 'health', 'safety', 'security'
        ]
        self.sensitive_metrics = [
            'mental_health', 'addiction_score', 'emotional_state'
        ]

    def review_experiment(self, experiment_config):
        """
␣␣␣␣␣␣␣␣Automated␣ethics␣review␣of␣experiment␣configuration.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣--------
␣␣␣␣␣␣␣␣dict␣:␣Review␣results␣and␣recommendations
␣␣␣␣␣␣␣␣"""
        flags = []

        # Check for high-risk keywords in description
        description = experiment_config.get('description', '').lower()
        for keyword in self.high_risk_keywords:
            if keyword in description:
                flags.append({
```

```
                    'severity': 'high',
                    'type': 'high_risk_keyword',
                    'message': f"High-risk keyword detected: '{keyword}'",
                    'action': 'Requires human review'
                })

        # Check for sensitive metrics
        metrics = experiment_config.get('metrics', {})
        all_metrics = (metrics.get('primary', []) +
                       metrics.get('secondary', []))

        for metric in all_metrics:
            if metric in self.sensitive_metrics:
                flags.append({
                    'severity': 'high',
                    'type': 'sensitive_metric',
                    'message': f"Sensitive metric: '{metric}'",
                    'action': 'Requires ethics board approval'
                })

        # Check for degraded experiences
        variants = experiment_config.get('variants', [])
        for variant in variants:
            if variant.get('intentionally_degraded', False):
                flags.append({
                    'severity': 'medium',
                    'type': 'degraded_experience',
                    'message': f"Variant {variant['name']} intentionally degraded",
                    'action': 'Document justification'
                })

        # Check target population
        targeting = experiment_config.get('targeting', {})
        if 'age_max' in targeting and targeting['age_max'] < 18:
            flags.append({
                'severity': 'critical',
                'type': 'minors_targeted',
                'message': 'Experiment targets minors',
                'action': 'Ethics board approval and parental consent required'
            })

        return {
            'approved': len([f for f in flags if f['severity'] in
                             ['high', 'critical']]) == 0,
            'flags': flags,
            'recommendation': self._get_recommendation(flags)
        }

    def _get_recommendation(self, flags):
        """Generate recommendation based on flags."""
        if not flags:
            return "No ethical concerns detected. Approved for launch."

        critical = [f for f in flags if f['severity'] == 'critical']
        high = [f for f in flags if f['severity'] == 'high']
```

```
83
84          if critical:
85              return "BLOCKED: Critical ethical issues. Ethics board
                    review required."
86          elif high:
87              return "CAUTION: High-risk experiment. Manager approval
                    required."
88          else:
89              return "PROCEED WITH CAUTION: Document ethical
                    considerations."
90
91  # Example usage
92  experiment = {
93      'description': 'Testing notification frequency to increase
                engagement',
94      'metrics': {
95          'primary': 'daily_active_users',
96          'secondary': ['session_duration', 'notification_clicks']
97      },
98      'variants': [
99          {'name': 'control', 'notification_freq': 1},
100         {'name': 'treatment', 'notification_freq': 10}
101     ]
102 }
103
104 checker = ExperimentEthicsChecker()
105 result = checker.review_experiment(experiment)
106
107 print(f"Approved: {result['approved']}")
108 print(f"Recommendation: {result['recommendation']}")
109 for flag in result['flags']:
110     print(f"  - {flag['message']}")
```

12.5 Privacy and Data Protection

12.5.1 Data Minimization

Collect only data necessary for the experiment:

- Define metrics clearly and collect only those
- Aggregate early when possible
- Delete raw data after analysis
- Anonymize or pseudonymize user identifiers

12.5.2 Regulatory Compliance

GDPR (General Data Protection Regulation):

- Right to access: Users can request their data
- Right to erasure: "Right to be forgotten"

- Data protection by design
- Lawful basis for processing

CCPA (California Consumer Privacy Act):

- Right to know what data is collected
- Right to delete personal information
- Right to opt out of sale
- Non-discrimination for exercising rights

12.5.3 Internal Data Governance

1. **Access controls:** Limit who can view individual-level data
2. **Audit trails:** Log data access and analysis
3. **Retention policies:** Delete data after specified period
4. **Anonymization:** Remove personally identifiable information
5. **Differential privacy:** Add noise to protect individual privacy

12.6 Transparency and Communication

12.6.1 Internal Transparency

Experiment registry:

- Centralized catalog of all experiments
- Pre-registration of hypotheses
- Documentation of methods and results
- Sharing of negative results

Benefits:

- Prevents duplicate testing
- Enables meta-analysis
- Builds institutional knowledge
- Reduces publication bias

12.6.2 External Transparency

User-facing:

- Clear terms of service
- Privacy policies mentioning experimentation
- Opt-out mechanisms where appropriate
- Transparency reports

Academic:

- Publishing findings in peer-reviewed venues
- Open-sourcing methodologies
- Sharing datasets (when appropriate)
- Contributing to field knowledge

12.7 Building Experimentation Culture

12.7.1 Cultural Principles

1. **Data-informed, not data-driven:** Balance data with judgment
2. **Embrace failure:** Learn from experiments that don't succeed
3. **Democratize experimentation:** Enable all teams to test
4. **Question assumptions:** Challenge conventional wisdom
5. **Iterate rapidly:** Small, frequent experiments over big bets
6. **Share learnings:** Disseminate insights across organization

12.7.2 Organizational Structure

Centralized model:

- Dedicated experimentation team
- Ensures methodological rigor
- Can become bottleneck

Decentralized model:

- Each team runs own experiments
- Faster iteration
- Risk of inconsistent standards

Hybrid model (recommended):

- Central platform and guidelines
- Distributed execution
- Centers of excellence provide support

12.7.3 Training and Education

Invest in capability building:

- Statistical literacy training
- Experimentation best practices workshops
- Case study discussions
- Mentorship programs
- Documentation and runbooks

12.8 Best Practices Checklist

12.8.1 Design Phase

☐ Clear hypothesis stated

☐ Primary metric identified

☐ Sample size calculated

☐ Guardrail metrics defined

☐ Ethical review completed

☐ Success criteria pre-specified

12.8.2 Implementation Phase

☐ Randomization mechanism tested

☐ Instrumentation validated

☐ SRM monitoring configured

☐ Logging working correctly

☐ Configuration documented

12.8.3 Analysis Phase

- ☐ Pre-analysis quality checks passed
- ☐ Multiple testing corrections applied
- ☐ Subgroup analysis planned
- ☐ Confidence intervals reported
- ☐ Effect sizes calculated
- ☐ Business impact estimated

12.8.4 Decision Phase

- ☐ Results communicated clearly
- ☐ Statistical and practical significance assessed
- ☐ Stakeholders aligned
- ☐ Implementation plan created
- ☐ Learnings documented

12.9 Common Pitfalls and How to Avoid Them

Pitfall	Consequence	Prevention
No pre-specified hypothesis	P-hacking, false positives	Pre-register experiments
Peeking at results	Inflated Type I error	Sequential testing methods
Ignoring guardrails	Harm to user experience	Monitor health metrics
Small sample sizes	Underpowered tests	Power analysis upfront
Short duration	Missing long-term effects	Run for full business cycles
No documentation	Lost institutional knowledge	Experiment registry

12.10 Summary

Ethical and effective experimentation requires:

- Commitment to ethical principles and user welfare
- Robust governance and review processes
- Privacy protection and regulatory compliance
- Transparency internally and externally
- Strong organizational culture supporting experimentation

- Continuous learning and improvement

A/B testing is powerful but carries responsibility. Practitioners must balance innovation with ethics, rigor with pragmatism, and data with judgment.

12.11 Further Reading

- **?**: Controversial study highlighting ethical issues
- **?**: Ethics of tech experimentation
- **?**: Building experimentation culture
- IRB guidelines: Institutional Review Board frameworks

12.12 Exercises

1. Create an ethics review framework for your organization
2. Analyze a controversial experiment (e.g., Facebook emotional contagion study)
3. Design an experimentation training program
4. Build an experiment registry system
5. Draft transparency report for external publication

"The goal is to turn data into information, and information into insight."
— Carly Fiorina

Mathematical Proofs

.1 Mathematical Proofs

This appendix provides rigorous mathematical proofs for key theorems referenced throughout the book.

.1.1 Proof: Unbiasedness of Sample Mean

Theorem .1. Let $X_1, X_2, \ldots, X_n$ be independent and identically distributed random variables with $\mathbb{E}[X_i] = \mu$. Then the sample mean $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$ is an unbiased estimator of μ.

Proof. By linearity of expectation:

$$\mathbb{E}[\bar{X}] = \mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right] \tag{1}$$

$$= \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[X_i] \tag{2}$$

$$= \frac{1}{n}\sum_{i=1}^{n} \mu \tag{3}$$

$$= \frac{1}{n} \cdot n\mu \tag{4}$$

$$= \mu \tag{5}$$

Therefore, $\bar{X}$ is an unbiased estimator of μ. □

.1.2 Proof: Variance of Sample Mean

Theorem .2. For independent random variables $X_1, \ldots, X_n$ with $\mathrm{Var}(X_i) = \sigma^2$:

$$\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n} \tag{6}$$

Proof. Using independence:

$$\begin{aligned}
\text{Var}(\bar{X}) &= \text{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) && (7)\\
&= \frac{1}{n^2}\text{Var}\left(\sum_{i=1}^{n} X_i\right) && (8)\\
&= \frac{1}{n^2}\sum_{i=1}^{n}\text{Var}(X_i) \quad \text{(by independence)} && (9)\\
&= \frac{1}{n^2}\sum_{i=1}^{n}\sigma^2 && (10)\\
&= \frac{1}{n^2}\cdot n\sigma^2 && (11)\\
&= \frac{\sigma^2}{n} && (12)
\end{aligned}$$

□

.1.3 Proof: Beta-Binomial Conjugacy

Theorem .3. If $p \sim \text{Beta}(\alpha, \beta)$ and $X|p \sim \text{Binomial}(n, p)$, then:

$$p|X \sim \text{Beta}(\alpha + X, \beta + n - X) \tag{13}$$

Proof. By Bayes' theorem:

$$f(p|X) \propto f(X|p) \cdot f(p) \tag{14}$$

The likelihood is:

$$f(X|p) = \binom{n}{X} p^X (1-p)^{n-X} \tag{15}$$

The prior is:

$$f(p) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1} \tag{16}$$

The posterior is proportional to:

$$\begin{aligned}
f(p|X) &\propto p^X(1-p)^{n-X} \cdot p^{\alpha-1}(1-p)^{\beta-1} && (17)\\
&= p^{X+\alpha-1}(1-p)^{n-X+\beta-1} && (18)
\end{aligned}$$

This is the kernel of a $\text{Beta}(\alpha + X, \beta + n - X)$ distribution. □

.1.4 Proof: Sample Size Formula for Proportions

Theorem .4. For comparing two proportions p_1 and p_2 with significance level α and power $1 - \beta$:

$$n = \frac{(z_{\alpha/2} + z_{\beta})^2[p_1(1-p_1) + p_2(1-p_2)]}{(p_2 - p_1)^2} \tag{19}$$

Proof. Under $H_0 : p_1 = p_2 = p$, using pooled proportion:

$$Z_0 = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{2p(1-p)/n}} \sim \mathcal{N}(0,1) \tag{20}$$

Under $H_1 : p_1 \neq p_2$:

$$Z_1 = \frac{(\hat{p}_2 - \hat{p}_1) - (p_2 - p_1)}{\sqrt{[p_1(1-p_1) + p_2(1-p_2)]/n}} \sim \mathcal{N}(0,1) \tag{21}$$

Rejection region: $|\hat{p}_2 - \hat{p}_1| > z_{\alpha/2}\sqrt{2p(1-p)/n}$
For power $1-\beta$, we require:

$$P\left(|\hat{p}_2 - \hat{p}_1| > z_{\alpha/2}\sqrt{2p(1-p)/n} \mid H_1\right) = 1 - \beta \tag{22}$$

Assuming $p_2 > p_1$ and approximating $p \approx (p_1 + p_2)/2$:

$$\frac{(p_2 - p_1) - z_{\alpha/2}\sqrt{2p(1-p)/n}}{\sqrt{[p_1(1-p_1) + p_2(1-p_2)]/n}} = z_\beta \tag{23}$$

Solving for n yields the formula above (with slight approximation). □

.1.5 Proof: CUPED Variance Reduction

Theorem .5. For adjusted outcome $Y^* = Y - \theta(X - \mathbb{E}[X])$ with $\theta = \frac{\text{Cov}(Y,X)}{\text{Var}(X)}$:

$$\text{Var}(Y^*) = \text{Var}(Y)(1 - \rho^2) \tag{24}$$

where $\rho = \text{Cor}(Y, X)$.

Proof.

$$\begin{aligned}
\text{Var}(Y^*) &= \text{Var}(Y - \theta(X - \mathbb{E}[X])) && (25)\\
&= \text{Var}(Y) + \theta^2\text{Var}(X) - 2\theta\text{Cov}(Y,X) && (26)\\
&= \text{Var}(Y) + \frac{\text{Cov}(Y,X)^2}{\text{Var}(X)^2}\text{Var}(X) - 2\frac{\text{Cov}(Y,X)}{\text{Var}(X)}\text{Cov}(Y,X) && (27)\\
&= \text{Var}(Y) + \frac{\text{Cov}(Y,X)^2}{\text{Var}(X)} - 2\frac{\text{Cov}(Y,X)^2}{\text{Var}(X)} && (28)\\
&= \text{Var}(Y) - \frac{\text{Cov}(Y,X)^2}{\text{Var}(X)} && (29)\\
&= \text{Var}(Y)\left(1 - \frac{\text{Cov}(Y,X)^2}{\text{Var}(Y)\text{Var}(X)}\right) && (30)\\
&= \text{Var}(Y)(1 - \rho^2) && (31)
\end{aligned}$$

□

.1.6 Central Limit Theorem

Theorem .6 (Lindeberg-Lévy CLT)**.** Let $X_1, X_2, \ldots$ be i.i.d. random variables with $\mathbb{E}[X_i] = \mu$ and $\text{Var}(X_i) = \sigma^2 < \infty$. Then:

$$\frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1) \tag{32}$$

The full proof requires characteristic functions and is beyond scope, but the key idea is that standardized sums of i.i.d. random variables converge in distribution to the standard normal distribution.

.1.7 Bonferroni Inequality

Theorem .7 (Bonferroni Correction)**.** For m hypothesis tests, each at level α/m, the family-wise error rate satisfies:

$$\text{FWER} = P(\text{Reject at least one true } H_0) \leq \alpha \tag{33}$$

Proof. Let R_i be the event that test i rejects a true null hypothesis. By Boole's inequality:

$$\begin{aligned} \text{FWER} &= P\left(\bigcup_{i=1}^{m} R_i\right) && (34)\\ &\leq \sum_{i=1}^{m} P(R_i) && (35)\\ &= \sum_{i=1}^{m} \frac{\alpha}{m} && (36)\\ &= \alpha && (37) \end{aligned}$$

□

.1.8 Delta Method

Theorem .8 (Delta Method)**.** Let $\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$ and g be differentiable at θ with $g'(\theta) \neq 0$. Then:

$$\sqrt{n}(g(\hat{\theta}) - g(\theta)) \xrightarrow{d} \mathcal{N}(0, [g'(\theta)]^2\sigma^2) \tag{38}$$

Application: Useful for transformations like log odds ratios or relative lifts.

Proof Sketch. By Taylor expansion around θ:

$$g(\hat{\theta}) \approx g(\theta) + g'(\theta)(\hat{\theta} - \theta) \tag{39}$$

Therefore:

$$\sqrt{n}(g(\hat{\theta}) - g(\theta)) \approx g'(\theta)\sqrt{n}(\hat{\theta} - \theta) \tag{40}$$

Since $\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$ and multiplication by constant preserves normality, the result follows. □

Code Examples

.2 Complete Code Examples

This appendix provides complete, production-ready code implementations for common A/B testing tasks.

.2.1 Comprehensive A/B Test Analysis Class (Python)

Listing 2: Automated Ethical Guardrails

```
import numpy as np
import pandas as pd
from scipy import stats
from dataclasses import dataclass
from typing import Optional, Dict, List

@dataclass
class ABTestResult:
    """Container for A/B test results."""
    control_mean: float
    treatment_mean: float
    difference: float
    relative_lift: float
    standard_error: float
    confidence_interval: tuple
    p_value: float
    statistically_significant: bool
    test_statistic: float
    sample_size_control: int
    sample_size_treatment: int

class ABTestAnalyzer:
    """
    Comprehensive A/B test analysis.

    Supports:
    - Proportions (conversion rates)
    - Continuous metrics (revenue, time, etc.)
    - Multiple testing corrections
    - Subgroup analysis
    """

    def __init__(self, alpha=0.05):
        self.alpha = alpha

    def test_proportions(self, control_conversions, control_total,
```

```
                          treatment_conversions, treatment_total,
                          method='z_test'):
        """
        Test difference in proportions.

        Parameters:
        -----------
        control_conversions : int
            Number of successes in control
        control_total : int
            Total observations in control
        treatment_conversions : int
            Number of successes in treatment
        treatment_total : int
            Total observations in treatment
        method : str
            'z_test', 'chi_squared', or 'fisher_exact'

        Returns:
        --------
        ABTestResult : Test results
        """
        p1 = control_conversions / control_total
        p2 = treatment_conversions / treatment_total

        if method == 'z_test':
            # Pooled proportion
            p_pool = ((control_conversions + treatment_conversions) /
                      (control_total + treatment_total))

            # Standard error under null
            se_null = np.sqrt(p_pool * (1 - p_pool) *
                              (1/control_total + 1/treatment_total))

            # Test statistic
            z_stat = (p2 - p1) / se_null
            p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))

            # Confidence interval (unpooled)
            se_unpooled = np.sqrt(p1*(1-p1)/control_total +
                                  p2*(1-p2)/treatment_total)
            z_crit = stats.norm.ppf(1 - self.alpha/2)
            ci = (p2 - p1 - z_crit * se_unpooled,
                  p2 - p1 + z_crit * se_unpooled)

            test_stat = z_stat
            se = se_unpooled

        elif method == 'chi_squared':
            table = [[control_conversions, control_total - control_
                conversions],
                     [treatment_conversions, treatment_total - treatment
                         _conversions]]
            chi2, p_value, dof, expected = stats.chi2_contingency(table
                )
            test_stat = chi2

            # CI from z-test
```

```
            se = np.sqrt(p1*(1-p1)/control_total +
                         p2*(1-p2)/treatment_total)
            z_crit = stats.norm.ppf(1 - self.alpha/2)
            ci = (p2 - p1 - z_crit * se, p2 - p1 + z_crit * se)

        elif method == 'fisher_exact':
            table = [[control_conversions, control_total - control_
                conversions],
                     [treatment_conversions, treatment_total - treatment
                         _conversions]]
            _, p_value = stats.fisher_exact(table)
            test_stat = None

            # CI from z-test
            se = np.sqrt(p1*(1-p1)/control_total +
                         p2*(1-p2)/treatment_total)
            z_crit = stats.norm.ppf(1 - self.alpha/2)
            ci = (p2 - p1 - z_crit * se, p2 - p1 + z_crit * se)

        return ABTestResult(
            control_mean=p1,
            treatment_mean=p2,
            difference=p2 - p1,
            relative_lift=(p2 - p1) / p1 if p1 > 0 else np.nan,
            standard_error=se,
            confidence_interval=ci,
            p_value=p_value,
            statistically_significant=p_value < self.alpha,
            test_statistic=test_stat,
            sample_size_control=control_total,
            sample_size_treatment=treatment_total
        )

    def test_means(self, control_data, treatment_data,
                   equal_variance=False):
        """
␣␣␣␣␣␣␣␣Test␣difference␣in␣means.

␣␣␣␣␣␣␣␣Parameters:
␣␣␣␣␣␣␣␣-----------
␣␣␣␣␣␣␣␣control_data␣:␣array-like
␣␣␣␣␣␣␣␣␣␣␣␣Control␣group␣values
␣␣␣␣␣␣␣␣treatment_data␣:␣array-like
␣␣␣␣␣␣␣␣␣␣␣␣Treatment␣group␣values
␣␣␣␣␣␣␣␣equal_variance␣:␣bool
␣␣␣␣␣␣␣␣␣␣␣␣Assume␣equal␣variances␣(default␣False,␣uses␣Welch's␣t-test)

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣--------
␣␣␣␣␣␣␣␣ABTestResult␣:␣Test␣results
␣␣␣␣␣␣␣␣"""
        control_data = np.array(control_data)
        treatment_data = np.array(treatment_data)

        n1 = len(control_data)
        n2 = len(treatment_data)
        mean1 = np.mean(control_data)
        mean2 = np.mean(treatment_data)
```

```
148         var1 = np.var(control_data, ddof=1)
149         var2 = np.var(treatment_data, ddof=1)
150 
151         # T-test
152         t_stat, p_value = stats.ttest_ind(treatment_data, control_data,
153                                           equal_var=equal_variance)
154 
155         # Standard error and CI
156         se = np.sqrt(var1/n1 + var2/n2)
157 
158         if equal_variance:
159             df = n1 + n2 - 2
160         else:
161             # Welch-Satterthwaite degrees of freedom
162             df = ((var1/n1 + var2/n2)**2 /
163                   ((var1/n1)**2/(n1-1) + (var2/n2)**2/(n2-1)))
164 
165         t_crit = stats.t.ppf(1 - self.alpha/2, df)
166         ci = (mean2 - mean1 - t_crit * se, mean2 - mean1 + t_crit * se)
167 
168         return ABTestResult(
169             control_mean=mean1,
170             treatment_mean=mean2,
171             difference=mean2 - mean1,
172             relative_lift=(mean2 - mean1) / mean1 if mean1 > 0 else np.
                nan,
173             standard_error=se,
174             confidence_interval=ci,
175             p_value=p_value,
176             statistically_significant=p_value < self.alpha,
177             test_statistic=t_stat,
178             sample_size_control=n1,
179             sample_size_treatment=n2
180         )
181 
182     def calculate_sample_size_proportions(self, p1, p2, power=0.80):
183         """
184 ␣␣␣␣␣␣␣␣Calculate␣required␣sample␣size␣for␣proportion␣test.
185 
186 ␣␣␣␣␣␣␣␣Parameters:
187 ␣␣␣␣␣␣␣␣-----------
188 ␣␣␣␣␣␣␣␣p1␣:␣float
189 ␣␣␣␣␣␣␣␣␣␣␣␣Control␣proportion
190 ␣␣␣␣␣␣␣␣p2␣:␣float
191 ␣␣␣␣␣␣␣␣␣␣␣␣Treatment␣proportion
192 ␣␣␣␣␣␣␣␣power␣:␣float
193 ␣␣␣␣␣␣␣␣␣␣␣␣Desired␣statistical␣power
194 
195 ␣␣␣␣␣␣␣␣Returns:
196 ␣␣␣␣␣␣␣␣--------
197 ␣␣␣␣␣␣␣␣dict␣:␣Sample␣size␣requirements
198 ␣␣␣␣␣␣␣␣"""
199         z_alpha = stats.norm.ppf(1 - self.alpha/2)
200         z_beta = stats.norm.ppf(power)
201 
202         variance = p1*(1-p1) + p2*(1-p2)
203         effect = (p2 - p1)**2
204 
```

```
        n_per_group = int(np.ceil(
            ((z_alpha + z_beta)**2 * variance) / effect
        ))

        return {
            'n_per_group': n_per_group,
            'n_total': 2 * n_per_group,
            'p1': p1,
            'p2': p2,
            'effect_absolute': p2 - p1,
            'effect_relative': (p2 - p1) / p1,
            'alpha': self.alpha,
            'power': power
        }

    def format_results(self, result: ABTestResult) -> str:
        """Format results for reporting."""
        report = f"""
A/B Test Results
================
Control Mean: {result.control_mean:.4f}
Treatment Mean: {result.treatment_mean:.4f}

Difference: {result.difference:.4f}
Relative Lift: {result.relative_lift:.2%}
Standard Error: {result.standard_error:.4f}

95% Confidence Interval: [{result.confidence_interval[0]:.4f}, {result.confidence_interval[1]:.4f}]

Test Statistic: {result.test_statistic:.4f}
P-value: {result.p_value:.4f}
Statistically Significant: {'Yes' if result.statistically_significant else 'No'}

Sample Sizes:
  Control: {result.sample_size_control:,}
  Treatment: {result.sample_size_treatment:,}
"""
        return report

# Example usage
if __name__ == "__main__":
    analyzer = ABTestAnalyzer(alpha=0.05)

    # Test proportions
    result_prop = analyzer.test_proportions(
        control_conversions=80,
        control_total=1000,
        treatment_conversions=95,
        treatment_total=1000
    )

    print(analyzer.format_results(result_prop))

    # Calculate sample size
    sample_size = analyzer.calculate_sample_size_proportions(
        p1=0.08,
```

```
        p2=0.09,
        power=0.80
    )

    print("\nSample Size Calculation:")
    print(f"  Required per group: {sample_size['n_per_group']:,}")
    print(f"  Total required: {sample_size['n_total']:,}")
```

.2.2 Experiment Analysis Pipeline (R)

Listing 3: Automated Ethical Guardrails

```
library(tidyverse)
library(broom)

# Comprehensive experiment analysis pipeline
analyze_experiment <- function(data, metric, variant_col = "variant",
                               alpha = 0.05) {
  #' Analyze A/B test experiment
  #'
  #' @param data DataFrame with experiment data
  #' @param metric Character, name of metric column
  #' @param variant_col Character, name of variant column
  #' @param alpha Numeric, significance level
  #'
  #' @return List with analysis results

  # 1. Pre-analysis checks
  cat("=== Pre-Analysis Checks ===\n")

  # Sample ratio mismatch
  variant_counts <- table(data[[variant_col]])
  srm_test <- chisq.test(variant_counts)
  cat(sprintf("SRM p-value: %.4f %s\n", srm_test$p.value,
              ifelse(srm_test$p.value < 0.01, "(WARNING)", "(OK)")))

  # 2. Summary statistics
  cat("\n=== Summary Statistics ===\n")
  summary_stats <- data %>%
    group_by(!!sym(variant_col)) %>%
    summarise(
      n = n(),
      mean = mean(!!sym(metric), na.rm = TRUE),
      median = median(!!sym(metric), na.rm = TRUE),
      sd = sd(!!sym(metric), na.rm = TRUE),
      se = sd / sqrt(n),
      .groups = 'drop'
    )

  print(summary_stats)

  # 3. Statistical test
  cat("\n=== Statistical Test ===\n")

  variants <- unique(data[[variant_col]])
  control <- data %>% filter(!!sym(variant_col) == variants[1]) %>%
    pull(!!sym(metric))
```

```r
  treatment <- data %>% filter(!!sym(variant_col) == variants[2]) %>%
    pull(!!sym(metric))

  # Welch's t-test (robust to unequal variances)
  test_result <- t.test(treatment, control, var.equal = FALSE)

  cat(sprintf("Test statistic: %.4f\n", test_result$statistic))
  cat(sprintf("P-value: %.4f\n", test_result$p.value))
  cat(sprintf("Significant: %s\n",
              ifelse(test_result$p.value < alpha, "Yes", "No")))

  # 4. Effect sizes
  cat("\n=== Effect Sizes ===\n")

  mean_control <- mean(control, na.rm = TRUE)
  mean_treatment <- mean(treatment, na.rm = TRUE)
  diff <- mean_treatment - mean_control
  relative_lift <- diff / mean_control

  cat(sprintf("Absolute difference: %.4f\n", diff))
  cat(sprintf("Relative lift: %.2f%%\n", relative_lift * 100))
  cat(sprintf("95%% CI: [%.4f, %.4f]\n",
              test_result$conf.int[1], test_result$conf.int[2]))

  # 5. Visualizations
  p1 <- ggplot(data, aes_string(x = variant_col, y = metric)) +
    geom_boxplot() +
    geom_jitter(alpha = 0.1, width = 0.2) +
    labs(title = "Distribution by Variant",
         x = "Variant", y = metric) +
    theme_minimal()

  # Density plot
  p2 <- ggplot(data, aes_string(x = metric, fill = variant_col)) +
    geom_density(alpha = 0.5) +
    labs(title = "Density Plot by Variant",
         x = metric, fill = "Variant") +
    theme_minimal()

  # Return results
  list(
    srm_check = list(p_value = srm_test$p.value,
                     passed = srm_test$p.value >= 0.01),
    summary = summary_stats,
    test = test_result,
    effect_size = list(
      absolute = diff,
      relative = relative_lift,
      ci = test_result$conf.int
    ),
    plots = list(boxplot = p1, density = p2)
  )
}

# Bayesian analysis function
bayesian_ab_test <- function(x_control, n_control, x_treatment, n_
    treatment,
                             prior_alpha = 1, prior_beta = 1,
```

```
                                n_sim = 100000) {
  #' Bayesian A/B test for proportions
  #'
  #' @param x_control Conversions in control
  #' @param n_control Sample size control
  #' @param x_treatment Conversions in treatment
  #' @param n_treatment Sample size treatment
  #' @param prior_alpha Beta prior alpha
  #' @param prior_beta Beta prior beta
  #' @param n_sim Number of Monte Carlo simulations
  #'
  #' @return List with Bayesian results

  # Posterior parameters
  post_alpha_control <- prior_alpha + x_control
  post_beta_control <- prior_beta + n_control - x_control
  post_alpha_treatment <- prior_alpha + x_treatment
  post_beta_treatment <- prior_beta + n_treatment - x_treatment

  # Simulate from posteriors
  p_control <- rbeta(n_sim, post_alpha_control, post_beta_control)
  p_treatment <- rbeta(n_sim, post_alpha_treatment, post_beta_treatment
    )

  # Probability treatment > control
  prob_treatment_better <- mean(p_treatment > p_control)

  # Expected lift
  lift <- p_treatment / p_control - 1
  expected_lift <- mean(lift)
  lift_ci <- quantile(lift, c(0.025, 0.975))

  # Expected loss
  loss_control <- mean(pmax(0, p_treatment - p_control))
  loss_treatment <- mean(pmax(0, p_control - p_treatment))

  list(
    prob_treatment_better = prob_treatment_better,
    expected_lift = expected_lift,
    lift_ci = lift_ci,
    loss = list(
      control = loss_control,
      treatment = loss_treatment,
      recommendation = ifelse(loss_control < loss_treatment,
                              "Control", "Treatment")
    )
  )
}

# Example usage:
# data <- read_csv("experiment_data.csv")
# results <- analyze_experiment(data, metric = "conversion")
# print(results$plots$boxplot)
```

.2.3 Experiment Simulation Framework

Listing 4: Automated Ethical Guardrails

```
import numpy as np
import pandas as pd
from typing import Callable

class ExperimentSimulator:
    """Simulate␣A/B␣tests␣for␣power␣analysis␣and␣methodology␣validation."""

    def __init__(self, seed=None):
        if seed is not None:
            np.random.seed(seed)

    def simulate_proportions(self, n_control, n_treatment,
                             p_control, p_treatment,
                             n_simulations=1000, alpha=0.05):
        """
␣␣␣␣␣␣␣␣Simulate␣A/B␣tests␣with␣binary␣outcomes.

␣␣␣␣␣␣␣␣Returns:
␣␣␣␣␣␣␣␣--------
␣␣␣␣␣␣␣␣dict␣:␣Simulation␣results␣including␣empirical␣power␣and␣Type␣I␣error
␣␣␣␣␣␣␣␣"""
        results = []

        for _ in range(n_simulations):
            # Generate data
            control = np.random.binomial(1, p_control, n_control)
            treatment = np.random.binomial(1, p_treatment, n_treatment)

            # Test
            p1 = np.mean(control)
            p2 = np.mean(treatment)

            p_pooled = (np.sum(control) + np.sum(treatment)) / (n_control + n_treatment)
            se = np.sqrt(p_pooled * (1 - p_pooled) * (1/n_control + 1/n_treatment))

            z = (p2 - p1) / se
            p_value = 2 * (1 - stats.norm.cdf(abs(z)))

            results.append({
                'p_value': p_value,
                'reject': p_value < alpha,
                'p1_hat': p1,
                'p2_hat': p2,
                'effect_estimate': p2 - p1
            })

        results_df = pd.DataFrame(results)

        # Calculate metrics
        empirical_power = results_df['reject'].mean()
        type_i_error = empirical_power if p_control == p_treatment else None

```

```
        return {
            'results': results_df,
            'empirical_power': empirical_power,
            'type_i_error': type_i_error,
            'mean_p_value': results_df['p_value'].mean(),
            'significant_count': results_df['reject'].sum()
        }

# Example: Verify power calculation
simulator = ExperimentSimulator(seed=42)
sim_result = simulator.simulate_proportions(
    n_control=1000,
    n_treatment=1000,
    p_control=0.10,
    p_treatment=0.12,
    n_simulations=10000,
    alpha=0.05
)

print(f"Empirical power: {sim_result['empirical_power']:.3f}")
print(f"Significant tests: {sim_result['significant_count']}/{10000}")
```

Statistical Tables

.3 Statistical Tables

This appendix provides reference tables for common statistical distributions and critical values used in A/B testing.

.3.1 Standard Normal Distribution (Z) Critical Values

Confidence Level	Significance (α)	$z_{\alpha/2}$
90%	0.10	1.645
95%	0.05	1.960
98%	0.02	2.326
99%	0.01	2.576
99.5%	0.005	2.807
99.9%	0.001	3.291

.3.2 Student's t-Distribution Critical Values ($t_{\alpha/2,df}$)

For two-sided tests at $\alpha = 0.05$:

Degrees of Freedom	$t_{0.025,df}$
5	2.571
10	2.228
15	2.131
20	2.086
25	2.060
30	2.042
40	2.021
50	2.009
60	2.000
80	1.990
100	1.984
∞	1.960

.3.3 Chi-Squared Distribution Critical Values ($\chi^2_{\alpha,df}$)

For significance level $\alpha = 0.05$:

Degrees of Freedom	$\chi^2_{0.05,df}$
1	3.841
2	5.991
3	7.815
4	9.488
5	11.070
6	12.592
7	14.067
8	15.507
9	16.919
10	18.307
15	24.996
20	31.410
25	37.652
30	43.773

.3.4 Sample Size Requirements (Two Proportions)

For $\alpha = 0.05$ (two-sided), power = 0.80, and baseline proportion p_1:

	Baseline Proportion (p_1)				
Relative Lift	**0.05**	**0.10**	**0.15**	**0.20**	**0.25**
5%	62,448	30,081	19,718	14,616	11,540
10%	15,396	7,337	4,784	3,540	2,790
15%	6,772	3,213	2,092	1,547	1,218
20%	3,788	1,793	1,166	862	678
25%	2,405	1,137	739	546	429
30%	1,650	780	507	374	294

Note: Values show sample size per variant.

.3.5 Minimum Detectable Effect (MDE)

For $\alpha = 0.05$, power = 0.80, and sample size n per variant:

	Sample Size per Variant			
Baseline (p_1)	**1,000**	**5,000**	**10,000**	**25,000**
0.05	3.95%	1.76%	1.24%	0.79%
0.10	5.32%	2.38%	1.68%	1.06%
0.15	6.31%	2.82%	2.00%	1.26%
0.20	7.08%	3.17%	2.24%	1.42%
0.25	7.66%	3.42%	2.42%	1.53%

Note: Values show absolute minimum detectable effect (e.g., 0.05 = 5 percentage points).

.3.6 Bonferroni Corrected Significance Levels

For overall $\alpha = 0.05$:

Number of Tests (m)	Corrected $\alpha^* = \alpha/m$
2	0.0250
3	0.0167
4	0.0125
5	0.0100
6	0.0083
8	0.0063
10	0.0050
15	0.0033
20	0.0025
25	0.0020
50	0.0010
100	0.0005

.3.7 Power vs. Sample Size

For two-proportion test with $p_1 = 0.10$, $p_2 = 0.12$ (20% relative lift), $\alpha = 0.05$:

Sample Size per Variant	Statistical Power
1,000	0.17
2,000	0.27
3,000	0.35
4,000	0.42
5,000	0.49
7,500	0.62
10,000	0.72
15,000	0.85
20,000	0.92
25,000	0.96
30,000	0.98

.3.8 Beta Distribution Quantiles

For Beta(a, b) distributions commonly used as priors:

Distribution	Mean	Mode	2.5% Quantile	97.5% Quantile
Beta(1, 1)	0.500	-	0.025	0.975
Beta(2, 2)	0.500	0.500	0.082	0.918
Beta(5, 5)	0.500	0.500	0.186	0.814
Beta(1, 9)	0.100	0.000	0.003	0.352
Beta(10, 90)	0.100	0.099	0.050	0.165
Beta(50, 450)	0.100	0.099	0.071	0.133

.3.9 Conversion Rate Effect Size Examples

Real-world context for effect sizes at different baseline rates:

Baseline	5% Lift	10% Lift	20% Lift	50% Lift
1%	1.05%	1.10%	1.20%	1.50%
5%	5.25%	5.50%	6.00%	7.50%
10%	10.50%	11.00%	12.00%	15.00%
20%	21.00%	22.00%	24.00%	30.00%
50%	52.50%	55.00%	60.00%	75.00%

.3.10 Design Effect for Cluster Randomization

Design effect $= 1 + (m - 1)\rho$ where m is cluster size and ρ is ICC:

	Intraclass Correlation (ρ)			
Cluster Size (m)	**0.01**	**0.05**	**0.10**	**0.20**
5	1.04	1.20	1.40	1.80
10	1.09	1.45	1.90	2.80
20	1.19	1.95	2.90	4.80
50	1.49	3.45	5.90	10.80
100	1.99	5.95	10.90	20.80

.3.11 Quick Reference Formulas

Standard Error for Proportion:

$$SE(\hat{p}) = \sqrt{\frac{p(1-p)}{n}}$$

Confidence Interval for Proportion:

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Z-Test Statistic for Proportions:

$$Z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\bar{p}(1-\bar{p})(1/n_1 + 1/n_2)}}$$

Sample Size for Proportions:

$$n = \frac{(z_{\alpha/2} + z_\beta)^2[p_1(1-p_1) + p_2(1-p_2)]}{(p_2 - p_1)^2}$$

Relative Lift:

$$\text{Relative Lift} = \frac{p_2 - p_1}{p_1} = \frac{p_2}{p_1} - 1$$

Minimum Detectable Effect:

$$\text{MDE} \approx (z_{\alpha/2} + z_\beta)\sqrt{\frac{2p(1-p)}{n}}$$

.3.12 Decision Matrix

Scenario	Recommended Approach
Small sample ($n < 100$)	Fisher's exact test or permutation test
Large sample, proportions	Z-test or chi-squared test
Continuous metric, normal	T-test (Welch's preferred)
Continuous metric, skewed	Transform data, use robust statistics, or permutation test
Multiple metrics ($m < 10$)	Bonferroni or Holm-Bonferroni correction
Multiple metrics ($m > 10$)	Benjamini-Hochberg FDR control
Sequential testing	Group sequential with spending function
Rare events ($p < 0.01$)	Longer duration, proxy metrics, or exact tests
Network effects	Cluster randomization with appropriate analysis